

Agentic Coding : 从 Vibe Coding 到 可托付的工程系统

让 AI 在目标、上下文、边界与验收之内，持续完成可检查的工程任务

CodeNow

CHINESE · DIGITAL READING EDITION



目录

第一章：什么是 Agentic Coding 10

1.1 Vibe Coding 的价值和边界 10

Vibe Coding 的价值：先让想法跑起来 11

Demo 之后，工程债开始浮出水面 12

中间过程不透明，风险被推到最后 14

缺的不是更多生成，而是承接结构 15

1.2 Agentic Coding 的定义 16

核心变化不是提示词变长 18

Codex 已经内置了一套成熟框架 19

文件约束代码质量 20

Skills 让经验可以被复用 22

验收让结果合你的需求 23

1.3 Agentic Coding 和 Vibe Coding 的核心区别 23

一句话可以直接生成，也可以触发工程理解 24

系统先读取上下文，再对齐意图 24

文档不是输入负担，而是系统回写的位置 26

真正的差异：谁来维护解题空间 27

从生成结果到维护项目状态 29

1.4 Agentic Coding 解决了什么问题？ 29

它解决 Demo 之后没人接住变化的问题 30

它解决五类债后移的问题 31

它解决代码规模扩大后控制力下降的问题 32

它解决 Agent Loop 跑久以后会漂移的问题 32

它解决完成信号不可信的问题 34

它解决开发成本失控的问题 35

它解决 AI 自主性无法委托的问题 36

1.5 人和 AI 的职责分工 37

人负责意图和业务意义	38
产品和设计负责把判断变成用户能理解的状态	39
人负责让意图进入中间过程	39
协作不是分工交接，而是持续意图对齐	42
人负责边界、权限和预算	42
人负责验收和接受风险	43
团队负责共享意图板，而不是共享感觉	44
AI 负责读取上下文和选择路径	46
AI 负责实现、验证、修复和回写	46
人负责 Review、经验沉淀和最终问责	47

第二章：控制和协作的核心文件 49

2.1 AGENTS.md：项目长期工作协议	49
它回答的不是“这次做什么”	52
长期有效，才值得写进去	53
一份够用的 AGENTS.md 长什么样	54
它应该短、准、可执行	56
不同目录可以有不同规则	57
AGENTS.md 需要从真实错误中生长	58
它不是万能安全网	59
2.2 CONTEXT.md：让 AI 拿到正确的世界	59
它回答“这个项目里的词指什么”	60
术语表怎么写	61
多人协作时，术语还要对齐角色视角	62
不要让它吞掉其他文件	63
旧资料会误导 AI，但答案不是再加一块清单	65
术语也会变，要及时回写	66
它不替代读取代码和设计	66
CONTEXT.md 让 Prompt 变轻	67
2.3 PRD.md：定义目标和边界	67
PRD 是工程合同，但工作在目标层	68

一份轻量 PRD 可以长什么样	69
这五个部分各自钉住一个判断	72
PRD 要把抽象目标压成成功现象	74
把非目标写进 PRD 的好处	75
当项目增长：PRD 也可以按模块拆分	76
一份 PRD 是否合格	77
2.4 DESIGN.md：让设计系统进入工程过程	77
把设计取向写成工程语言	78
一份 DESIGN.md 包含什么	80
DESIGN.md 还可以包含什么？	81
DESIGN.md 可以写「不要这样做」	82
DESIGN.md 只承接设计依据	83
进入 TODO 和验收	83
DESIGN.md 也会过期	84
2.5 TODO.md：记录当前执行状态	84
TODO 不应该只写实现动作	85
一个可执行的 TODO 长什么样	86
TODO 要能恢复任务	87
状态比勾选更重要	88
阻塞要写出来	89
任务粒度决定 Review 成本	90
TODO 是长任务的控制面	91
TODO 是回写的第一站	91
一个 TODO 项是否合格	92
2.6 ACCEPTANCE.md：把完成变成可检查场景	93
验收不是最后盖章	93
抽象要求要拆成场景	93
场景要覆盖不可接受结果	95
测试是验收证据之一	96
验收也要说明不覆盖什么	97
ACCEPTANCE.md 要成为完成标准的事实源	98

PRD 和 acceptance 可以住在同一个模块文件夹	98
一份轻量 ACCEPTANCE.md 可以长什么样	100
验收文件也需要回写	101
一份 ACCEPTANCE.md 是否合格	102
2.7 六个文件如何协同	103
一条需求，多份文件各写一部分	104
执行之后，按事实归位回写	106
冲突要被暴露，而不是被代码吞掉	107
按任务风险决定文件出场强度	108
第二章的收束	108

第三章：搭建自己的 Agentic Coding 脚手架 110

3.1 为什么需要搭建自己的脚手架	110
脚手架不是把文件自动生成出来	111
从工厂模型看脚手架	111
为什么值得投入	113
脚手架改变人的控制点	113
从最小脚手架开始	114
3.2 基于现成经验和自己的角色构建 Skills	115
Skill 是动态上下文，不是更长 Prompt	115
Skill 解决哪几类问题	116
Skill 通常包含什么	117
先使用现成 Skills，再写自己的	118
把 LeadFlow 的经验写成项目 Skill	120
什么时候才该写自己的 Skill	121
Skill 的边界	122
3.3 基于任务复杂度设计 SubAgents	123
SubAgent 是受控分工，不是更聪明的 AI	124
先问是不是 Skill 就够了	124
规划和执行有时也要分离	125
LeadFlow 可以先建哪些 SubAgents	126

什么时候不要用 SubAgent	127
写 SubAgent 前问五个问题	128
3.4 基于需求设计 Rules	129
Rule 是静态上下文，不是知识仓库	130
薄内核：AGENTS.md 只放最稳定的规则	131
Rule 应该从需求和风险里长出来	132
Rule 要有明确作用域	134
什么时候不要写 Rule	134
LeadFlow 的最小 Rules 集	135
Rules 也要被维护	136
写 Rule 前的五个问题	137
3.5 MCP 与 Hooks：外部接入与执行拦截	137
MCP：把外部系统变成可授权的上下文	138
Hooks：把关键检查放到生命周期节点	139
各层都要留下证据	141
从工具清单到工作系统	142

第四章：完成 LeadFlow 第一版 144

4.1 从 0 到 1：用 PRD 和 DESIGN 完成 LeadFlow MVP	144
有文件和没有文件，Codex 的行为差在哪里	145
PRD 和 DESIGN：对实现路径影响最大的一组文件	146
先用脱敏示例数据跑通流程，再考虑真实接入	147
把 PRD 和 DESIGN 交给 Codex	149
MVP 跑起来之后	150
项目完成后大致长什么样	151
4.2 任务进入：从反馈、问题和评论进入下一轮 TODO	151
反馈先不要急着实现	152
任务来源不只来自聊天	153
先判断它应该落到哪里	154
把反馈压成本轮最小变化	155
用评论保留上下文，而不是只保留情绪	157

终端错误也要翻译成任务 158

`/status`、`/review` 和 `worktree` 是入口辅助，不是任务本身 159

让 Codex 帮你整理入口 159

从 4.2 交棒给 4.3，至少要确认四件事 160

4.3 开发执行：三类信号，如何知道 Codex 走在正确路上 161

先说清楚：这和普通 AI 改代码有什么不同 162

委托颗粒度：什么可以让 Codex 连续做，什么必须停下来 163

小步计划要和信号对应 165

信号一：命令反馈证明机器层面是否成立 165

信号二：页面状态证明用户现场是否成立 166

信号三：diff 范围证明委托边界是否还在 167

权限提示是风险边界，不是打扰 168

早期失控信号：什么时候必须介入 169

Skills 和 Rules：不要抢戏，但要进入正确位置 170

自检报告要说明证据强度 170

交棒给 4.4 前，至少确认五件事 171

4.4 验收回写：验证、Debug、Git 封存，再进入下一轮 172

从自检报告进入正式验收 173

证据要支撑具体主张 174

让浏览器检查带着场景走 175

日志和终端要证明没有越过隐私边界 176

验收失败时进入 Debug，而不是重写 177

复验要回到原场景 178

回写 TODO：把执行状态留给下一轮 178

回写验收和设计：只改事实变化的地方 179

长期规则只沉淀反复出现的问题 180

最终 diff / review：确认范围没有变形 181

Git commit 不是完成证据 181

结束时要留下四类东西 183

下一轮从哪里来 184

第五章：我是如何构建项目冷启动的脚手架	185
5.1 CodeNow 整个脚手架里有什么	185
`AGENTS.md`：薄入口，而不是超级说明书	187
`.codex/skills/`：把生命周期里的关键动作做成节点	188
`.codex/agents/`：专家 worker	189
`.codex/rules/`：长期边界	190
真正撑起 CodeNow 的，是状态交接	192
5.2 整个冷启动的设计思路是什么	193
01：为什么只生成一个 prd.md	194
02：为什么准备阶段不能碰代码	197
03：为什么首版开发是一个独立类别	198
update-prd：为什么事实只能在开发后写	200
设计的真正目的	201
5.3 冷启动之后，日常迭代的四个节点是怎样分工的	202
四个节点，四道隔离墙	202
有顺序，但不自动链	204
`project-iterate`：在实现时就定好"做完是什么意思"	205
`project-verify`：让"打勾"这个动作有可核查的含义	206
`project-debug`：先收束根因，再修复，而不是"改一下试试"	207
`update-prd`：让 Codex 下一轮进来时读到的是真实的项目状态	208
git commit 是这套流水线的同步锁	209
设计自己的系统时，先抓五个维度	210
5.4 如何设计 `AGENTS.md` 和 rules	210
`AGENTS.md`：接上现场，走对路径	211
`lifecycle.md`：同一句话在不同阶段是不同任务	213
`kernel.md`：每轮都要进入上下文的最低铁律	214
`acceptance-contract.md`：把"做完"从感觉里拿出来	214
`todo-writeback.md`：让下一轮不用翻聊天记录	215
`verification-levels.md`：自检要轻，验收要硬	216
`runtime-contract.md`：脚手架自己也会漂移	217
`coach.md`：新手路径与执行流程分开走	217

rules 的设计方法：先写失败，再写边界	218
5.5 如果你要设计自己的 Agentic Coding 脚手架，应该考虑什么	219
不要先设计目录，先找到失控点	219
先把任务入口、验证和边界做扎实	221
迁移到你的项目之前，先问六个问题	222
一条克制的建设顺序	226
每一层都要有边界，也要有删除标准	229
CodeNow 不是答案，而是一种写法	231
结尾：把能力变成可以托付的系统	232

什么是 Agentic Coding

1.1 Vibe Coding 的价值和边界

一支三五人的销售团队，往往要同时应对好几件让人头疼的事：线索从表单、社群、活动、转介绍和私信里进来，渠道各不相同；客户沟通散在邮件、电话纪要和聊天记录里，没有统一的地方汇总。销售成员想知道客户到底提过什么需求；负责人想看到哪些机会正在变冷、下一步该重点跟谁；管理层想了解整体进展，不只靠口头汇报；而每个人都担心：哪些承诺不能随便说出口，哪些内容不能直接发给客户。于是，一个很自然的念头出现了：能不能做一个轻量 CRM，让 AI 帮忙整理线索、总结需求、提示风险，并给出下一步跟进建议？

今天我们来一起做一个 CRM 工具，作者会把这个工具命名为 LeadFlow。它的第一版听起来并不复杂：导入销售线索和客户沟通记录，展示客户详情，让 AI 基于已有记录生成总结和建议。但只要稍微往真实使用靠近一步，问题就会变得尖锐：客户没有提预算时，AI 能不能猜；AI 建议看起来很确定时，你会不会把它当成客户承诺；手机号、邮箱和合同金额能不能进入日志；一条建议被生成、被确认、被执行、被对外发送，界面上该不该是同一种状态。

在定义 Agentic Coding 之前，先看更常见的起点。很多人第一次认真体验 AI 编程，不是从工程规则开始，而是从一个很自然的愿望开始：我有一个想法，我想马上看见它。Vibe Coding 正是把这个愿望变成软件原型的一种方式。

Vibe Coding 的价值：先让想法跑起来

2025 年 2 月，Andrej Karpathy 用“Vibe Coding”描述了一种新的编程体验：人主要用自然语言提出要求，接受 AI 的修改，遇到错误就把错误继续交给 AI，有时甚至暂时不再关注代码本身。这个词后来被更宽泛地使用，用来指代那种凭直觉、语言和感觉推动 AI 生成软件的过程。它最适合周末实验、一次性工具、早期原型和产品方向探索。

在 LeadFlow 的起点上，Vibe Coding 式的请求通常非常直接，例如你可以在 Google 的 AI Studio 上这样对 Gemini 说：

帮我做一个销售线索管理工具，可以导入客户线索，AI 自动总结客户需求，并给出下一步跟进建议。

几分钟后，一个页面出现了。左边是线索列表，右边是客户详情，中间有客户来源、跟进阶段、AI 总结、意向评分和下一步建议。它能点击，能切换状态，甚至还能生成一段像销售助理写出的跟进建议。

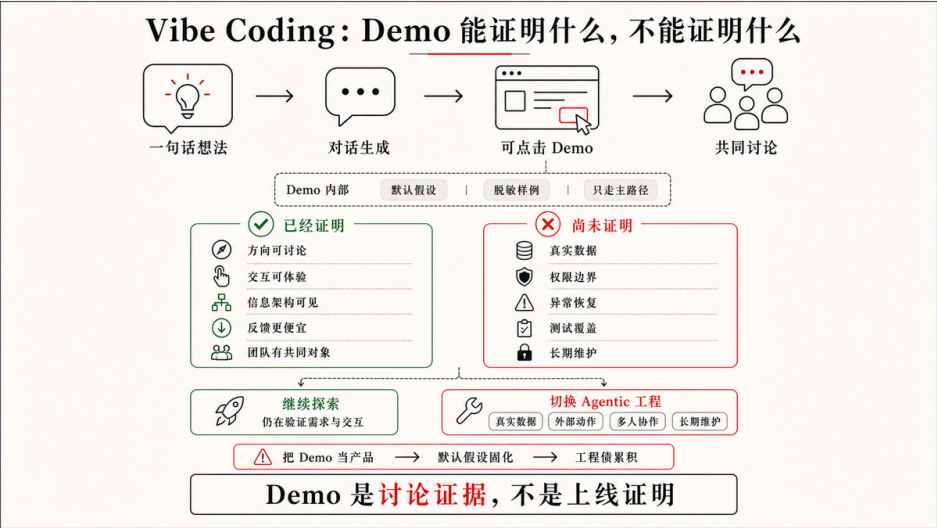


图 1-1 LeadFlow 第一版 Vibe Demo 界面

过去，一个想法要变成可点击的软件，需要产品草图、设计稿、技术选型、页面结构、数据模型、前后端实现和一堆调试。现在，你只要把感觉说出来，AI 就能把它变成一个看得见、摸得着、能继续讨论的东西。

Vibe Coding 降低的是软件表达的门槛。人不必先证明自己会写代码，才能开始表达自己想要的软件；团队也不必在第一天就把所有需求、边界和架构想清楚，才能判断这个方向有没有价值。对 LeadFlow 这种还在探索阶段的产品来说，先看到一个 Demo，往往比先写二十页需求文档更有效。

这也是为什么我很愿意承认 Vibe Coding 的价值。它特别适合做自己的工具：一个内部脚本、一个内容整理器、一个临时看板、一个只服务自己的自动化流程。你知道它的数据从哪里来，知道哪里只是临时拼的，知道坏了以后自己能不能接受。这个阶段的目标不是商业级交付，而是让想法出现、让自己用起来、让问题变得可讨论。

但这个边界也要说清楚。个人工具可以靠开发者自己的记忆和容忍度维持，但 LeadFlow 从第一天起就要承接真实线索、真实销售动作、客户隐私和对外承诺，它不能只靠“看起来能用”推进。Vibe Coding 可以帮你把工具做出来，但它默认不会替你建立需求边界、设计状态、权限策略、验收证据和长期维护秩序。

Demo 之后，工程债开始浮出水面

你看完 LeadFlow 的 Demo，觉得方向对了，于是开始补充更真实的要求：

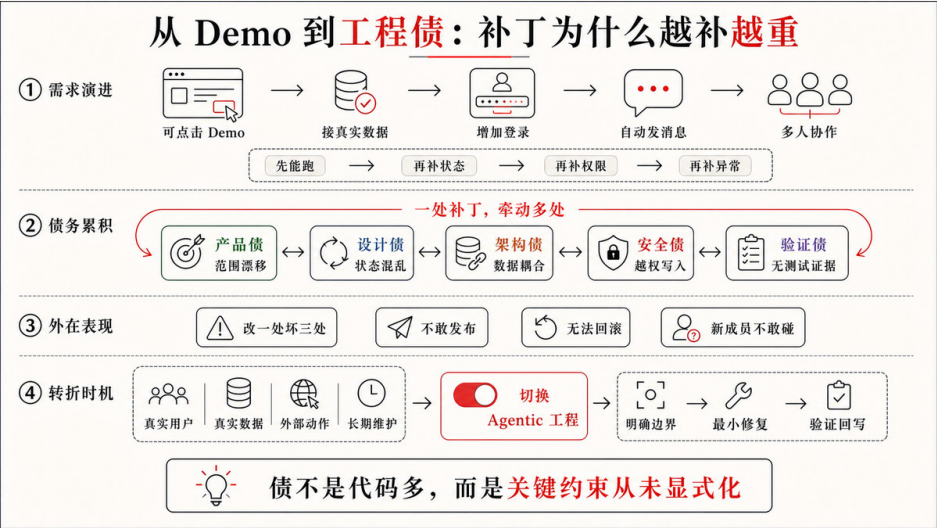
AI 总结必须基于真实沟通记录。
客户没有提预算时不要猜预算。
下一步建议要先由销售确认，不能自动对客户承诺。
客户手机号、邮箱和合同金额不能进日志。
页面要清楚区分“AI 建议”“销售已确认”和“已对外承诺”。
后面可能还要接真实 CRM 和团队权限。

这时，LeadFlow 可能开始暴露一些你在意的问题：AI 总结到底来自哪条沟通记录，还是模型凭常识补全？意向评分是基于客户行为、销售阶段，还是一个好看的数字？“建议下周发送报价”是一条销售草稿，还是系统已经默认可以对客户发出承诺？如果客户资料缺少关键字段，界面是提示“信息不足”，还是

继续显示一段看似确定的建议？

如果说上面这些疑问关心的是“输出能不能信”，那么继续让 AI 修改，还会带来另一类问题：代码内部正在变乱。它确实还能继续生成代码，可以加一个来源标签，也可以补一个“确认建议”按钮，还可以再接一个“导入 CRM”的入口。每一小步看起来都在前进，但代码内部可能已经出现了三套线索结构、两套阶段判断、一组没人再使用的假数据，以及一堆只为让页面跑起来而临时拼上的状态。

输出不可信、结构在变乱，这两类问题合起来，就是 AI 时代的工程债。它不是因为某一行代码看起来不够漂亮，而是因为每一轮生成都在局部补洞，却没有一个稳定的工程结构承接这些洞。需求边界没有稳定，设计状态没有定义，架构理由没有留下，隐私和权限没有进入约束，验收和安全检查被拖到以后，代码为什么这样组织也没有人能解释清楚。



这些问题不会因为 AI 继续生成而自然消失，反而会越积越高。生成速度越快，这些没有被接住的债务就堆得越多，直到某一天团队发现软件已经“看起来很完整”，却越来越不敢改。

代码规模会把这个问题放大。几百行、几千行的个人工具，即使结构粗糙，人也可能靠记忆和搜索把它修回来；一旦项目进入数万行级别，尤其超过三万行代码以后，真正困难的就不再是“AI 能不能继续写”，而是人和 AI 是否还能共同理解哪些结构是当前有效的，哪些路径已经废弃，哪些行为不能破坏，哪些测试和页面结果才说明改动可信。三万行不是一个科学分界线，而是作者认为的一个工程经验上的警戒线：当代码已经大到不能靠感觉通盘掌握时，继续用不审核代码、不维护上下文、不沉淀验收的方式推进，控制力会明显下降。

中间过程不透明，风险被推到最后

Vibe Coding 还有一个容易被忽略的问题：中间过程经常不透明。人看到的是一个能跑的页面、一段顺畅的回答、一个看起来更智能的交互，但很难知道 AI 在中间补了哪些假设、绕过了哪些失败、修改了哪些无关文件，又把哪些风险藏进了默认行为里。

对 CRM 工具来说，这种不透明尤其危险。AI 可能为了让建议看起来完整，默认补出客户预算；为了让页面不报错，继续显示示例客户；为了让日志方便调试，把手机号和邮箱一起打印出来；为了让状态流转顺畅，把“销售已确认”和“已对外承诺”混成同一个字段。界面越像一个成熟产品，这些中间假设越容易被误认为已经经过设计和验证。

人如果只在最后看结果，就只能做一种粗粒度判断：这个页面感觉对不对。可是工程风险往往发生在结果形成的过程中：AI 读了哪些上下文，依据了哪份规则，选择了哪条实现路径，跑了哪些检查，失败时怎么处理，越过边界时有没有停下来请求确认？只看最后的结果，这些问题都无从回答，人的控制就会退化成事后猜测。

这也是很多团队在 AI 编程里感到不安的原因。问题不只是“AI 会不会写错代码”，而是“我不知道它为了得到这个结果到底做了什么”。当过程看不见，团

队 Review 时就只能盯着最终页面和最终 diff，这时候很难判断哪些风险已经被覆盖，哪些风险只是暂时没有暴露。

缺的不是更多生成，而是承接结构

把前面的两个问题放在一起看，真正危险的地方就清楚了：一边是工程债在增加，一边是中间过程看不见。继续让 AI “再生成一点”当然还能推动页面变完整，但它不能自动回答建议基于什么、状态如何流转、哪些信息不能泄露、哪些检查已经做过、哪些风险应该停下来让人判断。

如果你想解决这个问题，你缺的不是一个更聪明的补全动作，而是一套能承接 AI 执行的工程结构。它要把目标、上下文、设计约束、边界、权限、验收、反馈和回写组织起来，让 AI 的行动不再只靠当下对话里的感觉，而是进入一个可观察、可限制、可验证的工作过程。后面我们会把这种承接结构称为 Harness 工程。

Harness 工程可以先理解成“把 AI 的能力接进工程现场的那套结构”。在这套结构里，模型负责理解和生成，工具负责读写、命令、浏览器和外部连接，规则给出长期约束，权限和审批划定影响范围，验收标准定义什么时候算做完，测试、日志、截图和 diff 则把结果变成可核对的证据。没有这套结构，AI 仍然能生成代码，但每一轮都像在临场发挥；有了它，AI 才能在边界内持续行动，并把结果交回给人审查。

把这套承接结构建起来、让 AI 真正在里面工作，正是 Agentic Coding 要做的事。它不是否定 Vibe Coding，而是在方向初步成立之后，把软件修改放回真实工程环境里，让 Codex 这类 Coding Agent 在目标、上下文、设计约束、边界、权限和验收标准内完成一轮可检查的工程任务。

读到这里，你可能会有一个担心：既然这套承接结构要管目标、上下文、设计约束、边界、权限和验收，那是不是意味着，我以后让 AI 做事，每次都得先把这一长串东西写成规范，才能开始？

恰恰相反。承接结构的意义，就是把这些长期约束从每一次对话里搬出去，沉淀进项目本身。所以在真实项目里，人提需求往往还是一句话，甚至更短。

同样是做 LeadFlow，你说出口的可能仍然是：

帮我做一个销售线索管理工具，要有 AI 总结和跟进建议。

区别不在这句话有多长，而在它背后有没有那套承接结构。约束越早沉淀进项目，人需要在当下说清的细节反而越少，系统即使只听到一句话也能接住；同一句话进入系统后，是被直接当成生成指令，还是先被项目上下文解释、校准，再变成可验证、可交接的状态，正是后面两节要展开的事。

Vibe Coding 让想法更快出现，Agentic Coding 让已经出现的方向进入真实工程。理解这一点之后，下一节才适合正式定义 Agentic Coding：它到底委托给 AI 什么，又要求人把哪些控制点提前放进系统里。

1.2 Agentic Coding 的定义

上一节的结尾已经把话题带到 Agentic Coding 的门口：Vibe Coding 能很快把 CRM 这样的想法变成可见的 Demo，但 Demo 之后真正缺的，是一套能承接 AI 执行的工程结构，也就是 Harness；让 AI 真正在这套结构里工作，就是 Agentic Coding。问题是，“在承接结构里工作”还只是一个轮廓。它没有说清楚：人到底交出什么，AI 又在什么条件下完成什么、对什么负责。

所以，定义 Agentic Coding 时，不能只看“人怎样提示 AI”。真正要看的，是 AI 在什么工作系统里写代码。在这本书里，作者认为 Agentic Coding 可以这样定义：

人用目标、上下文、设计约束、边界、权限、规则和验收标准发起工程委托；Coding Agent 在一套内置的 Harness 工程底座中读取项目、选择路径、修改代码、运行验证、修复失败，并把证据和风险交还给人判断。

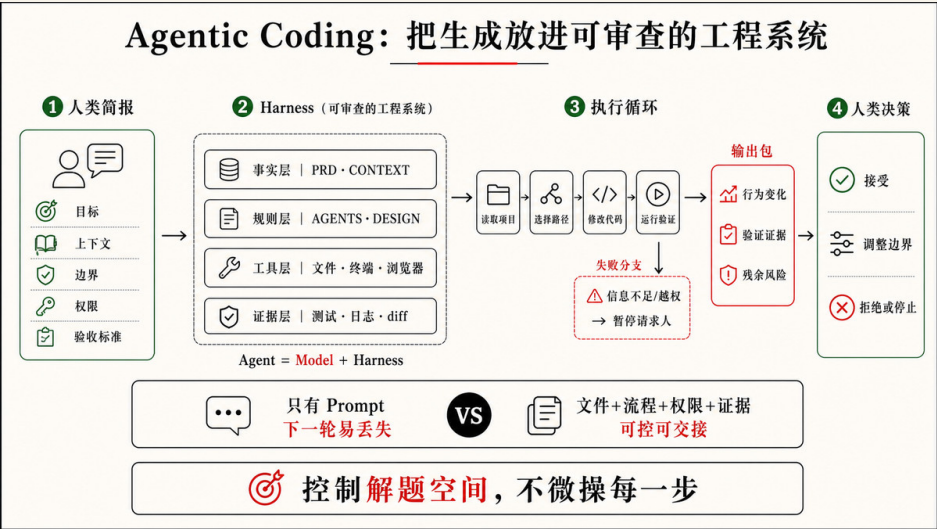


图 1-2 Agentic Coding 的定义

这个定义故意把“写代码”放在中间，而不是放在全部。真实项目里，一次代码变更还要面对现有架构、历史代码、运行环境、设计状态、团队规则、权限边界、测试证据和验收标准。Agentic Coding 要解决的，不是让 AI 生成更多代码，而是让 AI 生成的代码被一套工程系统约束，最后更接近人的真实需求。

这里可以借用一个更直观的公式：

Agent = Model + Harness

模型当然重要，它提供理解、推理和生成能力。但在真实工程里，模型不是整个系统。指令、规则、上下文、工具、沙箱、审批、Hooks、Skills、测试、日志、diff、可观测性和人的 Review，才共同决定这个模型能不能把一件事做完、做对、做得可审查。Google 的一份 AI 工程 playbook 《The New SDLC With Vibe Coding》里用“模型只占 10%”来提醒团队不要把所有成败都归因于模型强弱。这个比例不必被理解成精确数字，它真正有价值的地方在于指出：团队最能控制、最应该投入的，往往是模型周围那套 Harness。

核心变化不是提示词变长

上面这个定义一口气列了目标、上下文、设计约束、边界、权限、规则 and 验收标准。只看这串清单，很容易以为 Agentic Coding 就是把它们整理成一段更完整的 Prompt：

```
目标是什么？
上下文是什么？
限制是什么？
完成标准是什么？
```

这些当然有用。一个清楚的任务表达，永远比一句“帮我做得更智能”可靠。但如果 Agentic Coding 只停留在这几行提示词，它很快会遇到上限。

因为 Prompt 是一次性的。它只能在当前对话里提醒 AI：这次要注意什么。可真实项目里的很多要求不是一次性的，比如 CRM 里“AI 建议必须带来源”“不能编造客户预算”“手机号和邮箱不能进日志”“销售确认前不能标为已执行”。这些要求如果每次都靠人临时输入，迟早会漏掉。

Agentic Coding 的关键，是把这类判断从临时对话里移出来，放进 AI 能持续读取、执行和回写的系统里，例如：

把一次性任务留在 Prompt 或 `TODO.md` 里；把长期工作协议放进 `AGENTS.md`；当前可信事实进入 `CONTEXT.md`；产品目标、非目标和成功现象进入 `PRD.md`；视觉 Token、UI 技术栈、组件策略、界面流程、状态和风险表达进入 `DESIGN.md`；什么结果可以接受，进入 `ACCEPTANCE.md`；重复出现的工作流，可以做成 Skill；别人已经封装好的成熟流程，也可以作为 Skill 下载、安装和复用；需要连接外部工具，可以用 MCP；需要在生命周期里自动检查，可以用 Hook；需要团队分发，就可以进一步打包成 Plugin 等等。

这时，AI 写代码时面对的就不再只是一段自然语言，而是一组可以共同约束它行动的工程对象。

Codex 已经内置了一套成熟框架

这也是为什么本书会以 Codex 为主要样本。Codex 不是一个只会回答问题的代码模型。OpenAI 当前文档把 Codex 定位为面向软件开发的 Coding Agent：它可以写代码，理解陌生代码库，审查代码，调试修复问题，并自动化一部分开发任务。更重要的是，Codex 已经把 Agentic Coding 需要的几层能力组织在一起。

可以把 Codex 的内置框架粗略理解成这样：

```
项目规则和上下文
→ 任务路由与策略判断
→ 文件、终端、浏览器、MCP 等工具执行
→ 沙箱与审批边界
→ Skills / Plugins / Hooks 等可复用机制
→ 测试、日志、截图、diff 和最终汇报
→ 人的反馈、Review 和最终验收
```

这套框架让 Codex 进入项目时不是空手猜测。它会读取项目规则，例如官方文档建议放在 `AGENTS.md` 里的仓库结构、常用命令、工程约束、完成标准和禁区。它能根据任务判断自己应该答疑、改代码、调试、查文档、跑验证，还是请求授权。它可以读文件、搜索代码、执行命令、编辑文件、查看 diff、操作浏览器，并把失败结果带回下一轮判断。它的自主性也被沙箱和审批限制：在已授权范围内可以连续推进，触碰网络、越界文件、危险命令或高风险动作时必须停下来请求确认。

所以，当一次 AI 编程结果不理想时，第一反应不应该只是“换个更强模型”。模型可能确实不够强，但很多失败其实来自 Harness：规则太虚，验收太粗，上下文过期，工具不可用，沙箱和权限没有设计好，或者没有留下可复查的验证证据。Agentic Coding 的工程性，就体现在这些可调整、可审查、可沉淀的部分上。

2026 年，Karpathy 在与 Stephanie Zhan 的一场访谈里，把话题从 Vibe Coding 推向 Agentic Engineering。这个转向背后的判断很重要：当 AI 生成代码的能力已经足够强，真正的瓶颈就不再只是“能不能写出来”，而是“怎样设计一套 AI 可以持续参与、可观察、可验证、可审批、可回写的系统”。

从这个角度看，Codex 已经提供了一套可直接使用的 Agentic Engineering 底座，也就是上一节说的那种承接 AI 执行的 Harness 工程。你不必第一天就自己发明工作区、工具调用、权限审批、Skills、MCP、Hooks、验证和 Review 机制。你可以先站在 Codex 这套现成底座上，把自己的目标、项目规则、业务约束和验收标准放进去。

文件约束代码质量

Agentic Coding 的一个核心动作，是把“你希望 AI 怎么写代码”外部化，让产品判断、架构偏好和风险边界在实现之前就写进项目，而不是留给 AI 在补空白时临场决定。AI 补出来的常常能跑、也像回事，却不一定是你想要的那条路。如果你只说：

帮我接入 AI 跟进建议。

AI 可能真的会做出一个建议卡片。它可能顺手接入模型 API，也可能加一个“发送给客户”的按钮，还可能为了让页面看起来完整，在没有沟通记录时显示一段通用销售话术。页面会变丰富，但代码质量和产品风险不一定更好。

换成 Agentic Coding，约束不再集中在一段 Prompt 里，而是更早进入系统的不同文件。比如，同样是“做 AI 下一步跟进建议”，它在项目里可能被拆成这样：

PRD.md

- 产品目标：帮助销售基于真实沟通记录判断下一步跟进，而不是让 AI 生成听起来专业的话术。
- 成功现象：销售能在客户详情页看到可追溯的 AI 建议，并在确认后决定是否执行。
- 非目标：不做完整 CRM，不做自动外呼 / 自动邮件，不让 AI 直接对客户作出承诺。

CONTEXT.md

- 当前版本只使用脱敏样例线索和沟通记录。
- 当前沟通记录支持 leadId、channel、timestamp、author、body 等字段。
- 当前来源粒度是沟通记录级，真实 CRM 同步尚未启用。
- 旧静态建议文案不再作为当前产品事实。

DESIGN.md

- 设计系统方向：轻量 CRM 后台，信息密度高于营销页，界面应克制、清晰、便于扫描。
- 视觉 token：主色、表面色、正文色、间距、圆角、字号和行高使用统一 token，不在组件里随意写死。

- UI 技术栈：Tailwind + CSS variables，图标库统一使用 lucide-react，复杂弹层和下拉遵循同一组件策略。
- 客户事实、AI 建议和销售确认必须在界面上分区展示。
- AI 建议卡片默认是 draft / waiting for confirmation 状态。
- 来源入口必须靠近建议正文，方便销售展开复查。
- 无沟通记录时显示信息不足状态，不展示通用销售话术。
- MVP 主按钮是“确认为跟进计划”，不是“发送给客户”。

AGENTS.md

- 默认使用脱敏样例数据，真实客户数据、真实 CRM、外部模型调用和自动发送能力都需要明确授权。
- 不得把手机号、邮箱、合同金额、授权信息或完整客户原文写入日志。
- AI 建议相关改动必须保留来源、确认状态和隐私边界。
- 交付时说明行为变化、验证证据、未验证项和残余风险。

TODO.md

- [] 客户详情页展示 AI 下一步建议
- 依赖：当前线索有可读取的沟通记录。
- 设计：见 `DESIGN.md#AI-Suggestion-Card`
- 验收：见 `ACCEPTANCE.md#AI建议必须显示来源`
- 状态：未开始
- [] 无沟通记录时显示信息不足
- 验收：见 `ACCEPTANCE.md#沟通记录不足时不生成建议`
- 状态：未开始

ACCEPTANCE.md

- Given 某条线索有沟通记录，When 系统生成下一步建议，Then 建议必须关联来源记录并允许展开查看。
- Given 某条线索没有沟通记录，When 用户查看 AI 建议区域，Then 页面显示信息不足，不生成客户需求或预算判断。
- Given 销售尚未确认建议，When 页面展示建议状态，Then 建议不得出现在已执行跟进记录中。

这些内容不一定要在第一天全部写满，也不一定每个项目都照抄这六个文件，但是它们是关键：目标、事实、设计系统、长期规则、当前任务和验收标准被放到了不同位置，各自承担不同职责。它们不是一段被拆行的 Prompt，而是项目系统的一部分。

这里没有规定 Codex 先改哪个文件、后写哪个函数。人控制的不是每一步操作，而是解题空间。Codex 仍然可以自己读取代码、判断现有结构、选择实现路径和运行验证；但它的选择会被这些文件里的目标、事实、设计系统、规则、任务状态和验收场景约束。

这就是 AGENTS.md、TODO.md、ACCEPTANCE.md 等文件的意义。它们不是文档洁癖，而是把代码质量要求变成 AI 可以读取的工程条件。没有这些外部状态，质量只能靠人最后 Review 时发现问题；有了这些状态，质量可以提前进入 AI 的执行过程。

Skills 让经验可以被复用

文件主要承载项目状态和长期规则。Skills 承载的是另一类东西：一套可复用的工作流。

如果你每次都对 AI 说：

做 UI 改动前先看设计约束；
启动页面后检查桌面和移动端；
截图看有没有重叠和溢出；
改完后说明视觉验证结果。

这就不应该永远停留在 Prompt 里。它可以变成一个 UI 验收 Skill。以后遇到类似任务，Codex 只要判断任务匹配这个 Skill，就能读取完整流程、参考资料和脚本。

更重要的是，Skill 不只来自你自己。Codex 支持系统内置 Skill、用户本地 Skill、仓库 Skill，也可以通过安装器或插件使用别人已经写好的 Skill。换句话说，Agentic Coding 的经验来源可以是个人踩坑后的沉淀，也可以是团队共享的流程，还可以是社区或第三方已经封装好的成熟做法。

这会改变 AI 编程的学习方式。过去，你要把所有经验都写进自己的提示词；现在，你可以把一部分经验安装进工作环境，让 Codex 在合适任务里按需加载。Skill 不是更长的 Prompt，而是别人或自己沉淀下来的可复用经验。

当然，Skill 也不是越多越好。一个坏 Skill 会把过期流程变成新的噪音，一个太重的 Skill 会让小任务背上不必要的流程成本。判断一个 Skill 是否值得使用，仍然要回到任务风险：它是否减少重复解释，是否提高质量，是否让验收更清楚，是否让 Codex 少走错路。

验收让结果合你的需求

最后，Agentic Coding 必须用验收收口。AI 写出代码、页面能打开、测试变绿，都只说明某个动作发生过，并不说明这次结果符合人的需求、覆盖了关键风险。所以在开发 LeadFlow 的时候，“AI 跟进建议做完了”不是一个合格的完成标准；“建议必须带来源、确认前不得标为已执行、手机号和邮箱不进日志，并说清未验证项”，才更接近 Agentic Coding 的完成。

这些验收条件会反过来约束 Codex 的实现：建议卡片做出来但没有来源，它不该停；隐私日志没有检查，它应该说明证据缺口；测试无法运行，它不能把“没跑”包装成“通过”。完成信号为什么不能只靠感觉、验收强度又如何匹配风险，后面会专门展开；这里只需先记住一点：验收是定义的一部分，不是事后追加的环节。

从这个角度看，Agentic Coding 的核心不是让 AI 更自由，而是让 AI 的自由发生在可检查的边界内。Codex 负责读取、实现、验证、修复和汇报；人负责定义目标、边界、规则、权限、验收和最终责任。

第一章先把定义讲到这里：Vibe Coding 让想法出现，Agentic Coding 借助 Codex 这样的成熟框架，把想法放进目标、文件、Skills、权限和验收组成的工程系统里，让一轮代码变化可控、可验、可交接。下一节把两者正面摆在一起比较：你会看到，它们的差别不在谁更高级，而在同样一句话交给系统之后，谁还在维护那个可以继续解释、验证和交接的解题空间。

1.3 Agentic Coding 和 Vibe Coding 的核心区别

前两节已经反复说到同一点：区别不在这句话有多长。现在把 Vibe Coding 和 Agentic Coding 正面放在一起，要追的就是另一个问题——同样一句话交给系统，接下来会发生什么不同。

真实开发里，人提需求经常仍然是一句话。产品经理、设计师、工程负责人或业务同学不会每次都先写好 `PRD.md`、`TODO.md`、`DESIGN.md` 和 `ACCEPTANCE.md`，再把任务交给 AI。更多时候，人只是指出一个问题：

这个建议区域还不够好，帮我优化一下，让它更智能、更像销售助手。

差别就在系统接到这句话以后做什么。

一句话可以直接生成，也可以触发工程理解

在 Vibe Coding 里，这句话很容易被直接当成生成指令。AI 会围绕“更智能”“更像销售助手”这些可见目标继续补全页面：改卡片样式，加一个醒目的 AI 图标，调整建议文案，生成一个“高意向客户”标签，或者让建议语气更像资深销售。

这些修改不一定错。它们可能让 LeadFlow 看起来更完整，也可能帮助团队继续判断产品感觉。但它们主要回应的是当前对话里的表层表达：这个区域看起来不够好，所以把它做得更像一个好看的 AI 销售助手。

Agentic Coding 里，同样一句话不应该立刻滑向代码生成。它首先会被当成一个需要解释的工程信号：用户说“不够好”，到底是视觉层级不清，还是建议缺少来源；说“更智能”，到底是要更强的推荐逻辑，还是要避免没有依据的销售话术；说“销售助手”，到底是帮助销售判断下一步，还是替销售自动对外承诺。

所以，Agentic Coding 的第一步不是把一句话扩写成更长 Prompt，而是让 Codex 进入项目上下文，检查这句话和当前工程状态之间的差距。

系统先读取上下文，再对齐意图

假设 LeadFlow 的客户详情页已经能显示 AI 建议，但建议没有来源，也没有销售确认状态。用户只说了刚才那一句。一个更接近 Agentic Coding 的流程，会先自动读取项目里的相关对象：

PRD.md

- LeadFlow 的目标是帮助销售基于真实沟通记录判断下一步跟进。
- AI 不能替销售对客户作出承诺。

CONTEXT.md

- 当前版本只使用仓库里的脱敏样例线索和沟通记录。
- 当前沟通记录支持 leadId、channel、timestamp、author、body 等字段。
- 真实 CRM 同步尚未启用。

DESIGN.md

- 页面使用既定设计系统 token：主色、表面色、正文色、间距、圆角和字号不在组件里临时发明。
- UI 技术栈与图标策略已经约定，AI 建议区域使用统一组件气质和 lucide-react 图标，不用 emoji 或混用多套图标。
- AI 建议、客户事实和销售确认分区展示。
- 建议卡片默认是待确认状态。

- 来源入口应该靠近建议正文。

AGENTS.md

- 不接真实 CRM，不自动发送邮件或短信。
- 手机号、邮箱和合同金额不得进入日志。
- 交付时说明改动范围、验证证据、未验证项和残余风险。

ACCEPTANCE.md

- AI 建议必须能展开来源。
- 无沟通记录时不得生成客户需求或预算判断。
- 销售尚未确认时，建议不能标为已执行。

读完这些内容以后，Codex 对那句“更智能、更像销售助手”的理解会发生变化。它不再把任务理解成“让卡片更像 AI 功能”，而会把它理解成：“线索详情页的 AI 下一步建议需要变成可追溯、可确认、不会误导销售的工程行为。”

如果项目文件已经清楚，Codex 可以直接进入实现计划。如果上下文还不够清楚，它应该先把理解暴露出来，和人对齐意图：

我理解这次优化不是单纯调整视觉，而是补齐 AI 建议的来源和销售确认状态。
我会先保持只使用脱敏样例数据，不接真实 CRM，也不增加自动发送能力。
我会把 TODO 和验收场景补齐，然后实现客户详情页建议卡片的来源展开与待确认状态。

这里的“对齐”不一定意味着停下来开会。任务足够明确时，Codex 可以继续推进；目标、边界或产品判断存在歧义时，它应该把歧义交还给人。关键是，AI 的下一步行动不再只由一句话里的形容词驱动，而是由项目事实、设计系统、长期规则和验收场景共同校准。



文档不是输入负担，而是系统回写的位置

很多人看到 PRD.md、TODO.md、DESIGN.md、ACCEPTANCE.md，会下意识觉得 Agentic Coding 增加了人的文档负担。好像人必须先把所有内容写进文件，AI 才能开始工作。

更合理的理解是：这些文件不是每次任务的前置作业，而是系统吸收需求、沉淀判断和回写状态的位置。

同样是刚才那句需求，Codex 在理解意图以后，可能会自动补出这一轮需要的任务状态：

```
TODO.md
- [] 线索详情页展示带来源的 AI 下一步建议
  - 设计：见 `DESIGN.md#AI-Suggestion-Card`
  - 验收：见 `ACCEPTANCE.md#AI建议必须显示来源`
  - 状态：未开始

DESIGN.md
- AI 建议卡片遵守当前设计系统的颜色、字体、间距、圆角和图标策略。
- AI 建议卡片包含建议正文、来源入口和销售确认状态。
- 默认状态为待确认，不显示为已执行。
- 无沟通记录时展示信息不足，不生成通用销售话术。

ACCEPTANCE.md
- Given 线索有沟通记录，When 展示下一步建议，Then 建议必须能展开来源。
```

```
- Given 线索没有沟通记录，When 查看 AI 建议区域，Then 不生成客户需求或预算判断。
- Given 销售尚未确认，When 展示建议状态，Then 建议不能标为已执行。
```

这一步不是把用户的一句话机械拆成多个文件，而是把一次模糊反馈转成项目可以继续使用的工程状态。`TODO.md` 记录当前要做什么，`DESIGN.md` 记录视觉系统、组件策略、界面状态和产品判断，`ACCEPTANCE.md` 记录怎样才算完成。以后再有人修改 AI 建议区域，Codex 不必重新猜测这些判断；项目本身已经留下了可读取的约束。

当然，文档回写也不是越多越好。小修小补不必把每个念头都写进长期规则；涉及产品目标、数据边界、设计状态、验收标准或团队协作的判断，才值得沉淀。Agentic Coding 要避免的不是“一句话需求”，而是“一句话直接变成代码，过程中没有上下文校准，也没有状态留下”。

真正的差异：谁来维护解题空间

还有一个更贴近日常体验的说法：Vibe Coding 很像“我自己开发一个工具，只是把敲代码交给 AI”，而且很多时候人并不真正审核代码。你关心的是它能不能跑、页面像不像、感觉对不对。Agentic Coding 则更像“我指挥 AI 来干活”：AI 读取上下文、规划、实现、运行检查、审查自己的 diff、说明证据和风险；人不再逐行指挥实现，但会保留最终 Review 和风险接受权。

可以把两者的差异放在一张表里：

表 1-1 Vibe Coding 与 Agentic Coding 的核心区别对照

维度	Vibe Coding	Agentic Coding
需求入口	可以是一句话、一个感觉、一个画面	也可以是一句话、一个反馈、一个问题
系统反应	直接围绕当前表达生成结果	先读取上下文，解释意图，再决定是否回写文档和执行
任务理解	主要来自当前对话	来自当前对话加项目文件、代码、规则、设计系统和验收
文档角色	通常不是执行过程的一部分	是需求吸收、状态回写和后续协作的位置
代码态度	能跑起来就先接受，可能很少阅读生成代码	AI 先自检和解释变更，人再审查关键 diff、证据和风险
审查方式	主要靠体验页面、复制错误、继续让 AI 修	测试、日志、页面检查、diff / review 和人工最终判断共同收口
AI 的职责	生成方案、补全界面、快速试错	读取环境、对齐意图、更新状态、实现、验证、修复、汇报
人的职责	描述感觉、体验结果、挑选方向	判断意图、确认边界、接受或修正系统对需求的理解
完成信号	看起来更像想要的东西	行为满足验收，有证据，状态已回写，风险已说明
适用范围	原型、个人工具、一次性脚本、早期探索	真实项目、团队协作、商业交付、长期维护
主要风险	把原型误认为交付	上下文过期、回写错误、验收太粗或权限边界不清

这张表的重点是第一行：Agentic Coding 并不要求用户每次都用工工程语言说话。它真正要求的是，系统不能只停留在用户那一句的表面，而要把这句话放回项目里解释。

人仍然可以用自然语言表达问题。Codex 需要做的，是从自然语言进入工程现场：读已有文件，看当前代码，发现缺失约束，必要时更新 PRD、TODO、设计系统和验收，再用这些对象约束实现。这样，一句话不会消失在一次生成里，而会变成项目状态的一部分。

从生成结果到维护项目状态

因此，这一节要强调的核心区别，不是 Vibe Coding 和 Agentic Coding 哪个更高级，也不是用户应该先用哪一种。更重要的区别是：Vibe Coding 擅长把一句话变成可见结果，Agentic Coding 要把一句话变成被上下文校准过、能进入项目状态、可以继续验证和交接的工程意图。

当 LeadFlow 还只是一个想法时，“帮我做一个 AI 销售线索工具”可以直接生成 Demo。当 LeadFlow 已经有代码、数据、设计系统、隐私规则和验收场景时，“让建议更智能”就不能只驱动一个更漂亮的卡片。它应该触发一次工程理解：当前建议缺什么，哪些文件已经定义过边界，哪些文档需要补齐，哪些行为必须验证，哪些风险需要交还给人。

下一节要回答的是 Agentic Coding 到底解决什么问题。人不需要把每一步施工指令都写出来，但系统必须能把一句话放回工程上下文，维护任务状态，并在约束内完成实现、验证和汇报。只有看清这些问题，人和 AI 的职责分工才不会停留在抽象口号里。

1.4 Agentic Coding 解决了什么问题？

前面三节已经把边界讲清楚了：Vibe Coding 让想法快速出现，Agentic Coding 让一句自然语言需求进入项目上下文、设计系统、规则、权限和验收组成的工程系统。现在可以进一步回答一个更直接的问题：既然 AI 已经能写代码，为什么还需要 Agentic Coding？

它解决的不是“代码由谁敲出来”这种问题。AI 生成代码已经足够快，很多时候快到让人来不及判断。真正的问题在后面：Demo 之后变化能不能被接住，五类工程债会不会不断后移，代码规模扩大后还能不能控制，Agent Loop 跑久以后会不会漂移，完成信号是否可信，开发成本是否可控，AI 的自主性是否真的可以委托。这些问题合在一起，才是 Agentic Coding 的问题域。

它解决 Demo 之后没人接住变化的问题

AI 编程最容易让人误判的地方，不是它生成得不够像，而是它生成得太像了。你说“帮我做一个销售线索管理工具，可以导入客户线索，AI 自动总结客户需求，并给出下一步跟进建议”，几分钟后 LeadFlow 的 Demo 就出现了。左边是线索列表，右边是客户详情，中间有 AI 总结、意向评分和下一步建议，按钮也能点。

这个时刻很有价值。它让一个模糊想法变得可见，也让产品经理、设计师、销售和工程师可以讨论方向。但 Demo 回答的是“这个方向看起来成立吗”，真实工程回答的是另一组问题：AI 总结是否基于真实沟通记录，客户没有提预算时会不会被编造预算，下一步建议是销售草稿还是对外承诺，手机号和邮箱会不会进入日志，无沟通记录、字段缺失、AI 分析失败时界面如何表现，后续接真实 CRM、团队权限和发布流程时系统还能不能继续改。

这两组问题都重要，但不是同一组问题。Demo 证明方向，不证明边界。真实工程的标志也不是代码量变大，而是系统开始承担后果：销售会相信建议，负责人会根据阶段看机会，客户信息可能包含隐私，后续修改会影响已有行为，别人也可能接手维护。

第二轮需求最容易暴露第一轮债务。你让 AI 加来源引用，才发现建议没有和沟通记录绑定；让它加人工确认，才发现界面只有一个看起来很确定的建议卡片；让它加隐私边界，才发现客户原文、手机号和邮箱可能混在日志里；让它接真实 CRM，才发现线索导入、去重、阶段判断和跟进状态各走一套临时逻辑。

所以，Agentic Coding 解决的第一类问题，是让 Demo 找到的方向进入一个能承接变化的工程结构。它不要求每个原型都马上变成严肃项目，也不否定快速探索。它只是在原型开始面对真实数据、真实用户、团队协作、长期维护或外部交付时，提醒我们换一种完成标准。

这套完成标准不是：

AI 已经生成了一堆代码。

而是：

目标、设计、边界、权限和验收已经清楚；
实现进入了现有工程；
关键风险有证据；
未验证项和残余风险被说明；
下一轮修改能从当前状态继续。

它解决五类债后移的问题

从 Demo 到可交付产品，最容易犯的错误，是继续让 AI 生成更多功能。多一个按钮、多一个页面、多一个导入入口、多一个 AI 建议选项，都会让项目看起来更完整。但如果底层问题没有补上，功能越多，后面的维护压力越大。

Agentic Coding 关注的是五类债：

表 1-2 Agentic Coding 关注的五类工程债

债务	常见表现	Agentic Coding 如何缓解
需求债	目标、范围、非目标不清楚	把本轮目标、边界和完成标准外部化
设计债	流程、状态、风险表达和视觉系统不稳定	把关键界面状态、设计系统约束和设计验收点外部化
架构债	临时实现堆叠，边界越来越糊	先读项目，复用模式，小步修改，必要时请求人判断
验收债	看起来能跑，但关键行为没有证据	用验收场景和风险匹配的证据说明完成
责任债	授权、验收、风险接受不清楚	用权限、审批、Review 和残余风险汇报收口

这五类债很少单独出现。需求债会放大设计债，目标不清楚，界面就只能用常见模式填空。设计债会放大架构债，状态没有定义，代码就会在组件里临时拼状态。架构债会放大验收债，边界混乱，验证就很难覆盖真实行为。验收债会放大责任债，没有证据，人只能凭感觉接受结果。责任债又会反过来放大需求债，没人明确哪些风险不可接受，下一轮需求就继续膨胀。

这就是为什么很多 AI 项目早期很快，后期却越来越慢。不是 AI 突然不聪明了，而是前面的未决问题开始收账。Agentic Coding 不能自动消灭这些债务，但能让债务更早暴露、更小范围扩散、更容易被偿还。它让每一轮生成都

进入一个更清楚的工程循环：目标清楚，设计可读，边界可见，实现可审查，证据可复查，风险可归属，状态可延续。

它解决代码规模扩大后控制力下降的问题

很多人第一次感到 Vibe Coding 失控，不是在第一个 Demo，而是在第十几轮修改之后。功能还在增加，页面还在变完整，AI 也还能继续补代码，但人已经很难回答几个基本问题：现在到底有几套数据结构，哪个状态字段是权威的，哪些 mock 还在被使用，哪些判断只是为了演示临时写死，哪些测试覆盖了真实风险，哪些代码没有人敢删。

项目规模越大，这个问题越明显。几千行代码时，人还能靠搜索、记忆和局部阅读兜住；一旦进入数万行，尤其超过三万行代码，继续用“我说感觉，AI 直接生成，我不认真审代码”的方式推进，就很容易把项目变成一个看似能跑、实际无人能完整解释的系统。这里的三万行不是精确阈值，而是一个工程经验上的警报：当代码量已经超过单个人靠感觉通盘掌握的范围，就必须把控制方式从“看结果像不像”切换到“上下文、边界、验证和 Review 是否接得住”。

Agentic Coding 对规模的回答，不是要求人重新逐行读完所有代码，而是让 AI 在项目结构内工作：先读相关上下文，复用已有模式，限制影响范围，运行贴近风险的验证，审查 diff 是否越界，并把未验证项说清楚。规模变大以后，人更需要的是这种可分层、可回溯、可停止的控制方式，而不是继续要求 AI 一口气把项目做完。

它解决 Agent Loop 跑久以后会漂移的问题

一次 AI 编程任务表面上像一段对话：人发出请求，AI 读取上下文，修改文件，运行验证，修复失败，最后汇报结果。但当任务变复杂以后，它更像一个循环：

理解目标

- 观察工程状态
- 选择下一步动作
- 修改或验证
- 读取反馈
- 更新判断
- 继续推进或停下

真正的问题不是 Loop 能不能跑起来。很多 Coding Agent 已经可以连续执行相当长时间。真正的问题是：它运行很久以后，人和项目还知不知道它为什么走到这里。

Loop 失控通常不是突然发生的。更常见的情况是，每一步看起来都有理由，整体却慢慢偏离。LeadFlow 本来只是让 AI 下一步建议带来源，AI 先整理建议模型，接着补确认状态，又发现销售阶段字段不统一，于是顺手调整线索导入。最后 diff 变成了跨导入、阶段判断、AI 分析、详情页和测试的大改动。单看每一步都合理，整体看已经偏离本轮目标。

Agentic Coding 给 Loop 提供最小控制信号：

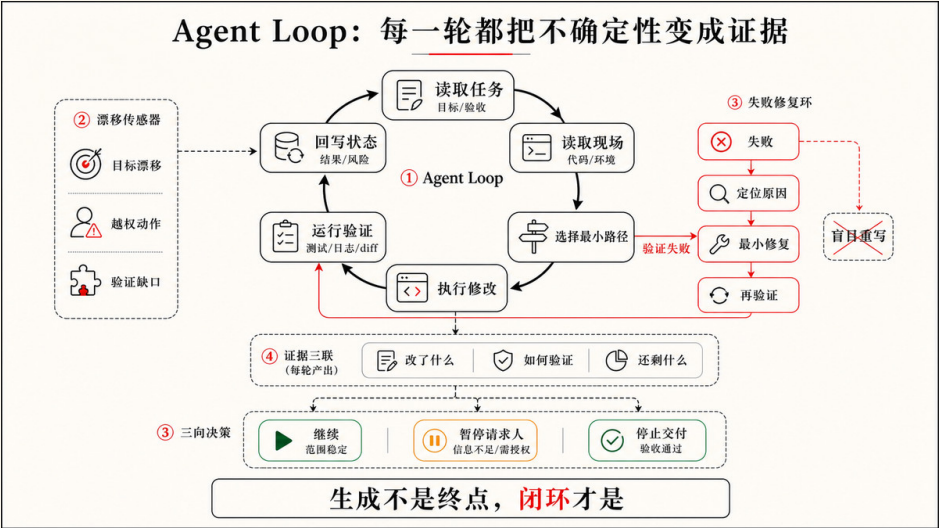
用户意图

- 任务路由
- 规格 / 设计 / 验收
- 实现
- 验证
- 状态回写
- 漂移检查

任务路由回答“这次到底是哪一类工作”：解释概念、实现功能、调试失败、补验证证据、同步文档、设计走查、Review，还是需要先澄清目标。规格、设计和验收回答“什么状态才算完成”。实现要求 AI 在现有工程里做最小必要修改。验证把失败、页面异常、权限拒绝和用户反馈带回循环。状态回写让下一轮能从这里继续。漂移检查则确认目标、设计、代码、文档、证据和责任没有分叉。

长运行还需要继续、暂停和停止条件。失败原因明确、修复仍在当前范围内、不需要新增权限时，AI 可以继续推进。发现目标有歧义、需要接入真实 CRM、需要安装新依赖、diff 扩大到多个模块、设计状态可能误导销售时，AI 应该暂停并请求人判断。验收条件满足、证据足以支持本轮交付、未验证项和残余风险已经说明时，AI 应该停止，而不是为了追求更完整继续扩张。所以，

Agentic Coding 要解决的不是“让 AI 一直做下去”，而是“让 AI 在正确的循环里做下去”。



它解决完成信号不可信的问题

AI 写出代码，只能说明动作发生过。页面能打开，只能说明某条路径没有明显崩掉。测试通过，也只能说明测试覆盖的行为成立。真正的问题是：这次结果是否符合人的需求，是否覆盖了关键风险，是否有证据支撑。

LeadFlow 里，“AI 跟进建议做完了”不是一个合格的完成信号。更接近 Agentic Coding 的验收会是：

有沟通记录时，AI 建议能展开来源；
没有沟通记录时，不生成客户需求或预算判断；
销售确认前，建议保持“待确认”状态；
手机号和邮箱不出现在日志；
相关测试、页面检查或人工审查完成；
未验证项和残余风险需要说明。

这些验收条件会反过来约束实现。建议卡片做出来但没有来源，它不该停。页面看起来正常但隐私日志没有检查，它应该说明证据缺口。测试无法运行，它不能把“没跑”包装成“通过”，而要把未验证项交还给人判断。

这也是为什么 Agentic Coding 要把“感觉完成”转换成“可检查完成”。完成不是一句总结，也不是一个漂亮界面，而是一组能支撑有限主张的证据。第四章会继续展开验收对齐、证据类型、验证强度和最终汇报；在第一章里，先建立这个判断就够了：可信不是没有风险，而是关键风险被看见、被解释，并由人决定是否接受。

它解决开发成本失控的问题

谈 AI 编程成本，很多人第一反应是模型调用费用。那当然是成本，但只是很小的一部分。真实项目里的成本还包括 AI 反复读取上下文的成本，多次尝试错误路径的成本，安装依赖、启动服务、跑 CI 的成本，人做 Review 和纠偏的成本，设计师反复走查状态偏差的成本，过度验证拖慢交付的成本，验证不足导致返工的成本，以及代码债、设计债、验收债和责任债后移的成本。真正昂贵的，常常是没有控制的循环。

从成本结构看，Vibe Coding 很像低前期投入、高后期维护的模式。刚开始只需要一个 AI 工具和几句提示，几乎没有流程成本；但如果它一路被推到真实项目，后面的运营成本会快速出现：反复把大段无结构内容塞进上下文、让 AI 修自己没有验证过的错误、让工程师倒查几个月前生成的临时代码、在生产后补安全和隐私漏洞。它前期便宜，后期不一定便宜。

Agentic Coding 则相反。它一开始要投入更多：写清项目规则，准备上下文，设计验收，建立测试和 Review 习惯，必要时沉淀 Skills 和 Hooks。看起来慢了一点，但这些投入会降低后续每一轮的试错成本。AI 不必每次从零猜，团队也不必每次从最终页面倒推风险在哪里。

不验证不是便宜，只是把成本推到更晚、更难定位、更难回滚的位置。但过度验证也会制造成本：小文案改动被完整发布流程拖慢，无关历史失败阻塞当前任务，AI 为了追求全绿扩大修改范围，团队最后开始忽略冗长的验证报告。

专业的做法不是永远多验证，而是让验证强度匹配风险。人和 AI 需要能讨论验证成本：这次任务风险有多高，哪些证据足以支撑交付，哪些检查可以省略，哪些未验证项必须升级或留到下一轮。低风险改动可能只需要 diff 和目标

页面检查；日常功能改动可能需要相关测试、页面冒烟测试和日志摘要；涉及发布、权限、真实数据或跨模块改动时，则需要更完整的验收检查、产物验证、人工 Review 和残余风险登记。具体怎样按任务选择证据、怎样组织证据链，留到第四章展开。

成本控制还包括上下文预算和 Loop 预算。低风险小任务，不必读取整个项目历史；涉及验收、设计状态、权限、真实数据或多模块修改时，再加载更完整的规则、PRD、DESIGN、TODO、ACCEPTANCE 和验证记录。相关测试失败且原因明确时，AI 可以自主修复并复验；连续多次失败仍无法定位时，它应该停止并汇报根因假设；发现需要新增依赖、联网或读取真实客户数据时，先请求确认。省成本不是让 AI 少干活，而是让它少走错路。

它解决 AI 自主性无法委托的问题

很多人担心规则、边界、设计和验收会限制 AI。这个担心很自然。我们习惯把“能力强”和“限制少”联系在一起，好像权限越大，AI 越能发挥。

但真实工程里往往相反。没有约束的 AI 看起来自由，实际只能靠猜；有清楚约束的 AI，反而更容易被委托。

如果你只说：

帮我把 LeadFlow 做得更智能一点。

AI 仍然会工作。它会根据常见产品模式补空白：自动判断销售阶段、推荐高意向客户、生成邮件草稿、接真实 CRM，甚至加一键发送能力。这些想法可能有价值，也可能超出当前风险边界。

让它变得可委托的，是项目里已经立着这一轮要守的约束。它们大多写在 `AGENTS.md`、`PRD.md`、`DESIGN.md` 和 `ACCEPTANCE.md` 里，这一轮只需补上少量边界：

本轮只处理仓库里的脱敏样例线索。
不接真实 CRM。
不做自动发送。
优先复用现有客户详情页和线索数据结构。
AI 建议没有来源时不得生成。
建议、客户事实和销售确认必须在界面上区分。
安装依赖、联网和读取外部客户数据前先确认。
完成后说明验证证据和未覆盖项。

这些约束没有替 AI 写代码，也没有规定每一步操作。它们大多留在项目文件里，由系统持续提供，人不必每轮重述。它们做的只是把不可接受的方向排除掉，把人愿意承担的风险范围说清楚。在这个范围内，AI 反而可以更大胆地行动：它知道可以读取哪些文件，可以改哪些模块，可以运行哪些检查，可以在测试失败时继续修复，也知道到达什么边界必须停下来问人。这就是可委托能力：不是“你想怎么做都行”，而是“在这个明确范围内，你可以自己推进”。

把这几类问题放在一起看，Agentic Coding 解决的不是 AI 会不会写代码，而是 AI 写代码这件事能不能进入真实工程。它让 Demo 之后的变化被接住，让五类债更早显影，让数万行之后的项目仍然有控制点，让长运行 Loop 不容易漂移，让完成信号变成证据，让验证、上下文和循环强度贴近风险，也让约束变成可委托能力的边界。

如果说 Vibe Coding 的完成信号是“这个方向有感觉”，那么 Agentic Coding 的完成信号应该是“这次工程变化在约束内完成，有证据，有回写，有未验证项说明，也有人愿意对残余风险负责”。下一节的人机分工，就要回答这件事具体由谁承担。

1.5 人和 AI 的职责分工

上一节把 Agentic Coding 要解决的问题摊开了：Demo 之后没人接住变化，五类债不断后移，Agent Loop 跑久以后漂移，完成信号不可信，成本失控，自主性也难以委托。到这里，一个更现实的问题就会出现：如果 AI 可以读代码、改文件、运行测试、根据报错继续修复，人到底还负责什么？

最容易想到的答案，是人负责下命令、看进度，在 AI 走错时告诉它下一步怎么改。但如果人的主要工作仍然是指定文件、函数、命令和修改步骤，我们

只是把键盘操作换成了语言操作。AI 写代码更快了，人依然被困在执行细节里，还额外承担了指令和检查的成本。

Agentic Coding 带来的变化，不是让人退出工程，而是让人的控制点上移。人不再需要亲自决定每一个实现动作，而要负责那些决定任务方向、风险和责任的判断：为什么做、做到什么程度、哪些设计状态不能误导用户、哪些边界不能跨、什么证据足以接受，团队应该共享哪些状态，出现后果时由谁负责。

用一个更直接的说法，Vibe Coding 更像是“我自己做开发，只是代码由 AI 帮我写”，它的危险在于人经常没有真正审代码，只是在页面上觉得差不多就继续下一步。Agentic Coding 更像是“我指挥 AI 做一轮工程任务”：AI 先读上下文、实现、运行验证、审查自己的变化并整理证据，人最后审目标是否达成、范围是否越界、风险是否能接受。人的工作不是减少到没有，而是从盯着每一行代码，转成设计任务、边界、验证和交付判断。

人负责意图和业务意义

AI 可以分析方案，却不能替人决定什么问题值得解决。“做一个 AI 销售工具”只是一个解决方案，真正的问题可能是销售团队跟进线索太慢，运营不知道哪些客户值得优先联系，负责人看不到机会风险，或者产品团队想从客户沟通中发现需求信号。

这些目标会导向完全不同的系统。如果重点是销售效率，下一步行动、跟进状态和提醒机制更重要；如果重点是管理判断，销售阶段、机会金额和风险提示更重要；如果重点是产品洞察，客户原话、需求来源和主题归类更重要。AI 可以帮你整理信息、提出选项、比较方案，但最终要解决哪个问题、服务谁、接受什么代价，仍然是人的决定。

产品和设计负责把判断变成用户能理解的状态

在 AI 项目里，产品和设计也不是最后来“补页面”的角色。它们承担的是另一类判断：人的业务意图怎样变成用户能理解、AI 能实现、团队能验收的状态。LeadFlow 里的关键风险，很多都发生在界面表达上：AI 建议看起来太确定，销售就可能把它当成客户事实；“发送跟进”按钮太突出，用户就可能误以为系统已经完成确认；风险提示藏得太深，负责人就看不到这条建议还没有证据。

这就是为什么本书会强调设计。不是因为 Agentic Coding 要把每个开发者都训练成设计师，而是因为在 AI 产品里，界面状态本身就是产品判断和风险边界。哪些信息应该优先出现在客户详情页，AI 总结、AI 推测、客户事实和销售确认如何区分，“待确认”“已确认”“已执行”“存在风险”分别怎样展示，无沟通记录、字段缺失、AI 分析失败时如何提示，哪些操作需要二次确认，哪些建议不能被设计成默认行动，这些都不是视觉装饰，而是在决定用户会怎样理解系统。

这些判断还要能进入工程。它们需要在 DESIGN.md 里变成用户流程、界面状态、风险表达、组件策略和必要的设计验收点。具体的颜色、字体、间距、圆角、图标库和响应式规则，下一章会单独展开；这一节先记住一点就够了：设计师的重要性不在于“让 AI 写出的页面更好看”，而在于帮助团队把产品判断变成可读、可实现、可验证的状态。

人负责让意图进入中间过程

很多人使用 AI 编程时，默认采用一种熟悉模式：

```
人提出需求
→ AI 长时间执行
→ AI 交回结果
→ 人最后检查
```

这个模式在小任务里可以工作。比如改一个标题、整理一组线索字段说明、补一段简单样式，AI 很快就能给出结果，人也能一眼看出对不对。但任务一复杂，这个模式就开始危险，因为人真正的意图往往不只在开头那句话里。它还藏在范围选择里，藏在业务优先级里，藏在设计状态里，藏在对风险的

承受度里，也藏在“这次先不要做那么大”的克制里。

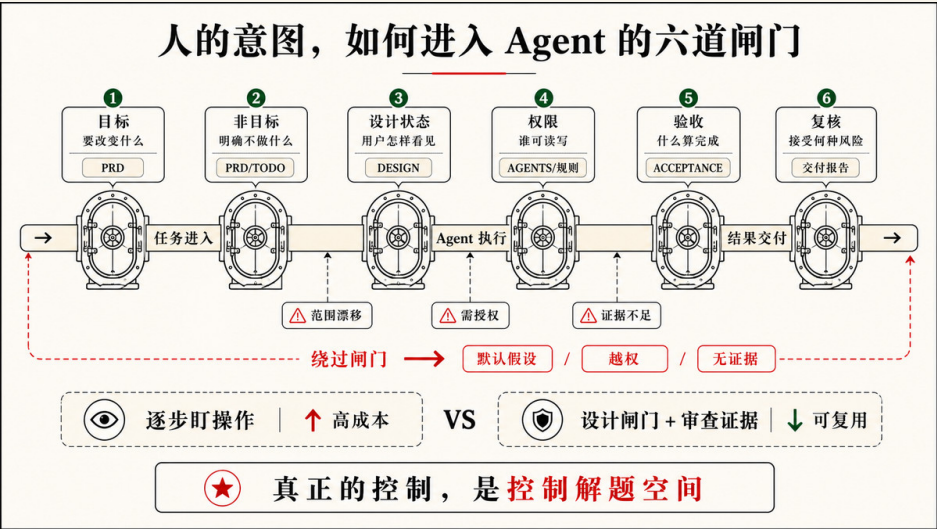
如果这些意图只在任务结束时才被拿来检查，很多偏差已经变成了代码、依赖、页面状态、文档和风险。你让 AI 给 CRM 工具做 AI 跟进建议，它可能真的写出建议卡片，顺手接入模型调用，再加一个“发送跟进邮件”按钮。最后你打开 diff，才发现它接入了真实模型 API，日志里有手机号和邮箱，没有区分 AI 建议、客户事实和销售确认，还把“建议发送报价”设计得像已经可以对客户承诺。

Agentic Coding 要改变的正是这一点。它不是要求人盯着每一次搜索、打开文件、修改代码和运行命令。那样人又会变回执行瓶颈。它要做的是让关键判断可见、可纠正、可追溯。

人的意图至少应该在六个位置进入过程：

表 1-3 人的意图进入过程的六个位置

位置	人要控制什么
开始前	目标、范围和完成信号是否清楚
设计时	流程、状态、信息层级和风险表达是否清楚
计划时	AI 准备读取哪些上下文、预计影响哪些模块
执行中	失败反馈是否改变判断，是否仍在范围内
触碰边界时	是否需要权限、审批或人工取舍
交付时	证据是否足够，残余风险是否可接受



过程透明，不等于过程监控。真正需要透明的，不是所有动作，而是会影响结果的判断。例如，AI 可以先说明：“本轮只做脱敏样例线索，不接真实 CRM，不做自动发送；我会检查线索详情页、沟通记录结构、AI 建议展示和现有验证；如果现有结构无法表达待确认状态，我会先说明最小模型变更；验证会覆盖有来源建议、无沟通记录不生成、确认前不标为已执行，以及隐私字段不进日志。”

这段计划不需要列出每个命令，却暴露了范围、设计状态、上下文入口、架构克制和验收重点。人可以在这里纠偏：“这次先不要做发送能力，只展示建议草稿和人工确认状态。”这条反馈改变了目标和风险，但没有接管实现。

随着 Harness 变得可靠，人还会从更贴近代码的指挥者，逐步变成更高层的协调者。指挥者会盯着某个文件、某段逻辑、某个错误，实时纠正 AI；协调者则定义目标、拆出合适任务、让一个或多个 Agent 在边界内执行，再周期性检查结果。两种模式都需要，但项目越成熟，人越不应该把自己困在每一次文件修改里。真正宝贵的注意力，要留给规格、架构、验证、Review 和风险接受。

协作不是分工交接，而是持续意图对齐

在团队里，这件事不只是“人和 AI 怎么分工”。产品经理、设计师、工程师和负责人，也不能把 AI 编程理解成一条线性的交接流程：产品给需求，设计给页面，工程交给 AI，管理者最后看结果。

更接近 Agentic Coding 的协作方式，是围绕意图持续对齐。目标对齐阶段，产品和设计要说清用户问题、核心场景和原型方向，工程要暴露可行性和技术风险，负责人要决定优先级、资源和节奏。规格共创阶段，团队把这些判断写进 PRD、设计状态、TODO 和验收标准，同时留下决策记录和协作规则。工程拆解阶段，AI 可以帮助拆任务、读代码、提出实现路径，但范围变化、依赖协调和风险升级仍然需要人判断。验证反馈阶段，团队要看测试、页面、用户反馈和风险复盘，而不是只看 AI 是否说“完成”。能力沉淀阶段，有效的需求模板、案例库、SOP、Harness、测试用例和反馈记录，应该进入下一轮可复用的工作系统。

这套协作方式的核心是：AI 编程的组织能力，不来自某个角色把自己的部分做完就扔给下一个人，而来自意图清晰、上下文共享、验证闭环和经验复用。人没有退出开发，只是从低层施工上移到定义目标、设计边界、建立验证、审查风险和沉淀能力。

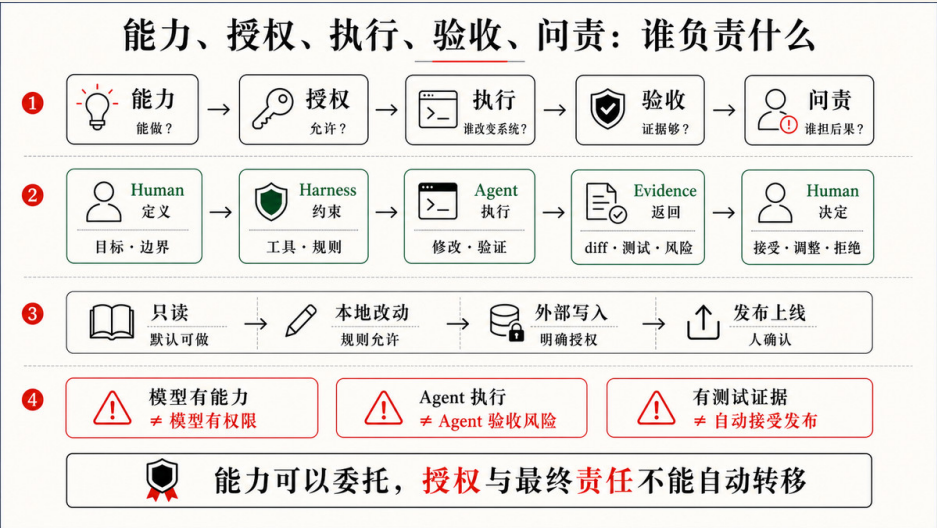
人负责边界、权限和预算

目标告诉 AI 去哪里，边界告诉它不能为了到达目标而牺牲什么。一个 Agentic Coding 任务通常需要人说清本轮允许影响哪些模块和行为，哪些客户数据、外部系统和真实环境不能碰，是否允许联网、安装依赖、调用付费 API，是否可以修改测试、配置、脚手架或 Git 状态，哪些动作必须先请求审批，哪些残余风险不能带着交付。

这些不是形式要求，它们决定 AI 的自主空间。低风险、可逆、容易检查的动作，可以让 AI 在任务范围内自主执行，例如搜索文件、阅读上下文、修改相关代码、运行已有测试。涉及真实客户数据、隐私、安全、付费服务、大规模迁移和生产发布时，则需要更明确的审批和更强证据。

人还要定义预算。这里的预算不只是钱，也包括验证强度、上下文加载范围、连续修复次数、是否允许并行探索、什么时候升级到人工 Review。人不一定告诉 AI 先跑哪个命令，但可以说：“这是日常功能级风险，请验证 AI 建议来源、无沟通记录状态、待确认状态和隐私日志，不做发布级检查。”AI 则应该在这个预算内寻找最贴近风险的证据。

能力、授权和问责必须分开。AI 技术上能做，不代表当前任务允许它做；当前任务允许它做，也不代表最终责任转移给 AI。当 AI Agent 越来越像一个可以独立行动的参与者时，团队最需要追问的往往不再只是“它是不是 AI”，而是“是谁让它这样做的”。



人的隐含期待	可观察的验收结果
建议要有依据	客户详情页的 AI 建议能展开来源沟通记录
资料不足时不要硬补	无沟通记录时不生成客户需求总结，页面显示信息不足
不能替销售对外承诺	销售确认前，建议保持“待确认”，不进入已执行记录
推测不能冒充事实	AI 推测和客户事实在界面上分区展示
隐私不能外泄	手机号、邮箱和合同金额不出现在日志
完成要有证据	相关测试、页面检查或人工审查已完成且可复查

测试、日志、截图和运行结果可以帮助检查这些条件，但“哪些条件值得被检查”仍然来自人的判断。工具可以回答代码是否通过，不能替人决定这个结果是否真的改善了工作。

最终是否接受残余风险，也必须由人决定。比如真实 CRM 还没接入、多角色权限还没覆盖、页面只用脱敏样例检查过、日志脱敏只验证了本地路径，这些未验证项不一定阻止交付，但必须被看见。AI 可以给出证据和建议，人负责决定是否接受、升级或收缩范围。

团队负责共享意图板，而不是共享感觉

一个人临时用 AI 写代码，可以靠记忆补很多空白。你知道上一轮为什么这么改，知道哪些线索是假数据，知道哪些功能只是演示，知道某个失败检查暂时无关，也知道自己其实还没准备接入真实 CRM。这些信息不一定写在项目里，但它们在脑子里。

多人协作时，这种方式很快失效。另一个开发者接手时，不知道你和 AI 上一轮达成了什么默契。设计师看到页面时，不知道哪些状态已经实现，哪些只是占位。销售同事看到“下一步建议”时，不知道它能不能真的拿去跟客户沟通。新的 AI 会话进入项目时，也不知道哪些约束是长期有效，哪些只是上一轮对话中的临时判断。

团队需要的不是共享感觉，而是一张可以被人和 AI 共同读取的意图板。它不一定真的是一个独立工具，也可以由 PRD、DESIGN、TODO、ACCEPTANCE、反馈记录、SOP 和验证证据共同组成。关键是，团队围绕同一组对象讨论目标、范围、设计状态、任务进度、验收证据和风险，而不

是各自在脑子里维护一份“我以为”。

表 1-5 团队需要共享的七类状态

共享对象	解决什么问题
当前目标	这轮到底推进什么，不推进什么
设计状态	用户流程、界面状态、风险表达和视觉系统是否已确认
任务状态	哪些已完成、阻塞、跳过或留到下一轮
长期规则	哪些约束不应每次靠人提醒
验收证据	完成凭什么成立，哪些风险被覆盖
风险和决策	哪些取舍已经被别人接受，哪些仍待确认
权威上下文	当代码、文档、对话和口头说法冲突时，以什么为准

这里还要特别区分三件事：建议、决策和事实。AI 可以建议“未来可以接入 CRM”“可以自动生成报价草稿”“可以按意向评分推荐跟进顺序”，但建议不是决策。决策需要人或团队承担后果，例如“首版不接入真实 CRM”“AI 建议必须由销售人工确认”“涉及预算、报价和承诺的内容不得标为客户事实”。事实又是另一层，比如“当前代码只支持 samples/ 下的脱敏线索”“AI 建议可以展开来源沟通记录”“手机号和邮箱脱敏检查已经通过”。事实应该能被代码、测试、文档或验证记录支持。

如果这三者混在一起，项目会很快失真。AI 把建议写成已决策，文档把未来计划写成已实现，团队把一次讨论中的方向当成已经验收的事实。Agentic Coding 要求状态可共享，本质上就是要求这些层次分开。

共享意图板不等于文档越多越好。它的目标不是保存所有信息，而是保存下一轮行动必须相信的信息：稳定、可定位、可更新、可区分、可审查。这样，团队协作就不再全靠会议记忆和聊天印象，AI 的工作也能被审查，而不是只被相信。

AI 负责读取上下文和选择路径

当目标、设计、边界、预算和验收已经给出，AI 就不应该继续等待一份精确到文件和行号的施工说明。它应该先观察工程现场：项目规则对修改范围和验证有什么要求，相似页面或模块使用了哪些模式，任务相关代码、数据、测试和文档在哪里，`DESIGN.md` 对 UI 技术栈、视觉 token、组件状态和风险提示有什么约束，当前 Git diff 是否已有用户修改，错误日志、失败测试或浏览器状态暴露了什么新事实。

然后，它需要在边界内选择路径。它可以判断应该复用哪个已有抽象，修改集中在哪些模块，哪些行为需要补测试，哪些检查最贴近风险，当前方案是否出现范围扩张。人不必预先猜中正确文件，人更应该关心的是 AI 选择路径时是否忠于目标、上下文、设计和边界。

AI 负责实现、验证、修复和回写

Agentic Coding 中的 AI 不只是生成一段代码。一次完整推进更像这样：

```
读取目标与项目规则
→ 搜索相关代码和业务上下文
→ 对齐设计状态和验收标准
→ 形成当前计划
→ 做最小必要修改
→ 运行相关检查
→ 读取失败信息
→ 修正实现或调整方案
→ 初步审查 diff 范围与风险
→ 回写必要状态
→ 汇报证据、未知和风险
```

这里最重要的是反馈循环。AI 第一次实现出错并不可怕，真实工程里很多根因只有在运行、比较和失败以后才会显现。可靠性不来自 AI 从不犯错，而来自错误能够被观察、影响能够被限制、方案能够被修正。

如果测试失败、页面行为不对、diff 超出范围、权限被拒绝，AI 应该把这些结果当作下一轮输入。能在原范围内修复，就继续修；触及目标歧义、设计取舍、权限边界或高风险判断，就把决定交还给人。

状态回写也是 AI 的责任之一。任务做到哪里，哪些验收通过，哪些场景跳过，哪些设计状态被确认，哪些规则被新增，哪些风险被接受，哪些问题留到下一轮，都应该回到项目的共享状态。对话历史不是项目长期记忆，项目文件和证据才是。

人负责 Review、经验沉淀和最终问责

AI 的最终回复不能代替人的 Review。最终回复是 AI 对工作的解释，diff 是工程状态实际发生的变化，测试和页面结果则说明这些变化产生了什么后果。三者放在一起，人才能判断这次委托是否可以接受。

Agentic Coding 里的 Review 应该分两层。第一层是 AI 自己做：它要检查 diff 是否只改了本轮相关范围，是否引入了不必要依赖，是否触碰隐私和外部系统，验证证据是否足以支撑它的结论。第二层才是人的最终 Review：人不需要重演 AI 的每一步，但要判断最初的目标有没有被解决，关键风险有没有被覆盖，哪些未知需要接受、收缩或升级。

人的 Review 可以很轻量，但至少要看五件事：行为是否解决了最初的问题，而不是只让某个检查变绿；范围是否保持在委托边界内，有没有覆盖用户原有改动；证据实际覆盖了哪些风险；未知场景和残余风险是否可以接受；哪些动作是在既定权限内完成的，哪些动作需要人确认。

人还负责把经验留下来。如果一次任务中发现“AI 建议必须引用沟通记录来源”，这条经验不应该只留在聊天记录里。它可以进入产品规格、设计状态、验收场景和测试。如果“首版只能使用脱敏样例线索”是长期规则，它可以进入 **AGENTS.md**。如果接入真实 CRM、安装生产依赖和调用付费模型每次都需要审批，这些边界也应该进入稳定的工作规则。

经验被沉淀以后，下一次 Agent 不必依赖某个人恰好记得提醒。项目本身开始承载团队的目标、设计、边界和质量判断，这也是从一次 AI 对话走向工作系统的关键一步。新的想法仍然可以回到 Vibe 阶段快速探索；一旦方向变清楚，就应该交给 Agentic Coding 的任务、边界、验证和回写机制继续推进。

把这些职责合在一起，人和 AI 的分工就清楚了：人负责目标、意义、设计、边界、上下文权威、权限、预算、验收、共享状态、风险、品味、伦理和最终问责；AI 负责读取、搜索、规划、实现、验证、修复、回写状态、整理证据和汇报风险。

这不是让人的作用变小，而是让人的注意力从低层操作回到高层判断。人仍然控制方向，只是不再通过逐步操纵每一个动作来控制。更准确地说，人设计 AI 可以自主工作的环境，并在价值、风险、授权和验收这些关键位置保留决定权。

第一章到这里，完成了概念边界：Vibe Coding 发现方向，Agentic Coding 在工程底座上完成这一轮可验证变化，它解决的是 Demo 之后工程状态如何被接住的问题；人定义目标、设计、边界、状态和责任，AI 在边界内推进工程闭环。后面的核心文件章节，会把这些职责落到 `AGENTS.md`、`CONTEXT.md`、`PRD.md`、`DESIGN.md`、`TODO.md` 和 `ACCEPTANCE.md` 这些可共享对象里。

控制和协作的核心文件

2.1 AGENTS.md：项目长期工作协议

LeadFlow 的 Vibe Demo 一句话就可以让 AI 生成出来：一个能点击的线索列表，一个客户详情页，一段看起来像销售建议的 AI 输出。方向看起来对，但第一章说过，Demo 之后有两类问题开始积累：工程债和中间过程不透明。继续靠感觉推进，每一轮修改只是在局部补洞；进入真实工程，就要在改代码之前，先把判断放到项目里——用 AI 能读取的格式，而不是只存在于某次对话和人的记忆里。

第一章最后讲到，人和 AI 的职责最终要落到 `AGENTS.md`、`CONTEXT.md`、`PRD.md`、`DESIGN.md`、`TODO.md` 和 `ACCEPTANCE.md` 这几个可共享对象上。在逐个拆解它们之前，先回答一个更基础的问题：为什么要用多个文件和 Coding Agent 协作，而不是把所有事情都写进一次 Prompt？

原因不在于“文档更正式”，而在于 Coding Agent 面对的不是一次问答，而是一个会持续变化的工程世界。它要读取仓库，理解产品目标，遵守项目边界，修改代码，运行验证，处理中途失败，还要把完成证据和剩余风险交还给人。Prompt 很适合表达本轮意图，却不适合长期保存项目规则、当前事实、设计约束、执行状态和验收标准。把所有信息都塞进一段对话里，短期看起

来方便，任务一长就会变成一团难以判断的混合物：哪些规则长期有效，哪些只是本轮偏好，哪些事实已经过期，哪些标准才算真正完成。

如果借用上下文工程里的说法，项目规则更接近“静态上下文”：它们应该在 Agent 进入项目时稳定可见，但也因为会被反复读取，所以必须足够短、足够准、足够能改变行动。当前任务、历史讨论和详细背景则更适合按需进入动态上下文。`AGENTS.md` 的第一层价值，就是把最小且长期有效的静态上下文放在仓库里，而不是让每一轮 Prompt 重新背一遍项目规矩。

多个文件的价值，是把不同类型的工程状态放到不同位置。长期工作协议不要和本轮任务混在一起，当前事实不要和旧资料混在一起，产品边界不要被拆成实现步骤，设计语义不要只留在截图里，完成标准也不要只出现在最终回复中。这样做不是为了让别人多写文档，而是为了让 Coding Agent 可以稳定读取、按需引用、执行后回写，并且让下一轮任务和团队成员都知道该相信哪一份状态。

本章介绍的文件组合，也不是所有项目必须照抄的标准目录。每个人都可以根据自己的项目、团队和风险来改名、合并、拆分这些文件，甚至用别的结构替代它们。相对来说，`AGENTS.md` 和 `TODO.md` 更接近 Agentic Coding 的基础入口：前者告诉 Agent 在这个项目里长期怎样工作，后者记录当前任务推进到哪里、下一步从哪里恢复。至于 `CONTEXT.md`、`PRD.md`、`DESIGN.md` 和 `ACCEPTANCE.md`，是我在大量实践中总结出来的一组高频协作文件，用来承载最容易混乱的四类状态：可信事实、产品目标、设计约束和完成标准。

所以，读这一章时不要先记文件名。真正要记住的是：Coding Agent 需要的不只是当前指令，还需要一组可读取、可更新、可审查的外部状态。文件名可以变，目录可以变，生成方式也可以变；但长期规则、当前事实、产品边界、设计约束、执行状态和验收标准，如果全部挤在 Prompt、聊天记录和人的记忆里，Agentic Coding 就很难从一次聪明的生成，变成可持续的工程协作。

这也是第一章说的 Harness 在项目里的落点之一。模型负责推理和生成，Harness 负责把规则、上下文、工具、验证和回写组织成可重复的工作系统；而这些文件，就是团队最容易维护、最容易版本化、也最容易被 Codex 读

取的一部分 Harness。

AGENTS.md 可以先理解为仓库根目录里的项目长期工作协议。它写给 Coding Agent，不写某一版产品需求，也不记录当前任务清单，而是保存这个项目长期有效的工作方式：先读哪些入口、哪些边界不能碰、哪些动作要确认、交付时要给出什么证据。第一章说，Agentic Coding 不是让人微操 AI 的每一步，而是把人的目标、边界、权限、验收和共享状态放进工作系统里；

AGENTS.md 就是把其中一部分长期控制点落到仓库里的文件。

为什么需要这样一份文件？因为如果长期规则只留在每一轮 Prompt 里，协作很快会变成一种奇怪的记忆劳动。产品经理记得风险，设计师记得状态，工程师记得实现边界，销售记得业务禁区；但新的 AI 会话进入仓库时，只能看到这一轮你刚刚输入的几句话。

在 LeadFlow 里，这种记忆劳动很容易变成一串重复提醒。如果你每次让 AI 改功能，都要重新强调：

客户手机号和邮箱不能进入日志。
AI 建议不能自动发送给客户。
AI 不能编造客户预算。
涉及真实 CRM 数据要先确认权限。
改了产品行为要同步 TODO 和验收。
完成后要说明验证证据和未覆盖风险。

这说明问题已经不在 Prompt。上面这些要求不是本轮临时偏好，而是项目长期工作协议。它们需要一个稳定、可被 AI 反复读取的位置，而不是每次靠人临时补一句。

落到 LeadFlow 这样的项目，这份启动协议先放在仓库根目录，再在里面指向其他重要文件的位置。其他文件可以在根目录，也可以放进 `docs/`、`product/` 或团队约定的其他位置；关键是根目录这份协议能告诉 AI 去哪里读它们，或者在它们尚未存在时，知道应该通过哪类 Skill 或准备流程把它们生成出来。

CRM 类产品的几类判断尤其容易散开：线索的定义有历史叫法，AI 建议的边界有业务争议，隐私字段有合规要求，外发权限有审批流程。LeadFlow 刚做完 Vibe Demo，这些判断还都分散在对话记录和团队记忆里。第一步是把它们找到稳定落点——**AGENTS.md** 放在根目录，指向 **PRD.md**（首版边界）、

CONTEXT.md (统一叫法)、ACCEPTANCE.md (验收场景)：

```
LeadFlow/  
  AGENTS.md  
  PRD.md  
  CONTEXT.md  
  DESIGN.md  
  ACCEPTANCE.md  
  TODO.md  
src/  
samples/
```

根目录的 **AGENTS.md** 承担的是项目级工作协议。以后 Codex 从这个仓库开始任务时，就能先看到这些长期规则：哪些文件是入口，哪些数据不能碰，哪些动作要审批，完成后要交代哪些验证证据。后面如果某个高风险目录需要更细规则，再在那个目录下补一个局部 **AGENTS.md**；但第一步，先把根目录这份写清楚。

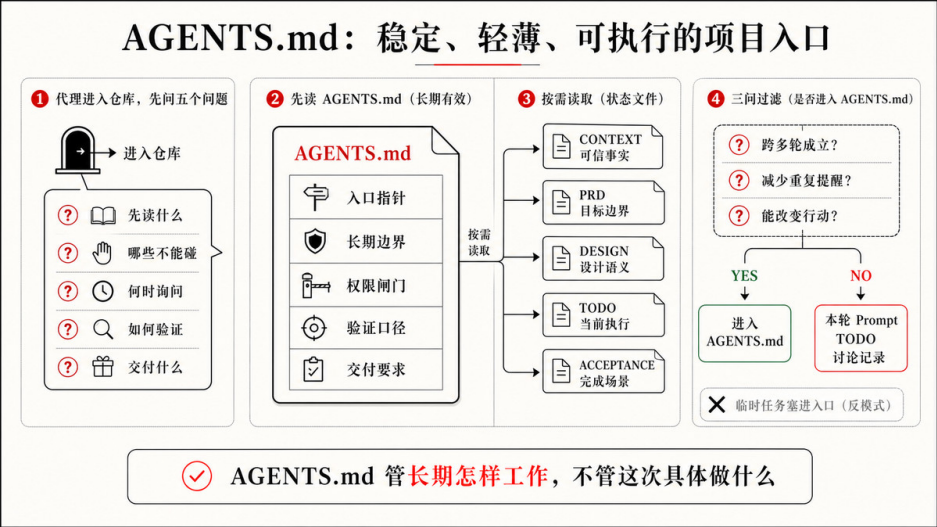
它回答的不是“这次做什么”

AGENTS.md 最容易被写错的地方，是被当成当前需求清单。例如：

- 本周把 AI 建议卡片放到客户详情页右侧。
- 这次按钮文案先叫“生成跟进建议”。
- 当前先不做销售阶段筛选。

这些信息可能有用，但它们不适合长期放在 **AGENTS.md**。它们更适合进入当前需求、**TODO.md** 或一次性的任务说明。**AGENTS.md** 保存的是另一类信息：无论这一轮改列表、下一轮改详情页，还是再下一轮改 AI 建议，AI 都应该遵守的项目工作方式。

这和 **PRD.md**、**DESIGN.md**、**TODO.md** 的职责不同。**PRD.md** 解释这一版为什么做、做给谁、核心目标和非目标是什么。**DESIGN.md** 解释视觉系统、组件策略、用户流程、界面状态和风险表达怎么处理。**TODO.md** 记录当前推进到哪里。**AGENTS.md** 则回答：无论本轮任务是什么，在这个项目里都应该怎样行动。



长期有效，才值得写进去

判断一条内容是否适合进入 AGENTS.md，可以先问三个问题。它们不是格式检查，而是在判断这条信息是否真的值得变成长期协议：

它是否跨多轮任务都成立？
它是否能减少反复提醒？
它是否足够明确，AI 能据此改变行动？

顺着这三个问题看，LeadFlow 里适合写入的，通常是本轮任务换了也仍然需要 AI 先知道的边界：

- 项目入口和重要目录；
- 当前项目的默认数据边界；
- 长期安全、隐私和质量规则；
- 常用命令和验证口径；
- 需要审批的动作；
- 交付时必须说明的信息；
- 修改后需要回写哪些状态。

这时不必先写一整份模板，只要把能改变行动的几条规则写清楚：

- 产品目标见 `PRD.md`，当前事实见 `CONTEXT.md`，设计系统见 `DESIGN.md`，活跃任务见 `TODO.md`，验收场景见 `A`
- 除非当前任务明确允许，不得读取、导入或同步真实 CRM 数据。
- 除非当前任务明确要求，不得新增自动客户消息。
- 涉及 AI 建议的改动必须保留来源记录，记录不足时不得编造客户事实。
- 安装依赖、使用网络访问、接触真实 CRM 服务或新增外部集成前，先询问。
- 交付时汇总行为变更、验证证据、未验证项与剩余风险。

这些规则不会因为本轮改列表、下轮改详情页、再下轮改 AI 建议而失效。它们描述的是项目长期工作方式。

反过来，不适合写入的内容也很清楚，它们通常只属于某一轮任务或某一次讨论：

- 本轮临时任务；
- 一次性文案偏好；
- 已过期的实验方案；
- 尚未决定的产品想法；
- 大段业务背景；
- 当前 TODO 里的细节进度。

如果把这些都塞进去，**AGENTS.md** 很快会变成垃圾桶。AI 每轮都读到一堆旧限制、旧愿望和旧实验，反而更难判断什么真正重要。

一份够用的 AGENTS.md 长什么样

LeadFlow 有一个写 **AGENTS.md** 时特有的风险：CRM 工具的产品边界天然模糊，“客户关系管理”能容纳的功能几乎没有上限。你说“加 AI 建议”，AI 可能顺手接入外部模型、补自动外发、加权限系统。第一章里你已经攒着三类限制——隐私字段不能进日志、建议必须有来源、不能自动对客户承诺——这三条是跨任务都要遵守的协议，不是某一轮改建议卡片时才临时想起来的细节。把它们写进 **AGENTS.md**，才能让每次进入仓库的 AI 先知道边界在哪里，再动手实现。

AGENTS.md 不需要很长，更不需要把产品规则、设计状态和验收细则都搬进来。它更像一个薄入口：写清协作口径、读取指针、长期工作规则、长期边界，以及验证与交付口径，把具体的产品和设计判断指向对应文件。一份够用的版本可以是这样：

```
# AGENTS.md
```

```
## 项目协作说明
```

- 用中文回复。
- 本项目是 LeadFlow，一个面向脱敏销售线索的内部工作台。
- 后续任务以 `TODO.md` 为准；尚未生成 `TODO.md` 时不算错误。

```
## 开始工作前按需读取
```

- 产品目标与非目标：`PRD.md`
- 当前可信事实与术语：`CONTEXT.md`
- 设计系统与界面状态：`DESIGN.md`（仅 UI 相关任务）
- 当前活跃任务与进度：`TODO.md`

- 完成标准与验收场景：`ACCEPTANCE.md`

```
## 长期工作规则
```

- 改变产品行为前，先读相关上下文文件，再动手。
- 优先沿用现有项目模式，变更范围限定在当前活跃任务内。
- 安装依赖、联网、接触真实 CRM 或新增外部集成前，先询问。

```
## 长期边界（跨任务不变）
```

- 默认只使用脱敏样例数据；不得记录客户手机号、邮箱、合同金额或原始消息正文。
- 涉及 AI 建议的改动必须保留来源记录，记录不足时不得编造客户事实。
- 真实 CRM 接入、外部模型调用和自动外发能力需要明确授权。

```
## 验证与交付
```

- 按改动风险选择最小但足够的验证；涉及 AI 建议、来源、隐私或确认流程时，回到 `ACCEPTANCE.md` 对应场景确认。
- 涉及真实 CRM、权限或外发消息时，采用发布级验证并报告残余风险。
- 交付时说明行为变更、验证证据、未验证项与剩余风险。
- 产品行为、设计状态或验收状态变化时，更新对应文件，或说明为何无需更新。

这份文件让 AI 进入项目时知道几件关键事：

- 当前项目的长期边界；
- 哪些文件分别承载目标、设计、任务、验收和上下文；
- 哪些动作不能默认自主执行；

- 完成后要用什么信息交还给人。

这些信息已经足够让很多任务少走弯路。它不替代 `PRD.md`、`DESIGN.md`、`TODO.md` 和 `ACCEPTANCE.md`，只负责把第一章说的长期边界和共享意图，放到 AI 每次进入项目时都会先看到的位置。

它应该短、准、可执行

`AGENTS.md` 不是百科全书。它不是为了把所有项目知识装进去，而是为了把 AI 每次进入项目时必须先遵守的工作协议放在一起。好的 `AGENTS.md` 有三个特点。

第一，短。它只保存跨任务都成立的规则。详细背景、当前任务、历史决策、设计状态和验收场景，应该放在更合适的文件里。根文件太长，AI 每轮都会背着大量无关信息行动，重要规则反而被淹没。

第二，准。不要写“保持代码质量”“注意用户体验”“回答要准确”这种好听但不可执行的话。更好的写法是：

- 除非当前任务明确要求，不得新增真实 CRM 集成。
- AI 建议须保留来源记录与销售确认状态。
- 不得记录电话、邮箱、合同金额或原始客户消息。
- AI 建议相关的 UI 状态与设计系统变更须遵循 `DESIGN.md`。

这些规则能改变 AI 的行动：它会少编造、少越界，也更容易知道该回到哪份设计或验收文件里确认边界。

第三，可执行。一条规则最好能让 AI 判断：我现在是否触碰了它？如果触碰了，应该继续、停止、请求确认，还是补充证据？例如：

- 安装依赖或使用网络访问前，先询问。

这条规则很清楚。AI 一旦准备安装新包，就知道要停下来。它不需要猜“为了完成任务可不可以先装一个库”，也不需要等到人 Review 时才发现权限已经越过。

不同目录可以有不同规则

一个项目里，并非所有规则都适用于所有文件。根目录的 `AGENTS.md` 可以写全局工作协议。某些高风险目录，还可以有更局部的规则。

这不只是组织习惯，也和 Codex 读取规则的方式有关。在当前 Codex 官方说明里，Codex 会在开始工作前读取 `AGENTS.md`，并按全局、项目根目录到当前工作目录的顺序合并指令；更靠近当前目录的文件出现在后面，因此可以覆盖更早的通用规则。

这给了一个很朴素的写法：根目录只写全项目都成立的协议，目录级 `AGENTS.md` 只写本目录的差异规则，不要复制整份根文件。比如 AI 调用目录可以这样写：

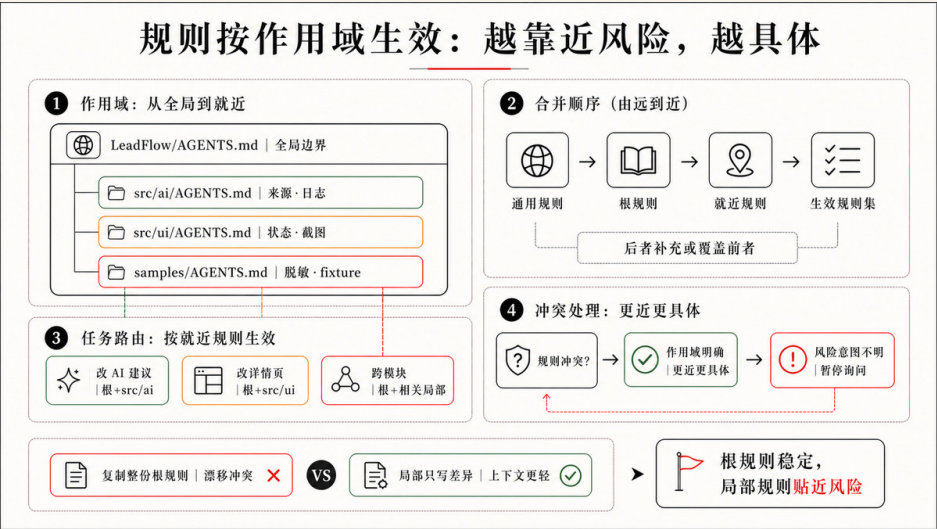
```
# src/ai/AGENTS.md
```

- 不得记录完整 prompt、API 密钥、token 或授权头。
- 任何建议生成相关变更须保留 `sourcelds`。
- 信息不足行为的变更须有验收覆盖。

其中 `sourcelds` 这条规则直接回应了第一章已经出现的问题：LeadFlow 的 AI 总结到底来自哪条沟通记录，还是模型凭感觉补全的？把这条要求放进 `src/ai/AGENTS.md`，AI 每次改建议生成逻辑时都先知道：来源不能省，信息不足时要呈现边界状态，而不是继续显示一段看起来确定的建议。

这样做有两个好处。第一，规则更贴近风险。样例数据目录可以关心脱敏，AI 调用目录关心日志和来源，详情页目录关心状态表达。根目录不必塞进所有细节。第二，AI 更容易判断当前任务需要读取哪些规则。改样例数据时，看样例目录规则；改 AI 建议时，看 AI 目录规则；做跨模块任务时，再把规则合起来。

这也是 Agentic Coding 中“上下文不是越多越好”的延续。规则也要按作用域管理；越靠近具体风险，规则越应该具体，越靠近项目根目录，规则越应该稳定。



，但要把反复出现的风险、边界和验收判断沉淀成系统能读取的规则。

它不是万能安全网

有了 `AGENTS.md`，不代表可以完全放手。它仍然会过期：首版不接真实 CRM，到了内测阶段可能会变成“接入真实 CRM 必须经过脱敏、权限和日志检查”。它也可能冲突：根目录说“UI 变更要浏览器检查”，某个包没有页面入口，AI 需要结合当前任务判断，无法判断时应该问人。它还可能太多：规则越堆越厚，后来的 AI 会读到一堆历史包袱，定期清理过期规则本身就是 Agentic Engineering 的一部分。

更重要的是，`AGENTS.md` 只告诉 AI 应该怎样工作，不证明这次真的做到了。写了“AI 建议必须保留来源”，不代表代码真的保留了来源；`AGENTS.md` 让 AI 知道长期要求，`ACCEPTANCE.md` 和验证证据才说明这次是否满足要求。所以，`AGENTS.md` 是长期协议，不是验收凭证。

下一节的 `CONTEXT.md`，则回答另一个问题：长期工作协议之外，这个项目里的那些词到底指什么，团队和 AI 怎样才能说同一套语言？

2.2 CONTEXT.md：让 AI 拿到正确的世界

上一节讲 `AGENTS.md`，它回答的是“在这个项目里，AI 应该怎样工作”。LeadFlow 的 `AGENTS.md` 里，最重要的几条规则都绕着同一类词打转：AI 建议必须保留来源、确认前不得标为已执行、销售确认是人工动作而非系统自动执行。但这几条规则要有用，前提是 AI 和团队对这些词指的是同一件事——一个 Coding Agent 进入仓库以后，光知道工作规则还不够。它还需要和团队对齐一件更基础的事：这个项目里的那些词到底指什么。谁算“线索”，谁算“客户”，“AI 建议”和“已执行跟进”是不是一回事，“销售确认”是人点的还是系统自动做的——这些概念如果每个人理解都不一样，AI 写出来的代码、命名、文案和测试就会跟着各说各话。

这就是 `CONTEXT.md` 的位置。它不是更长的背景资料，也不是把所有聊天记录、会议纪要和旧 PRD 汇总给 AI。它要解决的问题很集中：在这个项目里，每个关键概念叫什么、指什么、不该和哪些近义词混用。第一章说过，Agentic

Coding 的关键是让 AI 在项目上下文里行动；`CONTEXT.md` 就是先帮 AI 和团队站到同一套语言上，而不是让它在不同人的叫法、旧命名和代码之间自己猜这个词指的是哪个东西。

它回答“这个项目里的词指什么”

CRM 类产品有一个特别棘手的语言问题。“客户关系管理”这个词涵盖的概念范围太宽，而 CRM 软件的历史又很长，不同产品对同一个词的定义可能完全不同：Lead 是线索还是潜在客户？Opportunity 和 Deal 是 synonym 吗？

Contact 和 Account 的边界在哪里？每个公司、每个销售团队，都有自己的历史叫法。当你开始建 LeadFlow 时，这种混乱不会自动消失——它会悄悄渗进代码命名、界面文案、任务描述和 AI 的工作上下文，让同一件事在不同位置变成三种说法。

真实项目从来不是一片干净的新世界。LeadFlow 从一个评分卡片起步，逐步加入脱敏线索、沟通记录、AI 总结，以及“AI 建议”“销售确认”“已执行跟进”这些状态。每加一层，项目里就多出几个新概念，也多出几种叫法。在销售跟进工具里，叫法散乱有真实的业务后果：AI 可能把草稿建议渲染成已执行动作，销售看到界面以为系统已对客户承诺；晨会里销售运营看到的“跟进提醒”，究竟对应 suggestion 还是 confirmation，说不准。

麻烦的是，这些概念往往没人统一过名字。销售嘴里的“客户”，在产品那里叫“线索”，在代码里写成 `account`，到测试场景里又变回“联系人”。AI 写代码时不只会猜功能，也会猜名字：它可能把 lead 当成 customer，把 suggestion 当成 action，把“销售确认”理解成系统自动执行，于是同一个东西在数据模型、接口、页面文案和测试里被命名成好几种样子。短期代码能跑，长期项目语言越来越散，最后连人自己都要先花时间确认“我们说的是不是同一个东西”。

`CONTEXT.md` 就是来收住这件事的。它不负责把项目现实完整抄一遍，只回答一个很具体的问题：在这个项目里，每个关键概念叫什么、指什么、不该和谁混用。这件事别的文件都不专门管——代码里有名字，但代码里的名字可能正在漂移，甚至本身就是错的；只有 `CONTEXT.md` 专门站出来，当那个“叫法的裁判”。这个起点应该薄而准，不必、也不应该变成另一个大文档。

术语表怎么写

LeadFlow 里，最值得写进术语表的，正是第一章已经反复追问、却还没收成项目正式用语的四组概念：线索（Lead）、沟通记录（Communication Record）、AI 建议（AI Suggestion）和销售确认（Sales Confirmation）。它们决定的不只是命名——建议算不算对外承诺、来源算不算可追溯、确认算不算人工授权，都由这四组概念的边界决定。在 B2B 销售跟进场景里，这四组概念一旦混用，产品可信度就会出问题。

把这件事落到文件上，CONTEXT.md 的主体就是一块术语表，而且要有主见。它不收录“按钮”“错误”“状态”这种通用编程词，只收录本项目特有、容易混淆、会影响产品行为的概念。一条好的定义除了说清这个词指什么，最好也点明它不该和哪些近义词混用：

```
## Language

**Lead ( 线索 )** :
一条进入销售流程、尚未转成正式商机的潜在客户记录。
_Avoid_: Customer, account, contact

**Communication Record ( 沟通记录 )** :
销售与线索相关的邮件、电话纪要、聊天摘要或会议记录，是 AI 总结和建议的来源。
_Avoid_: message, note, transcript

**AI Suggestion ( AI 建议 )** :
模型基于沟通记录生成的草稿式跟进建议，需要销售确认后才能进入跟进计划。
_Avoid_: action, task, commitment

**Sales Confirmation ( 销售确认 )** :
销售用户对 AI 建议作出的人工接受、编辑、拒绝或执行动作。
_Avoid_: auto approval, executed follow-up
```

这类定义看起来很基础，却能明显降低后续漂移。AI 读到它以后，就不会随手把 AI 建议命名成 followUpAction，也不会把“销售确认”理解成系统自动审批。领域语言统一以后，PRD、DESIGN、TODO、ACCEPTANCE、代码类型和测试场景才更容易说同一件事。

多人协作时，术语还要对齐角色视角

如果项目只有一个人在做，术语混乱有时还能靠个人记忆补上。多人协作以后，这个问题会明显放大：销售说“客户”，产品说“线索”，设计稿里写“客户卡片”，工程代码里叫 `account`，测试场景里又写“联系人”。大家以为自己在讨论同一个对象，实际脑子里的边界并不一致。AI 进入这种项目时，会把这些差异一起读进去，然后在命名、状态、页面文案和验收场景里继续放大混乱。

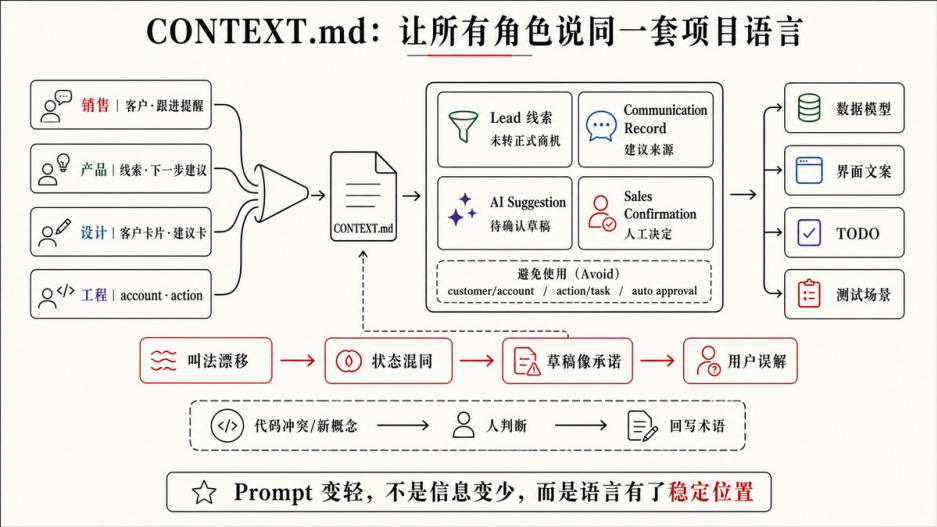
所以，`CONTEXT.md` 也可以记录不同岗位对同一概念的常用叫法，并把它们收敛成项目正式用语。这里要注意边界：它不是岗位黑话词典，不需要保存销售、产品、设计、工程和测试的所有表达；只有当某个叫法差异会影响数据模型、界面状态、任务拆解或验收判断时，才值得写进去。以 `LeadFlow` 为例，可以在术语表里补充“角色叫法”：

```
**Lead ( 线索 )** :
进入销售流程、尚未转成正式商机的潜在客户记录。
角色叫法：销售常说“客户”，产品常说“线索”，设计稿中可能写“客户卡片”，工程统一使用 `lead`。
_Avoid_: Customer, account, contact

**AI Suggestion ( AI 建议 )** :
模型基于沟通记录生成的草稿式跟进建议，需要销售确认后才能进入跟进计划。
角色叫法：产品可能说“下一步建议”，销售可能说“跟进提醒”，设计中表现为建议卡片，工程统一使用 `aiSuggestion`。
_Avoid_: task, action, commitment

**Sales Confirmation ( 销售确认 )** :
销售用户对 AI 建议作出的人工接受、编辑、拒绝或执行动作。
角色叫法：销售可能说“采纳建议”，产品可能说“人工确认”，测试场景统一写“销售确认”。
_Avoid_: auto approval, executed follow-up
```

这类信息对 AI 很有用。它让 AI 知道，当销售问“能不能对这条线索发报价邮件”时，销售运营打开的是 `Lead` 详情页里的 `AI Suggestion` 状态，而不是另起一个 `CustomerMessage` 入口；“客户卡片”对应 `lead`，“跟进提醒”对应 `aiSuggestion`，不会被混成两个不同模块。它也能帮助团队在 `Review` 时发现语言偏差：如果代码里出现 `executedSuggestion`，但 `CONTEXT.md` 明确说销售确认不等于已执行跟进，这就是需要停下来校准的信号。



不要让它吞掉其他文件

术语表越薄，越不容易过期，这正是它最大的好处。但真实项目里，你会很想往 **CONTEXT.md** 里再多塞一点：当前数据源在哪、线索有哪些字段、这一版还差哪些能力、哪些方案已经暂缓……这些信息确实有用，可一旦塞进来，**CONTEXT.md** 就会慢慢变成第二个当前状态垃圾桶，重新染上 **AGENTS.md** 那种“什么都往里堆”的毛病。

这也是上下文工程里最容易被误解的地方。好的上下文不是把资料倒满窗口，而是给 Agent 一份高密度、高信号的载荷：少到不会淹没真正重要的判断，多到足以避免它用常识乱补。**CONTEXT.md** 越克制，AI 越容易把它当作项目语言的事实源；它一旦混进字段清单、旧方案、会议纪要和当前进度，就会把“统一语言”这件事稀释掉。

更要命的是，这些信息几乎都另有更权威的家，而且变化得比术语快得多：

- 数据源在哪——仓库本身就是事实。**samples/leads/** 在不在，AI 读一下目录就知道，写错了立刻被发现，没必要在文档里再抄一份会过期的指针。
- 线索有哪些字段——代码里的类型才是事实。把字段表抄进 **CONTEXT.md**，只是多了一份注定要和代码打架的副本。

- 还差哪些能力、哪些方向本轮不做——这是非目标，属于 PRD.md。
- 默认只用脱敏样例、不碰真实客户数据——这是长期边界，属于 AGENTS.md。

所以更清楚的分工是：功能为什么做，写进 PRD.md；用户怎样看见状态、组件如何表达风险，写进 DESIGN.md；这轮做到哪里，写进 TODO.md；什么结果可以接受，写进 ACCEPTANCE.md 或模块验收文件；为什么选择某个难回退架构方案，写进 ADR 或决策记录；当前数据源和数据模型，回到仓库和代码本身去读；CONTEXT.md 只负责一件别人不管的事——告诉 AI 这个项目里的语言应该怎样理解。顺着这个边界，一份 LeadFlow CONTEXT.md 可以就这么薄：

```
# CONTEXT.md
```

```
## Language
```

```
**Lead ( 线索 )** :
```

进入销售流程、尚未转成正式商机的潜在客户记录，不等于已成交客户。

角色叫法：销售常说“客户”，产品常说“线索”，设计稿中可能写“客户卡片”，工程统一使用 `lead`。

Avoid: customer, account, contact

```
**Communication Record ( 沟通记录 )** :
```

销售与线索相关的邮件、电话纪要、聊天摘要或会议记录，是 AI 总结和建议的来源。

Avoid: message, note, transcript

```
**AI Suggestion ( AI 建议 )** :
```

模型基于沟通记录生成的草稿式跟进建议，需要销售确认后才能进入跟进计划。

角色叫法：产品可能说“下一步建议”，销售可能说“跟进提醒”，工程统一使用 `aiSuggestion`。

Avoid: task, action, commitment, executed follow-up

```
**Sales Confirmation ( 销售确认 )** :
```

销售用户对 AI 建议作出的人工接受、编辑、拒绝或执行动作，不是系统自动审批。

Avoid: auto approval

这份文件的重点不是完整，而是可靠。它只把 AI 最容易猜错的语言固定下来，把数据源、字段、目标、状态和验收都留给对应位置。这样，AI 进入任务时先拿到一套对齐的语言，再去仓库读数据、读 PRD 的目标、读 DESIGN 的状态、读 TODO 的任务和验收文件里的完成标准。

旧资料会误导 AI，但答案不是再加一块清单

有一类信息看起来很想塞进 `CONTEXT.md`：过期资料。很多项目失控，不是因为 AI 没看到资料，而是因为它看到了太多旧资料。旧文件不会因为被废弃就立刻消失，旧截图也不会自动失效，旧实现还可能在仓库里继续被搜索命中。对 LeadFlow 来说，这正是 Vibe Demo 留下的遗产：`legacy/mockLeads.ts` 里的静态假线索、暂缓的自动外发邮件方案、还没通过权限评估的 CRM adapter，以及早期那版界面里把 AI 建议和已执行动作显示成同一种样式的截图——那时候没有草稿状态，销售无从区分“建议”和“已承诺”。这些遗留物看起来很正式，也都可能把 AI 带偏。

但解决办法不是在 `CONTEXT.md` 里再开一块“不要再相信什么”的清单——那又会变回状态垃圾桶。更稳的做法，是让每类废弃信息回到它该去的位置：

- 废弃的叫法（不要再叫它 X）→ 放进术语表的 `_Avoid_`，这本来就是 `CONTEXT.md` 的职责。例如在 `AI Suggestion` 下写明 `_Avoid_: executed follow-up`，AI 就不会再把草稿建议命名成已执行动作。
- 废弃的功能或方案（自动外发邮件已暂缓、不做完整 CRM 替代）→ 这是非目标，写进 `PRD.md`。
- 废弃的数据或文件（`legacy/mockLeads.ts` 不是活跃数据）→ 由 `AGENTS.md` 的数据边界兜底（“默认只用脱敏样例数据”），必要时直接在那个文件或目录里标注废弃，让 AI 一读到它就看到提示。

这样，“旧资料会误导 AI”这个真问题并没有被忽略，只是不再靠 `CONTEXT.md` 一个文件硬扛。`CONTEXT.md` 仍然只做一件事：把当前该用的语言说清楚，顺带用 `_Avoid_` 标掉不该再用的旧叫法。

术语也会变，要及时回写

`AGENTS.md` 相对稳定，`CONTEXT.md` 里的语言其实也会动。项目里冒出一个新概念（比如把“已执行跟进”正式拆出来），某个词换了官方叫法，原来列在 `_Avoid_` 里的词反而被扶正成正式术语，或者发现代码里的命名和术语表已经对不上——这些都应该写回 `CONTEXT.md`。LeadFlow 从 Vibe Demo 到工程化，“下一步建议”逐渐拆成了 `aiSuggestion` 加 `confirmationStatus` 两个字段；如果术语表没跟上，AI 改建议卡片时仍会按旧叫法命名，草稿状态和已执行状态混在一起，建议卡片在界面上又变成“看起来已承诺”的样子。

这也是第一章说的状态回写。术语变了但文档没变，项目就会出现两套语言：人嘴里早就改叫“沟通记录”，`CONTEXT.md` 还写着“消息”；代码里已经统一用 `aiSuggestion`，术语表却仍把它和 `followUpAction` 并列。这种语言上的不一致迟早会渗进命名、文案和测试。

更新 `CONTEXT.md` 不等于把所有讨论都写进去。它只需要维护会影响下一轮命名判断的那部分语言：新出现的概念、改掉的叫法、升级或淘汰的 `_Avoid_`，以及必须回到代码确认的命名冲突。它的目标是让这套语言不只活在人的脑子里，同时也不去抢 PRD、TODO、设计和验收的职责。

它不替代读取代码和设计

`CONTEXT.md` 给 AI 一套初始语言，但它不是不可质疑的真理。如果 `CONTEXT.md` 说“`AI Suggestion` 在销售确认前只是草稿”，而代码里却出现了 `executedSuggestion` 这样的类型或字段，AI 不应该假装没看见——在 LeadFlow 这类销售工具里，这个命名冲突直接影响产品可信度：销售看到“已执行”样式，会以为系统已对客户作出承诺。它应该把冲突暴露出来：术语表说销售确认不等于已执行，代码命名却把两者混在一起，这要么是实现走偏，要么是术语该更新——无论哪种，都需要人来判断，而不是 AI 自己挑一个继续写。

同样，如果 `CONTEXT.md` 把“销售确认前建议是草稿”说清楚了，而 `DESIGN.md` 进一步定义了草稿、已确认、已拒绝、存在风险四种状态和对应组件表达，AI 也不能只凭这套语言就自由发挥界面。`CONTEXT.md` 统一项目的语言，`DESIGN.md` 说明用户怎样看见这个项目，代码、测试、页面和日志则提供当前

证据。

所以，更准确的说法是：**CONTEXT.md** 统一当前该用的语言，而不是充当最终裁判。它和代码、设计、验收冲突时，AI 应该把冲突交还给任务目标和人类判断，而不是静默挑一个最容易实现的版本。

CONTEXT.md 让 Prompt 变轻

有了这套文件，人每次就不需要把背景重新讲一遍。原来你可能要在 Prompt 里反复交代：“这里说的‘客户’其实是线索，不是已成交客户；‘AI 建议’是草稿，销售确认之前别当成已执行；‘沟通记录’才是建议的来源，不是随手写的备注……”这种对词的反复澄清最磨人，因为它每轮都要重来；当这些约定有了稳定位置——工作规则在 **AGENTS.md**，目标在 **PRD.md**，语言在 **CONTEXT.md**——任务入口就可以变轻：

请先读取 `AGENTS.md` 和 `CONTEXT.md`，再在客户详情页展开 AI 建议的来源记录，确认 `aiSuggestion` 在销售确认前只

Prompt 变短，不是因为信息少了，而是因为信息有了稳定位置。**AGENTS.md** 告诉 AI 在这个项目里怎样工作，**CONTEXT.md** 告诉 AI 这个项目里的词该怎么理解；下一节的 **PRD.md** 则继续回答另一类问题：这一版为什么做，服务谁，什么结果不可偏离。

2.3 PRD.md：定义目标和边界

AGENTS.md 把长期工作协议放进了仓库，**CONTEXT.md** 把项目语言统一了下来。这两份文件能帮助 AI 知道怎么行动、用词不跑偏，但还没有回答一个更产品层的问题：这一版为什么要做，服务谁，什么价值必须成立，哪些方向本轮明确不做。这个问题如果不被写清楚，AI 就会在实现时替人做产品判断。

这就是 **PRD.md** 的位置，它是把“目标”和“非目标”从人的脑子里拿出来，变成 AI、产品、设计和工程都能读取的产品合同。第一章说过，人不应该微操 AI 的每一步；但人必须定义解题空间，**PRD.md** 正是定义这块空间的文件。

在 Agentic Coding 里，`PRD.md` 还有一个容易被忽略的作用：它让规划和编码可以分开。一个会话可以负责把模糊想法、访谈材料和业务判断整理成 PRD；另一个会话再基于 PRD、设计和验收进入实现。这样做不会让流程变重。它的作用是避免编码会话背着整段产品讨论、临时假设和未决想法直接开写。PRD 把已经确认的产品判断沉淀下来，把还没确认的问题标出来，让后续 Codex 接手时读到的是一个可执行的目标空间，而不是一团长对话里的情绪和猜测。比起直接对 AI 说“帮我做个销售工具”就进入实现，先把产品判断压成 PRD 再开发，能在项目一开始就排除大量不想要的实现方向。等做出来再纠偏，代价往往远超当初写 PRD 的时间。

PRD 是工程合同，但工作在目标层

`PRD.md` 是产品工程的第一层文件。说“工程”不是比喻：PRD 是会被 AI 读取、被设计引用、被验收对照的活文档；产品意图偏移、功能边界扩张、未决问题被悄悄填充之后，它都需要被更新。不维护的 PRD 会过期，过期的 PRD 会让 AI 按旧判断实现新需求，跟没有 PRD 的效果相差无几。

但 PRD 工作在目标层，不是实现层。它定义三件事：为谁做（目标用户和核心场景）、做出什么结果才算成立（价值闭环和成功现象）、哪些边界不能越（范围约束和不可接受结果）。至于每个组件怎么写、每个函数叫什么、目录怎么拆，那些属于实现层，由 Codex 在读取项目、`DESIGN.md`、`TODO.md`、代码结构和验收标准之后自己选择。

LeadFlow 的产品目标是帮助销售在跟进线索前快速理解客户背景、最近沟通、潜在需求和下一步建议，同时避免 AI 把推测包装成客户事实。这个目标和“生成一段听起来专业的销售话术”有本质区别，它不会告诉 AI 创建几个组件，却会影响后续所有实现判断：建议必须能回到沟通记录，信息不足时不能硬补，销售确认前不能变成已执行动作，隐私字段不能进入日志或模型请求记录。

`PRD.md` 定义目标和边界，`DESIGN.md` 承接界面设计（视觉 token、组件状态、交互流程），两者各自工作在不同问题上。PRD 如果越位写进实现细节，人会重新陷入微操 AI 的模式；如果写得太虚，AI 又只能按常见产品套路补空白。好的 PRD 停在目标层：它明确产品成立的价值闭环、风险边界和不可接受结果，把实现判断留给下游。

一份轻量 PRD 可以长什么样

对一个刚从 Vibe Demo 进入工程化开发的项目，PRD 不需要一开始就写成几十页。更实用的做法，是用固定的轻量结构把产品从“想法”压成“可执行的 MVP”。这种结构至少要包含产品名称、Slogan、工具形态、用户画像、场景故事、核心功能 MVP、交互流程和 ASCII 流程图；其中目标与非目标不一定非要单独起大标题，但必须在用户场景、MVP 功能和流程边界里被清楚表达。

写这份 PRD 之前，先问六个问题，分别对应 PRD 要回答的六个核心维度，把模糊愿望逐步压成产品判断：

1. 工具形态：这个产品以哪种形态交付？（桌面网页 / 移动网页 / 桌面软件 / 浏览器插件）

2. 目标用户：第一版最优先服务哪一类具体人群（具体到能排除相近但不同的人群）？

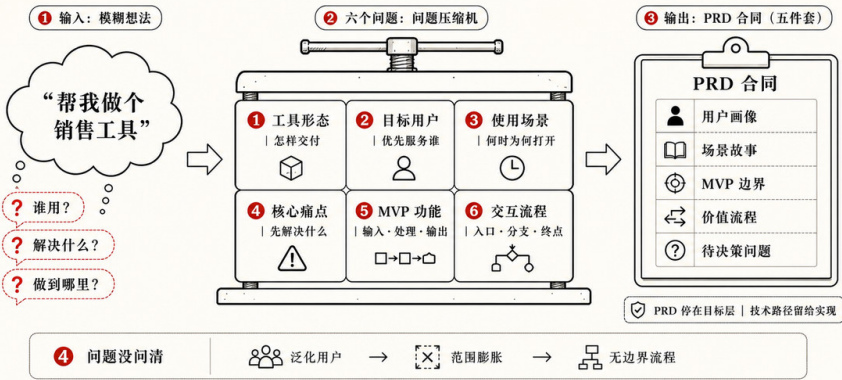
3. 使用场景：用户在什么时刻、因为什么触发、打开产品完成什么核心动作？

4. 核心痛点：在已确认的用户和场景里，最需要先解决的具体问题是什么？

5. MVP 功能：首版需要哪 3-4 个功能模块，每个模块的输入、处理、输出分别是什么？

6. 交互流程：用户从入口到获得价值，最自然的操作路径经过哪些步骤和分支？

六个问题，把模糊想法压成可执行 PRD



★ PRD 不是愿望清单，而是对解题空间的**产品合同**

六个问题问完，再开始写文档。以下面的内容为例：

产品名称: LeadFlow
Slogan: 让销售跟进有依据, AI 建议可追溯
工具形态: 网页应用, 优先在桌面浏览器使用

【用户画像】
周若琳 (化名) , 29 岁, B2B SaaS 销售运营, 负责协助 8 人销售团队整理每天新增线索和客户沟通记录。她不缺 CRM 表格

【场景故事】
场景一：上午 9:30, 周若琳准备销售晨会。她打开 LeadFlow , 导入昨天整理好的脱敏线索样本, 列表自动显示客户公司、线
场景二：下午 4 点, 一位销售询问某条线索是否可以直接发送报价邮件。周若琳在 LeadFlow 中打开该客户详情, 发现沟通记

【核心功能 MVP】
- 线索导入与列表：支持导入本地脱敏样例线索, 在线索列表中展示公司、阶段、负责人、优先级、最近沟通时间和风险提示

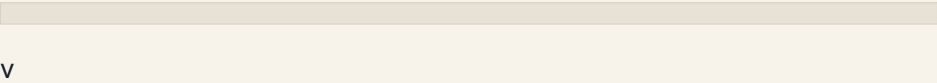
- 客户详情与沟通记录：点击线索后进入详情页, 展示客户基本信息、销售阶段和关联沟通记录；沟通记录是 AI 总结和建议的
- AI 总结与下一步建议：基于当前线索的沟通记录生成客户需求摘要、风险提示和下一步跟进建议；建议必须保留来源记录,
- 销售确认与边界状态：AI 建议默认是草稿, 销售确认前不得进入已执行跟进记录；首版不做自动外呼、自动邮件、报价审批

【交互流程】
- 用户进入 LeadFlow 主界面, 导入或选择本地脱敏样例线索, 系统展示线索列表和基础筛选。
- 用户点击某条线索, 进入客户详情页, 查看客户事实、销售阶段和沟通记录。
- 系统基于沟通记录生成 AI 摘要和下一步建议；若记录不足, 展示信息不足状态, 不生成建议。
- 用户展开 AI 建议的来源记录, 判断建议是否可信, 并选择确认、编辑或拒绝该建议。
- 系统记录本地确认状态, 更新页面展示；不写回真实 CRM, 也不向客户发送消息。

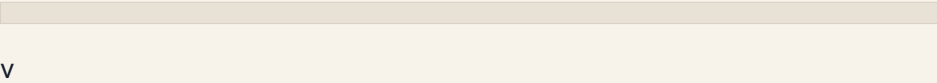
【ASCII 流程图】

[1. 线索入口]

导入/选择脱敏样例线索



线索列表: 公司 / 阶段 / 负责人 / 最近沟通 / 风险提示



[2. 客户详情]

客户事实区

+-- 基本信息

+-- 销售阶段

+-- 沟通记录时间线

V

[3. AI 分析]

沟通记录 -> AI 摘要 + 下一步建议

+--> 有足够记录: 显示建议 + sourceIds

+--> 记录不足: 显示信息不足状态

V

[4. 销售确认]

展开来源记录

+--> 确认/编辑建议 -> 本地待跟进计划

+--> 拒绝建议 -> 保留拒绝状态

V

不自动发送消息 / 不写回真实 CRM

这份 PRD 的价值在于把目标和非目标写进了同一条价值路径里，格式是否整齐是次要的。读者能看见用户是谁、为什么需要这个工具、核心场景是什么、首版做哪些功能、哪些能力明确不做，以及用户如何从线索入口走到 AI 建议和销售确认。Codex 读到它以后，也更容易判断一个实现是否偏离了产品边界。

这五个部分各自钉住一个判断

产品名称、Slogan 和工具形态先把“做什么形态的东西”交代清楚，后面五个部分才是真正约束实现的地方。它们各自钉住一个产品判断；少了任何一个，AI 都得自己脑补那一环，而脑补的方向往往偏离首版意图。

用户画像回答“为谁做、他真正卡在哪”。LeadFlow 的画像不是“一个销售”，而是周若琳：管 8 人团队，不缺 CRM 表格也不缺主观判断，真正难的是线索分散、记录不全、不知道先跟谁。有了它，AI 和团队在取舍功能时有一个真人可对照，知道该优化“先跟谁”，而不是再加一个花哨的评分。没有它，AI 只能按“通用销售工具”补全需求，很容易把 LeadFlow 做成一个大而全、却没解决周若琳问题的 CRM。

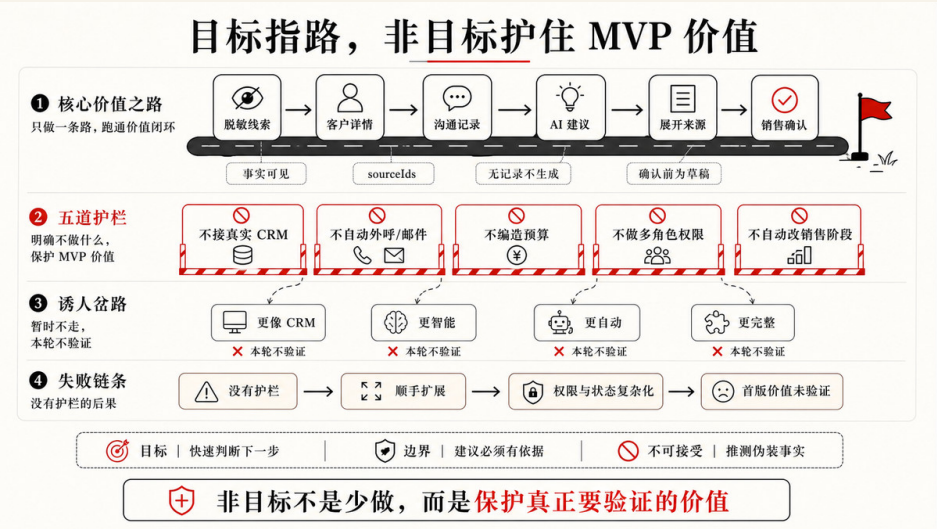
场景故事把画像放进具体的时间和动作里，演示产品在真实一刻怎么被用。场景一让“导入→看详情→读 AI 建议→展开来源→标记待确认”变成一条看得见的行为序列；场景二专门演示边界：没有预算信息时，系统提示“无法生成建议”，而不是编一段通用报价话术。有了它，抽象功能落到可观察的行为上，连“该拒绝、该降级”的时刻都被写了出来。没有它，功能停留在名词清单，AI 不知道这些功能怎么串起来，也容易把“信息不足”当成要补全的缺陷去硬生成。

核心功能 MVP 列出本轮真正要做的功能，并把边界贴在每一条后面。CRM 功能空间里，每条能力都可以向外延伸：“AI 建议”可以扩成真实模型调用，“沟通记录”可以扩成外部同步；把边界贴在功能后面，是把“做什么”和“做到哪里”绑在一起的方式。它把愿望收敛成可实现、可验收的范围：“AI 总结与下一步建议”后面就跟着“建议必须保留来源记录，信息不足时不得生成通用销售话术”。有了它，团队和 AI 都清楚这一版做到哪里、哪些明确不做。没有它，PRD 就退回愿望清单——一串听起来都对、却没法告诉 AI 本轮该收敛到哪的功能罗列，范围很容易失控成“更像 CRM”的大工程。

交互流程用文字写出用户从入口到完成一次任务的步骤，说明功能的先后、状态流转和分支。LeadFlow 里，“AI 建议”和“已执行跟进”一旦混成同一种状态，销售就会把草稿当成对客户的承诺；流程文字把“确认前只是草稿”这条岔口写死，比口头约定可靠得多。它让“导入线索→进入详情→生成或拒绝建议→确认或拒绝→记录本地状态”连成一条能走通的路径，也标出“记录不足就展示信息不足状态”这样的岔口。有了它，AI 知道页面之间怎么走、状态怎么流转。没有它，AI 容易把每个功能各自实现成孤岛，串不成一条用户真正走得完的路，或者自行编排出一套偏离意图的流程。

ASCII 流程图把交互流程画成一眼能看到分支和终点的结构图。LeadFlow 里，“记录不足”分支是产品的核心安全阀，阻止 AI 在没有依据时硬生成建议；这种边界藏在文字里容易被读漏，画成图就很难被忽略。它的价值是把关键分叉和边界固化下来：AI 分析分成“有足够记录”和“记录不足”两条路，销售确认分成“确认 / 编辑”和“拒绝”两条路，最后一行明确写着“不自动发送消息 / 不写回真实 CRM”。有了它，这些分支和边界比埋在文字里更难被读漏。没有它，AI 很可能只实现一条顺利的主干路径，把降级、拒绝和不可越界的终点当成次要细节漏掉。

把这五部分连起来，PRD 是在回答一条链：为谁做 → 在什么场景用 → 本轮做哪些功能 → 怎样走完一次任务 → 关键分叉和边界在哪。每一环都替 AI 钉住一个本来要靠它自己猜的判断；任何一环缺席，AI 都会用最常见的产品套路补上那个缺口，而这正是首版最容易跑偏的地方。



PRD 要把抽象目标压成成功现象

“LeadFlow 要好用”太抽象，“AI 建议要可信”也太抽象。PRD 需要继续追问：什么现象出现时，我们可以说这个版本目标成立？这个追问会把产品目标压成可观察的结果，让后面的 `TODO.md` 和 `ACCEPTANCE.md` 有东西可拆。

Vibe Demo 不会替你留下「做了什么」的清单——中间假设、临时假数据和绕过的失败，往往只留在那次生成过程里。但对着 Demo 走查一遍，几类失真仍然看得出来：建议没有来源、评分看起来像真数、草稿建议和已执行动作混用同一种界面状态、手机号可能出现在日志里。难的是这些判断当时还散落在对话和界面印象里，并没有写进任何文件。PRD 要做的，是把已经隐约看见的问题压成可观察的成功现象——失真的反面，就是这一版目标成立的条件。对 LeadFlow 来说，成功现象可以包括：

- 线索列表来自脱敏样例，没有页面假数据。
- 用户能进入客户详情，看到客户基本信息、销售阶段和沟通记录。
- AI 总结和下一步建议都能回到来源记录。
- 没有沟通记录时，系统明确提示无法生成建议。
- 销售确认前，建议不会被标记为已执行或已对外承诺。

- 客户手机号、邮箱和合同金额不会出现在日志或模型请求记录中。
- 无线索、无沟通记录、AI 分析失败时，页面都有可理解的状态。

这些还不是完整验收场景。PRD 层的成功现象更像结果边界，说明“我们要看到什么才相信方向成立”；后面的 `ACCEPTANCE.md` 会把它们拆成 Given / When / Then 或其他可检查场景。这个分工很重要：PRD 不负责替验收写完所有测试，但如果 PRD 连成功现象都没有，AI 就会用“页面能跑”“卡片出现了”“功能看起来有了”来判断完成。

把非目标写进 PRD 的好处

LeadFlow 里，你说“让销售更方便跟进”，AI 可能补提醒、补自动邮件、补客户分层、补 CRM 同步；这位销售运营真正卡在“不知道先跟谁”，补邮件和分层不是首版要解决的事，但目标句里看不出来。目标告诉 AI 往哪里走，非目标告诉 AI 不要把任务扩大到哪里。对 LeadFlow 的首版来说，“不接入真实 CRM、不做自动外呼、不让 AI 直接对客户作出承诺”这些非目标会极大改变 AI 的实现路径：没有它们，AI 可能为了“更像 CRM”去做权限系统，为了“更智能”去接外部服务；有了它们，AI 知道哪些方向本轮不该碰。很多任务失控，不是因为目标写错，而是因为非目标缺席。

把非目标写进 PRD，第一个好处是它和目标共用一条价值路径。AI 读到“基于沟通记录生成下一步建议”这条功能时，紧接着就读到“信息不足时不得生成通用销售话术”。目标和它的边界一起出现，AI 就不容易只接收“要做什么”，却漏掉“做到哪里为止”。如果把非目标抽到文档末尾另起一节，它很容易变成没人对照的死清单。

第二个好处是非目标成了可被读取和审查的产品合同，而不是停留在某个人脑子里的默契。团队里任何人都能在 PRD 里看到本轮明确不做什么，AI 实现时也有据可依：遇到“顺手接入真实 CRM”“再加一个自动发送按钮”这类诱惑，它能回到 PRD 判断这属于本轮边界之外，而不是自己替人做产品决定。

第三个好处是非目标让范围收缩这件事被显式记录下来。真实项目里，“这版先不做”常常是慎重的产品判断；写进 PRD，它就从一次口头约定变成可追溯的决定。下一轮要把某个非目标转成目标时，团队是在修改一份白纸黑字的

合同，而不是在回忆“当时到底说没说过不做这个”。

所以，非目标不需要在 PRD 里单独占一个标题，但它必须贴着目标、贴着功能、贴着流程，被 AI 和团队一起读到，这也是前面那份轻量 PRD 把非目标折进价值路径、没有另起一节清单的原因。

当项目增长：PRD 也可以按模块拆分

单一 PRD.md 适合首版和小项目。但当功能模块增多、团队拆分、或某个功能的产品说明开始撑满单个文件时，更合理的做法是把 PRD 拆成模块体系，各模块的产品细节分别放进 prd/<模块>/README.md。

对 LeadFlow 来说，ai-suggestion 和 lead-list 是两个需要单独成模块的原因：AI 建议模块承担来源追溯、确认状态和隐私边界，是产品风险最集中的地方；线索列表模块处理导入、筛选和阶段展示，边界相对清晰。两者的验收场景完全不同，放在一个文件里会让审查视角混淆：AI 建议的“记录不足不得生成建议”和线索列表的“优先级排序”，不应该共用同一份完成标准。目录可能长这样：

```
prd/
  README.md      # 模块索引，指向各子模块
  ai-suggestion/
    README.md    # AI 建议模块的产品能力说明
    technical.md # 实现路径、数据结构、代码位置
    acceptance.md # 验收契约 ( GWT 场景 )
  lead-list/
    README.md    # 线索列表模块
    technical.md
    acceptance.md
```

这个结构有一个值得注意的特性：同一模块的产品边界 (README.md) 和完成标准 (acceptance.md) 放在同一个文件夹里。AI 处理 ai-suggestion 模块时，不需要跨文件夹猜这个模块的验收标准在哪里；团队下一轮迭代时，也能在同一个地方读到“这个模块要做什么”和“怎么验证它做对了”。这不是格式习惯，而是让产品意图和完成证据保持共址，不会随着项目变大而分离。

单文件 PRD 是起点，不是终点。LeadFlow 首版可以先用根目录 PRD.md 把产品合同写清楚；等 AI 建议、线索管理、销售确认各自功能稳定以后，再把产

品目标和验收场景按模块归位。模块拆分的时机是功能边界清晰之后，而不是项目一开始。

一份 PRD 是否合格

检查 PRD 时，真正的标准是它能否改变 AI 的行动，有没有功能列表是次要的。可以逐条对照：

- 是否说明了目标用户和核心场景
- 是否说清这一版要验证什么价值
- 是否把本轮做什么和不做什么写成明确边界
- 是否把抽象目标压成可观察的成功现象
- 是否把待决策问题标出来
- 是否避免规定过细实现

LeadFlow 的 PRD 如果能回答这些问题，AI 就知道首版是为这位用户解决“不知道先跟谁”的问题，而不是做一个功能齐全的 CRM 替代品。但产品目标确定之后，还有一个问题没有解决：这个工作台长什么样，她晨会前三分钟怎样扫完线索状态，AI 建议是草稿还是已确认怎样一眼分清。这是 DESIGN.md 要继续承担的工作。

2.4 DESIGN.md：让设计系统进入工程过程

PRD.md 已经说清 LeadFlow 这一版为谁做、做什么、不做什么。产品目标定下来之后，Vibe Demo 里那类视觉债会重新冒出来：AI 建议区和客户事实区视觉上平级，草稿建议和已执行跟进用的是同一种状态样式，来源入口藏得深，「信息不足」时页面给不出一个有独立意义的边界态。这些问题 PRD.md 管不了，它们需要另一份文件回答：这个产品长什么样，用户怎样一步步操作，视觉 token 和组件状态由谁统一。

这就是 DESIGN.md 的位置。它只覆盖两件事：视觉设计和交互设计。视觉设计回答产品长什么样，包括颜色、字体、间距、圆角、阴影、图标库和组件视觉规格；交互设计回答用户怎样操作，包括页面结构、信息层级、关键流

程、组件在不同情境下的状态，以及响应式行为。`DESIGN.md` 把这两类判断写成 Codex 可以读取、实现和检查的对象。

生成 `DESIGN.md` 不一定要从零开始写。Google Labs 的 `Stitch` 提供了一个更直观的入口：上传你喜欢的界面截图，它会自动识别颜色、字体、布局和风格，生成一份可以直接导出的 `DESIGN.md`。作者认为，从截图开始是一个很低摩擦的起点。

把设计取向写成工程语言

设计系统的工作，是把视觉和交互判断从人的脑子里拿出来，写成 token 和规则。说「LeadFlow 要克制、便于扫描、客户事实优先于 AI 建议」是设计取向；但取向停在这里，AI 仍然不知道底色用什么色值、字体层级怎么分、建议区和事实区谁在视觉上更高。把这些判断落成工程语言，才不用每轮对话里再提醒一次。视觉判断如此，交互判断同样：组件有哪些状态、信息怎样分层、哪些操作放在主操作位，也需要落成可执行的描述，Codex 才不会只做出流程跑通却状态缺失的页面。

如果我们将 LeadFlow 的视觉方向指向「克制、便于扫描、客户事实优先于 AI 建议」，那么这意味着颜色 token 必须用 off-white 底色减轻长工作会话的视觉疲劳，字体层级必须能支撑列表快速扫描，AI 建议区必须在视觉上低于客户事实区。这些判断写进 `DESIGN.md` 之后，就不需要每一轮对话里再提醒一次「别做成营销页」。

以 LeadFlow 的设计系统 frontmatter 为例：

```
colors:
  surface: '#f8f9fa'
  on-surface: '#191c1d'
  primary: '#630ed4'
  secondary: '#b90538'
  status-high: '#F43F5E'
  status-medium: '#FB923C'
  status-low: '#10B981'
```

```
typography:
  body-md:
    fontFamily: Inter
    fontSize: 16px
    fontWeight: '400'
```

```
  lineHeight: '1.5'
label-md:
  fontFamily: Inter
  fontSize: 14px
  fontWeight: '600'
  letterSpacing: 0.01em
headline-xl:
  fontFamily: Inter
  fontSize: 48px
  fontWeight: '700'
```

```
spacing:
  base: 8px
  gutter: 24px
```

```
card-padding: 24px
container-max-width: 1280px
```

这些值看起来像细节，实际上会改变实现路径。对 Codex 来说，`status-high: '#F43F5E'` 不是「找一个红色差不多就行」，`base: 8px` 也不是「留白舒服一点」。它们应该进入 CSS variables、Tailwind 配置和组件样式，工程师 Review 时也能据此判断这次实现是否沿用了设计系统，还是又在某个卡片里临时写死了一组新的 hex 和间距。说「按钮用红色」很脆弱，因为下一轮 AI 可能换一个红色，再下一轮又在另一个组件里写死一组渐变；token 是可复用、可检查、可扩展的工程语言，它让视觉判断不再依赖每一轮对话里的提醒。

一份 DESIGN.md 包含什么

设计系统的标准模块不多，但每个模块各有语义。生成 DESIGN.md 不一定要从零开始写——Google Labs 的 [Stitch](#) 提供了一个低摩擦的入口：上传界面截图，它会自动识别颜色、字体、布局 and 风格，生成一份可以直接导出的 DESIGN.md。以下各模块说明以 [Stitch](#) 生成的 Design System 为参考：

品牌与风格说明气质和视觉方向的依据。不是「现代简洁」这种无效描述，而是从产品语境推出来的具体取向：LeadFlow 是销售和运营反复打开的 CRM 工作台，视觉方向以信息密度优先，专业感大于冲击感，整体风格融合 Minimalism 与 Glassmorphism，用柔和的 mesh 渐变缓解线索管理的视觉压力。

颜色把调色板写成 token 并说明各色的语义和禁用规则：mesh 渐变色只用于背景装饰，不用于传达状态；状态色（红 / 橙 / 绿）只用于线索优先级，不作装饰；颜色一律走 token，禁止在组件里写死 hex 值。

字体定义全站字体系统：LeadFlow 全站使用 Inter，通过字重和字号实现层级，不依赖颜色区分。body-md（16px / 400）是基准阅读字号，label-md（14px / 600）用于「最近跟进」「预算状态」等元数据，headline-xl（48px / 700）只用于极少数页面标题，而销售运营扫线索状态依赖的是 body 和 label，不是 hero 标题。

布局与间距固定核心页面结构：桌面端采用列表 + 详情分栏，列表 4 列，详情与 AI 建议 8 列，便于晨会前三分钟快速扫列表并点入详情；间距基准 8px，所有 padding 和 margin 取倍数；1024px 以下列表收起为抽屉；768px 以下切换为独立纵向视图，触控目标高度 ≥ 44px。

层次与深度规定视觉层级如何建立：页面底层 #f8f9fa，主内容卡片白底 + 极淡阴影，AI 建议区半透明白色（80%）+ 20px backdrop blur，客户事实区在视觉上高于 AI 建议区，建议区不得占据主要视觉焦点。这条规则是产品设计原则的界面表达：在视觉层级上强化「事实上，建议在下」。

形状规定圆角语义：按钮和输入框 rounded-lg（16px），大卡片和详情面板 rounded-xl（24px），状态徽章完整圆角（999px）与可操作按钮形状区分，避

免用户把状态标签误读成可点击入口。

组件写清技术栈和策略：样式用 Tailwind CSS + CSS 变量，图标全站唯一 `lucide-react`（禁止 emoji 充当功能图标），媒体使用本地 SVG 占位，MVP 不引入图表库。

设计验收收口哪些变更需要截图或浏览器检查：修改 AI 建议相关组件时，验收必须覆盖信息不足、待确认、已确认、来源展开和错误五种状态；关键页面在桌面（ $\geq 1024\text{px}$ ）和窄屏（ $\leq 768\text{px}$ ）下正常展示。

DESIGN.md 还可以包含什么？

从工程实践来看，还有几类设计判断值得写进 `DESIGN.md`——不是因为它们更高级，而是因为不写清楚，AI 在实现时就会自己发明。

图标规范不只是指定图标库，还要写清不同使用情境下的尺寸语义。导航图标、行内操作图标、空状态引导图标尺寸不同、语义不同；纯图标按钮必须提供 `aria-label`，状态图标必须配文案，不能只靠颜色传达含义。这类细节不写进设计文件，Codex 补按钮时就会临时猜一个尺寸，不同组件里很快出现三四套图标大小。

媒体与占位图说明项目的媒体策略：首版 MVP 用本地占位还是远程 CDN，图片加载失败的 fallback 是什么，头像用 initials 还是真实照片，所有图片是否需要固定宽高比以防止 CLS（布局偏移）。这些判断如果不事先说清，Codex 容易在一个组件里用本地 SVG、另一个组件里写死远程 URL，或者加载失败时让页面布局塌掉。

数据状态的视觉表达在接入 AI 或外部 API 的项目里尤其需要写明：`FIXTURE`（本地样例数据）、`LIVE`（真实 API 调用）、`LIVE_UNCONFIGURED`（Key 未配置时的降级态）各自应该以什么界面表达呈现——哪种状态需要「示例数据」角标，哪种状态只显示配置引导而不展示内容。这种区分放在 PRD 里太细，放在代码注释里太散，写进 `DESIGN.md` 才能让 Codex 实现各个组件时保持一致，不会在一处加角标、另一处直接展示 mock 输出、再一处什么都不说明。

无障碍基线只需要几条规则就能避免大量被动返工：颜色对比度最低标准（正文 4.5:1，大字 3:1），焦点环样式（不得设为 `outline: none`），状态信息不能只靠颜色传达。写进 `DESIGN.md` 之后，Codex 实现表单和按钮时会主动遵守，而不是等到 Review 才发现整个项目没有焦点样式。

动效与过渡是容易被 AI 随意发挥的区域。不写明规则，Codex 可能在每个组件里装饰性地加入不同时长的 transition，或者在加载状态里混用 spinner 和 skeleton。设计文件应该说清几件事：骨架屏还是 spinner（骨架屏适合内容形状可预测的区域，spinner 适合操作等待）；transition 的基准时长（进出场动效建议 150-200ms，页面切换 250-300ms）；必须声明 `prefers-reduced-motion` 支持，对运动敏感的用户应该关闭所有非必要过渡。这些不是精雕细琢的动效设计，而是防止页面运动感失控的基础规则。

DESIGN.md 可以写「不要这样做」

`DESIGN.md` 除了写应该出现什么，还应该写不应该出现什么。AI 很擅长为了让页面「看起来完整」补一些视觉元素，而这类补全往往会悄悄破坏设计系统的一致性。负面约束尤其适合给 AI，因为它们把不可接受的实现方式提前排除掉：

需要避免的实现方式

- 不得在组件里写死 hex 颜色，颜色一律走 token。
- 不得混用图标库，或用 emoji 充当功能图标。
- 不得为某个页面临时新增一套间距尺度。
- 空态不得显示假数据或灰色占位骗术。
- 重要提示不得只藏在悬浮 tooltip 里。
- 加载时不得让布局发生明显跳动。
- 不得用装饰性「意向评分」或进度条暗示数据可信度。
- 不得把 AI 建议区的视觉层级抬过客户事实区。
- 不得用同一按钮样式表达「确认草稿」和「已对外发送」。
- 记录不足时不得硬补通用销售话术或报价建议。

前六条管视觉系统纪律，后四条管 CRM 信任边界。AI 仍然可以自己选择组件结构、状态文案和布局实现，但它知道哪些视觉捷径不可接受。

DESIGN.md 只承接设计依据

PRD.md 决定产品要达成什么价值、服务谁；DESIGN.md 决定这个产品长什么样、用户怎样操作它。两者承接但不重叠。

「AI 建议默认是草稿，确认前不得标为已执行」——这是 PRD.md 的产品边界。DESIGN.md 把这条事实翻译成界面判断：草稿、已确认、已拒绝三种状态视觉区分；客户事实区层级高于 AI 建议区；来源入口在卡片内可见，不藏进 tooltip。前者是产品判断，后者是界面判断。DESIGN.md 不需要复述 PRD.md 的产品规则，它只从产品事实推出界面应该怎么做，让 Codex 读 DESIGN.md 时拿到的是设计约束，而不是产品规则的翻版。

DESIGN.md：把产品边界翻译成界面行为

1 边界翻译（从 PRD 到界面行为）

PRD facts

DESIGN rules

evidence

事实优先于建议	→	事实区层级更高	→	桌面/窄屏截图
确认前只是草稿	→	五种状态可区分	→	五态检查
建议必须有来源	→	来源入口可见	→	来源可展开
不展示假可信度	→	空态不放假数据	→	边界态检查

2 设计工作台（统一构件语言）

颜色 token

Aa 字体层级

88 Spx 间距

圆角语义

唯一图标库

→ CSS变量/Tailwind → 共享组件

3 反模式（禁止区）

✗ 写死 hex

✗ 混用图标

✗ 临时间距

✗ AI 区抢主视觉

4 责任链（从目标到证明）

PRD
为什么/做到哪

→

DESIGN
长什么样/怎样操作

→

TODO
本轮拆解

→

ACCEPTANCE
如何证明

设计取向只有变成规则与状态，才进入工程

这也是把两个文件分开的原因。PRD.md 如果承担所有界面细节，会越来越像杂物间；DESIGN.md 如果承担产品目标，设计师会被迫解释业务范围，工程师也很难判断哪些是产品边界、哪些是界面表达。职责分开以后，Codex 才能在正确的位置读取正确类型的判断。

进入 TODO 和验收

DESIGN.md 不应该只对设计师有用。只要任务涉及视觉系统、组件状态、页面布局或交互流程，TODO.md 里的任务项就可以直接引用它：

82

- [] AI 建议区域覆盖信息不足、待确认和来源展开
- 设计：见 `DESIGN.md#组件`、`DESIGN.md#层次与深度`
- 验收：见 `ACCEPTANCE.md#记录不足时不生成建议`、`ACCEPTANCE.md#建议待确认态与来源可见`

这样设计就不再只是「做得像不像」，而是进入当前任务的执行链条，成为 Codex 可以据此拆任务、实现和检查的外部状态。修后端字段映射、调整导入脚本、补日志脱敏规则这类任务不需要把 **DESIGN.md** 拉进来；设计文件只在它真正影响结果的地方出场。

DESIGN.md 也会过期

设计文件同样需要回写。技术栈变化、颜色或字体 token 调整、组件新增了一种状态——例如 LeadFlow 把「信息不足」从通用错误改成独立态——或者设计走查发现销售仍把草稿误读成已执行，**DESIGN.md** 都应该更新。如果代码和页面变了而 **DESIGN.md** 没变，下一轮 Codex 会继续按旧设计实现，视觉债就会重新回来。**DESIGN.md** 不是设计师写完就冻结的说明书，而是团队共享的设计系统状态；只要界面判断变了，它就应该跟着变。

设计系统进入项目以后，仍然需要被拆成当前这一轮可执行、可验证的任务，这就是 **TODO.md** 要承担的工作。

2.5 TODO.md：记录当前执行状态

LeadFlow 从 Demo 进入工程化后，「AI 跟进建议」往往跨很多轮推进：先改哪一块、做到哪里、哪里失败过、哪里要等人拍板、什么证据足以打勾——这些执行层问题，如果只留在会话里，新 Agent 很容易从「组件已写好」倒推出「功能已完成」，把来源缺失、假建议和确认状态混淆重新带回来，还会顺手把 CRM 同步、自动发信等宽范围改动当成合理下一步。**TODO.md** 记录的就是这层外部执行状态：把 PRD 的目标和 ACCEPTANCE 的标准拆成可推进、可中断、可恢复、可审查的任务；它不是备忘录或道德打卡，没有它，AI 就会在「做完整个 LeadFlow」的大目标里自行规划路径，拆错时把范围、验收和风险一起带偏。

长任务研究里有一个反复出现的结论：当上下文会被截断、任务会跨很多轮推进时，Agent 需要一个能保留重要事实、策略和下一步的外部记录，否则它很容易重复错误、忘记失败原因，或在看似合理的循环里漂移。Codex 官方文档也提醒复杂工作更适合拆成更小、更聚焦、可验证、可 Review 的步骤，并把 **AGENTS.md** 放在任务开始前就会读取的位置，用来提供稳定的项目指导。

把这几个事实放回本章，结论就很清楚：**AGENTS.md** 和 **TODO.md** 是让 Agent 完成长任务的两类关键文件。**AGENTS.md** 让 Agent 每次进入项目时知道长期工作协议，**TODO.md** 让 Agent 在一轮漫长、可中断的任务里知道当前执行状态。前者回答“在这个项目里应该怎样工作”，后者回答“这件事现在推进到哪里”。如果只有 **AGENTS.md**，AI 知道规矩，却不知道本轮进度；如果只有 **TODO**，AI 知道任务，却可能忘记长期边界。两者合在一起，长任务才不只是一个长 Prompt，而是一个可持续推进的工程循环。

从 workflow 角度看，**TODO.md** 还是计划的交接物。复杂任务里，前一轮分析会积累大量上下文和判断：哪些文件相关，哪些路径被排除，哪些验证失败过，哪些风险必须先收口。如果这些只留在会话里，下一轮执行 Agent 只能重新读、重新猜、重新犯同样的错；写进 **TODO**，它就从一次临时计划变成可被继续执行、可被 Review、也可被修正的项目状态。

TODO 不应该只写实现动作

Agentic Coding 里的 **TODO**，不应只列施工动作，而应从 PRD 的目标、**DESIGN** 的状态和 **ACCEPTANCE** 的条件拆出行为结果。AI 仍然会创建组件、写函数、改数据结构，但这些只是实现路径；**TODO** 关心的是项目状态是否向目标移动。更好的任务表达应该像这样：

- [] 线索列表能展示阶段、负责人、优先级和最近沟通时间
- [] 客户详情页能展示当前线索的沟通记录
- [] AI 下一步建议必须显示来源记录
- [] 沟通记录不足时不生成建议
- [] 销售确认前建议处于“待确认”状态
- [] 客户手机号和邮箱不进入日志
- [] 相关验证证据已记录

这类 TODO 给 AI 留出了实现空间，也给人留下了 Review 依据。它没有指定必须创建哪个组件、函数叫什么、目录怎样拆，却直接对应 Demo 之后那几条硬约束：销售运营和销售能在界面上看见什么、什么风险不能被引入、什么证据应该留下。

一个可执行的 TODO 长什么样

Demo 能点按钮，不等于工程上已经有可恢复的闭环。LeadFlow 首版要把「AI 跟进建议」从演示功能收成 MVP：销售能在详情页看到沟通记录，能在确认前分辨建议来源，也不会无记录时看到一段像真分析的话术。**TODO.md** 就是把这一轮收敛目标拆成当前状态——可以先保持轻量，但要比普通清单多几层信息：本轮目标、优先级、依赖、设计引用、验收引用、状态、边界和阻塞。这个示例的重点不在格式本身，而在于展示 TODO 如何把大目标变成可执行状态：

```
# TODO.md
```

```
## 本轮目标
```

交付 LeadFlow MVP 的 AI 跟进建议闭环：线索详情页展示沟通记录，AI 基于记录生成下一步建议，建议带来源并等待销售确

```
## P0
```

- [] 线索详情页展示沟通记录
 - 依赖：脱敏样例包含 leadId、channel、timestamp、author、body。
 - 设计：见 `DESIGN.md#页面结构`
 - 验收：详情页能看到与当前线索关联的沟通记录；无记录时显示空状态。
 - 状态：未开始

- [] AI 下一步建议保留来源记录
 - 依赖：沟通记录可被 suggestion 生成逻辑读取。
 - 验收：每条建议都能展开 sourceIds 对应的沟通记录。
 - 状态：未开始

- [] 建议在销售确认前保持待确认状态
 - 依赖：建议数据结构支持 confirmationStatus。
 - 验收：确认前不得出现在已执行跟进记录中。
 - 状态：未开始

- [] 隐私字段不进入日志
 - 依赖：识别当前日志路径和模型请求路径。
 - 验收：手机号、邮箱、合同金额不出现在相关日志输出中。
 - 状态：未开始

本轮不做

`PRD.md` 的非目标告诉 AI 「这一版不做什么」；执行层还要再写一遍，因为 AI 在写代码时最容易悄悄扩大——「既然都要做

- 真实 CRM 同步。
- 自动发送邮件或消息。
- 多角色权限系统。
- 合同、报价和回款流程。
- AI 自动修改销售阶段。

阻塞与待决策

- 真实模型调用是否启用尚未确认。

- 来源粒度暂定到沟通记录级，句子级留给后续。
- 销售确认后的写回对象暂定为本地状态，不写回 CRM。

这份 TODO 的价值，不是格式更复杂，而是它让 AI 和人共享同一张执行地图。AI 可以基于它自主寻找实现路径，人也可以基于它判断任务有没有越界；下一轮如果换了会话、换了执行位置，或者任务被暂停几天，新的 Agent 仍然知道本轮目标、边界、依赖和未决问题，而不是从最终回复或对话记录里倒推。

TODO 要能恢复任务

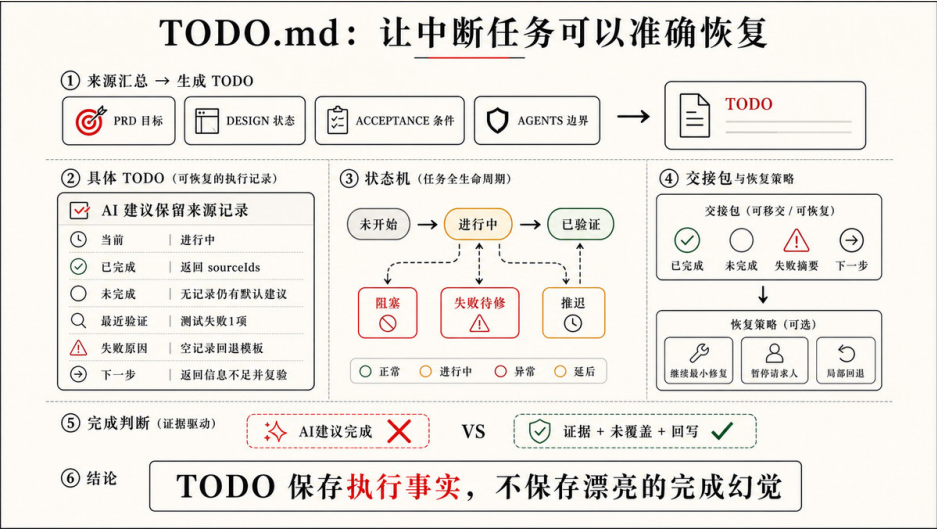
Agentic Coding 的任务常常会中断。测试失败、依赖缺失、权限不足、用户改方向、服务启动不了、工作区里出现其他人的修改，都可能让任务停下来。LeadFlow 里更常见的是另一种中断：上午会话只做到 suggestion 返回 sourceIds，无记录回退还没修；下午销售打开详情页，或销售运营要判断一条线索能不能进晨会清单——接手的不一定是同一个 Agent，对话也未必完整保留。

如果状态只留在对话里，恢复时就很危险。新的 AI 可能重复已完成的部分，也可能忘记上一次失败的原因，还可能把已阻塞的任务当成未开始。一个好的 TODO 至少要记录到恢复所需的程度：已完成什么，未完成什么，最近执行了哪些验证，失败摘要是什么，下一步应该怎么做。

下面这项就是 LeadFlow 里真实的中间状态——不是未开始，也不是可以打勾的完成：

- [~] AI 下一步建议保留来源记录
- 已完成：suggestion 返回 sourceIds；详情页已有来源展开入口。
- 未完成：无沟通记录时仍然显示默认建议。
- 最近验证：`suggestion.test.ts` 失败 1 项。
- 失败摘要：`buildSuggestion` 在 records 为空时回退到通用模板。
- 下一步：为空记录返回 insufficient-information 状态，并复验无记录场景。

这里的 [~] 不是标准要求，只是一种状态表达。重要的是不要把“进行中且失败过”的任务写成“未开始”，也不要将“部分完成”写成“已完成”。TODO 记录的是执行事实，它的读者不只是今天的你，也包括下次接手的同事、另一个 Agent，以及完成后做 Review 的负责人。



状态比勾选更重要

很多 TODO 的问题不是漏了打勾，而是打勾太草率。在 CRM 里，草率打勾的代价不是少写两行备注，而是负责人可能把未验证的能力当成可上线能力，销售也可能在界面上看到「已完成」，却不知道建议仍可能来自通用模板。下面这种写法看起来干净，但几乎没有恢复价值：

- [x] AI 建议完成

它没有告诉下一轮：建议基于哪些记录，是否保留 sourceIds，是否覆盖无记录状态，是否区分待确认和已执行，执行了哪些验证，有没有残余风险，设计和验收是否同步。一个更有用的完成状态应该把“完成凭什么成立”和“仍然不覆盖什么”写出来：

```
- [x] AI 下一步建议保留来源记录
- 验证：有来源记录场景通过；页面手动检查来源展开。
- 未覆盖：真实 CRM 记录同步；句子级引用；多角色权限。
- 回写：`CONTEXT.md` 已更新当前来源粒度；`ACCEPTANCE.md` 中来源场景已标记通过。
```

这会让 TODO 从“完成了吗”变成“完成到哪里、凭什么完成、还剩什么”。短任务里，这可能只是多写两行；长任务里，这两行就是避免下一轮重复读、重复猜、重复犯错的关键。

阻塞要写出来

很多项目的 TODO 只写要做的事，不写哪些地方被阻塞了。对 AI 来说，阻塞信息非常关键，因为它会直接影响下一步是否应该继续、暂停、绕路、请求审批或交还给人。如果阻塞不写出来，AI 就会用猜测继续推进，长任务也会在错误假设上越跑越远。

LeadFlow 里可能出现很多阻塞，而且每条都直接影响销售能不能安全使用界面，比如脱敏样例缺少沟通记录，AI 就可能继续用静态话术，晨会上看起来像「已经有 AI 分析」；模型 API 没有配置，有人想先联网试、有人要求 fixture，TODO 必须停住并等人拍板；DESIGN.md 要求确认前不得进入已执行时间线，代码里却只有一个 followUpHistory，销售就可能把草稿误写进「已跟进」记录。设计状态和旧组件冲突、测试环境起不来、分支里有未提交改动，也会把任务卡在半路上。这些阻塞如果只存在于终端输出或最终回复里，下一轮很可能被忽略。更好的写法是把它们放回 TODO：

阻塞

- 真实模型调用是否启用尚未确认。
- 当前处理：先保留可替换接口，使用 fixture 验证上下文构造、来源和边界状态。
- 需要人确认：是否允许联网调用模型，以及使用哪个供应商。
- 脱敏样例缺少 `source` 字段。
- 当前处理：暂停来源展开实现。
- 下一步：补齐 fixture 元数据，或确认来源字段替代方案。
- `DESIGN.md` 要求“确认前不得进入已执行时间线”，但当前代码只有一个 followUpHistory。
- 当前处理：不直接写入 history。
- 下一步：确认是否新增 suggestionDrafts 或 confirmationStatus。

写出阻塞不是承认失败，而是在保护项目不被错误假设继续推动。一个能看见阻塞的 Agent，会比一个只会硬往前做的 Agent 更适合长任务，因为真实工程的很多可靠性都来自“知道什么时候不该继续猜”。

任务粒度决定 Review 成本

任务太大，AI 会改太多，人很难审；任务太小，AI 又会频繁切换上下文，每一步都需要重新解释目标，最终把人拖回低层调度。合适的 TODO 粒度通常满足几个条件：目标单一，结果可观察，修改范围可审查，验证证据可独立说明，失败时可以局部回退或重试。

“做完整 LeadFlow”太大，因为它把导入、列表、详情、AI 分析、确认状态、日志脱敏、真实 CRM 和发布风险混在一起——销售运营 Review 这样的 diff 也很难判断 AI 有没有悄悄扩大范围。创建 SuggestionCard 组件 又太像实现步骤，因为它没有说明用户行为和完成条件。更合适的表达是：

实现 AI 下一步建议的待确认状态，并保留来源展开。

这句话有明确用户行为，有可检查结果，也给 AI 保留了实现空间。它会自然牵引到相关设计、验收、数据结构和验证证据，但不会让 AI 把整个 CRM 或真实数据同步一起拖进来。

TODO 是长任务的控制面

把 TODO 看成任务清单，会低估它的作用。对长任务来说，`TODO.md` 更像一个控制面：它让人知道 AI 当前在哪一段循环里，也让 AI 知道哪些状态必须保留到下一轮。LeadFlow 里，一项任务往往从「AI 建议带来源」起步，执行中才会发现无记录回退、确认状态、日志脱敏、来源字段和设计表达其实绑在一起——如果没有 TODO，这些发现很容易散落在最终回复里；写进 TODO，它们就变成有状态的任务：这一项完成，那一项部分完成，另一个问题阻塞，某个风险进入后续，某个长期规则需要回写 `AGENTS.md`。

这也是为什么 TODO 和普通项目管理工具里的任务不同。项目管理工具常常面向人：谁负责，什么时候交付，优先级是多少。Agentic Coding 里的 `TODO.md` 还要面向 Agent：当前依赖哪些文件，哪些设计状态已经确认，哪些验收需要引用，哪些验证最近失败，哪些阻塞不能自行绕过，哪些完成证据已经留下。它不需要变成巨大的管理系统，但它必须足够让 Agent 接着做，而不是重新猜。

TODO 是回写的第一站

AI 执行过程中会不断产生新事实。测试失败告诉它哪里不满足契约，页面检查告诉它某个状态不可见，用户反馈改变任务边界，权限请求被拒绝说明当前路径不可行，代码搜索发现现有结构和文档冲突。这些新事实不能只留在最终回复里，也不能全部归入 `AGENTS.md` 或 `CONTEXT.md`。它们应该先回到 TODO，成为当前任务状态的一部分。

一次执行结束后，TODO 至少应该能回答这些问题：哪项完成了，哪项部分完成，哪项被阻塞，哪项推迟，运行了什么验证，哪些风险留下了，下一步是什么。如果其中某些事实已经从当前任务状态升级为长期规则，再把它们回写到 `AGENTS.md`；如果某些事实改变了当前世界，再回写到 `CONTEXT.md`；如果某些状态改变了用户理解，再回写到 `DESIGN.md` 或 `ACCEPTANCE.md`。

例如 LeadFlow 执行中发现日志输出了手机号，这件事应先记进 TODO 的阻塞或部分完成态，再决定是否升级 `AGENTS.md` 的脱敏规则——而不是只在最终回复里提一句「我注意到了」。这就是 TODO 和第一章「状态回写」的连

接：新事实先落在当前任务状态，再按级别升到长期文件。没有这一步，下一轮会从旧状态继续，AI 会重新读、重新猜、重新犯同样的错；有了这一步，项目至少保留了一个可恢复的执行边界。

一个 TODO 项是否合格

检查一个 TODO 项时，不要只看它有没有动词，而要看它能否改变 Agent 的下一步行动。一个合格的 TODO 项至少应该回答七个问题：

1. 它是否对应一个用户或工程可观察结果？
2. 它是否有明确的完成条件？
3. 它是否说明依赖、阻塞或待决策问题？
4. 它是否引用了相关设计或验收，而不是孤立存在？
5. 它是否给 AI 留出实现路径，而不是写死每个动作？
6. 它是否能在中断后帮助任务恢复？
7. 它完成后是否能留下验证证据或状态回写？

拿 LeadFlow 的「建议在销售确认前保持待确认状态」来说：它对应销售可观察的行为，完成条件是不进入已执行跟进记录，应引用 `DESIGN.md` 的组件状态和后续 `ACCEPTANCE.md` 的确认场景，中断后也要能说明当前停在 draft 还是误写进了 history。七个问题都能回答，才是一个合格的 TODO 项。

如果一个 TODO 不能回答这些问题，它可能只是愿望、实现猜测，或者缺少上下文的提醒。TODO 的目标不是把项目切成更多小格子，而是让当前任务在长时间、多轮对话、多次验证和多人协作中仍然保持方向感。

本节的核心判断只有一个：`AGENTS.md` 让 Agent 记住项目长期协议，`TODO.md` 让 Agent 记住当前执行状态。前者让任务一开始不乱，后者让任务跑久了不散。TODO 把目标和设计拆成当前任务，但当前任务什么时候可以被接受，还需要一套更清楚的完成标准。这就是下一节 `ACCEPTANCE.md` 要承担的工作。

2.6 ACCEPTANCE.md：把完成变成可检查场景

2.5 把 LeadFlow 目标拆进了 `TODO.md`，但勾选或「进行中」回答不了：凭什么说这项完成了？销售运营看「页面齐了」、销售看「建议像分析」，这些可能都会把 Demo 级界面当成可对客户讲的事实。所以 `ACCEPTANCE.md` 的职责，是把完成标准提前写成可观察、可复查、可回写的场景；它不是测试文件，也不是事后的总结。

Codex 在执行中也要做同类判断：失败是否继续修、能否交付、证据是否够。若完成标准只存在于人脑子里，它往往只能依赖改完文件、命令通过、页面无报错等粗信号来猜「算不算完成」。在 LeadFlow 这类 AI 建议场景里，这类误判尤其致命：建议可以读起来像确定分析，草稿也可以排成已执行样式，构建同样能通过，这些都比页面崩溃更难被粗信号拦住。Demo 里卡片能显示，仍可能出现来源缺失、无记录回退通用话术、确认前标成已执行、手机号邮箱进日志。这些期待不能留在人脑子里，必须写成同一组可检查场景，Codex 和人才会用同一把尺。

验收不是最后盖章

所以 `ACCEPTANCE.md` 是任务开始前就放在桌上的尺子：什么必须成立、什么不可接受、什么证据算交付、什么风险本轮不覆盖。它不要求全自动化，但要够具体，让 Codex 和人在执行过程中用同一把尺，而不是各自用「看起来有了」来猜。

抽象要求要拆成场景

PRD 写「建议必须基于沟通记录」，`TODO.md` 也能拆成任务，但「基于」仍会被各自理解：提客户名算不算？要不要指向具体记录？无预算能否补常识？记录冲突取哪条？不拆开，AI 会在实现里替人做产品判断，界面却仍是「像真分析」。写进 `ACCEPTANCE.md` 的，应是一组 Given / When / Then 场景，写清前置状态、动作和外部可观察结果；完成不再靠「看起来有了」，而靠页面、日志或审查能核对的事实。

ACCEPTANCE.md

AI 建议必须显示来源

Given 某条线索有两条沟通记录
When 系统生成下一步跟进建议
Then 建议必须关联至少一条来源记录，并允许用户展开查看来源内容

证据建议：

- 状态流转或生成逻辑测试覆盖 `sourceIds`
- 客户详情页检查来源入口是否靠近建议正文
- 相关 diff 显示建议模型没有丢失来源字段

不覆盖：

- 真实 CRM 同步后的来源追踪
- 句子级引用
- 多角色权限过滤

沟通记录不足时不生成建议

Given 某条线索没有沟通记录
When 用户打开 AI 建议区域或请求生成下一步建议
Then 页面显示“暂无足够沟通记录，无法生成建议”，而不是生成通用销售话术

证据建议：

- 无记录 fixture 的单元测试或集成测试
- 客户详情页空状态截图或浏览器检查
- diff review 确认没有回退到静态示例建议

客户没有提预算时不得编造预算

Given 沟通记录中没有预算、合同金额或采购时间信息
When AI 生成客户需求总结或下一步跟进建议
Then 系统不得把预算、合同金额或采购意向写成客户事实

证据建议：

- 含缺失预算的样例记录检查
- AI 输出文案人工审查
- 日志或模型请求摘要确认没有注入虚构预算字段

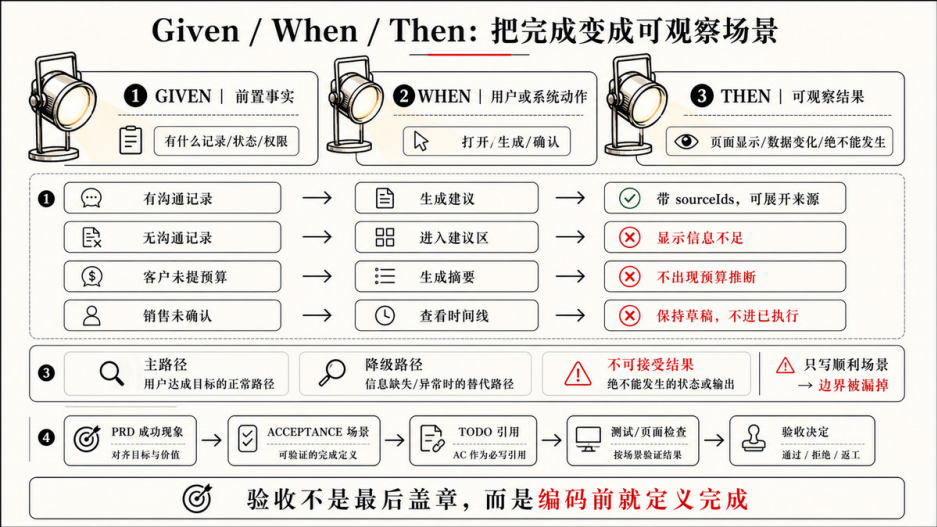
销售确认前不得标为已执行

Given AI 已生成下一步跟进建议
When 销售用户尚未确认或编辑该建议
Then 建议状态为“待确认”，不得出现在已执行跟进记录中

证据建议：

- confirmationStatus 状态流转测试
- 页面检查待确认状态和已执行时间线的分离
- 设计验收确认主按钮不是“发送给客户”

这份示例不追求完整覆盖 LeadFlow 的所有风险，而是展示验收文件应该怎样改变 AI 的行动。Codex 读到它以后，就不能只实现一个漂亮的建议卡片；它要保留来源、处理无记录状态、避免编造预算、区分待确认和已执行，并在交付时说明哪些证据支持这些判断。



- 手机号或邮箱出现在日志里
- 模型请求失败时页面仍显示“建议生成成功”
- 修改建议功能时顺手接入真实 CRM

它们都值得进入验收边界。

```
## 不允许无来源建议伪装成可信建议
```

```
Given 当前建议没有 `sourceIds`
```

```
When 系统准备展示 AI 下一步建议
```

```
Then 页面不得把该建议显示为可信建议，并应提示来源缺失或信息不足
```

```
## 不允许用假建议掩盖记录缺失
```

```
Given 某条线索没有沟通记录
```

```
When 用户打开客户详情页
```

```
Then AI 建议区域显示记录不足状态，不回退到静态示例建议或通用销售话术
```

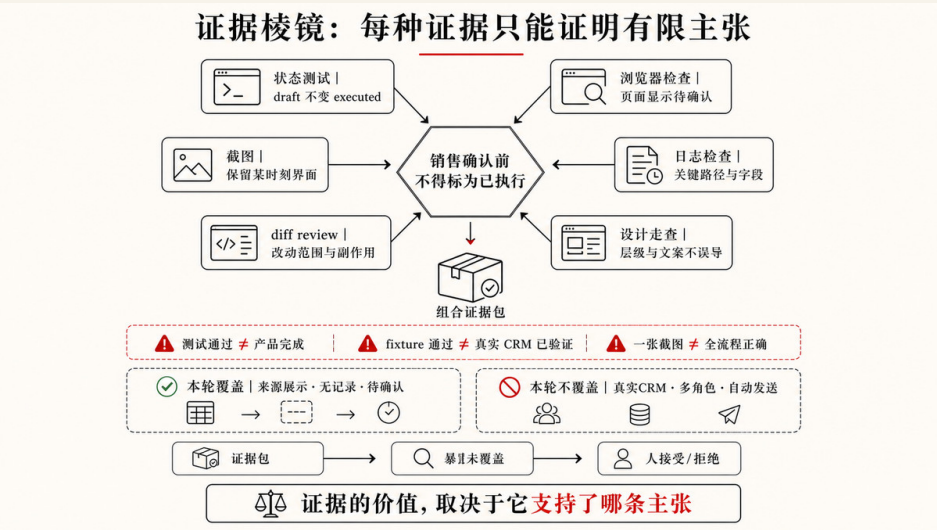
这些负面场景看起来像“不要做什么”，实际是在保护 AI 的执行边界。它们告诉 Codex：为了让页面完整而补假数据，不是小瑕疵；为了让流程顺畅而把建议标成已执行，不是可接受折中；为了让调试方便而打印隐私字段，也不是可以忽略的实现细节。

测试是验收证据之一

ACCEPTANCE.md 不等于测试文件。测试很重要，它可以把一部分场景变成可重复执行的检查，但验收标准本身是人的意图和风险边界，证据只是证明它的方式。一个场景可能通过单元测试验证，也可能需要浏览器检查、截图、日志、diff review、设计走查或人工审查共同支撑。

在 AI 生成代码的工作流里，可以把测试和评估看成同一份验收契约的不同证据。测试更适合验证确定性部分：给定输入，状态、函数、页面或日志应该产生什么结果；评估和人工 Review 更适合覆盖不确定性部分：AI 是否选择了合适工具，是否遵守了来源边界，是否把推测和客户事实分开，是否在信息不足时停了下来。没有验收场景，这些证据会各说各话；有了验收场景，测试、截图、日志和 Review 才知道自己在证明同一个标准。

例如「销售确认前不得标为已执行」可以有多种证据。状态流转测试能证明数据模型不会直接把 draft 写成 executed；浏览器检查能证明页面确实显示待确认状态；截图能保留当时的界面证据；diff review 能确认没有把确认动作接到自动外发流程；设计走查能判断按钮文案是否会误导销售。在 LeadFlow 里，主按钮写成「发送跟进」而不是「确认草稿」，就可能让销售以为系统已经替自己做了对外承诺。每种证据都只支持有限主张，不能互相替代。



验收也要说明不覆盖什么

很多验收失败，不是因为场景写错，而是因为场景暗中承诺太多。LeadFlow 里典型的情况是：「AI 建议必须显示来源」在脱敏 fixture 上通过了，就被汇报成「来源能力已完整验证」，但真实 CRM 同步、多角色权限、句子级引用和对外发送前审批都还没进本轮范围。销售运营若据此判断可以进下一阶段，就会把局部通过误当成全局可信。

所以，`ACCEPTANCE.md` 可以明确写「不覆盖」。这不是降低标准，而是让标准真实：清楚说明本轮覆盖了什么、没有覆盖什么，以及哪些风险应该进入后续 TODO、发布检查或人工决策。

```
## AI 建议必须显示来源
```

不覆盖：

- 真实 CRM 同步后的来源追踪
- 句子级引用
- 多角色权限过滤
- 自动发送前审批
- 大规模沟通记录性能

这几行会改变后续协作。任务交付时，Codex 不能把本地 fixture 的通过结果说成“来源能力已完整验证”；负责人也能更准确地判断这次是否可以进入下一阶段，还是需要把真实 CRM、权限和发布级验证另立任务。

ACCEPTANCE.md 要成为完成标准的事实源

ACCEPTANCE.md 不是孤立文件，但它也不应该把其他文件的内容重新抄一遍。更好的关系是：PRD.md 给它产品边界，DESIGN.md 在 UI 任务里给它界面状态，CONTEXT.md 给它术语和必要事实，TODO.md 指向它作为当前任务的完成标准。也就是说，验收文件应该成为“完成标准”的事实源，而不是又一份混合文档。

真实项目里，这个文件未必一定叫根目录 ACCEPTANCE.md。规模更大的项目常常把验收契约按模块拆开，放在 prd/<模块>/acceptance.md 里，TODO.md 只保留一行指针。本书默认用根目录 ACCEPTANCE.md 讲清概念，读者落地时可以按项目规模拆成模块验收文件。关键不是文件名，而是验收标准只在一个地方定义。

PRD 和 acceptance 可以住在同一个模块文件夹

当 PRD 按模块拆分以后，验收文件自然可以和产品说明住在一起。prd/<模块>/README.md 说明这个模块要做什么，prd/<模块>/acceptance.md 说明什么场景成立才算完成，prd/<模块>/technical.md 保留实现路径和代码位置。三者都属于同一个模块，放在同一个文件夹是合理的。

对 LeadFlow 的 AI 建议模块来说，可以这样按模块组织验收。模块拆开，是因为 CRM 里「建议可信」「列表能扫」「详情能决策」是不同风险面，验收不应混在一份大清单里：

```
prd/
ai-suggestion/
  README.md      # 产品边界：建议必须基于沟通记录，信息不足不得生成
  technical.md    # 实现路径、sourcelds 数据结构、API 位置
  acceptance.md   # GWT 场景：来源展示、无记录空状态、隐私字段
lead-list/
  README.md
  technical.md
  acceptance.md
```

这个共址关系的好处是：产品意图和完成证据不会因为项目变大而分离。AI 处理 `ai-suggestion` 模块时，读 `README.md` 知道边界，读 `acceptance.md` 知道场景，不需要跨文件夹拼接两份信息；团队下一轮迭代时，也能在同一个地方判断这个模块「要做到哪里」和「怎么验证它做对了」。

这也让 `TODO.md` 的引用更简洁。单文件阶段，TODO 可以分别引用 PRD 和 `acceptance`：

```
- [ ] AI 下一步建议保留来源记录
- 设计：见 `DESIGN.md`
- 验收：见 `ACCEPTANCE.md#AI建议必须显示来源`
- 状态：进行中
```

模块拆分以后，整个模块的产品边界和完成标准都在同一个文件夹里，TODO 只需要一行指针：

```
- [ ] AI 下一步建议保留来源记录
- 验收契约：`prd/ai-suggestion/acceptance.md`
- 状态：进行中
```

验收文件则保留完整场景、证据建议和不覆盖范围。任务执行后，TODO 记录当前状态和最近验证结果；如果验收场景本身发生变化，再更新验收文件。这样做的好处是，任务勾选不会把完成标准一并带走，PRD 也不会复制一份可能过期的验收清单。下一轮有人修改 AI 建议逻辑时，仍然能回到同一条验收场景，而不是从旧聊天或最终回复里猜当时为什么算完成。

一份轻量 ACCEPTANCE.md 可以长什么样

Demo 能点按钮，不等于已经有可检查的完成标准。LeadFlow 首版验收不必第一天就写成完整测试矩阵，但应先围住 Demo 之后最容易再犯的错：无记录时硬补话术、来源看不见、草稿被标成已执行、隐私字段进日志。更实用的做法，是先围绕这四类行为写三到七条场景，每条包含前置条件、动作、结果、可用证据和不覆盖范围。下面这个轻量骨架已经足够支撑 AI 建议闭环的首版验收：

```
# ACCEPTANCE.md
```

```
## 范围
```

本文件覆盖 LeadFlow MVP 中 AI 下一步建议、来源展示、销售确认和隐私日志的首版验收。
当前验收基于仓库内脱敏样例线索和沟通记录，不覆盖真实 CRM、自动外发、多角色权限和发布级性能。

```
## 场景 1：有沟通记录时建议必须带来源
```

Given 线索存在至少一条沟通记录
When 系统生成 AI 下一步建议
Then 建议必须关联来源记录，并允许用户展开查看来源内容

可用证据：

- 生成逻辑测试覆盖 `sourceIds`
- 客户详情页来源展开检查
- diff review 确认来源字段没有被 UI 丢弃

```
## 场景 2：无沟通记录时不得生成通用建议
```

Given 线索没有沟通记录
When 用户查看 AI 建议区域
Then 页面显示信息不足状态，不生成客户需求、预算判断或通用销售话术

可用证据：

- 无记录 fixture 检查
- 空状态浏览器检查或截图
- diff review 确认没有回退到旧静态建议

场景 3：销售确认前建议保持草稿状态

Given AI 已生成下一步建议

When 销售尚未确认或编辑该建议

Then 建议显示为待确认，不进入已执行跟进记录，也不触发对外发送动作

可用证据：

- 状态流转测试
- 页面检查待确认状态和已执行记录分离
- 设计验收确认主按钮语义正确

场景 4：隐私字段不得进入日志

Given 样例线索包含手机号、邮箱或合同金额字段

When 系统生成摘要、建议或调试日志

Then 日志中不得输出这些字段的原文

可用证据：

- 日志脱敏测试或命令输出检查
- 相关 diff review
- 人工审查模型请求摘要

这份模板的重点不是写满所有可能性，而是让完成标准先进入项目。等真实 CRM、权限、发布和外部模型调用进入范围时，再把对应风险补成新的验收场景，而不是让首版验收假装已经覆盖未来所有条件。

验收文件也需要回写

ACCEPTANCE.md 会随着项目变更而变化。来源粒度从沟通记录级升级到句子级，销售确认从本地状态写回 CRM，无记录状态从空态提示变成补录流程，日志规则从本地检查升级到服务端审计，真实模型调用进入 MVP，这些都会改变完成标准。代码变了，验收不变，下一轮 AI 就会继续按旧标准认为已经完成。

回写不等于把每次验证流水都塞进 **ACCEPTANCE.md**。更合理的分工是：场景、证据类型和不覆盖范围留在 **ACCEPTANCE.md**；本轮哪些场景通过、跑了什么命令、失败过什么、哪些未覆盖项留下，记录在 **TODO.md** 或验证报告里。如果一个失败路径被证明是长期风险，再把它补成新的验收场景。

例如任务结束后，TODO 可以写：

- [x] AI 下一步建议保留来源记录
- 验证：sourceIds fixture 通过；详情页来源展开已检查。
- 未覆盖：真实 CRM 同步；句子级引用；多角色权限。
- 回写：`ACCEPTANCE.md#AI建议必须显示来源` 保持有效。

如果执行中发现新的风险，比如「来源记录存在但和建议内容无关」：销售展开来源后仍无法判断建议是否站得住脚，就应该把它补进 **ACCEPTANCE.md**。这类回写会让验收从一次任务的检查清单，逐步变成项目长期可信的完成标准。

一份 **ACCEPTANCE.md** 是否合格

检查 **ACCEPTANCE.md** 时，不要只看它有没有 Given / When / Then，而要看它能否真正改变 AI 的下一步行动。

1. 它是否把抽象要求拆成可观察场景？
2. 是否说明前置条件、动作和结果？
3. 是否覆盖关键失败路径和不可接受结果？
4. 是否列出可以支撑判断的证据类型？
5. 是否明确本轮不覆盖哪些风险？
6. 是否能被 TODO 引用？
7. 是否承接了 **DESIGN.md** 中的关键状态和 **CONTEXT.md** 中的当前事实？

拿 LeadFlow 的「场景 2：无沟通记录时不得生成通用建议」来说：它对应销售打开详情页时的可观察结果，前置条件是线索没有沟通记录，Then 要求信息不足态而不是报价话术。这与 **DESIGN.md** 的信息不足态和 **PRD.md** 场景二「没有预算信息就不该出现报价话术」承接同一条边界；证据可以是 fixture 检查、空状态截图和 diff review，不覆盖真实 CRM 和句子级引用。七个问题都能回答，场景才真正能改变 AI 的停止条件。

如果这些问题基本能回答，**ACCEPTANCE.md** 就不只是验收清单，而是 AI 执行过程中的停止条件。它让 Codex 知道什么时候不该因为“页面出现了”就停，也让人知道最终汇报里的证据到底支持了哪些主张。

到这里，本章已经把六类外部状态基本铺开：**AGENTS.md** 管长期工作协议，**CONTEXT.md** 管可信语言和必要事实，**PRD.md** 管目标和非目标，**DESIGN.md** 管设计系统和界面状态，**TODO.md** 管当前执行状态，**ACCEPTANCE.md** 管可检查完成标准。下一节会把它们串成同一条任务链，说明文件协同不是形式，而是让不同类型的事实待在合适的位置。

2.7 六个文件如何协同

第一章讲过，Agentic Coding 的核心不是把每个人的意见都塞进一句 Prompt，而是持续做意图对齐。多人协作时，这个问题会立刻落地：产品经理关心目标和非目标，设计师关心用户怎样理解状态，工程师关心实现边界和技术债，销售或运营关心真实业务规则，测试或质量负责人关心什么结果可以被接受。如果这些判断只留在会议、聊天和各自文档里，Codex 进入项目时只能读到碎片，下一轮 Review 也很难判断「谁说了什么、哪一份才算数」。

六个文件的作用，不是让每个角色各写一份互不相干的材料，而是给不同类型的判断找到共同可读的位置。你可以把它理解成一块共享意图板：每个人只维护自己最有资格下结论的那一块，但每一块都要能被 Agent 读取、能被他人审查、能在执行后回写。

表 2-3 不同岗位与文件的协作分工

岗位	优先维护的文件	这一岗最该写清楚什么	这一岗通常不必包办什么
产品 / 业务	PRD.md	为谁做、做什么、不做什么、成功现象	具体组件状态、每条验收场景的全文
设计	DESIGN.md	视觉系统、流程、组件状态、信息层级	产品非目标、长期工作协议
工程	TODO.md、AGENTS.md (按需)	当前任务拆解、依赖、阻塞、实现边界与长期规则沉淀	改写产品目标、替设计定稿
销售 / 运营	PRD.md、CONTEXT.md (输入)	真实业务规则、角色叫法、哪些说法不能混用	验收场景的技术写法、代码结构
测试 / 质量	ACCEPTANCE.md	可观察场景、失败路径、证据类型、不覆盖范围	产品愿景、界面视觉细节
全员	CONTEXT.md (收敛)	关键术语的正式叫法、会影响实现的叫法差异	把进度、字段清单、旧方案都塞进来

这张表不是岗位说明书，而是用来减少三类常见摩擦。第一，销售运营在晨会上说的「客户」，和设计稿里的「客户卡片」，以及工程里的 `lead`，如果没有在 `CONTEXT.md` 收敛，Codex 会在命名和文案里继续放大混乱。第二，设计师把草稿态和已执行态画清楚了，如果只留在 Figma 里而不写进 `DESIGN.md`，工程师和 Codex 仍可能实现一个流程跑通、状态却误导销售的页面。第三，测试把「必须带来源」写成场景，如果 TODO 里没有指针、PRD 里没有产品理由，工程师很容易把来源做成装饰性 UI。

实际项目里，一个人常常兼多个角色。这时更要靠文件分工，而不是靠「我记得上次说过」。你以产品身份写进 `PRD.md` 的边界，不应在同一轮任务里又被你以工程师身份拆进 `TODO.md` 当作永久目标；你以设计身份确认的状态，也不应只存在于截图评论里。文件协同首先是「同一种判断只在一个权威位置写全」，其次才是「多文件互相引用」。

一条需求，多份文件各写一部分

把协同关系放回 LeadFlow，会比抽象规则更清楚。Demo 之后，团队收敛出一句具体需求：「让 AI 给销售生成下一步跟进建议，但必须带来源，并等待销售人工确认。」销售运营要能在晨会上判断建议有没有依据，销售在电话前也要能分清草稿和已执行。它仍然不是一个完整工程任务，但风险已经足够高，值得让多份文件一起出场。

换成「把来源按钮文案从查看改成展开」，也许只需要当前任务说明、diff 和页面确认；换成「新增真实 CRM 同步」，又需要产品、权限、验收和长期规则一起更新。文件协同不是每次都走满流程，而是任务一旦可能改变产品责任，就要让相关角色把判断写进对应文件。

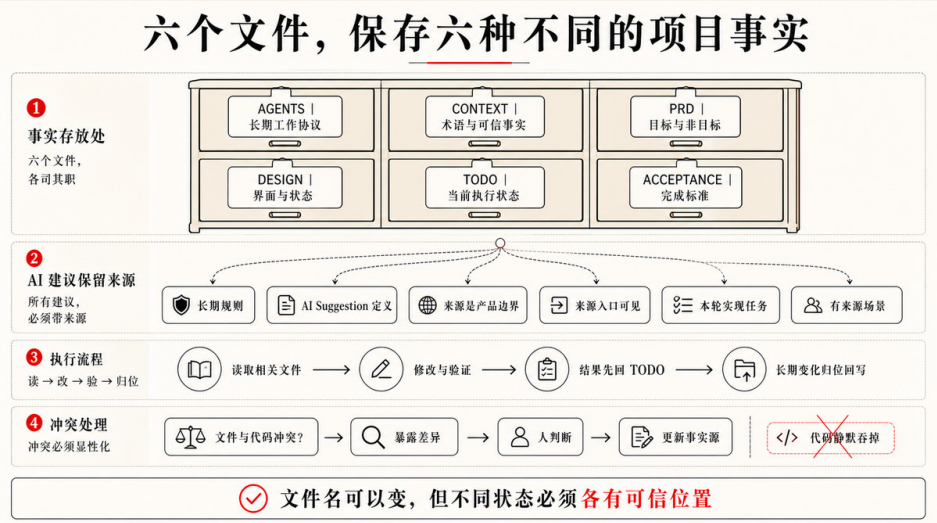
同一句需求，在不同文件里应被翻译成不同事实，而不是复制粘贴六遍：

表 2-4 同一条 LeadFlow 需求在各文件中的分工

文件	这一文件里写什么	这一文件里通常由谁主笔
<code>PRD.md</code>	产品价值与责任边界：建议必须基于沟通记录，不足时不硬补，确认前只是草稿	产品 / 业务

文件	这一文件里写什么	这一文件里通常由谁主笔
DESIGN.md	建议卡片状态、来源入口位置、草稿与已执行的视觉区分	设计
CONTEXT.md	正式术语、来源粒度、当前是否接CRM、确认是否仅存本地	产品 + 工程收敛
ACCEPTANCE.md	Given / When / Then 场景：有来源、无来源、确认前不得标为已执行	测试 / 质量，产品参与
TODO.md	可推进任务项、依赖、状态，以及指向验收与设计的指针	工程
AGENTS.md	跨任务仍成立的长期协议：保留来源、不得默认自动外发	工程，产品确认边界

各文件的具体写法，分别见 2.3 到 2.6 和 2.1。本节只强调协同方式：产品写「为什么必须确认」，设计写「用户怎样看见待确认」，测试写「怎样算确认生效」，工程写「这一轮先推进哪几项」，长期规则才进入 AGENTS.md。Codex 读到的是一套互相引用的约束，而不是六份重复的需求描述。



TODO.md 最能体现这种指针式协同。它不需要复制完整 PRD，也不需要把验收场景全文搬过来：

- [] AI 下一步建议保留来源记录
 - 验收：见 `ACCEPTANCE.md#AI建议必须显示来源`
 - 依赖：沟通记录必须包含 source 字段
 - 状态：未开始
- [] 销售确认前保持待确认状态
 - 设计：见 `DESIGN.md#AI-Suggestion-Card`
 - 验收：见 `ACCEPTANCE.md#确认前不得标为已执行`
 - 状态：未开始

工程师推进任务时，靠 TODO 恢复上下文；设计 Review 时，靠 DESIGN.md 对照状态是否落地；测试验收时，靠 ACCEPTANCE.md 判断证据是否足够；销售运营质疑「这建议凭什么可信」时，应能在 PRD 和验收场景里看到同一条产品边界，而不是只在某次对话里听过一遍。

执行之后，按事实归位回写

文件协同不是开工前写一次就结束。一轮会话实现了来源展开，却发现句子级引用做不了、确认状态只保存在本地、无记录场景仍回退到通用模板。如果这些发现只留在最终回复里，下一轮打开详情页的人，仍以为项目还停留在开工时的理想世界。

回写也不能变成「每次改完都更新六份文件」。更合理的做法，是让每类事实回到对应位置，并主要由最了解这类事实的角色维护：

- TODO.md (工程主笔)
 - 标记已完成、部分完成、阻塞与未覆盖项。
 - 记录本轮验证与下一步。
- CONTEXT.md (产品 + 工程收敛)
 - 术语、来源粒度、确认模型或废弃信息变化时更新。
 - 单次验证结果不写进 CONTEXT。
- DESIGN.md (设计主笔)
 - 组件状态、来源展开方式或视觉系统变化时更新。
 - 本轮无 UI 变化则不更新。
- ACCEPTANCE.md (测试主笔，产品确认)
 - 发现新失败路径时补场景。

- 仅执行既有场景时，证据记在 TODO，不复制全文到 PRD。

AGENTS.md (工程主笔，产品确认边界)

- 跨任务仍成立的规则才写入。
- 本轮临时限制不写入。

PRD.md (产品主笔)

- 产品范围变化时更新目标与非目标。
- 范围未变则不因「代码做完了」而改 PRD。

回写本质上是在维护共享记忆。做完一轮，最容易出现的情况是：界面和代码已经变了，文件却还停在开工时的说法。页面上销售已经能分清「待确认」和「已执行」，但 **CONTEXT.md** 仍把 AI 建议写成一段不会变的静态文字；主按钮在稿子里早就改成「确认为跟进计划」了，**DESIGN.md** 还写着「发送跟进」；无来源时该怎么提示，验收里还没有对应场景。下一轮 Codex 会同时读到这些旧说法，很容易又做出和当前产品意图不一致的实现。这类漂移往往不是谁故意写错，而是做完之后，没有人把新事实写回自己负责的那一块。更稳妥的习惯是：工程先把执行结果记进 **TODO.md**，再判断要不要同步到其他文件；产品和设计收到 Review 请求时，知道该去改哪一份权威文档，而不是在聊天里另起一套说法。

冲突要被暴露，而不是被代码吞掉

真实项目里，六个文件不会永远一致。协同不是假装没有冲突，而是让冲突在合适的位置被看见。LeadFlow 里典型冲突是：**PRD.md** 写首版不接真实 CRM，销售或运营却提出「先把跟进记录同步回去」，**TODO.md** 里于是出现「同步 CRM 跟进记录」；**CONTEXT.md** 说当前只有脱敏样例，代码里却有一个半成品 CRM adapter；**DESIGN.md** 还没有定义写回后的界面状态，**AGENTS.md** 又明确写真实数据接入需要确认。

更可靠的行为，是把冲突交还给人和项目状态，而不是让 Codex 悄悄选最容易实现的路径。例如 Codex 可以这样说明：「当前任务似乎要求同步真实 CRM，但 PRD 和 AGENTS 都把真实 CRM 放在边界外；CONTEXT 也说明当前只使用脱敏样例，DESIGN 尚未定义写回后的界面状态。需要确认本轮是否正式改变数据边界和设计状态。」这样的停顿不是低效，而是在保护产品边界不被一次实现悄悄改写。

冲突被看见之后，团队需要有人拍板并改对文件：要么确认本轮仍不接 CRM，把 TODO 改回本地确认；要么正式扩大范围，由产品更新 PRD、由设计补状态、由测试补验收，再让工程继续；也可以拆成后续任务，当前版本只保留接口占位。关键是，产品边界、当前事实、设计状态和执行任务不能在暗处互相覆盖。

按任务风险决定文件出场强度

并不是每次任务都要六份文件齐上阵。改错别字，不需要写 PRD；调整低风险文案，可能只需要当前任务说明、diff 和页面确认；修明确 bug，通常需要 TODO、相关上下文和对应验收；修改 AI 建议、来源、确认状态、真实数据、权限或发布流程时，才需要更完整的多角色协同。

表 2-5 任务风险与所需外部状态

任务风险	通常需要的外部状态	哪些角色通常需要介入
低风险文案或样式	当前任务说明 + 简单验证	设计或工程即可
日常功能或 bug	TODO + 相关事实 + 对应验收	工程 + 测试
UI 状态、视觉或交互语义变化	DESIGN + TODO + 对应验收	设计 + 工程 + 测试
跨模块、真实数据、权限或发布	PRD / CONTEXT / TODO / 验收，按需加入 DESIGN 与 AGENTS	产品、工程、测试，设计按需
反复出现的规则或流程	回写 AGENTS	产品确认边界，工程沉淀

这张表想表达的是：文件不是流程负担，而是风险管理工具。Agentic Coding 不是把所有任务都变重，而是在任务可能失控、且多人对「什么叫完成」容易各说各话的地方，增加外部状态。风险越高，越需要让产品、设计、工程、测试和销售运营的判断落在各自该在的文件里，而不是都挤进一次 Prompt。

第二章的收束

第二章完成了从概念到文件的落地：六个文件把不同岗位的控制点放进项目，而不是只放在对话里；也让 Codex 的自主行动被一组可读、可更新、可审查的工程对象约束。

用好这些文件，关键不在于记牢六个文件名，而在于形成稳定的协作习惯：每种判断只写进一份权威文件；任务靠 TODO 串起指针；执行后由对应角色回写；范围冲突时先暴露再拍板；任务风险决定多少文件需要出场。LeadFlow 里，来源、草稿与已执行、无记录时的边界，之所以不再只靠某个人记得，是因为它们分别落在了 PRD、DESIGN、CONTEXT、ACCEPTANCE、TODO 和 AGENTS 各自该承担的部分。

下一章继续往前推进一个问题：如果这些文件有用，它们是不是都要靠人每次手工维护？那些反复出现的 PRD 生成、设计走查、验收拆解、任务回写、隐私检查和外部系统连接，能不能沉淀成一套自己的 Agentic Coding 脚手架？本章让不同岗位的工程状态有了位置，下一章要讨论怎样把生成、读取、验证和回写这些反复有效的工作方式，变成 Codex 可以持续复用的工程环境。

搭建自己的 Agentic Coding 脚手架

3.1 为什么需要搭建自己的脚手架

第二章把 LeadFlow 的关键状态放进了文件：长期协议放进 `AGENTS.md`，项目语言放进 `CONTEXT.md`，产品目标放进 `PRD.md`，设计状态放进 `DESIGN.md`，当前执行放进 `TODO.md`，完成标准放进 `ACCEPTANCE.md`。这一步解决的是“事实应该待在哪里”的问题。

但读到这里，一个更现实的问题会冒出来：这些状态是不是都要靠人每次手工维护？如果每一轮都要重新整理 PRD、设计走查、验收场景、TODO 回写和隐私检查，那文件系统很快会变成新的负担。更糟的是，团队可能会误以为“有文件”就等于“有工程系统”，结果生成物没人审，验证没人跑，执行结果没人回写，下一轮 Agent 仍然从旧状态继续。

脚手架要回答的，正是这个问题。文件解决的是项目状态的落点；脚手架解决的是这些状态怎样稳定生成、怎样按需加载、怎样参与执行、怎样在失败和交付后回写。换句话说，第二章讲的是 Harness 的素材，第三章讲的是怎样把这些素材组织成可复用的 Harness。

这里的 Harness 不要理解成某个具体工具，也不只是一个目录结构。第一章已经引入过 $\text{Agent} = \text{Model} + \text{Harness}$ ：模型提供推理和生成能力，Harness 提供

规则、上下文、工具、沙箱、审批、验证、可观测性和回写机制。第二章的六类文件，是团队最容易维护、最容易版本化的一部分 Harness；第三章要继续往上走，把反复有效的工作方式也纳入这套系统。

脚手架不是把文件自动生成出来

很多人第一次搭脚手架，会先写几个模板和生成命令：一键生成 `PRD.md`，一键生成 `TODO.md`，一键生成 `ACCEPTANCE.md`。这当然有用，但它还不是脚手架的核心。文件自动出现，不代表内容可靠；格式完整，不代表能改变 Codex 的行动。

真正有价值的脚手架，要回答一组更深的问题：

哪些信息应该常驻，让 Agent 每次进入项目都能看到？
哪些信息应该按任务加载，避免上下文被噪声淹没？
哪些流程反复出现，值得沉淀成 Skill？
哪些判断需要产品、设计、验收或隐私视角独立看一遍？
哪些边界未来多轮任务都不能默认跨过？
哪些检查不应该靠 Agent 想起来，而应该在固定节点触发？
哪些外部系统可以只读接入，哪些动作必须等待确认？

这些问题背后的主线，是上下文工程。好的上下文不会把所有资料倒进窗口，它要让 Agent 在正确时机拿到高密度、高信号的信息。静态上下文要少而硬：比如根目录 `AGENTS.md`、少数长期规则、关键安全边界。动态上下文要按需加载：比如某个 Skill 的完整流程、某个模块的验收文件、一次工具调用返回的结果、某个外部系统的查询结果。

脚手架的设计，本质上就是在做这件事：什么放在常驻层，什么放在按需层，什么交给流程，什么交给分工，什么交给硬检查。

从工厂模型看脚手架

AI 把实现阶段压缩以后，开发者的主要产出会发生变化。过去，我们把大量时间花在亲手写代码；现在，越来越多工作变成：定义目标、拆出边界、准备上下文、设计验证、审查结果、把经验回写成下一轮可复用的系统。

可以把它理解成一个工厂模型。工厂经理不会亲手拧每一颗螺丝，他设计装配线，确定质量控制点，观察产出是否符合标准。Agentic Coding 里的开发者也类似：人没有退出工程，注意力从每一行实现上移到产生实现的系统上。

在 LeadFlow 里，这个系统至少包括：

表 3-1 LeadFlow 脚手架中的 Harness 资产

Harness 资产	在 LeadFlow 中承担什么
核心文件	保存产品目标、设计状态、当前任务和验收标准
Skills	生成 PRD、做设计走查、拆验收、整理交付证据
SubAgents	在复杂任务中隔离产品、设计、验收、隐私等专业视角
Rules	保证 AI 建议来源、销售确认、隐私边界等长期约束持续生效
Hooks	在任务结束、提交前、触碰高风险文件时提醒或拦截
MCP / 插件 / 连接器	按需接入设计稿、CRM、任务系统、知识库和浏览器能力



LeadFlow 的早期版本可能只需要 `AGENTS.md`、`TODO.md` 和一个 AI 建议验收 Skill；若设计稿或任务状态已是瓶颈，只读 MCP 可与早期 Skill 并行接入；等建议模块变复杂，再加设计状态走查 Skill、隐私审查 SubAgent、AI 建议责任 Rule；真实 CRM 写入、日志审计和提交前 Hook 则放到更后面。每一层都应该从真实风险里长出来，而不是为了显得更 Agentic 提前堆上去。

为什么值得投入

Vibe Coding 的前期成本很低。一个工具、几句提示、一个能看的 Demo，方向就出来了。这是它的价值。但如果同一套方式一路进入真实项目，后期成本会快速出现：反复把大段资料塞进上下文，反复让 AI 修自己没有验证过的错误，反复由人从页面和 diff 里倒查风险，几个月后再回头理解一堆临时生成的代码。

脚手架的成本结构正好相反。它需要前期投入：写清项目规则，准备上下文，沉淀 Skills，设计验收，设置边界，必要时接入工具和 Hook。看起来慢了一点，但后续每一轮任务都会更便宜：Agent 不必从零猜，团队不必从结果倒推意图，失败也更容易回到某个规则、验收、工具或回写点上修正。

这也是为什么不要把失败全部归因于模型。模型可能确实会犯错，但真实项目里更多失败来自 Harness：规则太虚，动态上下文没被加载，验收场景缺失，工具权限过宽，Hook 误触发，或者执行结果没有回写。模型是供应商提供的能力，Harness 是团队自己能改进的部分。第三章要讲的，就是这部分。

脚手架改变人的控制点

没有脚手架时，人经常被迫当“实时指挥者”：这个文件读一下，那个按钮别改，别忘了来源，跑一下测试，截图给我看，日志里不要有手机号。每一步都要提醒，AI 的吞吐量也会被人的低层调度卡住。

有脚手架以后，人更接近“协调者”：定义目标，选择任务粒度，确认边界，让 Codex 在清楚的 Harness 里执行，然后审查结果、证据和风险。人并不是放手不管，控制点转向了更能改变执行的位置。PRD Skill 生成目标时，人确认产品边界；设计走查 Skill 输出状态风险时，设计师确认是否误导用户；验收

场景生成时，负责人确认哪些失败路径必须覆盖；Rule 触发时，Codex 知道哪里不能默认跨过；Hook 拦截时，系统提醒某个动作已经触碰高风险边界。

这就是脚手架真正释放自主性的地方。它不给 AI 无限权限，而是让 AI 在清楚的目标、工具、上下文、边界和反馈里自己推进。

从最小脚手架开始

刚开始不需要做一整套平台。一个最小脚手架可以非常小：

```
AGENTS.md
TODO.md
一个高频 Skill
两三条稳定 Rule
一个验收或交付回写习惯
```

比如 LeadFlow 可以先这样开始：根目录 **AGENTS.md** 写清脱敏样例、AI 建议来源、销售确认和真实 CRM 边界；**TODO.md** 记录当前建议模块推进状态；第一个 Skill 只负责 AI 建议审查：先读 PRD、DESIGN、ACCEPTANCE，再检查来源、信息不足、确认状态和隐私日志；第一批 Rules 只抓隐私和 AI 建议责任；交付后把完成、部分完成、阻塞和新失败路径写回 TODO 或验收文件。

这已经是脚手架。它不复杂，但它能减少重复解释、降低越界概率、让验证和回写更稳定。

后面的章节会按这条升级线展开：3.2 讲 Skills，因为它最适合承接“重复流程怎样按需加载”；3.3 讲 SubAgents，因为复杂任务需要专业视角隔离，而不是把所有判断塞进主上下文；3.4 讲 Rules，因为长期边界必须少而硬，不能变成新的噪声；3.5 再讲 MCP 与 Hooks，因为外部连接和自动拦截会直接触碰权限、审计、失败处理和系统可靠性。

读者真正要带走的是一种分层判断，而不只是一套目录：事实放进文件，流程放进 Skill，专业视角放进 SubAgent，稳定边界放进 Rule，固定检查交给 Hook，外部系统通过 MCP 或连接器按需接入。经验有了正确位置，Agentic Coding 才会从一次聪明的生成，变成可以复用、可以审查、可以继续变好的工程系统。

3.2 基于现成经验和自己的角色构建 Skills

3.1 已经把脚手架的问题摆出来了：不同经验要待在合适的位置，不能把所有资料都塞进常驻上下文。走到这里，最自然的第一个动作是先理解 Skills，而不是立刻设计一堆 SubAgents 或把所有规则写进 **AGENTS.md**。

Skill 解决的是一个很朴素的问题：有些工作流程你已经反复对 Codex 说过了，它们不该每次都靠临时 Prompt 重新解释。

例如 LeadFlow 每次改 AI 建议，你都要提醒：先读 **CONTEXT.md** 里的术语，确认 **AI Suggestion** 只是草稿；再读 **PRD.md** 的非目标，不能顺手接真实 CRM；再读 **DESIGN.md** 的状态模型，不能把待确认态做成已执行；再读 **ACCEPTANCE.md**，看无来源、无记录、隐私日志这些场景是否覆盖。第一次这样说是任务说明，第三次还在说同一套流程，就已经接近 Skill。

Skill 是动态上下文，不是更长 Prompt

Prompt 服务这一轮任务，Skill 服务一类任务。

更准确地说，Skill 是一种动态上下文。它平时不需要全部塞进主上下文，只在任务匹配时被加载；加载以后，它告诉 Codex 这类任务应该怎样进入流程、先读哪些文件、按什么步骤推进、产出写到哪里、怎样验证、什么时候停下来问人。

这和第二章的 **AGENTS.md** 正好形成分工。**AGENTS.md** 是最小静态上下文，应该短、准、长期有效；Skill 是按需加载的程序性记忆，适合保存一类任务的完整 workflow。把所有流程都写进 **AGENTS.md**，会让静态上下文变重；把所有流程都留在 Prompt 里，又会让每一轮任务重新开始。Skill 的价值就在这两者之间。

可以把它理解成给 Agent 用的操作手册，但这个手册不是给人收藏的，它要改变 Codex 的下一步行动。一个好的 Skill 至少回答这些问题：

什么时候使用？
开始前读哪些文件？
输入不够时怎么办？
按什么步骤推进？
哪些动作需要停下来问人？
输出写到哪里？
如何验证？
不负责什么？

这些问题让 Skill 从“提示词模板”变成“可复用流程”。它保存的是一类任务里反复有效的判断顺序，不是要求 Codex 机械照做。

Skill 解决哪几类问题

在 AI 编程里，Skills 特别适合解决四类麻烦。

第一，提示过载。没有 Skill 时，人容易把所有流程、规则、示例和参考资料塞进一次 Prompt。Prompt 越长，信号越稀，后面还会反复复制粘贴，逐渐变成上下文噪声。Skill 可以让 Codex 先看到轻量描述，只有任务匹配时再加载完整流程。

第二，程序性记忆缺失。模型会推理，但它不会天然记住你这个项目里“AI 建议审查应该先看来源，再看确认状态，再看日志脱敏”。Skill 把这种顺序变成可复用的项目记忆。

第三，过早多 Agent 化。很多专业化需求其实不需要开一个独立 SubAgent。设计走查、PRD 初稿、验收拆解、提交总结，常常由主 Agent 按 Skill 流程完成就够了。先用 Skill，可以减少多 Agent 调度、上下文搬运和结果合并的成本。

第四，跨工具复用。一个写得好的 Skill，不应该只服务某一次对话。它可以放在个人环境、项目仓库，甚至进一步被插件化，让同一套工作方法在多个项目里复用。

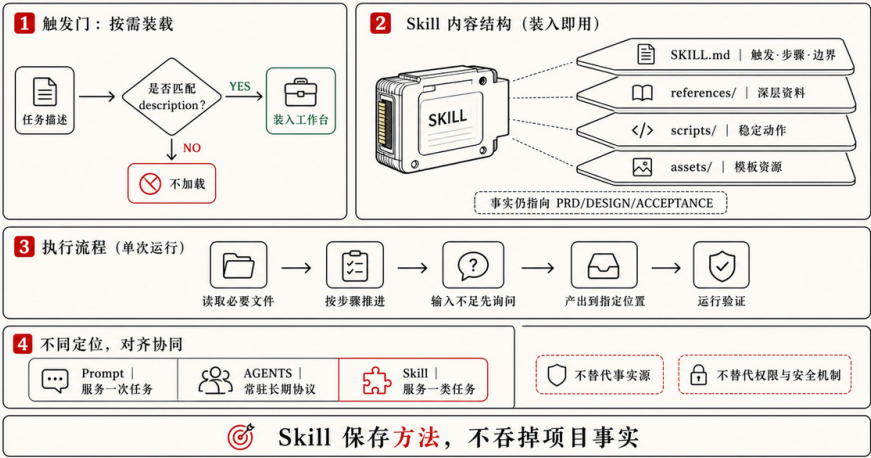
这就是为什么第三章先讲 Skills，再讲 SubAgents。很多时候，团队真正缺的是一套可按需加载的工作方法，不是更多“专家 Agent”。

Skill 通常包含什么

项目级 Skill 可以先放在 `.codex/skills/` 里：

```
.codex/  
skills/  
leadflow-ai-suggestion-review/  
  SKILL.md  
  references/  
  scripts/  
  assets/
```

Skill：按需加载的程序性记忆



真正必需的是 `SKILL.md`。它需要有清楚的 `name` 和 `description`，因为描述会影响 Codex 判断什么时候使用这个 Skill。描述太宽，低风险任务也会误触发；描述太窄，真正相关任务又可能错过。

`references/` 适合放深层资料：设计规范、验收写法、接口文档、示例输出、失败案例。不要把所有资料都写进 `SKILL.md`，否则 Skill 会变成另一份大 Prompt。更好的写法是：`SKILL.md` 告诉 Codex 什么时候打开哪些 reference。

`scripts/` 适合放稳定、重复、格式严格的动作：检查敏感字段、生成报告骨架、校验目录结构、整理截图、转换数据格式。越确定、越脆弱、越需要重复执行的步骤，越适合脚本；越需要业务理解和风险判断的部分，越应该留给 Codex 按流程处理。

`assets/` 可以放模板、样例和输出资源。不是每个 Skill 都需要这些目录。一个只有 `SKILL.md` 的轻量 Skill，如果能稳定减少重复解释，就已经成立。

先使用现成 Skills，再写自己的

第一次接触 Skills，不要急着写自己的“专家系统”。更好的顺序是：先用现成 Skills，观察它们怎样组织触发条件、步骤、边界和验证。

这和学编程时先用成熟库很像。你不需要第一天就写自己的框架，先看别人怎样拆能力，怎样写边界，怎样处理失败，才知道好 Skill 和长 Prompt 的区别。

现在可以先从 `skills.sh` (<<https://www.skills.sh/>>) 这样的 Agent Skills 目录看起。它把公开的 Skills 按主题和仓库组织起来，页面上通常会显示安装命令、摘要、`SKILL.md` 片段和仓库链接。安装方式一般形如：

```
npx skills add owner/repo
```

或者在某个仓库中只安装指定 Skill：

```
npx skills add https://github.com/owner/repo --skill skill-name
```

这里有一个重要提醒：不要把“能安装”当成“可以无脑信任”。安装前至少看三件事：这个 Skill 的触发范围是否清楚，是否包含会执行命令的脚本，仓库和 `SKILL.md` 是否符合你的项目边界。特别是涉及文件删除、外部服务、真实客户数据、密钥、Git 操作和生产环境的 Skill，一定要先看清它会引导 Codex 做什么。Skills 是能力扩展，不是安全豁免。

有几类现成 Skills 值得先看。

第一类是 UI 和设计质量相关的 Skill。比如 `ui-ux-pro-max` (<<https://www.skills.sh/nextlevelbuilder/ui-ux-pro-max-skill/ui-ux-pro-max>>)，它面向 Web 和移动应用的 UI/UX 设计判断，覆盖界面结构、视觉风格、配色、字体、交互、无障碍、响应式和常见产品类型。对 LeadFlow 这种 CRM 工作台来说，它的价值不是替代 `DESIGN.md`，而是在设计系统和界面状态已经

有方向时，帮助 Codex 做更专业的页面实现和体验走查。

例如你让 Codex 优化客户详情页里的 AI 建议卡片，它不应该只把卡片做得“更好看”。如果配合 UI/UX 类 Skill，它更容易检查：来源入口是否靠近建议文本，待确认状态是否足够清楚，主按钮是否会让销售误以为已经发送给客户，移动端是否能承载沟通记录展开，错误和空状态是否给了用户可恢复路径。Skill 在这里沉淀的是设计判断，而不是单一风格。

第二类是 Git 工作流相关的 Skill。比如 `git-commit` (<https://www.skills.sh/github/awesome-copilot/git-commit>)，它围绕 Conventional Commits 生成标准化提交信息，并根据实际 diff 判断类型、scope 和提交内容。对团队来说，这类 Skill 的价值不是“帮我少写一句 commit message”，而是把提交前的 diff 分析、逻辑分组、破坏性变更识别和安全边界变成稳定流程。

不过，Git 类 Skill 也要谨慎使用。它可以帮助整理变更，但不应该替人承担最终确认。特别是工作区里有用户未提交改动、生成文件、密钥文件或临时实验时，提交前仍然要看 `git status` 和 diff。一个好的 Git Skill 应该让提交更可审查，而不是让提交变成自动滑过的动作。

第三类是需求澄清和方案形成相关的 Skill。比如 `brainstorming` (<https://www.skills.sh/obra/superpowers/brainstorming>)，它强调在实现前通过结构化对话把想法变成设计和规格。它对 Agentic Coding 很有启发：很多失败不是代码写错，而是需求还没形成可以委托的形状，Codex 就已经开始实现。

把它放到 LeadFlow 里，你可以在“让销售助手更智能”这种模糊需求进入代码前，先让 Codex 和你澄清：更智能指的是建议更准确、来源更清楚、风险提示更强，还是销售确认流程更顺？哪些属于 MVP，哪些会越界到完整 CRM？哪些状态需要设计师确认？如果这些问题没答清，后面再多测试也只能证明一个不清楚的目标被实现了。

第四类是成熟作者或团队整理出来的一组 Skills。比如 `mattpocock/skills` (<https://www.skills.sh/mattpocock/skills>)，它不是单个流程，而是一组覆盖 PRD、TDD、诊断、架构改进、代码审查、写 Skill 等任务的能力集合。学习

这种集合，不是为了全量照搬，而是观察一个有经验的实践者怎样拆分能力边界：什么适合做成 `to-prd`，什么适合做成 `diagnose`，什么适合做成 `tdd`，什么应该保持为单独 Skill，而不是塞进一个超级助手。

第五类是项目级脚手架里的成组 Skills。比如 CodeNow (<https://github.com/xue160709/CodeNow>)，这是作者自己做的一套 Agentic Coding 脚手架样本，读者可以下载来体验一下。除了直接使用，它的价值还在于可以拆开观察如何构建 Agentic Coding 脚手架，而这部分内容会在第五章详细介绍。

这几类例子可以给读者一个实际起点：

表 3-2 常见重复问题与可参考的 Skill 类型

反复遇到的问题	可参考的 Skill 类型
页面能跑，但状态、层级和响应式不稳定	UI / UX 或 design review Skill
需求模糊，Codex 太早开始写代码	brainstorming / product discovery Skill
PRD、验收和 TODO 总要从头整理	planning / to-prd / acceptance Skill
bug 排查容易变成乱改	diagnose / systematic debugging Skill
提交信息随意，diff 没有逻辑分组	git-commit Skill
想观察多个 Skills 怎样组成项目生命周期	CodeNow 这类脚手架样本

作者建议初学者也不要一口气装太多 Skills。Skills 太多，触发描述会互相竞争，维护成本也会上升。先装少量高频、低风险、能立刻改善工作质量的 Skills，观察几轮，再决定哪些经验需要项目级沉淀。

把 LeadFlow 的经验写成项目 Skill

LeadFlow 最适合先沉淀的，是一个高风险、反复出现、输入输出稳定的流程：AI 建议审查，而不是“做 CRM”这种大而空的能力。

通用 UI/UX Skill 可以提醒对比度、触控区域、响应式和状态反馈，但它未必知道 LeadFlow 的特殊边界：AI 建议不能冒充客户事实，销售确认前不能显示为已执行，沟通记录不足时不能生成通用销售话术，手机号和邮箱不能进入日志。这些判断已经在第二章进入了 PRD、DESIGN、ACCEPTANCE 和

AGENTS。现在它们可以进一步沉淀成一个项目级 Skill。

一个轻量版本可以这样写：

```
---
name: leadflow-ai-suggestion-review
description: 当任务涉及 LeadFlow 的 AI 建议、来源展示、销售确认、客户详情页建议区域或相关日志时使用；不用于无关的
---

##  workflow

1. 阅读 `CONTEXT.md`，确认 Lead、Communication Record、AI Suggestion 和 Sales Confirmation 的定义。
2. 阅读 `PRD.md`，确认 AI 建议的当前范围、非目标和待决策问题。
3. 阅读 `DESIGN.md` 中关于 AI 建议卡片、来源展示、确认状态和信息不足态的小节。
4. 阅读 `ACCEPTANCE.md` 中关于来源、无记录、确认前状态和隐私日志的场景。
5. 检查相关代码、fixture 和日志路径。
6. 判断本次改动是否保留来源记录、是否让建议在销售确认前保持草稿、是否避免编造预算、合同金额或采购时间。
7. 如果发现新的失败路径，先建议补充 `ACCEPTANCE.md`，再判断任务是否可以标记完成。

8. 汇报行为变化、验证证据、未覆盖范围和需要回写的位置。

##  boundary

- 除非当前任务明确授权，不新增真实 CRM 同步。
- 不把 AI 建议当成客户事实。
- 不在日志中输出手机号、邮箱、合同金额或完整客户消息。
- 不替代人工最终验收。
```

这份 Skill 没有替代第二章的文件。它只是规定：当任务触及 AI 建议域时，应怎样把那些文件读起来、用起来、回写起来。事实仍然在 `PRD.md`、`DESIGN.md`、`CONTEXT.md`、`TODO.md` 和 `ACCEPTANCE.md` 里；Skill 保存的是流程。

这点非常重要。Skill 一旦开始保存项目事实，很快就会和文件系统打架。比如把完整设计系统复制进 Skill，下一次 `DESIGN.md` 更新后，Skill 就过期；把完整验收场景复制进 Skill，下一次 `ACCEPTANCE.md` 变化后，完成标准就分裂。好的 Skill 应该指向事实源，而不是吞掉事实源。

什么时候才该写自己的 Skill

判断一个流程是否值得写成 Skill，可以问五个问题。

1. 它是否反复出现？

- 2. 它是否有稳定输入和输出？
- 3. 它是否包含容易遗漏的步骤？
- 4. 它是否有可检查的完成方式？
- 5. 维护它的成本是否低于每次重新解释的成本？

可以把标准压成一句话：

当你连续三次对 Codex 说同一套流程，并且这套流程有稳定输入、步骤、输出和验证方式，就可以考虑把它沉淀成 Skill。

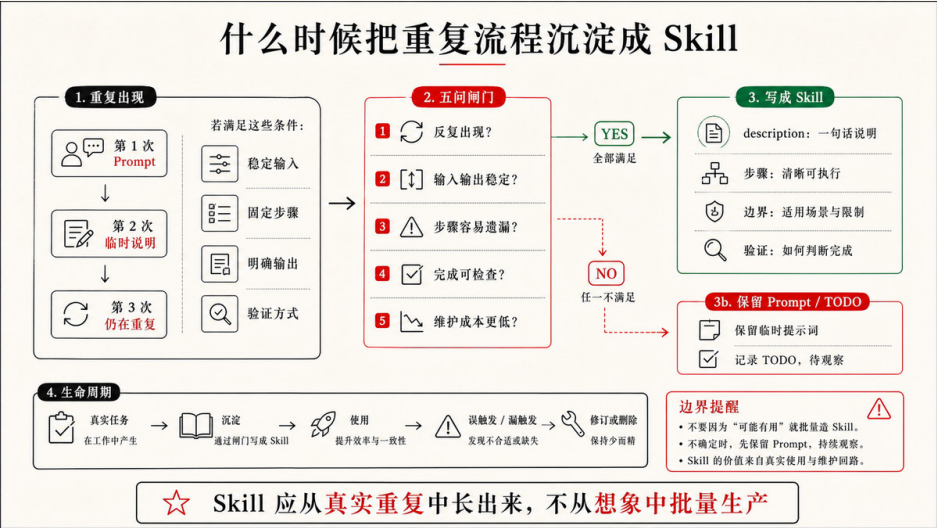


图 3-1 何时将流程沉淀成 Skill

这里的“三次”不是机械数字，它提醒你：Skill 应该从真实任务里长出来。第一次靠 Prompt 完成，第二次把流程写进 TODO 或临时说明，第三次还在重复，就说明它可能已经开始从一次性对话变成项目能力。

Skill 的边界

Skill 不能替代项目事实。它可以要求“先读 DESIGN”，但不应该保存完整设计系统；可以要求“检查验收场景”，但不应该复制验收文件。

Skill 也不能替代安全机制。它能提醒 Codex 不要读取真实 CRM、不要泄露手机号、不要运行危险命令，但高风险动作仍然需要权限、审批、沙箱、Hook、测试或人工验收接住。

Skill 还会腐烂。LeadFlow 从脱敏样例进入真实 CRM 试点以后，旧 Skill 如果仍然写着“永远不接真实数据”，就会阻碍正确开发；AI 建议从本地草稿变成可写回 CRM 后，确认状态和验收场景也要跟着升级。一个好 Skill 写完以后也不会封存，它会在误触发、漏触发和真实失败中继续改。

所以，搭 Skills 文件夹时，不要追求数量。先接入少量现成 Skills，再观察哪些流程真正改变了 Codex 的读取、实现、验证和汇报。等你看到自己的重复流程，再写项目级 Skill；等项目级 Skill 跨项目稳定复用，再考虑插件化和团队分发。

下一节的 SubAgents，承接的是另一个问题：如果一件任务缺的不是流程，确实需要多个专业视角分头看，应该怎样拆分。Skill 管方法，SubAgent 管分工。两者都能让 Agent 变得更专业，但它们解决的不是同一种问题。

3.3 基于任务复杂度设计 SubAgents

Skills 讲完以后，很容易产生一个新冲动：既然 AI 可以按需加载流程，为什么不再多派几个 Agent，把产品、设计、验收、隐私、架构都交给各自的专家？主 Agent 只负责把它们的结论汇总起来，看起来就像一个小型 AI 团队。

这个想法很有吸引力，也很容易把脚手架做重。很多任务真正缺的，是一个清楚的 Skill、一个准确的验收场景，或者一条长期边界，不是更多 Agent。SubAgent 不是默认增强器，它是一种有成本的分工。

所以这一节先回答一个问题：

这件事真的需要分出去做吗？

SubAgent 是受控分工，不是更聪明的 AI

SubAgent 可以理解成主 Agent 临时委托出去的专业执行者。它通常运行在独立的上下文窗口里，负责一个清楚的切片：阅读一组文件、审查一个模块、检查一类风险、整理一批证据，最后把摘要、证据和建议交回主 Agent。

独立上下文是它的价值，也是它的风险。价值在于，子任务的搜索、日志、失败和局部推理不会挤满主上下文；风险在于，SubAgent 不会自动拥有主会话里的全部上下文。委派时必须把目标、输入、边界、输出格式和不能做的事写清楚，不能依赖“刚才我们聊过”。

主 Agent 仍然负责统筹。它决定是否拆分，给出任务说明，限制权限，接收结果，合并冲突，并向人汇报最终判断。SubAgent 负责把局部看清，不能替主 Agent 做最终取舍。

可以用一句话区分：

Skill 教 Agent 怎么做。
SubAgent 帮 Agent 分头看。
主 Agent 负责取舍和合并。

表 3-3 Skill 与 SubAgent 的边界

问题	Skill	SubAgent
更像什么	操作手册	专业同事
解决什么	这类任务按什么流程做	复杂任务里谁来独立看
主要价值	动态加载流程，减少重复解释	上下文隔离、并行审查、专业视角
主要成本	维护流程和触发条件	调度、上下文搬运、结果合并
最大误用	把事实复制进 Skill	把简单任务拆成小型官僚体系

先问是不是 Skill 就够了

很多团队一开始会过度多代理化：产品 Agent、设计 Agent、测试 Agent、安全 Agent、架构 Agent 全部建好，结果每个任务都要经历一轮复杂分派。问题并没有更清楚，只是变成了更多摘要、更多冲突和更多合并工作。

更稳的顺序是：先让主 Agent 通过 Skill 按需专业化。比如 LeadFlow 的 AI 建议审查，主 Agent 读一个 Skill 就能知道该检查来源、信息不足、销售确认、隐私日志和对应验收。如果任务只涉及一个建议卡片的小改动，这已经足够。

只有当任务变得更复杂，Skill 不再够用时，才考虑 SubAgent。判断时不要只看“这件事重要”，要看“独立上下文和独立视角能不能带来明显收益”。例如：

- 大量搜索或日志分析会污染主上下文；
- 产品、设计、隐私、验收之间可能有真实冲突；
- 某个风险需要只读审查，不希望它顺手改代码；
- 多个模块可以并行探索，最后由主 Agent 合并；
- 子任务过程很吵，但结果可以用结构化摘要表达。

如果这些条件不成立，优先写 Skill、Rule 或 TODO，而不是创建 SubAgent。

规划和执行有时也要分离

规划 Agent 和编码 Agent 有时应该分开——这个判断对 SubAgent 设计尤其有用。原因不在于规划者比执行者高级，在于规划阶段会积累大量上下文和假设。如果同一个会话从模糊需求一路跑到多文件实现，它可能把早期推测当成事实，也可能因为上下文太满而遗忘后面的关键验证。

更稳的做法是把计划变成一个可审查产物，再交给执行。LeadFlow 可以这样做：

规划阶段：

阅读 PRD / DESIGN / ACCEPTANCE
拆出建议来源、无记录状态、确认状态、隐私日志四个工作项
标出不做真实 CRM、不做自动外发
形成 TODO 或实施计划

执行阶段：

读取计划和必要上下文
做最小修改
运行对应验证
把结果回写 TODO 和验收证据

这不一定需要两个真实的配置化 SubAgent。小任务里，主 Agent 可以先规划再执行；大任务里，可以让一个规划型 SubAgent 只负责形成计划，让主 Agent 或执行型 SubAgent 按计划做。关键是计划本身要成为项目状态，而不是只留在一段长上下文里。

LeadFlow 可以先建哪些 SubAgents

LeadFlow 早期不需要很多 SubAgents。四个专业视角已经足够覆盖最容易失控的地方：

表 3-4 LeadFlow 建议先建的 SubAgent

SubAgent	什么时候使用	主要看什么	输出
leadflow-product-reviewer	新增 AI 建议、销售确认、跟进阶段或 CRM 范围变化时	是否越过 MVP，是否把建议变成承诺	产品风险、范围建议、待决策问题
leadflow-design-state-reviewer	修改客户详情页、建议卡片、状态徽标、主按钮时	客户事实、AI 推测、销售确认是否可区分	状态误读风险、页面证据建议
leadflow-acceptance-planner	验收场景缺失、含糊或无法执行时	能否写成可检查场景，是否遗漏失败路径	GWT 建议、证据类型、不覆盖范围
leadflow-privacy-auditor	涉及客户数据、日志、模型请求或外部 CRM 时	是否泄露隐私，是否默认接触真实数据	风险等级、证据位置、下一步建议

这些角色都应该先保持克制。比如 `leadflow-privacy-auditor` 默认只读，不改业务代码；`leadflow-acceptance-planner` 只提出验收场景建议，不直接宣布任务通过；`leadflow-design-state-reviewer` 只判断界面状态是否误导用户，不替设计师决定全新视觉方向。

一个轻量草案可以这样写：

leadflow-privacy-auditor

什么时候使用：

- 涉及客户数据、日志、AI 分析、模型请求或外部 CRM 集成时。

负责：

- 检查手机号、邮箱、合同金额、完整客户原文是否进入日志或请求摘要。
- 检查任务是否默认使用脱敏样例数据。
- 检查是否出现未经授权的真实 CRM 读取、上传或同步。

不负责：

- 不修改业务代码。
- 不决定产品范围。
- 不替代正式安全评审。

输出：

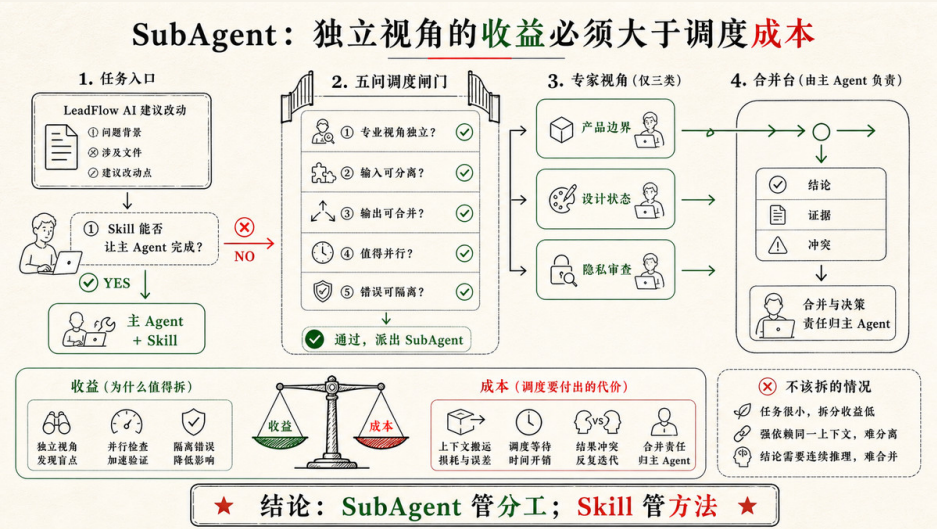
- 风险等级。
- 发现的问题。
- 证据位置。
- 建议下一步。

等触发条件、输出格式和边界在真实任务中稳定，再把它转成具体工具支持的 agent 配置。不同工具格式不同，本书统一用 `.codex/agents/` 作为教学口径，是为了先讲清分工设计，而不是被字段格式带跑。

什么时候不要用 SubAgent

SubAgent 是否值得用，可以用一个简单公式判断：

收益 > 调度成本 + 上下文搬运成本 + 结果合并成本



下面这些情况通常不适合拆：

- 任务很小，主 Agent 直接按 Skill 做更快；
- 任务需要和用户频繁来回确认；
- 子任务会同时修改 `TODO.md`、`package.json`、共享类型、全局样式或同一个组件；
- 只是流程说明，更适合写 Skill；
- 只是长期边界，更适合写 Rule；
- 必须强制执行，更适合 Hook、权限、审批或 CI；
- 输出没有结构，只是一段泛泛意见，最后仍要主 Agent 重做。

SubAgent 最适合做“过程复杂、结果可摘要”的工作。比如大型代码搜索、日志分析、只读安全检查、复杂验收规划、多方案产品评审。它不适合替主 Agent 承担最终决策，也不适合让人逃避清楚定义任务。

写 SubAgent 前问五个问题

在把一个角色写进 `agents/` 前，先问五个问题。

1. 它是不是稳定专业视角，而不是“更认真一点”的泛泛要求？

2. 它是否真的需要独立上下文，而不是主 Agent 按 Skill 就能完成？
3. 它是否只需要返回摘要和证据，而不是频繁与人对话？
4. 它的权限能否收窄到只读、只写证据或只改特定模块？
5. 主 Agent 能否合并它的输出，并处理冲突？

这五个问题背后是一条边界：SubAgent 沉淀的是专业分工，不是决策权。复杂任务里，多个视角可以分头看，但产品取舍、范围确认、风险接受和最终交付仍然要回到主 Agent 与人。

下一节的 Rules，承接的是另一个层次的问题。Skill 说“这类任务怎么做”，SubAgent 说“复杂任务谁来独立看”，Rule 则说“无论谁来做，都不能默认跨过哪条线”。如果这三层混在一起，脚手架会变重；分清以后，Codex 才能在合适的地方获得合适的约束。

3.4 基于需求设计 Rules

第二章把项目状态拆进了 AGENTS.md、CONTEXT.md、PRD.md、DESIGN.md、TODO.md 和 ACCEPTANCE.md。3.2 又把重复流程沉淀成 Skills，3.3 把复杂任务里的专业视角拆成 SubAgents。到了 Rules，我们要处理的是另一类问题：哪些边界不应该随着任务变化而消失。

在 Harness 里，这一层属于“静态上下文”。Rule 不是更长的 Prompt，也不是把所有偏好写进一份“AI 使用说明”。它是一组长期成立、稳定触发、能改变 Agent 行动的边界。Skill 可以按需加载，因为它回答“这类任务怎么做”；SubAgent 可以按需调用，因为它回答“这个复杂任务需要谁独立看”；Rule 更接近项目的常驻约束，因为它回答：

无论这次任务是谁来做、怎么做、做到哪一步，都不能默认跨过哪些线？

如果说第二章的文件是 Harness 的事实资产，那么 Rules 就是在这些事实资产之上建立的一层长期边界。它让 Codex 每次进入相似任务时，不必等人重新提醒“别把 AI 建议写成客户事实”“别改 UI 状态前忘记读设计稿”“别在交付时只说完成了而说不说验证证据”。这些判断应该被放到系统里。

但 Rules 也最容易写坏。很多团队一开始会把它当成保险箱：每出一次问题，就补一条；每想到一个偏好，就加一句；每担心 AI 误解，就再提醒一次。半年后，规则越来越多，真正重要的边界反而被淹没在一堆“曾经有道理”的句子里。

所以设计 Rules 的第一原则不是“写得更全”。第一步要先判断：

这条经验真的应该常驻在 Harness 里吗？

Rule 是静态上下文，不是知识仓库

Rules 的价值来自常驻，成本也来自常驻。凡是进入静态上下文的东西，都会在后续任务里影响 Codex 的判断。它可能让 Agent 更稳，也可能让上下文变吵、变慢、变矛盾。

因此，Rule 应该只收纳三类内容。

第一类是长期边界。比如密钥不能硬编码、真实客户数据不能进入日志、生产数据导入必须确认。这些不依赖某一次任务，未来每轮都成立。

第二类是高风险触发条件。比如修改 AI 建议、销售确认、对外发送、权限、账单、数据迁移、隐私日志时，必须先读哪些文件、检查哪些验收、在哪里停下来请求确认。

第三类是交付协议。比如涉及产品行为变化时要回写 `TODO.md` 或 `ACCEPTANCE.md`，涉及 UI 状态变化时要提供页面检查证据，涉及外部系统时要说明权限和数据来源。

这些内容适合放进 Rules，因为它们关心的是“这个项目长期不允许怎样做”，已经超出了某一轮任务偏好。反过来，产品范围、设计细节、当前任务状态、完整验收场景、操作流程，都不应该被 Rules 吞掉。

表 3-5 Rules 与其他 Harness 资产的边界

对象	应该保存什么	不应该被 Rules 抢走什么
PRD.md	目标、范围、非目标、成功标准	不要把完整产品规格复制进 Rule

对象	应该保存什么	不应该被 Rules 抢走什么
DESIGN.md	状态模型、交互、视觉系统、风险表达	不要把设计系统写成零散禁令
TODO.md	当前进度、阻塞、恢复点、最近验证	不要让 Rule 变成任务状态表
ACCEPTANCE.md	Given / When / Then、测试与评估证据	不要把完整验收场景藏进 Rule
Skill	一类任务的做法、步骤、输入输出	不要把流程步骤写成常驻规则
SubAgent	独立专业视角和受限上下文	不要用一条提醒代替必要的审查角色
Rule	长期边界、触发条件、交付协议	不要变成新的知识仓库

这个边界很关键。Rules 的职责不是把系统重新统一成一个文件；它要让第二章建立的事实源在正确时机被正确使用。

薄内核：AGENTS.md 只放最稳定的规则

最容易被所有工具读取的入口，往往是项目根部的 AGENTS.md。这让它很适合做 Harness 的“薄内核”：每次任务都应该看到的少数协议放在这里。

薄内核不是一句口号。它意味着 AGENTS.md 不应该承载整个项目的历史、全部业务背景和所有操作步骤。它只保存最稳定、最值得常驻的边界，例如：

```
# 项目工作协议

- 开始实现前，先读取与任务相关的 `TODO.md`、`PRD.md`、`DESIGN.md` 或 `ACCEPTANCE.md`。
- 不把 AI 生成的推测写成客户事实；涉及 AI 建议、客户记录和销售确认时，必须保留来源与责任边界。
- 涉及真实数据、外部系统写入、删除操作或自动对外发送时，先请求人确认。
- 交付时说明修改内容、验证证据、未验证部分和需要回写的状态。
```

这已经足够影响 Codex 的行动。它会提醒 Agent 先找事实源，识别高风险任务，区分客户事实与 AI 推测，并在交付时给出证据。

更细的规则可以拆到 .codex/rules/ 这样的概念目录中。这里的路径只是组织方式，不是强制标准：把长期边界按主题管理，而不是把所有内容堆在入口文件里。

```
.codex/  
rules/  
  privacy.md  
  ai-suggestion.md  
  design-state.md  
  delivery.md
```

privacy.md 管客户数据、日志、脱敏和外部服务；**ai-suggestion.md** 管建议来源、事实 / 推测区分和销售确认；**design-state.md** 管界面状态、空状态、错误状态和主操作语义；**delivery.md** 管 TODO 回写、验收证据和最终汇报。

这就是静态上下文的分层：**AGENTS.md** 是薄内核，主题 Rules 是可按任务加载的边界包。每一层都稳定，但不必每一层都在每个任务里展开。

Rule 应该从需求和风险里长出来

不要从抽象规范开始写 Rules。下面这些句子都对，但很难改变 Agent 的下一步行动：

- 注意代码质量。
- 保持用户体验一致。
- 修改后要测试。
- 不要引入风险。

它们没有说明什么叫质量，什么叫一致，哪些测试相关，什么风险必须停下来。Codex 读到以后可能会礼貌地遵守语气，却仍然按照默认经验继续做。

好的 Rule 应该来自真实需求和真实风险。先问四个问题：

- 这个项目里什么错误反复发生？
- 这个错误发生后有什么真实后果？
- Codex 在什么任务或文件变化时最容易触发它？
- 下一次触发时，Agent 应该先读什么、检查什么、在哪里停下来？

LeadFlow 的 AI 建议就是一个典型例子。产品目标允许 AI 给销售生成下一步建议，但建议必须来自客户沟通记录，不能冒充客户事实，也不能在销售确认前变成对外承诺。这个要求首先属于 **PRD.md** 和 **ACCEPTANCE.md**，因为它定义了产品目标和验收标准。

但如果后续每次修改 AI 建议区域，Codex 都容易为了让页面完整而生成一段通用销售话术，这就不只是单次验收问题了。它已经变成反复出现的行为风险，可以沉淀成 Rule：

AI 建议来源与责任规则

适用：

- 修改 AI 总结、下一步建议、来源展示、销售确认或客户详情页建议区域时。

风险：

- 无来源建议会被销售误认为客户事实或可执行承诺。

规则：

- 修改前先确认对应验收场景是否存在；缺失时先补验收或请求人确认。
- AI 建议必须尽可能关联来源记录。
- 沟通记录缺失或来源不可用时，显示信息不足状态，不生成通用销售话术。
- 客户没有明确提到预算、合同金额或采购时间时，不得写成客户事实。
- 销售确认前，建议不得进入已执行跟进记录，也不得触发对外发送。

交付：

- 说明来源场景、无记录场景和确认前状态是否被检查。

这条 Rule 的价值不在于它写得长，而在于它能改变行动。Codex 看到相关任务时，不应该只去改卡片样式或模型文案，而要先确认验收、检查来源、区分事实和推测，并在交付时说明证据。

再看设计侧。设计师可能反复发现，AI 容易把“确认为跟进计划”改成“发送给客户”，因为后者更像常见 CRM 主按钮。但在 LeadFlow 里，这会改变责任边界：用户可能以为销售确认前已经发生了对外动作。这个风险适合写成另一条 Rule：

设计状态规则

适用：

- 修改客户详情页、AI 建议卡片、状态徽标、空状态、错误状态或主操作按钮时。

规则：

- 改界面状态前先阅读 `DESIGN.md` 的状态模型和风险表达。
- 客户事实、AI 推测和销售确认决策必须在界面上可区分。
- 销售确认前，主按钮不得表达“发送给客户”或已对外承诺。
- 可能改变用户理解的 UI 变更，交付时说明页面检查、截图或人工走查证据。

需求给出目标，风险决定规则。Rule 的成熟度，取决于它是否能让下一次相似任务少犯同一种错。

Rule 要有明确作用域

没有作用域的规则最容易变重。比如：

- 所有 UI 改动都必须截图验证。

听起来很安全，但实际会制造两个问题。第一，很多 UI 改动很轻：改一个错别字、调整一个低风险标签、删掉一行无用文案，不一定需要截图。第二，截图只能证明某个画面当时长什么样，不能证明来源链路、状态流转或隐私日志正确。过宽的规则会让低风险任务变慢，也会让高风险任务获得一种虚假的安全感。

更好的写法，是把触发条件说清楚：

- 涉及 AI 建议卡片、确认状态、风险提示、空状态或错误状态的 UI 改动，交付时必须提供页面检查或截图证据。
- 纯文案、小样式且不改变用户理解的改动，可以只说明 diff 和目标页面检查结果。

作用域可以按几种方式设计：按任务类型，例如 AI 建议、日志、设计状态、验收回写；按文件路径，例如客户详情页、AI 生成逻辑、样例数据；按风险等级，例如真实数据、外部服务、权限、发布、自动外发；按生命周期，例如开发前、验证前、交付前。

好的 Rule 应该让 Codex 更容易判断“我现在是否触发了这条规则”。如果一条 Rule 什么任务都适用，它往往要么太抽象，要么太重。

什么时候不要写 Rule

不是所有经验都应该规则化。很多时候，不写 Rule 反而更专业。

如果它只服务本轮任务，写进当前 Prompt 或 **TODO.md** 就够了。比如“这次只改客户详情页，不动列表页”，这是当前任务边界，不是长期规则。

如果它定义产品范围，优先写进 **PRD.md**。比如“首版不支持自动外呼”，这是产品非目标，不应该藏在 Rules 里。

如果它定义界面结构、状态语义、视觉 token 或组件策略，优先写进 `DESIGN.md`。Rule 只负责要求相关任务读取 DESIGN，不负责替 DESIGN 保存设计系统。

如果它定义完成标准，优先写进 `ACCEPTANCE.md`。Rule 可以说“实现前先确认验收场景”，但不要把完整验收场景复制到 Rule 里。

如果它是一套重复流程，优先写成 Skill。比如“访谈到 PRD”“设计状态走查”“交付总结”，这些是做事方法，不是长期边界。

如果它需要独立专业视角，考虑 SubAgent。比如隐私审查、安全分析、大型验收复核，更像由一个受限角色独立看一遍，而不是只靠一条规则提醒。

如果它必须被强制执行，考虑 Hook、权限、审批、CI 或脚本门禁。Rule 能影响 Codex 的行为，但它不是硬安全层。比如“真实客户数据不得上传外部服务”当然应该写成 Rule，但如果项目真的处理真实数据，还需要权限隔离、日志脱敏、请求审计和人工确认接住。

这个判断会让 Rules 保持锋利。属于事实源的放回事实源，属于流程的放进 Skill，属于分工的交给 SubAgent，属于强制执行的交给 Hook 或权限系统。

LeadFlow 的最小 Rules 集

对 LeadFlow 这样的项目，不建议一开始就写几十条规则。更现实的起点，是先写少数高风险、反复触发、能改变行动的规则。

表 3-6 LeadFlow 初始 Rules 建议

Rule	适用场景	改变什么行动
<code>privacy.md</code>	处理客户记录、日志、样例数据、外部服务	禁止真实敏感信息进入日志、示例、模型请求和公开报告
<code>ai-suggestion.md</code>	修改 AI 总结、下一步建议、来源展示、销售确认	要求区分客户事实、AI 推测和销售确认，不生成无来源话术
<code>design-state.md</code>	修改详情页状态、空状态、错误状态、主按钮	要求先读 <code>DESIGN.md</code> ，保持状态语义和责任边界
<code>delivery.md</code>	任务完成、部分完成、阻塞或验收变化	要求说明验证证据、未验证部分，并回写 <code>TODO.md</code> 或验收事实

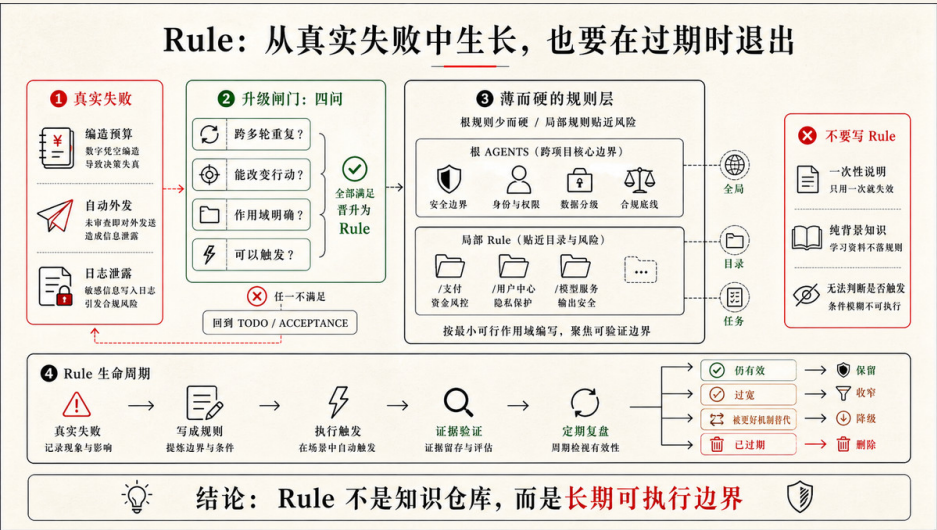
这四条规则已经覆盖 LeadFlow 最容易出问题的地方：客户隐私、AI 建议责任、界面状态语义、交付证据回写。它们不会让 Codex 知道更多业务细节，但会让 Codex 在关键节点做出更稳的选择。

Rules 也要被维护

Rules 不是写完就永久正确。项目会变，风险会变，团队的工作方式也会变。曾经重要的规则可能变成噪声，曾经只是一次提醒的经验也可能随着问题反复出现而升格为 Rule。

维护 Rules 可以采用一个简单节奏：

- 发现一次问题：先写进 TODO 或复盘记录。
- 第二次出现：检查是否已有事实源或 Skill 能覆盖。
- 多次出现且后果真实：提炼成 Rule。
- 规则长期没有触发：降低优先级、合并或删除。
- 规则与事实源冲突：以事实源为准，修正 Rule。



这也是 Harness 经济性的体现。静态规则有固定成本，动态 Skills 有调用成本，SubAgents 有调度成本，Hooks 有维护成本。成熟的团队不会把所有经验都立刻规则化，而是让经验在正确层级里沉淀。

写 Rule 前的五个问题

最后，可以用五个问题判断一条 Rule 是否值得写入：

1. 这条经验是否跨多轮任务长期成立？
2. 它是否能在下一次任务开始前改变 Codex 的行动？
3. 它是否有明确触发条件，而不是“永远注意一点”？
4. 它是否引用正确的事实源，而不是复制事实源？
5. 如果它失效，后果是否足够真实，值得占用静态上下文？

五个问题都回答得清楚，这条经验就可以进入 Rules。答不清，就先留在 Prompt、TODO、PRD、DESIGN、ACCEPTANCE、Skill 或 SubAgent 里。

到这里，第三章的升级线已经走过四层：第二章的文件负责保存事实，Skills 负责沉淀可复用流程，SubAgents 负责复杂任务里的上下文隔离，Rules 负责长期边界。3.5 接着讲 MCP 与 Hooks：只读 MCP 可与 Skills 并行接入外部事实，Hooks 则把关键检查固定到生命周期节点；各层应留下的证据贯穿始终，主要由第二章的核心文件与交付汇报承担。

3.5 MCP 与 Hooks：外部接入与执行拦截

到 3.4 为止，我们搭建的主要还是“仓库内的 Harness”：事实放进文件，流程放进 Skills，复杂任务用 SubAgents 分工，长期边界沉淀为 Rules。这个阶段已经足够支撑很多个人项目和早期团队项目。

但真实工程不会永远停留在仓库里。项目继续发展，就会连接设计稿、任务系统、知识库、数据库、浏览器、CRM、云环境和内部 API。Agent 不再只是改代码和读 Markdown，它会越来越多地接触外部系统，也会越来越多地在固定节点上被要求自动检查、自动记录、自动拦截。

这时，Harness 会从“上下文组织”升级成“执行系统”。

表 3-7 第三章的 Harness 分层

层次	解决的问题	成熟标志
核心文件	项目事实、当前状态、验收契约放在哪里	人不需要反复口头补背景
Skills	一类重复任务按什么流程做	高频流程可以按需加载和复用
SubAgents	复杂任务中哪些视角需要隔离	专业审查不再挤在同一个上下文里
Rules	哪些长期边界必须持续生效	高风险任务能在行动前被校准
MCP / 连接器	Agent 如何按权限访问外部系统	外部上下文不再靠复制粘贴进入会话
Hooks	哪些固定节点需要自动提醒或拦截	关键检查不再只靠 Agent 想起来

表中的排列便于对照各层职责。上面各层解决的是 Agent 如何执行；证据与复盘则横切于每一层。第二章的核心文件从第一天就开始承担大部分职责：最终汇报说明修改与风险，`TODO.md` 记录恢复点，`ACCEPTANCE.md` 保存验收场景，截图、测试输出和 diff 作为验证证据。接入 MCP 与 Hooks 以后，再把外部调用、触发记录和失败样例一并纳入同一套证据习惯。

MCP：把外部系统变成可授权的上下文

MCP 可以理解为一种让 AI 应用连接外部系统的协议层。它的核心目的不是知识堆叠；它让 Agent 能在需要时通过受控接口访问外部资源或工具：读取设计稿、查询任务系统、搜索知识库、读取数据库、调用内部 API、操作浏览器，等等。

在 Harness 里，MCP 的位置很清楚：它是外部上下文和外部动作的入口。

没有 MCP 或连接器时，外部信息通常靠人复制粘贴进 Prompt。设计稿、会议纪要、任务状态、CRM 样例、接口文档，都可能散落在聊天记录里。这样做能启动工作，但不可复用、不可审计，也容易过期。MCP 把这些外部来源变成工具接口，让 Agent 在任务需要时读取，而不是把所有东西一次性塞进上下文。

MCP 工具大致可以分成两类。

资源型工具只读取或检索，例如读取当前设计稿、查询某个任务状态、搜索知识库、读取脱敏样例数据。动作型工具会改变外部系统状态，例如创建任务、更新 CRM、发送消息、触发发布、写入数据库。两者风险完全不同，不

能放在同一个无差别的权限篮子里。

表 3-8 资源型 MCP 与动作型 MCP 的风险差异

类型	典型能力	主要风险	默认策略
资源型	读取设计稿、查任务、搜文档、读测试数据	读到不该读的数据，泄露到日志或模型请求	从只读、脱敏、最小范围开始
动作型	写 CRM、创建任务、发消息、触发部署	改变外部状态，误操作难回滚	默认需要人工确认和审计记录

一个 MCP 接入能不能进入项目，至少要回答六个问题：

- 1. 它解决的是外部事实读取，还是外部状态写入？
- 2. 它能否限制到只读、脱敏、测试环境或最小数据范围？
- 3. Agent 调用它时，人是否需要确认？
- 4. 返回内容会不会进入日志、模型请求、截图或最终报告？
- 5. 调用失败时，Agent 能否看见失败并回到人工路径？
- 6. 调用证据写在哪里，后续任务能否复用？

这些问题答不清，就先不要接。成熟的 Harness 会克制连接范围，只把真正改善上下文质量、验证质量或协作效率的外部系统接进来。

Hooks：把关键检查放到生命周期节点

Rules 会告诉 Codex 应该遵守什么边界，但 Rule 仍然是自然语言上下文。它能影响 Agent 的判断，却不能保证每个固定节点都被执行。Hooks 解决的就是这件事：把一些检查放到工具调用、文件变更、任务结束、提交前等生命周期节点上。

换句话说，Hook 是 Harness 的“执行拦截层”。

比如，Rule 可以写：

涉及 AI 建议逻辑时，交付前必须说明来源场景和无记录场景是否被检查。

Hook 则可以在文件变更或任务结束时自动提醒：

检测到 AI 建议相关文件变更，请确认来源字段、无记录状态和销售确认前状态是否已检查。

前者改变 Agent 的任务理解，后者改变执行过程中的固定节点。两者配合起来，才更接近工程系统。

Hooks 常见的触发点包括工具调用前、工具调用后、文件变更后、会话开始、会话结束、提交前。不同触发点适合不同用途。

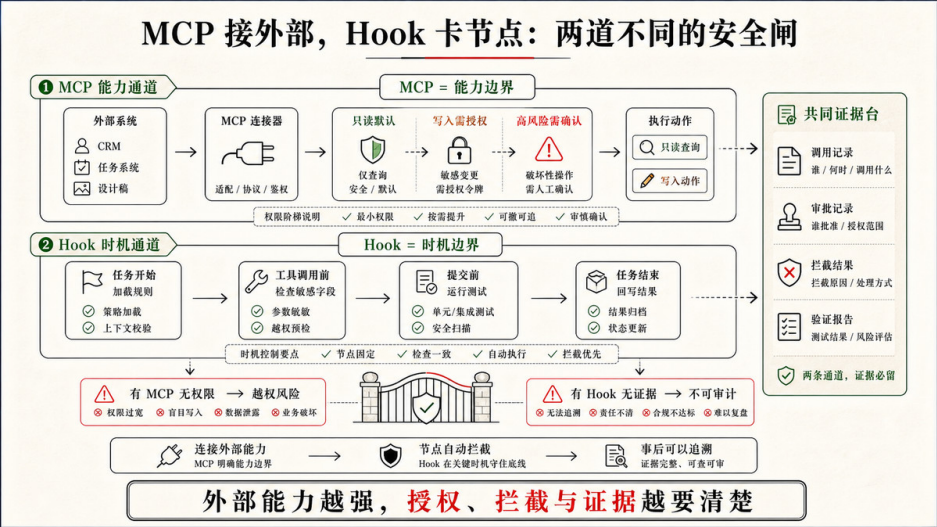
表 3-9 Hook 触发点与适用场景

触发点	适合做什么	LeadFlow 示例
工具调用前	拦截高风险动作	删除数据、访问真实客户路径、触发外发前要求确认
工具调用后	记录证据或提示下一步	运行测试后提示把结果写回 TODO
文件变更后	检查相关事实源是否同步	修改详情页状态时提醒查看 DESIGN.md
会话结束时	检查交付协议	确认是否说明验证证据、未验证部分和阻塞项
提交前	执行确定性门禁	格式检查、静态检查、敏感信息扫描

Hook 的行为也可以分层。提醒型 Hook 只输出提示，让 Agent 自己决定下一步；阻断型 Hook 返回失败，强制停止当前动作；副作用型 Hook 会写文件、发通知或调用外部系统。建议从提醒型开始，等误报率、触发范围和失败路径都稳定以后，再把少数高风险节点升级为阻断型。

LeadFlow 可以先有三个轻量 Hook。

- 1. 修改 AI 建议相关文件时，提醒检查来源、信息不足状态和销售确认前状态。
- 2. 修改客户详情页或设计系统文件时，提醒读取 `DESIGN.md` 的状态模型。
- 3. 任务结束时，提醒说明验证证据、未验证部分，以及是否需要回写 `TODO.md` / `ACCEPTANCE.md`。



这三类提醒覆盖的是高频遗漏点。它们不替 Codex 做判断，也不强行中断正常开发，只是把关键检查放回固定节点。等项目真的处理真实客户数据，再考虑用阻断型 Hook 拦截生产数据路径、硬编码密钥、真实外发或高风险删除。

Hook 写得好，会让团队少靠记忆；Hook 写得差，会让每个小改动都变慢。它比 Rule 更接近执行层，所以要更谨慎地控制触发条件、输入输出、退出语义和失败提示。不要用 Hook 表演安全感。只有当某个检查足够稳定、足够高频、足够可判断时，才值得自动化。

各层都要留下证据

Google 的 AI 工程经验里有一个很重要的判断：真正决定 Agent 质量的，不只是模型本身，还包括模型外部那套 Harness。放到工程团队里，这套 Harness 不能只会执行，还要能回答一组朴素问题：

- Codex 读了哪些关键文件？
它调用了哪些工具？
哪些命令成功，哪些失败？
验证证据在哪里？
哪些地方没有验证？
哪些 TODO、验收或设计状态被回写了？
下次接手时，能不能从记录恢复现场？

这些问题不对应某种要单独安装的 Harness 组件，而是对表 3-7 每一层的共同要求。核心文件留下事实与验收契约；Skill 执行后留下步骤产出与检查结论；SubAgent 审查后留下独立视角的判断；Rule 触发时，边界是否被遵守应能从交付中看出来；Hook 触发与 MCP 调用，进入团队阶段后应能查到记录。

早期不必追求复杂仪表盘。最终汇报、TODO.md、ACCEPTANCE.md、截图、测试输出和 diff review 通常就够。团队扩大以后，再按需补充工具调用日志、Hook 触发记录、MCP 调用审计、评估结果、失败样例、成本和延迟趋势——前提是有人真的会看、会据此改规则、改 Skill、收窄权限或补验收。

证据真正保护的是改进能力。没有记录时，团队只会说“AI 又没按我们想的做”；有记录时，才能判断问题到底出在哪一层：事实源缺失，Skill 流程不清，SubAgent 分工过重，Rule 触发范围太宽，Hook 误报，MCP 权限太大，还是验收证据不足。

这也是 Harness 思维和单次 Vibe Coding 的分界。Vibe Coding 可以靠当下感觉快速推进，Agentic Coding 则需要让推进过程留下可复用的痕迹，让团队能从一次失败里提炼规则、改进 Skill、收窄权限、补充验收，而不是下次继续用更长的 Prompt 祈祷。

从工具清单到工作系统

第三章真正想教的是一套分层判断，工具名只是入口。

当经验是项目事实，把它写进文件；当经验是一类流程，把它写成 Skill；当经验需要独立专业视角，把它拆给 SubAgent；当经验是长期边界，把它沉淀成 Rule；当经验需要固定节点执行，把它交给 Hook；当经验依赖外部系统，把它接成 MCP 或连接器。无论落在哪一层，都要问一句：这次执行留下了什么证据，下次接手能不能恢复现场。

这就是从“AI 帮我写代码”到“我在设计 Agentic Coding 工作系统”的转变。模型仍然重要，但模型只是一部分。真正让 Agent 稳定工作的，是模型外部这套 Harness：文件、规则、流程、工具、权限、执行节点，以及贯穿各层的证据

习惯。

读者能做出这些分层判断，就已经开始搭建自己的 Agentic Coding 脚手架，而不只是使用 AI 编程工具。

完成 LeadFlow 第一版

4.1 从 0 到 1：用 PRD 和 DESIGN 完成 LeadFlow MVP

第二章为 LeadFlow 写好了 `PRD.md` 和 `DESIGN.md`，第三章系统介绍了脚手架的各个层次。4.1 从这两份文件出发，先把 MVP 在本地跑起来。这里不是把第三章的脚手架撤回，而是先选择首轮开发真正需要的最小工程合同：目标、设计状态、长期规则、运行方式和验收边界先有落点，Skills、SubAgents、Rules、Hooks 和 MCP 等更厚的层次，等重复风险出现后再沉淀。

全章贯穿一条任务：AI 建议必须带来源，并且等待销售人工确认。4.1 的 MVP 不追求"把所有功能做齐"，而是先把这条最小价值链跑通：线索进来，沟通记录可查，AI 建议有来源，销售能确认或拒绝建议，建议在确认前保持草稿状态。后续 4.2 到 4.4 章节都围绕这条边界推进。

写第一行代码之前，先确定用哪套脚手架。

第一章的核心公式是 `Agent = Model + Harness`——模型提供生成能力，Harness 决定这次生成能不能被约束、可不可以被验证、能不能在下一轮继续。这套 Harness 在工程上落地为脚手架：核心文件记录产品边界、设计状态和长期

规则；Skills 把可重复 workflow 封装成 Codex 可以按需加载的经验；TODO 和验收标准追踪任务进展和可观察结果。

没有脚手架，你做的是 Vibe Coding——Codex 可以很快生成页面，但每次对话结束后，上下文消失，决策不留痕，下一轮再进来的人或 AI 都要重新猜测边界。有了脚手架，每一轮开发的产品判断、设计约束、任务状态和验收结果都沉淀在项目里，成为下一轮的工程条件。

所以，任何认真的项目，在第一次把任务交给 Codex 之前，都应该先确定自己的脚手架起点。起点可以很小，但不能没有。没有脚手架，Codex 只能跟着当前提示猜；有了最小脚手架，它至少知道目标写在哪里、边界从哪里读、执行状态往哪里回写、验收结果用什么证据证明。

你可以从 [skills.sh](#) 找到适合自己工作流的现成 Skills，比如项目冷启动、首版 MVP 开发、正式验收这类可重复 workflow，或者直接用作者整理的 [CodeNow](#) 脚手架里那套开箱即用的 Skills 组合；第五章会详细介绍它的完整设计思路。对于第四章的 LeadFlow，我们先不用一整套冷启动系统，而是使用一个足够现实的最小起点：

PRD.md	说明 LeadFlow 第一版为什么做、做给谁、做什么、不做什么
DESIGN.md	说明用户如何看见客户事实、AI 建议、来源和人工确认
AGENTS.md	说明 Codex 的长期工作规则、数据边界和汇报要求

Codex 开始开发以后，可以再把任务拆进 [TODO.md](#)，把关键风险沉淀进 [ACCEPTANCE.md](#)，把可重复流程做成 Skills。这里的判断标准不是“文件越多越专业”，而是“本轮需要哪些约束，才能让 Codex 的行动可校准”。首轮 MVP 不需要一开始就拥有完整工具链，但至少要让目标、边界、规则和完成信号有稳定位置。

有文件和没有文件，Codex 的行为差在哪里

这个差别比“怎么写提示词”更根本。同样一句话：

帮我完善 AI 建议卡片。

如果项目里没有 `PRD.md` 和 `DESIGN.md`，Codex 可能会按常见产品直觉行动：把“建议”写成系统结论，让按钮更像“发送给客户”，为了演示效果在组件里写死几条建议，甚至顺手预留真实 CRM 或邮件发送。它不是故意越界，而是没有可对照的项目事实，只能用通用模式补空白。

如果项目里已经有 `PRD.md`、`DESIGN.md` 和后续的 `ACCEPTANCE.md`，同一任务会被拉回 LeadFlow 的具体边界：

```
PRD.md      不自动发送消息，不接真实 CRM，不处理真实客户数据
DESIGN.md   建议是待确认草稿，来源入口靠近正文
ACCEPTANCE.md 无沟通记录时不得生成通用建议，确认前不得进入已执行记录
```

这不表示 Codex 一定不会犯错。更准确地说，文件让错误变得可见：当它准备加发送按钮时，你能指出这违反 PRD；当它展示无来源建议时，你能指出这违反 ACCEPTANCE；当它把“待确认草稿”做成系统结论时，你能指出这违反 DESIGN。文件的价值不是让 Codex 瞬间变聪明，而是给人和 Codex 一个共同的事实来源。

PRD 和 DESIGN：对实现路径影响最大的一组文件

2.3 已经讲过 `PRD.md` 的职责和写法，2.4 已经讲过 `DESIGN.md` 的职责和写法，本节不重复。到了实战里，判断一份 PRD 是否够用，要看它能不能改变 Codex 的下一步行动。如果 Codex 读不读这份 PRD 都会写出差不多的页面，那它还不够具体。

PRD 对 LeadFlow 实现路径影响最大的一节是 `数据与 API 策略`。没有它，Codex 可能为了“完成 AI 建议”直接在前端写死几条建议，或者把 `VITE_OPENAI_API_KEY` 放进浏览器环境；有了它，`fixture` 数据、`mock generator`、`server-side adapter` 和未配置态各自的职责才能分清。

LeadFlow 的 `DESIGN.md` 里，AI 建议区需要专门覆盖状态链、建议卡片交互，以及 `fixture` / `live` / 未配置三种数据模式的界面表达——CRM 工具里，销售对 AI 建议的信任程度，直接取决于界面能否把“待确认草稿”和“客户事实”区分开来。

组件

****AI 建议区状态****（CRM 工作台的核心状态链）

- 空态：无线索或无记录时引导，不显示假数据
- 加载态：骨架屏，布局不跳动
- 信息不足态（not_ready）：记录不足以生成建议，明确说明，不补通用话术
- 模拟态（mocked）：区域内显示「示例 AI 建议」标识
- 未配置态（live_unconfigured）：显示未配置提示，不展示任何建议内容
- 生成中（generating）：骨架屏保持建议区布局稳定
- 待确认态（draft）：草稿样式，带来源入口，视觉权重低于客户事实区
- 已确认 / 已拒绝态：与草稿态明确区分
- 错误态：明确提示 + 重试入口

****建议卡片交互****

- 主操作文案：「确认为跟进计划」，不写成「发送给客户」
- 次要操作：「编辑建议」「拒绝建议」「查看来源」
- 来源入口靠近建议正文，展开后显示关联沟通记录，不藏进 tooltip
- 确认前建议不得出现在「已执行跟进」时间线

****数据与演示****

- MVP 使用 FIXTURE 数据（`src/fixtures/`）；示例线索显示「示例数据」角标
- mock 模式下 AI 区域显示「示例 AI 建议，未调用真实模型」
- Key 未配置时显示「真实 AI 未配置，当前可使用示例建议预览流程」
- 真实 API 调用失败时展示失败状态和重试入口，不生成假建议兜底

先用脱敏示例数据跑通流程，再考虑真实接入

从 0 到 1 做 MVP，最容易犹豫的是：要不要一开始就接真实 AI 模型，要不要连真实 CRM，要不要上传真实客户数据。我的建议是先把这个问题拆开。

MVP 完全可以用开发者自己写好的一批脱敏示例数据来驱动页面——比如几条假线索、几条假沟通记录，AI 建议也用预先写好的示例返回，不真正调用 OpenAI 或任何模型。这不是偷懒，而是主动选择一个可控的开发世界：数据稳定、流程可重复、任何人打开都能复现同样的页面状态。

这时候需要想清楚四件事：

- 示例数据要单独存放，线索、沟通记录和来源标记都有稳定内容，每次运行结果一样，不依赖手动填写；
- 示例 AI 建议和未来真实 AI 建议要用同一套接口格式，这样等以后接入真实模型时，页面不需要重写；

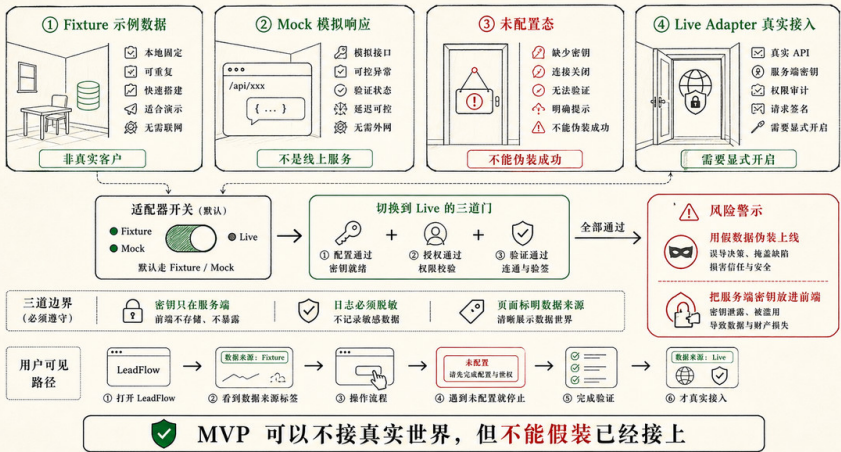
- 如果未来需要真实 AI Key，Key 只能放在服务端读取，不能直接写进前端代码或暴露给浏览器；
- 项目里留一个 `.env.example` 文件，把需要配置的项写清楚，让后来的人知道真实模式需要什么。

你可以让 Codex 在开发时遵守四条规则：

1. 默认用示例数据完成 MVP，不依赖真实外部服务。
2. 如果接入真实 AI，API Key 只能放在服务端环境变量中。
3. 没有配置 API Key 时，页面必须显示"未配置"状态，不能假装已经调用了真实模型。
4. 示例 AI 建议必须带来源标记，并且在页面上明确标识为"示例建议"。

这几条规则会让 MVP 更诚实。很多团队觉得用示例数据是"假的"，于是急着接真实 API；但真正危险的不是用示例数据，而是用了示例数据却没有说明——让页面看起来像在调用真实模型。一个清楚标识的示例世界，比一个半真半假的状态更适合 0 到 1：前者能让你验证信息架构、来源追溯、人工确认和各种失败状态；后者很容易把 Key 管理、权限、日志、隐私和不稳定的模型输出一起引入首版，让项目还没跑起来就被复杂度拖住。

诚实的 MVP：每一种“数据世界”都要被明确标注



把 PRD 和 DESIGN 交给 Codex

到这里，才轮到 Codex 写代码。这个顺序很重要：不是人把实现细节都规定完，而是人先把产品和设计边界放进项目，再让 Codex 自主选择实现路径。

你可以在项目根目录准备好 PRD.md 和 DESIGN.md，然后这样交给 Codex：

我们要从 0 到 1 完成 LeadFlow MVP。

请先阅读根目录的 PRD.md 和 DESIGN.md。不要额外引入本轮用不到的复杂流程或外部脚手架。你可以根据当前项目状态创

本轮目标：

1. 完成 LeadFlow MVP，可以在本地运行。
2. 使用 fixture/mock 数据展示线索列表、客户详情、沟通记录、AI 摘要、下一步建议、来源展开和人工确认。
3. 预留真实 LLM API 接入边界，但默认不依赖真实 API。
4. 没有 API Key 时必须显示未配置态，mock 建议必须标识为示例。
5. 不接真实 CRM，不处理真实客户敏感信息，不自动发送消息。

实现要求：

- 如果已有前端脚手架，沿用当前技术栈。
- 如果当前目录为空，请创建一个最小 React + TypeScript + Vite + Tailwind 项目。

- 把示例数据放在 fixtures 或等价目录。
- 把 mock suggestion 和未来 live suggestion 的接口分开。
- 添加 README，说明启动方式、mock/API 模式、已完成范围和未完成边界。

完成后请运行必要命令，例如 npm install、npm run dev、npm run build 或项目已有检查命令。最后汇报修改文件、启动方式

这个提示有几个设计点。第一，它要求 Codex 先读 PRD.md 和 DESIGN.md，而不是直接写代码。第二，它没有要求 Codex 使用某个特定脚手架，因为第四章要讲的是可迁移的工程委托方式，不是某个仓库的专属流程。第三，它明确 mock 和 API 的关系，避免 Codex 为了"更完整"直接接入真实外部服务。第四，它把 README 也列进交付物，因为 MVP 做出来以后，下一轮人和 Codex 都需要知道怎么运行、现在是什么模式、还有什么没有做。

如果 Codex 反过来问你"是否需要真实 API Key""是否要接真实 CRM""是否要加登录权限"，不要顺着它把范围越聊越大。用 PRD 里的边界回答它：

本轮先不接真实 CRM，不做登录权限。

真实 LLM API 可以预留服务端接口，但默认使用 mock。

请优先完成 fixture/mock 模式下的完整 MVP，并把 live 未配置态做出来。

这就是人和 Codex 的分工。Codex 可以决定文件结构、组件拆分、状态管理和样式实现；人要守住产品边界、数据边界和风险承受度。

MVP 跑起来之后

一个 MVP 做出来，产品从你脑子里的想象，变成了一个可以打开、可以点击、可以观察出问题的工程对象。你可以走通这条最小闭环：

打开应用

- > 看到示例线索列表
- > 点击线索进入客户详情
- > 查看客户事实和沟通记录
- > 看到 AI 摘要和下一步建议
- > 展开建议来源
- > 确认、编辑或拒绝建议
- > 看到示例数据 / 未配置 / 失败 / 无记录等边界状态

项目里通常会包含这些东西：

- 一个可运行的本地 Web App。
- 一组脱敏示例线索和沟通记录。
- 线索列表、客户详情、沟通记录时间线。
- AI 摘要和下一步建议卡片，带来源展开。
- 建议的草稿、已确认、已拒绝、信息不足、未配置等显示状态，在页面上清楚区分。
- 示例 AI 建议在页面上明确标识为"示例"，不伪装成真实模型输出。
- 真实 AI 的预留接入位置，且 API Key 不出现在浏览器端代码里。
- README 说明如何运行、当前用的是示例数据还是真实模式、还有什么没做。

这里的"完成"是 MVP 意义上的完成。它不代表 LeadFlow 已经可以接入真实客户数据，也不代表 AI 建议已经具备生产级准确性，更不代表销售可以对客户自动发送消息。它代表的是：产品的最小价值链已经跑通，数据和边界是诚实的，用户能在一个可见界面里理解客户事实、AI 建议、来源和人工确认之间的关系。

项目完成后大致长什么样

完成后，项目目录可能长成这样：

```
PRD.md
DESIGN.md
README.md
package.json
src/
  fixtures/
    leads.ts
  lib/
    ai/
      types.ts
      mockSuggestion.ts
      liveSuggestion.server.ts # 如当前技术栈支持服务端
  components/
    LeadList.tsx
```

```
LeadDetail.tsx
CommunicationTimeline.tsx
AiSuggestionCard.tsx
App.tsx
```

这个结构只是示例，不是标准答案。如果你用 Next.js、Remix、SvelteKit 或已有公司脚手架，目录会不同。判断标准不变：PRD 说明目标和边界，DESIGN 说明界面状态，示例数据保证流程可复现，真实 AI 的接入位置在哪里预留，README 说明后来的人和 Codex 如何继续。

4.2 任务进入：从反馈、问题和评论进入下一轮 TODO

MVP 跑起来以后，反馈就来了。问题在于，反馈本身还不是任务。“AI 建议看起来太像结论”“卡片再高级一点”“顺便把邮件也发出去”，这些话有价值，但直接扔给 Codex，结果往往是功能越加越多、边界越来越散，哪些是当前必须修复的问题，哪些只是界面观感，没有人说得清。

本节要做的是任务进入：先把反馈整理成可以执行、可以验证、也可以拒绝的 **TODO.md** 条目，让 Codex 不再根据“感觉”继续写，而是对着清楚的任务边界动手。

为什么不直接把反馈说给 Codex？因为 Codex 会立刻开始实现，而没有人先判断这条反馈该改什么、改多少、是否属于当前 MVP。TODO 的作用不是增加流程，而是在实现之前，先让任务有一个可以被拒绝、可以被拆小、可以被验证的形式。没有这一步，反馈就直接变成代码，边界在不知不觉中改掉了。

反馈先不要急着实现

MVP 跑起来以后，读者最常遇到的第一类反馈通常很短：

AI 建议看起来太像结论了。

这句话有价值，但它还不是一个工程任务。它可能意味着产品语义错了：AI 建议被写得像客户事实，而不是销售草稿。它也可能意味着设计状态不清楚：卡片没有把“待确认”显示出来。它还可能只是文案问题：建议标题叫“最佳下一步”，给人一种系统已经替销售做决定的感觉。三种解释对应不同修改范围，也对应不同验收方式。

如果直接让 Codex “优化一下 AI 建议”，它很可能会做出一组表面合理的改动：文案更销售化，按钮更醒目，甚至加一个“自动发送邮件”的动作。可这不一定解决问题，反而可能把 LeadFlow 的边界推坏。第二章已经说过，AI 建议在销售确认前只能是草稿，不能冒充客户事实，也不能自动对客户作出承诺。任务进入阶段，就是要把这种隐含边界重新拉到桌面上。

一个更好的进入方式，是先把反馈改写成判断问题：

反馈：AI 建议看起来太像结论。

需要判断：

1. 是否把 AI 推测和客户事实混在了一起？
2. 是否缺少“待确认 / 草稿”状态？
3. 主按钮或标题是否暗示系统已经替销售做决定？
4. 是否需要同步更新 DESIGN.md 的状态表达或 ACCEPTANCE.md 的验收场景？

这一步的价值不在于放慢节奏，而在于防止开发跑偏。每条反馈进代码之前，先确认它该落的位置：是 PRD 的产品判断、DESIGN 的状态表达，还是 ACCEPTANCE 的验收场景。

任务来源不只来自聊天

在 Codex app 这样的工作环境里，下一轮任务有很多入口。聊天里的用户反馈只是其中一种，甚至不一定是最准确的一种。

第一种是用户直接反馈。比如你看完 LeadFlow 以后说：“这个建议卡片太像系统结论了，应该更像销售待确认的草稿。”这类反馈来自人的产品判断，通常需要先回到 PRD、DESIGN 或 ACCEPTANCE 看边界是否已经存在。如果边界已经写清楚，那它就是实现或设计表达没有做到；如果边界没有写清楚，那它不应该只改代码，还要补文档。

第二种是浏览器里的页面反馈。Codex app 的内置浏览器可以打开本地开发页面，页面上出现的问题不再只能靠口头描述。你可以围绕某个区域给出反馈，例如“这里的来源入口太弱，销售不会点开”，或者“这个未配置态看起来像错误崩溃，不像可预期的 API 未配置”。这类反馈的价值在于它绑定了具体界面位置，比“优化一下详情页”更容易变成可执行任务。

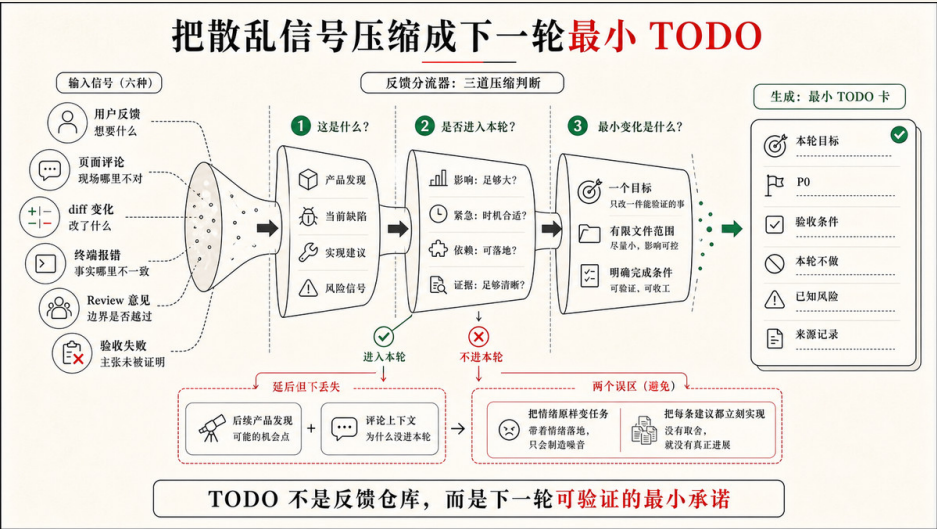
第三种是 diff 或 review 里的评论。Codex app 的 diff 面板显示当前项目或 worktree 的 Git diff，用户可以对具体改动留下行内评论，也可以按文件或修改块暂存或撤回。放到 LeadFlow 里，行内评论可能是：“这里把 mock 建议写死在组件里，会绕过 mock generator 和 sourceIds。”这种反馈不是产品感觉，而是针对具体实现边界的审查意见。

第四种是终端暴露的问题。内置终端绑定当前项目或 worktree，适合运行 `npm run dev`、`npm test`、`npm run build`、`lint` 或项目自带检查。终端错误不是噪音，它常常是下一轮任务的入口：类型检查失败说明数据模型和组件不一致，测试失败说明验收场景没有满足，dev server 起不来说明环境或依赖有问题。Codex 可以读取终端输出，把失败摘要带回下一步判断。

第五种是 `/review` 或人工审查发现的缺陷。`/review` 可以进入代码审查模式，检查未提交改动或和基准分支的差异。它不替代人的最终判断，但能帮你把“能跑”背后的风险显影出来：有没有行为回归，有没有缺少测试，有没有隐私字段进入日志，有没有把真实 CRM 接入悄悄拉进来。

第六种是 `TODO.md` 和 `ACCEPTANCE.md` 里的未完成项或失败场景。比如 4.1 的 MVP 已经把来源显示做出来了，但 `ACCEPTANCE.md` 里的“无沟通记录时不得生成通用建议”仍然失败。这个失败本身就是下一轮任务，不需要重新发明需求。

这些入口的共同点是：它们都不是代码本身，却都可以改变下一步开发。任务进入阶段要做的，就是把它们从“散落的信号”变成“可执行的任务”。



先判断它应该落到哪里

一条反馈进入项目以后，先不要问“要改哪个文件”，而要问它属于哪类变化。这个判断会决定 Codex 应该读取哪些上下文、更新哪些状态，以及本轮是否应该继续。

LeadFlow 的反馈可以用一张分流表判断落点：

表 4-1 LeadFlow 反馈分流对照表

反馈或问题	更像什么	应该先落到哪里
“AI 建议太像客户事实”	产品语义和设计状态问题	DESIGN.md、ACCEPTANCE.md、本轮 TODO.md
“来源入口太隐蔽”	UI 交互和设计验收问题	DESIGN.md、本轮 TODO.md

反馈或问题	更像什么	应该先落到哪里
“没有沟通记录还生成建议”	验收失败和实现缺陷	ACCEPTANCE.md、本轮 TODO.md
“把建议自动发邮件给客户”	可能改变产品范围	PRD.md 待决策或后续 TODO，本轮通常拒绝
“真实 CRM 数据也接进来”	数据边界和权限变化	PRD.md、CONTEXT.md、权限确认，通常另开任务
npm run build 失败	工程缺陷	本轮 TODO.md，附终端失败摘要
diff 中出现无关重构	范围漂移	review comment、本轮 TODO 或要求回退

这个表不是流程规范，而是提醒：同样一句反馈，可能进入不同事实源。低风险文案可以直接成为小 TODO；涉及 AI 建议来源、销售确认、真实数据、权限和对外动作的反馈，必须先回到产品边界和验收场景；明显越过 MVP 非目标的请求，要敢于拒绝或延期。

Agentic Coding 里的“拒绝”不是对用户说不，而是把任务放回正确位置。比如“希望自动给客户发跟进邮件”是一个真实的产品想法，但它不属于当前 MVP。4.1 的 PRD 已经明确不自动发送邮件、短信或外呼。如果本轮顺手实现，LeadFlow 的责任边界就被一次反馈悄悄改写了。更合理的处理是：

```
## 后续产品发现

- [] 评估“AI 建议转跟进邮件草稿”的产品范围
- 来源：用户反馈希望自动给客户发邮件。
- 当前判断：不进入本轮；不能自动发送。
- 需要决策：是否只生成邮件草稿、是否需要审批、是否记录来源、是否接入真实邮箱。
```

这样做既没有丢掉用户想法，也没有让本轮任务越界。项目继续前进，但边界没有被偷偷改掉。

把反馈压成本轮最小变化

假设现在有三条反馈同时出现：

1. AI 建议太像结论，销售容易误会已经可以执行。

2. 来源入口太弱，看不出建议来自哪条沟通记录。

3. 没有沟通记录的线索仍然显示了一条通用建议。

它们看起来都指向 AI 建议卡片，但不应该被写成“重做 AI 建议区域”。这个任务太大，容易把文案、状态、数据模型、设计系统、测试和布局全部卷进去；Codex 可能越改越多，人也很难 review。

更好的做法，是把它们压成一个本轮最小变化：

本轮目标：修正 AI 建议卡片的可信边界。

最小变化：

1. 建议必须显示为待确认草稿，而不是客户事实或已执行结论。
2. 来源入口靠近建议正文，并能展开对应沟通记录。
3. 无沟通记录时显示信息不足状态，不生成通用建议。

这三个变化属于同一条可信边界，彼此有关，也足够小。它们会影响 UI、状态和生成逻辑，但不会引入真实 CRM、邮件发送、多角色权限或生产级模型接入。Codex 可以围绕它们读取 `DESIGN.md`、`ACCEPTANCE.md`、相关组件和测试，而不是把整个 LeadFlow 重新设计一遍。

落到 `TODO.md` 里，可以这样写：

```
# TODO.md

## 本轮目标

修正 LeadFlow MVP 中 AI 建议卡片的可信边界：建议必须以待确认草稿呈现，来源入口可见，无沟通记录时不得生成通用建

## P0

- [] AI 建议以待确认草稿呈现
  - 来源：用户反馈“AI 建议太像结论”。
  - 设计：见 `DESIGN.md##AI 建议卡片状态`、`DESIGN.md#建议卡片交互`
  - 验收：见 `ACCEPTANCE.md#销售确认前不得标为已执行`
  - 完成条件：建议标题、状态徽标和主按钮都表达“待确认”，确认前不进入已执行记录。
  - 状态：未开始
```


- [] 来源入口靠近建议正文并可展开
 - 来源：浏览器反馈“来源入口太弱”。
 - 设计：见 `DESIGN.md#建议卡片交互`
 - 验收：见 `ACCEPTANCE.md#AI 建议必须显示来源`
 - 完成条件：每条建议能展开 sourceIds 对应沟通记录；来源缺失时不显示为可信建议。
 - 状态：未开始
-
- [] 无沟通记录时显示信息不足状态
 - 来源：验收失败“无记录线索仍显示通用建议”。
 - 验收：见 `ACCEPTANCE.md#沟通记录不足时不生成建议`
 - 完成条件：无记录 fixture 打开后不生成客户需求、预算判断或通用销售话术。
 - 状态：未开始

本轮不做

- 不接真实 CRM。
- 不自动发送邮件或消息。
- 不改变销售阶段判断模型。
- 不做句子级引用。
- 不引入新的 UI 框架或重做整体布局。

已知风险

- 如果当前 fixture 缺少 sourceIds，需要先补齐脱敏样例或明确来源替代字段。
- 如果现有组件把建议和已执行记录共用同一数据结构，可能需要先拆出 confirmationStatus。

这份 TODO 已经能改变 Codex 的执行方式。它给 Codex 划定了三条边界：任务是修正可信边界，不是美化卡片；改动范围是三个可观察结果，不是整张详情页；五件事明确放在本轮不做之外。

用评论保留上下文，而不是只保留情绪

评论的价值在于定位。一个好的页面评论或代码评论，能把人的判断绑定到具体对象上；一个坏的评论，只是把情绪换了一种写法。

在浏览器里看到 LeadFlow 页面时，你当然可以说“这里不对”。但更有用的评论是：

AI 建议卡片标题叫“推荐行动”，会让销售误以为这是系统结论。
请改成待确认语义，并确认按钮文案不能暗示已经发送或执行。

这条评论包含了位置、问题、原因和边界。Codex 读到它以后，知道要检查卡片标题、状态和按钮文案，也知道不能把主操作写成“发送给客户”。它仍然可以自己决定具体实现，但不会误解任务方向。

diff 里的行内评论也一样。不要只写“这里不好”，而要指出它破坏了什么工程条件：

这段 fallback 会在 sourceIds 为空时仍然展示建议。
这会违反 ACCEPTANCE.md 的“无来源不生成建议”场景。
请改成信息不足状态，并补对应 fixture 或测试。

这样的评论天然可以进入 TODO，因为它已经包含了验收指针和最小修复方向。它也能帮助 Codex 在修改时保持范围：修 fallback，不是重写整个建议系统。

终端错误也要翻译成任务

终端输出经常被当成临时故障：命令失败了，让 Codex 修一下。但在 Agentic Coding 里，失败输出也是项目状态的一部分。它告诉你当前世界和预期世界之间哪里对不上。

假设 4.1 之后运行 `npm run build`，出现这样的错误：

```
Type error: Property 'sourceIds' is missing in type 'Suggestion'
```

这不是单纯的 TypeScript 麻烦。它说明项目里至少存在两套关于 AI 建议的数据理解：一套验收和 UI 要求建议带来源，另一套类型或 fixture 还没有这个字段。如果 Codex 只为了通过构建把 `sourceIds?: string[]` 改成可选，然后 UI 遇到缺失来源时仍然显示建议，构建会绿，产品边界却坏了。

所以，终端错误进入 TODO 时，不应该只写：

```
- [] 修复 build 报错
```

更好的写法是：

```
- [] 对齐 Suggestion 类型、fixture 和 UI 的来源字段
- 来源：`npm run build` 失败，提示 `sourceIds` 缺失。
- 背景：AI 建议必须保留来源；缺少来源时不能显示为可信建议。
- 完成条件：类型、mock generator、fixture 和 UI 使用同一套来源字段；无来源时进入信息不足或来源缺失状态。
- 验证：`npm run build`；有来源和无来源两个页面场景。
- 状态：未开始
```

这就是把错误翻译成任务。命令失败给出的是症状，TODO 要记录的是工程状态该如何恢复。这样下一轮 Codex 不会为了“让命令通过”牺牲产品语义。

`/status`、`/review` 和 `worktree` 是入口辅助，不是任务本身

工具命令也容易被误解。比如 `/status` 可以显示线程、上下文和限制信息；`/review` 可以进入代码审查模式；Local / Worktree / Cloud 模式可以决定任务在哪里运行，Worktree 还能把修改隔离在单独 Git worktree 里。这些能力很有用，但它们本身不是任务定义。

什么时候看 `/status`？当任务跑了很久、上下文变重、你担心 Codex 已经读了太多东西或需要切分任务时，它可以帮助你判断是否该收束当前轮，或者把剩余问题写进 TODO 后另开一轮。

什么时候用 `/review`？当 MVP 已经能跑、diff 也不小，你需要从“实现者视角”切到“审查者视角”时。`/review` 给出的反馈不应直接被全部实现，而要先分流：哪些是 P0 缺陷，哪些是后续改善，哪些和当前目标无关，哪些需要人判断风险。

什么时候开 `worktree`？当反馈可能需要探索，但你又不想污染当前稳定 MVP 时。比如“真实 CRM 同步不可行”“AI 建议能否升级到邮件草稿”“是否要重构建议数据模型”，都可能适合放到隔离 `worktree` 里做 spike。但修一个明确的来源入口或无记录状态，不一定需要另开 `worktree`。工具选择仍然服务于任务风险，而不是为了显得流程高级。

让 Codex 帮你整理入口

任务进入不一定都要人手写。更实际的协作方式，是让 Codex 先读取反馈、页面评论、终端错误、diff comment 和已有 TODO，再帮你整理一版任务入口。人负责确认边界，Codex 负责把散落信号压成结构化 TODO。

可以这样发起：

LeadFlow 现在要进入下一轮迭代。

请先读取 PRD.md、DESIGN.md、TODO.md、ACCEPTANCE.md，以及当前未提交 diff 和最近终端输出。

我会给你几条反馈：

1. AI 建议看起来太像结论，不像销售待确认草稿。
2. 来源入口太弱，用户不容易知道建议来自哪条沟通记录。
3. 无沟通记录的线索仍然显示通用建议。

请先不要改代码。先完成任务进入整理：

- 判断每条反馈属于产品范围、设计状态、实现缺陷、验收缺口、后续想法还是应拒绝范围。
- 给出本轮最小可交付目标。
- 更新或起草 TODO.md 中的本轮任务，包含来源、设计/验收指针、完成条件、本轮不做、已知风险。
- 如果需要更新 DESIGN.md 或 ACCEPTANCE.md，请说明原因并只给出最小修改建议。

这个提示的关键是“先不要改代码”。Codex 当然可以自行推进，但任务入口阶段要先产出下一轮执行地图。地图不清楚，后面执行越积极，偏得越远。

整理完成后，人要检查三件事：

1. 本轮目标是否真的最小。
2. 是否把超出本轮目标的想法挡在本轮之外。
3. 每个 TODO 是否有可观察完成条件和验收指针。

如果这三件事都清楚，才进入 4.3 的开发执行。

从 4.2 交棒给 4.3，至少要确认四件事

4.2 不是一节“如何写任务清单”的理论。它的产物是 LeadFlow 下一轮开发已经有了可执行入口。具体来说，至少要确认四件事。

第一，反馈来源被记录。它可能来自用户聊天、浏览器评论、diff 行内评论、终端错误、*/review* 结果，或者失败的 acceptance 场景。记录来源不是为了形式完整，而是为了让后来的人知道这项任务为什么出现。

第二，本轮目标是一个具体的状态变化。不要把“优化 AI 建议区域”写成一个大纲，而要定义出一个可以交付的结果，例如“修正 AI 建议卡片的可信边界：待确认草稿、来源展开、无记录不生成建议”。

第三，边界被写清。当前不做真实 CRM、不自动发送邮件、不做句子级引用、不重做整体布局，这些不是附带说明，而是保护本轮 diff 不失控的围栏。

第四，完成条件已经可检查。每个 TODO 至少要有设计或验收指针、可观察结果、最近失败或风险，以及需要运行或观察的证据类型。它不一定提前写成完整测试，但必须让 Codex 知道怎样算完成，怎样算不该继续。

到这里，LeadFlow 的开发 loop 才真正准备好进入实现。4.1 让产品从想法变成可运行 MVP；4.2 让反馈从散落信号变成下一轮 TODO。下一节才轮到 Codex 执行这些任务，但我们看的重点不是它点了哪个按钮、跑了哪条命令，而是命令反馈、页面状态和 diff 范围这三类信号是否证明它仍然走在本轮边界内。

4.3 开发执行：三类信号，如何知道 Codex 走在正确路上

4.2 已经把反馈压成了本轮 TODO：修正 AI 建议卡片的可信边界。接下来很多人会把 `TODO.md` 直接交给 Codex，说“按这个做完”。这句话看起来很自然，但它把最重要的控制点藏掉了：人要如何知道 Codex 真的在完成本轮任务，而不是在一段看似流畅的执行里扩大范围、绕过验收、或者把未来功能提前做进来？

这一节不把重点放在“怎样打开终端、怎样点击浏览器、怎样看 diff”。这些动作本身容易误导方向——操作会了，判断未必跟上。真正需要学的是三类信号：

命令反馈：代码在机器层面是否站得住
页面状态：改动在用户现场是否成立
diff 范围：Codex 是否还在本轮委托边界内

权限提示、Skills 和 Rules 也会出现，但它们都服务于同一个问题：这次工程委托是否仍然受控。

先说清楚：这和普通 AI 改代码有什么不同

换成 Codex 之后，有一个很合理的疑问：分析任务、写代码、跑测试、看页面、查 diff，这不就是平时开发流程吗？为什么还要专门讲一章？

差别不在这些动作本身，而在动作的优先级变了。普通开发里，你看 diff，主要是在 review 自己或同事写的代码质量；Agentic Coding 里，你先看 diff，是为了确认委托边界有没有被遵守。普通开发里，你跑命令，是为了确认实现是否正确；Agentic Coding 里，你还要看 Codex 是否把一个命令通过误报成“验收完成”。普通开发里，页面看起来对了通常是好消息；Agentic Coding 里，页面看起来对了以后，还要确认它不是靠假数据、硬编码、或者越界功能撑出来的。

更本质的差别，是文件系统给 Codex 提供了可对照的事实来源。没有 PRD.md、DESIGN.md、TODO.md 和 ACCEPTANCE.md，Codex 收到“完善 AI 建议卡片”时，可能会按常见 SaaS 直觉做一个漂亮的推荐卡片，顺手加发送按钮，甚至把真实 CRM 接入也当成“完善”。有了这些文件，本轮任务就不再是一句模糊要求，而是一组可检查的边界：

PRD.md	不自动发送，不接真实 CRM，不处理真实客户数据
DESIGN.md	AI 建议是待确认草稿，来源入口靠近正文
TODO.md	本轮只修可信边界：草稿、来源、无记录不建议
ACCEPTANCE.md	用具体场景证明来源、无记录、确认前状态和隐私日志

这不保证 Codex 永远不会跑偏。它改变的是可观察性：当 Codex 准备加发送按钮时，人可以指回 PRD；当它把无记录状态写成通用话术时，可以指回 ACCEPTANCE；当它想重做整张详情页时，可以指回 TODO。Agentic Coding 的价值，不是让 AI 第一次就全对，而是让“跑偏”能被早一点看见、说清楚、拉回来。

工具说明：零代码读者先看什么 你不需要先学会所有命令或 Git 细节。先记住三件事：命令结果告诉你程序有没有明显错误；页面状态告诉你用户看到的東西是否合理；diff 告诉你 Codex 改了哪些文件，是否改过头。你要学的是判断，不是背命令。

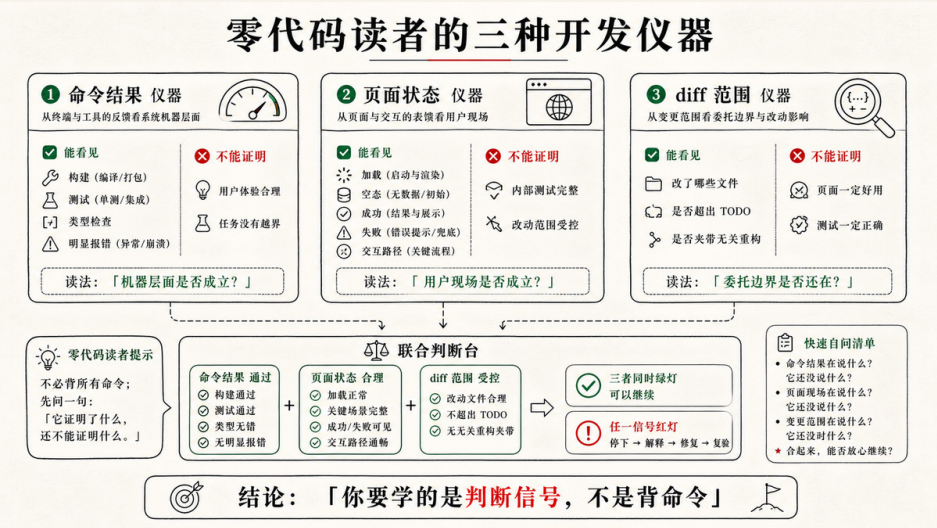


图 4-1 工具说明：零代码读者先看什么

委托颗粒度：什么可以让 Codex 连续做，什么必须停下来

"小步迭代"不是永远让 Codex 每改一行都汇报。真正的问题是任务颗粒度，这比"跑什么命令"更值得想清楚。

一项任务可以让 Codex 相对连续地做，通常满足五个条件：目标只对应一个本轮 TODO；验收场景已经写清；改动主要发生在当前工作区；失败可以本地复现和回退；不涉及真实数据、外部服务、费用、权限或对用户的真实动作。

LeadFlow 的"AI 建议卡片可信边界"基本符合这个条件。它会改类型、fixture、mock generator、组件和测试，但这些变化都围绕同一条用户可见纵切：销售看到的 AI 建议必须仍然是带来源、待确认、可拒绝的草稿。

需要分段介入的任务，通常有这些特征：同时改数据模型、核心状态、共享组件和验收文件；diff 会跨很多模块；失败原因不容易从本地命令看出；Codex 开始提出新增依赖、外部 API、登录权限、真实 CRM 或自动外发；或者本轮 TODO 里的"完成条件"本来就很含糊。

必须留给人判断的，不是代码细节，而是产品和风险边界：要不要接真实客户数据，要不要真的调用模型，要不要把建议变成邮件草稿，要不要进入销售已执行记录，要不要扩大 MVP 范围。这些决定不能因为 Codex 觉得“顺手”就进入实现。

可以把本轮委托写成这样：

请按 TODO.md 执行本轮 P0，但每一小步都要能停下来检查。

本轮目标是修正 AI 建议卡片可信边界：

1. AI 建议以待确认草稿呈现。

2. 来源入口靠近建议正文，并能展开对应沟通记录。

3. 无沟通记录时显示信息不足状态，不生成通用建议。

请先阅读 PRD.md、DESIGN.md、TODO.md、ACCEPTANCE.md，以及相关建议组件、fixture、mock generator 和测试。

先给出最小开发计划，说明每一步对应哪类信号：命令反馈、页面状态或 diff 范围。

这个提示没有让人接管实现，它只是把 Codex 的自主执行放进了可停止的检查点里。

工程师视角：小步不是低效，而是降低回滚成本 小步的价值不在"慢慢写"，而在保持 diff 可审查、失败可定位、产品边界可恢复。Agentic Coding 里最贵的不是让 Codex 多跑一轮，而是最后面对一个看似完成但无法解释的巨大 diff。

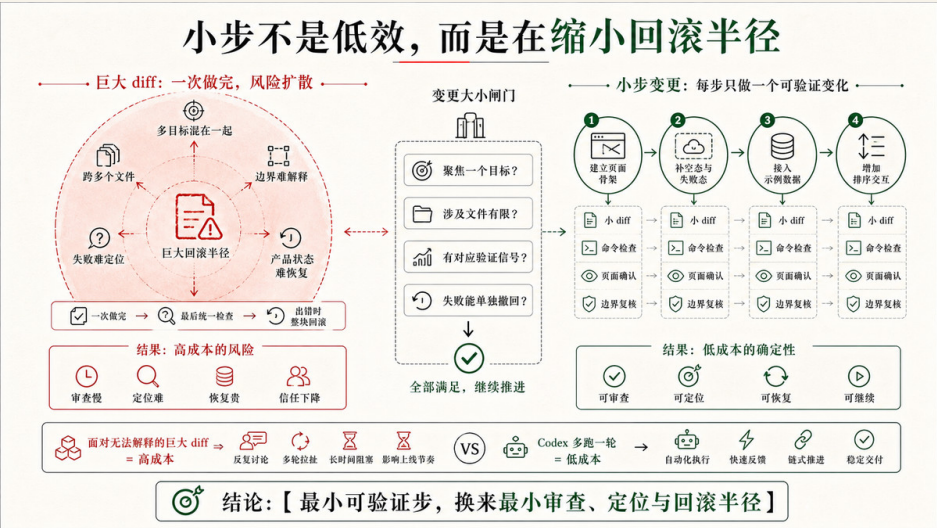


图 4-2 工程师视角：小步不是低效，而是降低回滚成本

小步计划要和信号对应

好的开发计划不需要长，但要能回答三个问题：第一，先改哪条最小纵切；第二，用什么证据知道这一步没坏；第三，什么时候人应该介入。

对 LeadFlow 这轮任务，一个合理计划可以是：

1. 对齐 suggestion 类型、fixture 和 mock generator。
信号：类型检查 / 相关测试；diff 是否集中在数据和生成逻辑。
2. 更新 AI 建议卡片 UI。
信号：浏览器里有记录、无记录、来源展开三条页面状态；diff 是否只触及建议卡片和必要样式。
3. 补确认交互。
信号：确认前不进入已执行记录；确认后只进入本地待跟进计划；不出现发送、外呼或 CRM 写回。
4. 运行项目已有检查。
信号：build、test 或 lint 的结果；失败时说明失败和修复范围。
5. 查看本轮 diff。
信号：未引入真实 CRM、自动外发、新 UI 框架或无关重构。

这类计划不是流程表演。它把每一步的完成感交给外部信号，而不是交给 Codex 的自我描述。Codex 可以自主选择组件拆分、状态管理和样式实现，但每一步都要回到同一个问题：AI 建议是否还保持来源、草稿和人工确认的责任边界。

信号一：命令反馈证明机器层面是否成立

命令反馈是最早出现的信号。它能告诉你项目是否还能启动、类型和构建是否明显失败、已有测试是否被破坏。不同项目命令不同，可能是 `npm run build`、`npm test`、`npm run lint`，也可能是项目自己的检查脚本。关键不是把每条命令都跑一遍，而是让 Codex 说明：跑了什么，为什么跑，没跑什么，不能证明什么。

比如 `npm run build` 通过，可以支撑“项目在构建层面没有明显类型或打包错误”。它不能支撑“来源展开正确”，也不能支撑“销售确认流程符合产品语义”。如果当前项目没有测试，Codex 就不能把“未发现报错”写成“已验证”。如果只跑了 build，就只能说 build 通过。

命令失败时，人要看的是 Codex 怎么修。假设类型检查提示某个 fixture 缺少 `sourceIds`，最差的修法是把 `sourceIds` 改成可选，然后让 UI 在缺失时仍然显示建议。这会让命令变绿，却把“建议必须带来源”的产品边界藏掉。更好的修法是回到任务边界：fixture 补来源，无来源时进入信息不足或来源缺失状态，UI 不把它展示为可信建议。

工具说明：终端输出看不懂怎么办 你可以不理解每一行错误。先让 Codex 用一句话解释“这条错误说明哪个工程事实不一致”，再让它说明“修复后用什么命令或页面状态证明”。不要只接受“已修复报错”。

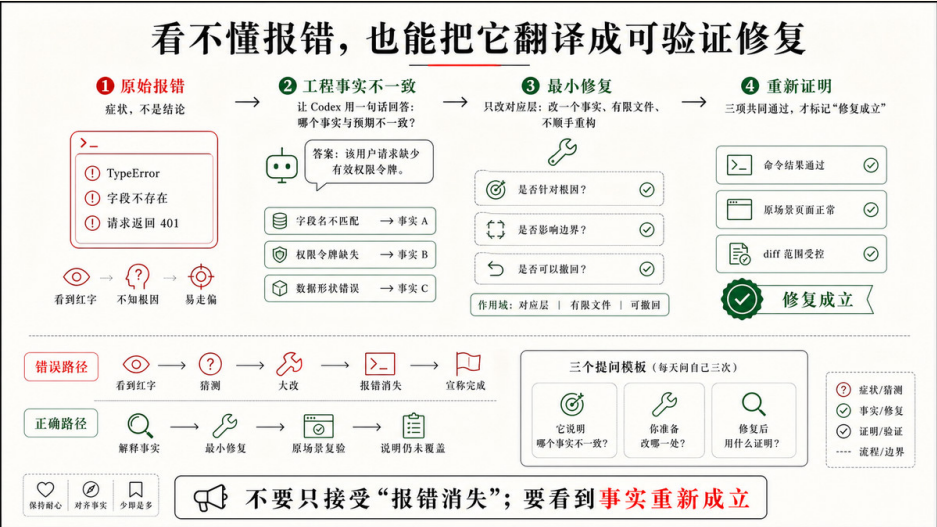


图 4-3 工具说明：终端输出看不懂怎么办

信号二：页面状态证明用户现场是否成立

命令让代码在机器层面站住，页面状态让它回到用户现场。LeadFlow 是销售工作台，很多问题只有打开页面、点过路径才会暴露：来源入口是否靠近建议正文，待确认状态是否清楚，无沟通记录时是否真的不生成建议，确认按钮是否暗示外发，未配置态是否诚实。

对本轮任务，浏览器检查不需要覆盖完整 CRM，只需要围绕验收风险走四条路径：

1. 打开有沟通记录的线索，AI 建议显示为待确认草稿。
2. 展开来源入口，能看到对应沟通记录。
3. 打开无沟通记录的线索，显示信息不足，不出现通用销售话术。
4. 点击确认建议，进入本地待跟进计划，不进入已执行记录，不触发外发动作。

页面状态有两个价值。第一，它能发现测试和类型看不到的问题，例如按钮文案让人误解、来源入口太弱、状态徽标不清楚。第二，它能把人的产品判断变成精确反馈，而不是一句“优化一下”。比如：

来源入口现在离建议正文太远，销售读完建议后不一定会看到。
请把入口放到建议正文下方，并保持“待确认”状态同时可见。

这句话比“卡片再明显一点”更适合交给 Codex。它有位置、有原因、有边界，也没有把任务扩大成重做详情页。

页面检查也有边界。浏览器里点通一次来源展开，不等于所有来源关系都正确；确认按钮肉眼可用，不等于状态流转已经被可重复测试覆盖；本地 mock 页面正确，也不等于真实 CRM 接入正确。这些边界要留到 4.4 的正式验收里写清楚。

信号三：diff 范围证明委托边界是否还在

diff 是 Agentic Coding 中最重要、也最容易被低估的信号。终端告诉你有没有明显错误，页面告诉你看起来是否合理，diff 告诉你 Codex 到底动了哪里。

在普通开发里，diff review 常常先看代码质量。这里要先看范围。对 LeadFlow 本轮任务，合理 diff 应该集中在 suggestion 类型、fixture / mock generator、AI 建议卡片、少量状态流转、相关测试和必要 README / TODO 说明。如果 diff 里出现真实 CRM 接入、邮件发送模块、全局布局重做、新 UI 框架、大规模重命名或权限系统，就要先停下来问：这是不是本轮完成条件必须要求的？

这也是 4.2、4.3、4.4 三处 diff 的分工：

- 4.2 看 diff：把评论和问题整理成下一轮 TODO。
- 4.3 看 diff：检查这一小步是否改过头。
- 4.4 看 diff：验收后封存前，确认最终版本边界干净。

发现越界时，不要等到最后再 review。可以让 Codex 解释这段改动与 TODO 的对应关系；解释不出来，就要求回退或拆到后续任务。一个典型指令是：

这部分 diff 引入了邮件发送路径，不属于本轮“待确认草稿、来源展开、无记录不建议”的完成条件。请回退这部分改动，并把“生成邮件草稿”记录为后续产品发现，不进入本轮实现。

工程师视角：先审委托边界，再审代码质量 Agentic Coding 里的 diff 检查有一个优先级变化：越界比 bug 更紧急。bug 通常还能在本轮局部修复；越界会改变任务本身，让后面的测试、页面检查和提交说明都失去清晰对象。

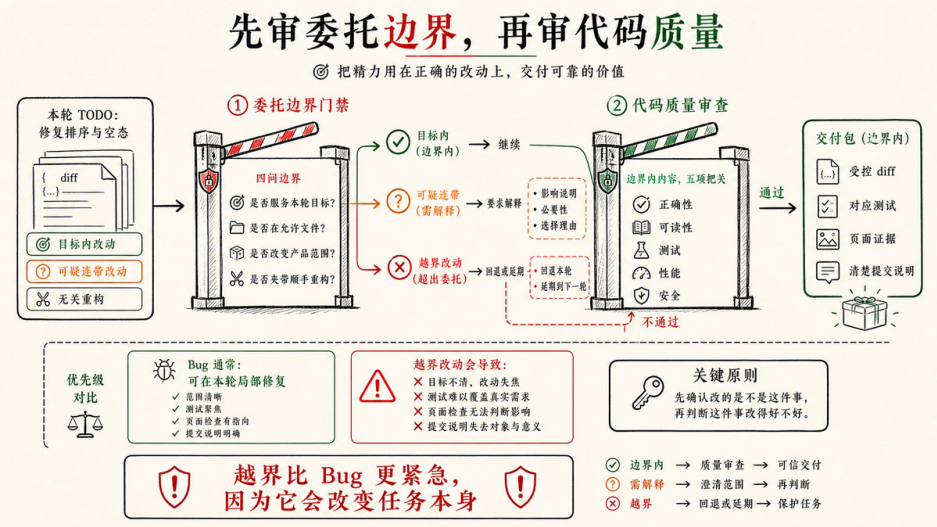


图 4-4 工程师视角：先审委托边界，再审代码质量

权限提示是风险边界，不是打扰

权限提示不属于三类常规信号，它更像一个边界警报。正常本地迭代通常发生在当前工作区：读文件、改组件、改 fixture、跑已有命令、打开本地页面、看 diff。真正值得停下来的，是那些把任务推向外部世界的动作：联网安装依赖、访问外部模型、写工作区之外的文件、读取真实客户数据、调用 CRM 或邮件服务、修改密钥或环境配置。

出现这类提示时，不要条件反射同意，也不要把它当成技术细节。直接问四件事：

1. 这一步是否属于本轮 TODO 的完成条件？
2. 有没有本地替代方式可以验证同一条风险？
3. 授权以后会带来什么副作用：外部请求、费用、日志、真实数据、权限变化？
4. 不授权时，本轮还能完成并诚实声明哪些范围？

比如本轮没有真实 API Key，Codex 不能因此把 mock 建议伪装成真实模型输出。它仍然可以完成 mock / 未配置态、来源展开、草稿确认和无记录状态的验证，只是必须在自检报告里说明：真实模型调用未验证。

早期失控信号：什么时候必须介入

Codex 失控通常不是突然发生的。它会先露出一些早期信号，这些信号比最终 bug 更早出现，也更容易被错过。

下面这些情况一出现，就应该暂停执行、收回边界：

计划开始包含本轮 TODO 没写的未来功能。
Codex 主动提出接真实 CRM、真实 LLM、邮件发送、登录权限等新依赖。
diff 从局部组件扩散到全局布局、路由、权限、数据层或样式系统。
为了让命令通过，Codex 弱化类型、放宽验收条件、删除测试或改写测试语义。
自检报告开始使用“基本完成”“应该可以”“简单验证过”这类模糊说法。
页面看起来更完整，但依赖硬编码数据、假成功态或与 PRD 非目标冲突的动作。
Codex 反复问与当前 TODO 无关的问题，说明任务边界已经被它重新想象。

这些信号不一定说明 Codex 做错了，它们说明人要重新行使控制权。介入方式不需要粗暴打断，可以很短：

请停在当前状态，不要继续扩展。
重新对照 TODO.md 和 PRD.md，列出当前 diff 中哪些改动直接服务本轮目标，哪些超出范围。
超出范围的改动先回退或移入后续 TODO。

这就是控制点上移后的实际样子。人不微操每一行代码，但要在目标、边界、授权、证据和停止条件上保持最终判断。

Skills 和 Rules：不要抢戏，但要进入正确位置

第三章已经讲过 Skills、SubAgents、Rules、Hooks 和 MCP。第四章不需要把这些概念再讲一遍，也不应该为了显得“脚手架完整”临时造一套流程。它们在 4.3 的正确出场方式，是作为任务执行前的短检查：

这次是否修改 AI 建议、来源、销售确认或日志？

-> 如果已有相关 Skill，先按它读取事实源和检查边界。

这次是否触及手机号、邮箱、沟通原文、API 请求或日志？

-> 如果已有隐私日志 Rule，按它检查输出和 diff。

这次是否只是修一个明确 UI 状态或 fixture 缺口？

-> 如果没有相关 Skill / Rule，直接按本节三类信号推进，不临时造脚手架。

这样就承接了第三章，又不会把第四章写成第二遍脚手架介绍。Skills 和 Rules 的价值，是减少重复提醒，提前带入稳定边界。它们不能替代本轮 TODO，也不能替代命令、页面和 diff 的开发信号。

如果执行中发现某条风险反复出现，比如 mock 建议总被写得像真实模型输出，或者调试日志总会打印完整沟通原文，不要在 4.3 立刻扩写一大段规则。先记录到 TODO 或 4.4 的回写判断里。等正式验收和复盘确认它是长期风险，再沉淀到 **AGENTS.md** 或 Rules。

自检报告要说明证据强度

一轮小步开发结束后，Codex 应该给出自检报告。自检报告的价值不是“宣布完成”，而是把当前状态、证据和边界交给别人，让下一步能进入正式验收。

一个合格的 4.3 自检报告可以这样写：

本轮完成：

- AI 建议卡片显示为待确认草稿。
- 来源入口移动到建议正文下方，并可展开对应沟通记录。
- 无沟通记录 fixture 显示信息不足状态，不再生成通用建议。

运行过的检查：

- npm run build 通过。
- suggestion 相关测试通过。
- 本地浏览器检查了有记录、无记录、来源展开、确认建议四条路径。

diff 范围：

- 修改 suggestion 类型、fixture、mock generator、AiSuggestionCard 和相关测试。
- 未改真实 CRM、邮件外发、权限系统和整体布局。

证据边界：

- build 只证明构建层面通过。
- 浏览器检查只覆盖本地 mock / fixture 路径。
- 真实 CRM、真实 LLM 请求日志和生产权限未验证。

仍需 4.4 正式验收：

- 对照 ACCEPTANCE.md 逐条确认来源、无记录、销售确认和隐私日志场景。
- 记录证据和未覆盖项。
- 决定是否回写 TODO / ACCEPTANCE / DESIGN。

注意这里没有把“我点过页面”写成“验收通过”。开发执行阶段的自检可以很有价值，它证明当前改动没有明显坏掉，也暴露了哪些证据已经有、哪些还没有。真正的正式验收、失败 Debug、状态回写和 Git 封存，要交给下一节。

交棒给 4.4 前，至少确认五件事

4.3 的产物不是一个完美功能，而是一次可审查的开发变化，已经完成到可以进入验收。进入 4.4 前，至少确认五件事。

第一，代码变化围绕本轮 TODO。AI 建议卡片、来源展开、无记录状态和确认状态被推进了，diff 没有偷偷引入真实 CRM、自动外发、多角色权限或整体重构。

第二，命令反馈有结论。运行了什么，通过了什么，失败过什么，失败如何处理，都不能只留在聊天情绪里。

第三，页面状态被看过。页面不是只在代码里存在，而是在本地地址上被打开、点击、观察过关键路径。

第四，diff 范围被初步审查。人或 Codex 至少知道哪些文件被改，哪些改动与目标对应，哪些需要回退或留到后续。

第五，自检报告说清证据强度。它区分了开发自检和正式验收，也说明了不能覆盖的风险。

这样，4.3 才没有掉回“AI 在黑箱里写完一大段代码”的老路。4.1 让 MVP 跑起来，4.2 让反馈进入 TODO，4.3 让开发变化被命令、页面和 diff 三类信号持续校准。下一节要做的，是把这些自检结果升级成验收证据，把失败导入 Debug，把状态回写到项目文件，最后用 Git 给这次变化封一个清楚的版本边界。

4.4 验收回写：验证、Debug、Git 封存，再进入下一轮

Codex 给出自检报告之后，很多人会直接接受。但自检报告只说“我刚改的东西大体成立”——它不能证明 `ACCEPTANCE.md` 的场景都被验过，不代表失败路径、隐私边界和设计语义都被收住，也不代表 Git 里没有混入无关重构。下一轮再改时，才发现所谓完成只是聊天里的一个乐观句子。

4.4 不是在重复讲测试、debug 或 commit 怎么做。这些动作本身不难，真正要改变的是验收优先级：先把 Codex 的自检转成可交接证据，再决定是否修复、回写和封存。测试、浏览器、日志、diff 和 Git 都只是证据与版本边界的载体。

本节要把“看起来做完了”变成可以交接的一次工程变化：

自检结果

- > 对照 `ACCEPTANCE` 做正式验收
- > 失败时进入 Debug
- > 最小修复 -> 复验
- > 回写 `TODO / ACCEPTANCE / PRD / DESIGN / CONTEXT`
- > 最终 diff / review
- > stage / commit / push / PR
- > 下一轮任务入口

从自检报告进入正式验收

正式验收的第一步，不是重新跑一遍所有命令，也不是把项目从头点一遍，而是回到 `ACCEPTANCE.md`。4.3 的自检报告已经告诉我们跑过哪些检查、看过哪些页面、diff 集中在哪些文件；4.4 要判断的是：这些证据是否足以支撑验收场景，哪里还缺证据，哪些场景需要补跑。

对 LeadFlow 这一轮来说，验收范围可以先锁在 AI 建议可信边界上，而不是顺手验完整 CRM：

1. 有沟通记录时，AI 建议必须显示来源。
2. 无沟通记录时，不得生成通用建议。
3. 销售确认前，建议保持待确认草稿，不进入已执行记录。
4. mock / API 未配置态必须诚实显示，不伪装成真实模型调用。
5. 手机号、邮箱、合同金额和沟通原文不得进入日志。

这些场景来自第二章的验收文件，不是 4.4 临时发明的检查清单。它们让 Codex 的验收动作有了方向：不是“尽量多测”，而是围绕本轮风险逐条采信。你可以这样要求 Codex 进入验收：

现在进入正式验收，不要继续扩展功能。

请读取 `ACCEPTANCE.md`、`TODO.md`、本轮自检报告和当前 diff。
只验收本轮目标：AI 建议卡片的可信边界。

请逐条执行或说明以下场景：

1. 有沟通记录时，建议能展开来源。
2. 无沟通记录时，不生成通用建议。
3. 销售确认前，建议不进入已执行记录。
4. API 未配置或 mock 模式在页面上有明确标识。
5. 日志或终端输出不包含手机号、邮箱、合同金额和沟通原文。

每条场景都要说明使用了什么证据：命令输出、测试、浏览器检查、截图、日志摘要、diff review 或人工判断。
如果某条没有覆盖，请明确写“未验证”，不要写成通过。

这段提示的重点是“不要继续扩展功能”和“未验证就写未验证”。验收阶段最怕两种漂移：一种是发现缺口后顺手扩大功能，另一种是证据不足却用笼统总结抹过去。前者让 diff 失控，后者让完成失真。

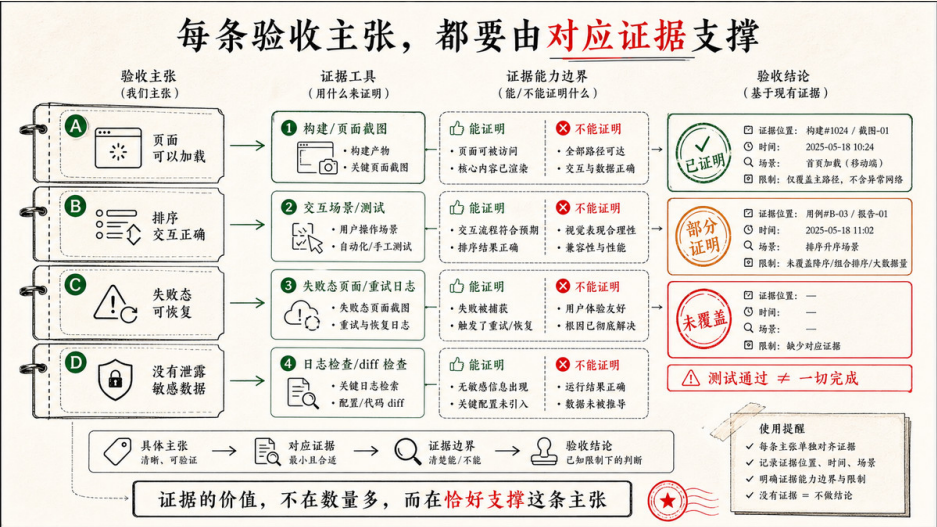
证据要支撑具体主张

验收证据不是越多越好，而是要能支撑具体主张。`npm run build` 通过，可以支撑“项目在构建层面没有明显类型或打包错误”；它不能支撑“来源展示正确”。浏览器里展开过一次来源，可以支撑“这个样例路径下来源入口可见且可用”；它不能支撑“所有来源关系都准确”。日志里没有看到手机号，可以支撑“本次检查的命令输出未暴露手机号”；它不能支撑“生产环境所有日志永久安全”。

对 LeadFlow 这一轮，可以把证据组合成一张很朴素的表：

表 4-2 LeadFlow 本轮验收证据与边界对照

验收主张	可用证据	证据边界
有沟通记录时建议带来源	suggestion 测试、浏览器来源展开、diff 中 <code>sourceIds</code> 保留	不证明真实 CRM 同步后的来源追踪
无沟通记录时不生成建议	无记录 fixture 测试、空状态浏览器检查	不证明所有外部数据缺失场景
确认前不进入已执行记录	状态流转测试、确认前后页面检查、diff review	不证明未来写回 CRM 的权限流程
mock / API 未配置态诚实显示	未配置环境页面检查、README 或 UI 文案检查	不证明真实模型质量
隐私字段不进日志	终端输出、console / server log 摘要、相关 diff review	不等于完整安全审计



这张表的作用不是给读者一个固定模板，而是训练一种判断：每条证据只能证明它能证明的事。Agentic Coding 的可信感，不来自“检查很多”，而来自“主张、证据和边界对得上”。

测试是重要证据，但不是唯一证据。AI 建议卡片这种任务，既需要自动测试，也需要浏览器检查，还需要设计语义判断和隐私日志检查。按钮文案是否会让销售误以为已经外发，往往不是单元测试能回答的；手机号是否进入日志，也不一定可靠页面截图发现。验收不是寻找一种万能证据，而是按风险组合证据。

让浏览器检查带着场景走

4.3 已经用浏览器做过自检，但正式验收里的浏览器检查要更像场景执行，而不是随手看看页面。你要让 Codex 或自己按 `ACCEPTANCE.md` 的前置条件切换样例线索，观察对应状态，并记录结论。

对 LeadFlow，可以按这几条路径执行：

场景 A：有沟通记录的线索

打开客户详情页 -> 查看 AI 建议 -> 展开来源 -> 确认来源内容来自对应沟通记录

场景 B：无沟通记录的线索

打开客户详情页 -> 查看 AI 建议区域 -> 确认显示信息不足，不出现通用销售话术

场景 C：销售尚未确认

打开 AI 建议 -> 检查状态为待确认 -> 检查已执行记录中没有该建议

场景 D：销售确认建议

点击确认 -> 检查进入本地待跟进计划 -> 确认没有触发邮件、短信、外呼或外部提交

场景 E：API 未配置

切换或保持未配置环境 -> 检查页面标识 mock / 未配置态 -> 确认没有伪装成真实模型输出

浏览器检查的价值在于它贴近用户现场。来源入口是否靠近建议正文，待确认状态是否清楚，未配置态是否像正常边界而不是错误崩溃，这些都需要在页面上看。Codex app 的内置浏览器适合这类本地页面预览和评论，但也要记住它的边界：它适合无需登录的本地开发页面和文件预览，不是用来验证真实 CRM 登录态或生产权限系统。涉及登录、扩展、真实浏览器会话或外部客户数据时，要重新选择工具和授权边界。

如果浏览器检查中发现问题，不要立刻把问题扩写成新需求。比如“来源展开后内容太长”可以先记录为设计改进，不一定阻塞本轮；“无沟通记录仍然生成建议”则直接违反本轮验收，必须进入 Debug。验收阶段要区分不完美和不通过，前者可能进入后续 TODO，后者必须修复或明确标为未完成。

日志和终端要证明没有越过隐私边界

LeadFlow 是销售线索工具，隐私边界不是附加题。哪怕本轮只使用脱敏 fixture，也要养成检查日志的习惯：AI 建议生成、mock / live adapter、错误处理和调试输出，都不应该把客户手机号、邮箱、合同金额或沟通原文直接打印出来。

这类验收不一定复杂。你可以让 Codex 检查最近终端输出、浏览器 console、服务端日志摘要和相关 diff：

请检查本轮改动是否新增或保留了可能泄露客户隐私的日志。

重点搜索：

- console.log / console.error / logger 调用
- suggestion request / response 的调试输出
- fixture 中的手机号、邮箱、合同金额、沟通原文
- API 失败时打印完整 payload 或 prompt 的路径

验收结论请区分：

1. 已检查并确认无敏感原文输出。
2. 发现问题并列出具体的文件位置。
3. 当前无法验证，例如没有运行 server log 或没有相关日志入口。

注意第三项很重要。没有日志入口，不等于日志安全；没有看到敏感字段，也不等于所有路径都不会泄露。正式验收不要求你夸大确定性，它要求你诚实说明已经检查了什么。如果本轮只覆盖本地 mock generator，就写清楚“不覆盖真实 LLM 请求日志”；如果未来接入真实 API，这条风险应该进入后续验收或安全检查。

验收失败时进入 Debug，而不是重写

验收失败不是坏事。它说明验收文件发挥了作用，把“看起来能用”背后的缺口暴露出来。真正危险的是失败后的动作太大：Codex 看到一个场景没过，就重构整个建议系统；人看到页面不对，就把任务改成“顺便把详情页重做一下”。这样会把失败从一个局部问题变成新的开发分支。

Debug 的正确入口，是复现、采证、定位最小原因、最小修复、复验。比如验收发现“确认建议后仍然显示未确认”，先不要改 UI 文案，也不要重做状态管理。先问四个问题：

1. 点击事件有没有触发？
2. 状态有没有从 draft 变成 confirmed？
3. 组件有没有读取新的状态？
4. 本地待跟进计划和已执行记录是否混用了同一字段？

如果问题在点击事件，就修事件绑定；如果问题在状态枚举，就修状态转换；如果问题在组件读取，就修渲染路径；如果问题在数据模型，就做最小拆分。Debug 不是为了证明 Codex 刚才做得不够好，而是让失败回到一个可修复的最小因果链。

再看另一个失败：无沟通记录的线索仍然显示一条通用建议。这里也不要直接“优化空状态”。更可能的原因有三种：fixture 没有真正覆盖无记录线索；mock generator 在 records 为空时走了 fallback；UI 在 suggestion 为空时渲染了旧静态示例。三种原因对应三种修法。只有定位清楚，修复才不会扩大。

可以这样让 Codex 进入 Debug：

正式验收失败：无沟通记录时仍然显示通用建议。

请进入 Debug，不要重构整个建议区域。

步骤：

1. 复现失败，并说明使用的 fixture / 页面路径。
2. 收集证据：相关测试、浏览器状态、mock generator 输出或组件渲染路径。
3. 判断最小原因：fixture、generator、状态流转还是 UI fallback。
4. 做最小修复。
5. 只复验这个失败场景以及相邻高风险场景：有记录来源展示、确认前草稿状态。

这段提示保留了 Codex 的自主性，但限制了修复半径。它不是让人手把手指挥每一行代码，而是告诉 Codex：失败属于哪条验收场景，修复不能顺手扩

大，本次复验要覆盖哪些相邻风险。

复验要回到原场景

Debug 修完以后，最容易犯的错误是看见页面恢复正常，就把任务勾掉。复验必须回到原来的失败场景，因为只有原场景能证明这次修复是否真的解决了验收缺口。

如果失败是“无沟通记录仍生成建议”，复验至少要重新打开无记录线索，确认信息不足状态出现，并确认没有通用话术；还要看有记录线索没有被修坏，来源展开仍然存在。如果失败是“确认前进入已执行记录”，复验要检查确认前和确认后的两个状态，而不是只看按钮能点击。如果失败是“日志打印了完整沟通原文”，复验要重新触发那条日志路径，而不是只搜索静态代码。

复验完成后，结论也要按场景写，而不是按文件写：

验收复验：

- [x] 无沟通记录时不生成通用建议
 - 证据：无记录 fixture 浏览器检查；`suggestion.empty.test` 通过。
 - 修复：mock generator 删除空记录 fallback，UI 改为信息不足状态。
 - 未覆盖：真实 CRM 返回空沟通数组时的服务端适配。
- [x] 有沟通记录时来源展开仍然可用
 - 证据：浏览器检查来源展开；相关测试通过。

这种记录方式让下一轮的人知道：我们修的不是某个文件，而是某个失败场景。文件会继续变化，验收场景才是任务完成的稳定坐标。

回写 TODO：把执行状态留给下一轮

验收和 Debug 完成后，不能只把结果留在聊天总结里。下一轮 Codex 不一定会完整读完这段对话，团队成员也不一定在场。**TODO.md** 是当前执行状态的控制面，它应该记录本轮哪些项完成、哪些失败、哪些延期、哪些转成后续任务。

LeadFlow 这一轮可以这样回写：

本轮目标

修正 LeadFlow MVP 中 AI 建议卡片的可信边界：建议必须以待确认草稿呈现，来源入口可见，无沟通记录时不得生成通用建

P0

- [x] AI 建议以待确认草稿呈现
- 验证：浏览器检查有记录线索；确认前未进入已执行记录。
- 证据：`npm run build` 通过；状态流转测试通过。
- 未覆盖：真实 CRM 写回后的确认权限。
- [x] 来源入口靠近建议正文并可展开
- 验证：浏览器展开来源，确认显示对应沟通记录。
- 证据：sourceIds fixture 测试通过；diff review 确认未丢失来源字段。

- 未覆盖：句子级引用。
- [x] 无沟通记录时显示信息不足状态
- 验证：无记录 fixture 页面检查；不生成通用建议。
- 证据：相关测试通过。
- 未覆盖：真实 CRM 返回空记录数组后的服务端适配。

后续

- [] 评估真实 LLM 接入时的脱敏请求日志和未配置态验收。
- [] 评估来源从沟通记录级升级到句子级引用。

这不是把最终报告复制进 TODO，而是留下最关键的恢复点。下一轮任务进入时，Codex 可以看到本轮到底完成到了哪里，哪些风险被留给后续，而不是从“上次好像已经做完了”开始猜。

回写验收和设计：只改事实变化的地方

状态回写不等于把所有文件都改一遍。4.4 要特别克制：TODO.md 记录本轮执行状态；ACCEPTANCE.md 记录长期完成标准；PRD.md 只有产品范围或非目标变化时才改；DESIGN.md 只有界面状态、设计系统或交互语义变化时才改；CONTEXT.md 只有业务事实、术语或外部约束变化时才改。

比如本轮如果只是把“推荐行动”改成“待确认跟进建议”，并且这原本就符合 DESIGN.md，不一定需要改设计文件。但如果验收中发现 DESIGN.md 没有定义“来源缺失”状态，而代码现在引入了 source_missing，那就应该补一条设计状态，避免下一轮 Codex 看到这个状态时不知道如何表达。

ACCEPTANCE.md 也一样。如果原来已经有“无沟通记录时不得生成建议”，本轮通过后不必把每次验证流水写进验收文件；只需要在 TODO 或验证记录里写结果。但如果 Debug 暴露出新的长期风险，例如“sourcelds 存在但和建议内容不对应”，那就值得补成新的验收场景：

```
## 来源记录必须和建议内容相关
```

```
Given 某条建议关联了来源记录
```

```
When 用户展开来源
```

```
Then 来源记录必须支持建议中的主要判断，不得使用无关沟通记录填充来源
```

```
可用证据：
```

- fixture 中包含多条语义不同的沟通记录
- 来源匹配逻辑测试或人工审查
- 浏览器检查来源展开内容

这种回写会让验收标准变强，而不是让文档变胖。反过来，如果某个想法只是“以后也许可以做”，就放进 TODO 后续，不要提前写进 PRD 和 ACCEPTANCE，免得把未来想象变成当前承诺。

长期规则只沉淀反复出现的问题

有些问题验收时会反复出现，比如 Codex 总想把 mock AI 建议写得像真实模型输出，或者总在调试时打印完整沟通记录。第一次出现时，先在本轮修掉，并把原因写进 TODO 或验收记录。只有当你确认这是会反复发生的风险，才值得沉淀进 AGENTS.md 或 Rules。

长期规则应该短而硬，能拦住具体风险：

- LeadFlow 中 mock AI 建议必须在 UI 上标识为示例，不得伪装成真实模型输出。
- 调试日志不得输出客户手机号、邮箱、合同金额或完整沟通原文。
- AI 建议没有来源记录时，不得显示为可信建议。

不要把一次验收里的全部经验都塞进长期规则。AGENTS.md 不是日记，Rules 也不是项目百科。它们适合承载稳定边界，而不是记录每次任务的细节。任务细节留在 TODO，完成标准留在 ACCEPTANCE，设计语义留在 DESIGN，长期风险才进入规则。

最终 diff / review：确认范围没有变形

正式验收通过以后，还要最后看一次 diff。这个动作看似重复，其实非常重要，因为 Debug 和复验过程中可能又引入新改动。你需要确认当前未提交变化仍然围绕本轮目标，没有混入无关重构、临时日志、实验代码、真实外部服务接入或被遗忘的调试文件。

这里再次看 diff，重点已经不同于 4.3。4.3 看的是“刚才那一小步有没有改过头”，4.4 看的是“准备封存的这一批变化是否能作为一个清楚版本交给下一轮”。如果工作区里混着用户手动修改、Codex 修改、临时调试文件和验收回写，就先把它们分清楚。哪些进入本次提交，哪些回退，哪些另记后续 TODO，必须在 commit 之前决定。

对 LeadFlow 这一轮，最终 diff 检查可以问：

1. 修改是否集中在 suggestion 类型、fixture、mock generator、AI 建议卡片、相关测试和必要 README / TODO 回写？
2. 是否有真实 CRM、邮件外发、权限系统、全局布局重构或新 UI 框架等越界改动？
3. 是否残留 console.log、临时调试按钮、硬编码客户原文或假 API 成功状态？
4. TODO / ACCEPTANCE / DESIGN 的回写是否和实际代码一致？
5. staged 和 unstaged 是否清楚，是否有不该进入本次提交的文件？

如果发现无关改动，不要因为验收已经通过就顺手带进去。可以按 hunk 回退，或者把它拆成后续 TODO。Git 封存的价值在于版本边界清楚；一次提交如果混进多个目标，下一轮回滚、审查和复盘都会变难。

Git commit 不是完成证据

最后才进入 Git。顺序不能反过来。commit 不是完成证据，它只是通过验证之后的版本封存。真正让提交可信的，是前面的验收、Debug、回写和 diff 审查。

如果本轮变化已经通过验收，可以在 review 面板或终端里 stage 需要保留的文件，写一个能表达工程状态的 commit message：

fix: tighten LeadFlow AI suggestion trust boundary

- show AI suggestions as draft follow-up plans before sales confirmation
- require source expansion for suggestion cards
- show insufficient-info state when communication records are missing
- document validation evidence and remaining CRM/live API gaps

这个 commit message 不需要文学化，但应该让后来的人知道这次封存的是哪条工程变化。它不是“update UI”，也不是“fix stuff”，而是“收紧 AI 建议信任边界”。如果团队需要远端协作，再 push 或创建 PR；如果只是本地学习项目，commit 本身已经给了一个可回退、可比较的版本点。

PR 也不是流程装饰。它适合把本轮变化交给别人审查：产品看 AI 建议语义，设计看状态和按钮，工程看 diff 和测试，负责人看未覆盖风险。PR 描述可以沿用 4.4 的验收结果：

本次变化

- AI 建议以待确认草稿呈现。
- 来源入口靠近建议正文并可展开。
- 无沟通记录时显示信息不足状态。

验收证据

- `npm run build` 通过。
- suggestion 相关测试通过。
- 浏览器检查：有记录来源展开、无记录空态、确认前草稿状态、mock / 未配置态。
- diff review：未引入真实 CRM、自动外发或新 UI 框架。

未覆盖

- 真实 CRM 同步后的来源追踪。
- 句子级引用。
- 真实 LLM 请求日志和权限错误处理。

这样的 PR 描述能让审查者从“看一堆改动”转成“检查一组主张和证据”。如果审查者留下评论，下一轮就从 4.2 重新进入：先判断评论属于缺陷、设计状态、验收缺口、后续想法还是拒绝范围，再进入小步开发。

结束时要留下四类东西

4.4 结束时，LeadFlow 不应该只是“AI 建议卡片更好了”。它应该留下四类东西。

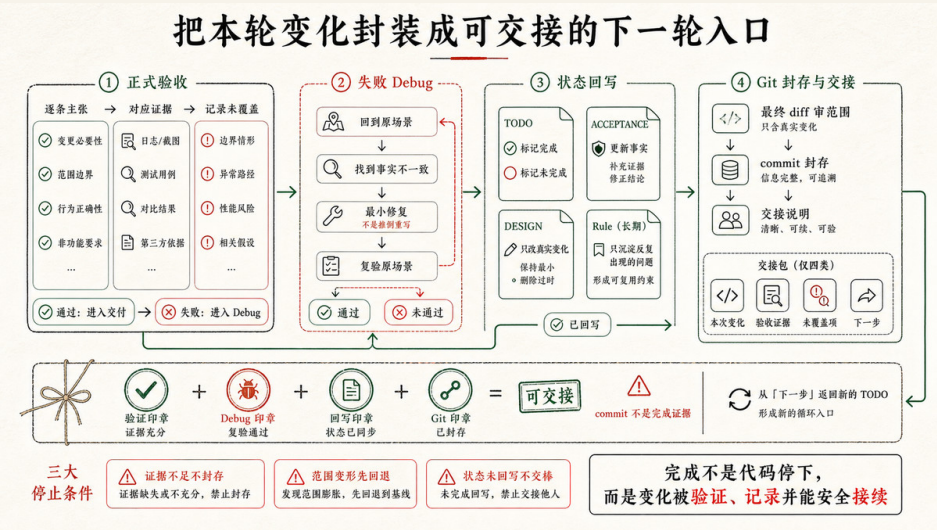
第一，代码变化。建议来源、无记录状态、确认状态和未配置态已经在代码中体现，diff 范围被审查过，无关改动被排除或拆到后续。

第二，验证证据。命令、测试、浏览器检查、日志摘要、diff review 和人工判断分别支撑了哪些验收主张，哪些没有覆盖，都被写清楚了。

第三，状态回写。`TODO.md` 记录本轮完成和后续，`ACCEPTANCE.md` 只在完成标准变化时更新，`DESIGN.md` / `PRD.md` / `CONTEXT.md` 只在对应事实变化时更新，长期规则只沉淀反复风险。

第四，版本边界。Git stage 和 commit 把本轮变化封存起来，必要时 push 或开 PR，让团队可以审查、回滚、比较和继续。

这四类东西合在一起，才是 Agentic Coding 里的“完成”。不是 AI 生成了代码，而是一次工程变化在约束内发生、被验证、被记录、被封存，并且下一轮知道从哪里继续。



下一轮从哪里来

4.4 不是终点，它是下一轮的入口整理器。验收通过后，下一轮可能来自新的用户反馈；验收未覆盖项可能进入后续 TODO；PR comment 可能变成本轮修复；真实 LLM 接入、句子级引用、CRM 写回、权限审计可能变成新的产品判断；也可能因为本轮已经足够，项目先停在一个可演示、可交接的 MVP。

这就是第 4 章完整 loop 的意义：

- 4.1 用 PRD 和 DESIGN 完成可运行 MVP
- 4.2 把反馈、问题和评论整理成下一轮 TODO
- 4.3 用命令反馈、页面状态和 diff 范围校准开发执行
- 4.4 用验收、Debug、回写和 Git 封存收口

下一轮再开始时，你不需要让 Codex 重新猜 LeadFlow 是什么，也不需要从聊天记录里翻“上次做到哪”。项目里已经有 PRD、DESIGN、TODO、ACCEPTANCE、README、代码、证据和 Git 历史。这些东西共同构成了一个可继续的工程现场。

Agentic Coding 的实战，不是让 AI 自动跑到天边，而是让每一次任务都有入口、有执行、有证据、有回写、有版本边界。只要这条 loop 能跑起来，一个 Vibe Demo 才开始变成可维护、可协作、可交付的工程项目。

这一轮 loop 跑通了，但它也暴露了一件事：靠人记很累。每次开始要提醒 Codex 先读 PRD、DESIGN 和 ACCEPTANCE；每次任务进入要手工整理 TODO；每次执行结束要区分哪些是自检、哪些需要正式验收；每次验收完成要逐一回写状态和证据；换一个新项目，这些准备动作还要从头再来。这套动作有效，但它没有被任何东西接住。第五章要回答的正是这个问题：能不能把这套手工 loop 做成一个脚手架，让新手也能按步骤走完，让老手不用每次重复解释？这就是 CodeNow 出现的原因。

我是如何构建项目冷启动的脚手架

5.1 CodeNow 整个脚手架里有什么

CodeNow 是作者我为零代码基础的人做的一套项目冷启动脚手架，已经开源在 <https://github.com/xue160709/CodeNow>。这一章我会以项目作者的身份，带你看清楚里面到底放了什么、每一层又在整个工作流里承担什么。读这一章时，我建议你顺手把仓库 clone 下来，或者直接在 GitHub 上打开，边读边对照——把文字和真实文件放在一起看，比单读文字更容易记住每一层在做什么。

在拆解结构之前，得先讲清楚我做 CodeNow 的愿景，因为它决定了这套脚手架为什么长成这样：让一个没有工程基础的人，也能无痛地把自己的想法做成一个真正能用的项目。这里的“无痛”，不是指点一下按钮、AI 就替你把活全干完，而是指你不必先学会写需求文档、拆开发任务、定验收标准、管项目状态——这些最容易让新手卡住、也最容易在中途失控的环节，CodeNow 把它们拆成一个个有人接手的步骤：你只用日常语言把想法说清楚，剩下的“先问什么、先做什么、什么时候还不能写代码、怎样才算做完”，由脚手架和 Codex 一步步接住。

这里有一点值得单独提：CodeNow 支持三种工具形态——网站（Next.js）、桌面软件（Electron）和浏览器插件（vite-web-extension）。这是 Lovable、AI Studio 这类主流 Vibe Coding 工具普遍做不到的，它们基本只能输出网站。你的产品应该是哪种形态，由你的想法和场景决定，不由工具边界决定。这个判断在冷启动最早的阶段就会被问清楚，也会决定后续整条流水线走哪个技术方向。

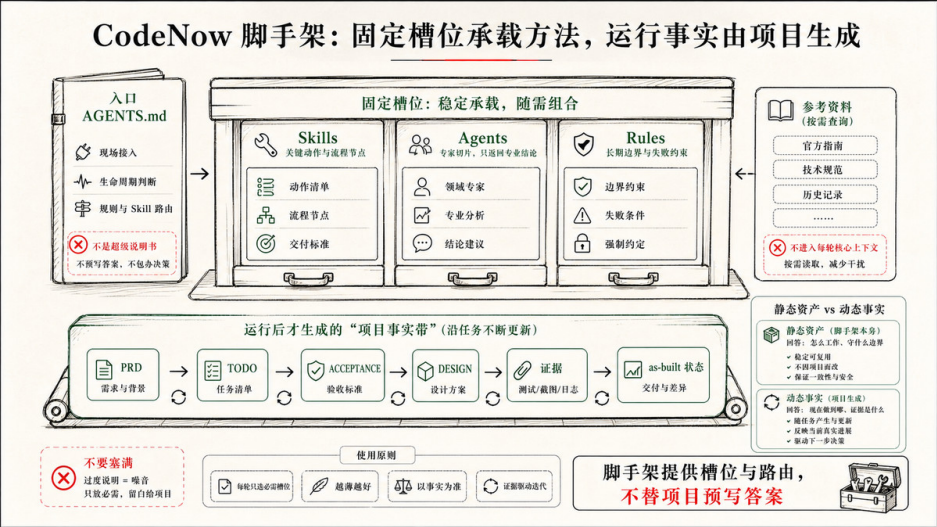
第一次打开这个仓库，你会看到 `AGENTS.md` 和 `.codex/{rules,skills,agents}` 这几层。我做 CodeNow 的起点，是几件在冷启动里反复失控的事：用户只有一句话想法，Codex 容易直接开始写代码，跳过产品意图的确认；写代码之前没有验收契约，做完没人知道算不算完成；首版跑通，自检就被当成正式验收打了勾；验收后的实现状态只活在聊天记录里，下一轮 Codex 进来就失忆了。仓库里这几层，各自对应其中一类问题——入口负责路由，rules 负责边界，skills 负责把任务变成可执行流程，agents 负责专业切片。

这也是我设计 CodeNow 时真正想搭的东西：与其把它看成一堆工具，不如把它看成一条状态交接的流水线。接下来这一章，我就顺着想法、计划、开发、验收、回写这条主线，带你逐层看每个目录在交接什么。

```
AGENTS.md
.codex/
  rules/
  skills/
  agents/
API文档/
```

这几层分别解决不同问题。`AGENTS.md` 是入口；`rules/` 保存长期边界；`skills/` 是具体 workflow 节点；`agents/` 是可被主会话派发的专家 worker；`API文档/` 则放着四份针对 SiliconFlow 的接口参考（LLM 文本生成、图片生成、ASR 语音转写，以及《如何接入 Token》）；CodeNow 默认以 SiliconFlow 作为 AI 能力供应商，当项目需要接 AI 或远程服务时，`api-integration-planner` 会按需来这里取材。

有一点要先提醒：你 clone 下来时，仓库里几乎只有这些脚手架文件，看不到 `prd/`、`TODO.md` 或 `design-system/`。这些不是漏掉了，而是它们本来就该由后面的 Skill 在你真正开一个项目时生成——脚手架交付的是“怎么干活”的约定，而不是某一个具体项目的产物。



AGENTS.md：薄入口，而不是超级说明书

AGENTS.md 是每一轮任务都会被加载进上下文的文件。这个特点决定了它的设计方向：不能把它当知识库来用。

一份胖的 AGENTS.md 会带来三个成本。一是 token，它有多长，就要在每一轮里反复付多少代价，和当前任务无关的内容也不例外。二是上下文腐坏，Codex 读入的内容越多，过期规则、陈旧示例或彼此冲突的说明越容易干扰当前任务的推理。三是维护失控，规则如果同时出现在 AGENTS.md 和各条 rule 文件里，改一处就要同时更新两处；哪一处落了，两份说明就开始矛盾。

所以 AGENTS.md 在 CodeNow 里承担的是导航职责，不是知识职责。它只需要回答三件事：这一轮任务先去读哪些文件？走到哪一步应该停下来等人确认？完成以后哪些状态必须写回去？

把这三件事做清楚，AGENTS.md 自然就薄。规则的细节放在各条 rule 文件里，Skill 的执行流程放在各 SKILL.md 里，验收标准放在 acceptance.md 里——AGENTS.md 只挂“什么时候去找哪个文件”的指针，不把内容内联进来。

这种设计有一个关键强制点：路由声明。CodeNow 要求 Codex 在开始任何实质性操作之前，先用一行写清楚本轮走哪条路：

路由判断：<触发依据> → 使用 <Skill / 轻量维护路径>

这不只是格式要求。它把"这是冷启动、准备、首版开发、debug、正式验收，还是轻量维护"明确摆到对话里，人一眼就能发现路线有没有选错——在 Codex 动手之前。

对熟悉这套流程的人，AGENTS.md 还留了一组省话口令，几个字就能锁定路径：

只读评估 → 只读文件给结论，不改文件
轻量维护 → 只改规则 / 文案 / 小样式，做自检
实现 TODO 第 x 项 → 按 project-iterate 执行
正式验收 / 打勾 → 按 project-verify 逐条回写证据
教学模式 → 零基础方式解释，不顺手改代码

AGENTS.md 的设计取舍就是入口文件的通用取舍：它越短，每轮的推理起点越干净；指针越准，Codex 找到正确方向的概率越高。

.codex/skills/：把生命周期里的关键动作做成节点

.codex/skills/ 目前有八个 Skill。其中七个各自对应项目生命周期里的一个具体动作，排在一起正好串成一条从“一个想法”到“一个可运行、可验收、可回写、还能持续迭代的项目”的主路径；剩下一个 ui-ux-pro-max 不属于这条主线，是外部接入的通用设计知识库，需要做 UI 时被前面几个 Skill 取用。

表 5-1 CodeNow Skill 一览

Skill	职责
01-mvp-prd-builder	把粗糙想法变成根目录 prd.md。它先吸收用户材料，再围绕工具形态、目标用户、使用场景、核心痛点、MVP 功能、交互流程做信息充分度判断；只在关键缺口上追问，最后形成叙事性的 MVP PRD。用户若上传参考图，还可在此阶段生成 design-system/DESIGN.md。
02-project-prepare	把 prd.md 变成开发前合同。它不写业务代码，而是通过多个规划 Agent 生成或更新 TODO.md、设计系统、架构规格、API 接入方案和模块 acceptance.md 验收契约，让 Codex 写代码前先知道做什么、怎么做、怎样算完成。
03-project-develop	首版 MVP 专用。它根据 TODO、PRD、设计系统和 acceptance 契约搭建可运行项目，安装依赖，实现核心功能，做 Codex 自检，并给出运行或安装说明。

Skill	职责
project-iterate	首版之后的日常迭代。已有代码时，它以 TODO.md 和模块 acceptance.md 为依据实现“下一项”、小改 UI/API 或按已定位根因做明确修复，默认只做 Codex 自检、不冒充正式验收。
project-debug	当页面空白、启动失败、数据不对、登录失败或行为异常但根因不明时使用。它强调复现、采集 console / network / server / 测试证据、最小修复和复验，而不是凭直觉改代码。
project-verify	正式验收节点。它按 prd/<module>/acceptance.md 的 GWT 场景逐条执行验证，写状态戳和证据路径，再同步 TODO checkbox。它不负责修 bug，也不凭自检给任务打勾。
update-prd	把实现后的真实状态同步回 prd/ 文档体系。它负责补齐模块 README / technical，记录 as-built 事实，把未完成或降级能力写回 TODO，并在 TODO 过长或模块闭环后归档已完成块。
ui-ux-pro-max	不是 CodeNow 原创的生命周期节点，而是外部接入的通用 UI/UX 设计知识库（大量风格、配色、字体搭配与 UX 准则）。需要做 UI 时， 01 / 02 / 03 / project-iterate 会检索它，把通用设计规则结合当前产品语境，沉淀进 design-system/DESIGN.md 。

主线上这几个 Skill 的边界同样重要。**01** 只让想法成形，不写验收契约；**02** 只准备开发合同，不写业务代码；**03** 只服务首版 MVP，不把每天的小改都当成重新冷启动；**project-iterate** 承接日常迭代，但默认只做自检、不替自己盖验收章；**project-debug** 只定位和修复根因不明的问题，不替代正式验收；**project-verify** 只验收和写证据，不顺手改产品范围；**update-prd** 只写当前真实实现，不把未来想做的东西粉饰成已经完成。

这就是 Skill 作为“ workflow 节点”的价值。Skill 定义的不只是 Codex 能做什么，也定义它不该做什么。

.codex/agents/：专家 worker

.codex/agents/ 里放的是一组 SubAgent 定义，每个都是只懂一件事的专家 worker：从冷启动时的产品成形，到准备阶段的需求拆解、架构规划、验收契约、UI 规划、API 接入，再到开发、验收、PRD 回写时的分片执行。它们由主会话在合适的阶段按需派发，自己不直接面对用户，也不能再往下派发子代理。

表 5-2 CodeNow SubAgent 一览

SubAgent	作用
mvp-shaper	01 阶段把用户材料整理成自治的 MVP 成形稿，判断用户场景、范围优先级、交互流程和 ASCII 流程。
mvp-reviewer	独立评审 mvp-shaper 的成形稿，检查一致性、范围克制和流程图信息密度，避免第一个方案刚自治就被写进 PRD。
requirement-analyst	把粗粒度 MVP 拆成可开发的功能规格、状态和数据读写。
architecture-planner	判断框架、模块边界、平台能力、安全边界和架构风险。
qa-acceptance-planner	为每个模块写 prd/<module>/acceptance.md，把做完标准变成 GWT 验收契约。
ui-interaction-planner	规划界面入口、信息架构、组件、状态、演示内容和响应式约束。
api-integration-planner	当项目涉及远程 API、AI、鉴权、Token、同步或第三方服务时，规划 API 清单、降级和 Key 边界。
module-developer	当多个模块真正独立时，按文件白名单实现某个模块；共享接口和最终合并仍由主会话负责。
acceptance-verifier	多模块正式验收时，每个 worker 只验一个模块的 acceptance 场景，只写自己的证据和状态戳，不碰 TODO。
prd-module-writer	多模块 PRD 回写时，只补自己模块的 README / technical / sync 状态；全局索引和 TODO 由主会话统一合并。

比起有哪些 worker，更值得注意的是它们之间的写入边界。主会话始终是唯一的编排者，SubAgent 只是叶子 worker：规划阶段可以并行问几个专家，但共享文件不能大家一起写；验收阶段可以按模块拆出去并行验，但 TODO checkbox 只能由主会话汇总后统一同步；PRD 回写可以把各模块文档分头写，可 prd/README.md、TODO 和全局同步状态始终只有主会话一个写入者。

如果没有这些边界，SubAgent 会很快变成“多个会话同时改一堆共享文件”。看起来效率高，实际上更容易把 TODO、PRD、验收状态和代码改乱。

.codex/rules/：长期边界

rules 放的是那些跨任务反复生效的长期边界，它更像一组护栏：平时不挡路，只有当前任务走到某个风险点时，Codex 才去读对应的那一条。可以把这些 rules 理解成七类边界。

表 5-3 CodeNow rules 一览

Rule	守护的边界
kernel.md	常驻薄内核。只保留最少铁律：改产品行为要回写 TODO，写功能前确认 acceptance，没做也要说明，先声明路由再执行。
lifecycle.md	生命周期和路由。它说明从冷启动、准备、首版开发、失败排查、正式验收、PRD 回写到轻量维护分别该走哪条路径。
runtime-contract.md	.codex/ 运行时契约。它规定 AGENTS.md、rules、skills、agents 的职责，以及 leaf worker、写入 allowlist / denylist 和迁移边界。
acceptance-contract.md	验收契约。它规定 prd/<module>/acceptance.md 是唯一验收事实源，TODO 只放任务和指针，测试清单和 test-results 只放证据。
todo-writeback.md	TODO 回写。它规定什么时候必须更新 TODO，什么时候可以不更新但要说明原因。
verification-levels.md	验证边界。它把 Codex 自检和正式验收分开，避免“跑过 lint / smoke”被误当成 acceptance 通过。
coach.md	教学模式。它服务新手解释和陪跑，不改变项目事实源。

这套 rules 的一个设计原则是：每条规则都要知道自己守的是什么。如果一条 rule 只是说“要认真”“要高质量”“要考虑用户体验”，它很难被执行，也很难被检查。好的 rule 应该能回答四个问题：

- 它守护哪类事实源？
它在什么任务中触发？
它要求 Codex 做什么、禁止做什么？
它什么时候可以不适用？

比如 acceptance-contract.md 守护的是“验收标准只能有一个事实源”。它要求准备阶段先写 acceptance，开发阶段按 acceptance 实现，正式验收阶段按 acceptance 盖状态戳；它同时禁止把验收标准复制进 TODO、README 或测试清单。这样下一个会话进来时，不会面对三份互相矛盾的“完成定义”。

再比如 verification-levels.md 守护的是“自检和正式验收不能混同”。Codex 可以在日常开发后跑 lint、test、typecheck、smoke，这些是自检；但只有 project-verify 按 acceptance 场景逐条执行并写证据，才算正式验收。这个区分看起来啰嗦，但它决定了 TODO checkbox 有没有可信含义。

真正撑起 CodeNow 的，是状态交接

如果把 CodeNow 看成目录清单，它会显得复杂。如果把它看成状态交接系统，它的结构就清楚了：

主线

- > 01-mvp-prd-builder -> prd.md
- > 02-project-prepare -> TODO.md + acceptance + design-system
- > 03-project-develop -> 可运行首版 + Codex 自检
- > project-verify -> 状态戳 + test-results + TODO checkbox
- > update-prd -> prd/ as-built 文档 + TODO 归档

异常分支（根因不明时）

- > project-debug -> 复现 + 定位 + 最小修复 -> 回到 project-iterate



AGENTS.md 负责让 Codex 进门后选对路；rules 负责在关键节点守住边界；skills 负责把复杂任务拆成可执行流程；agents 负责在合适的时候分担专业切片。

所以 CodeNow 整个脚手架里真正有的，不是一堆更高级的工具，而是一组明确的交接关系：

- 想法交给 PRD；
- PRD 交给准备阶段；

- 准备阶段交给 TODO、acceptance 和设计系统；
- 开发交给自检；
- 完成交给正式验收；
- 验收后的事实再交回 PRD 和 TODO。

如果只带走一个判断，我希望是这个：做自己的 Agentic Coding 脚手架，不是先复制 CodeNow 的目录，而是先问清楚你的项目里哪些状态最容易丢，哪些边界最容易混，哪些完成最需要证据。目录只是外壳，状态交接才是脚手架的骨架。

5.2 整个冷启动的设计思路是什么

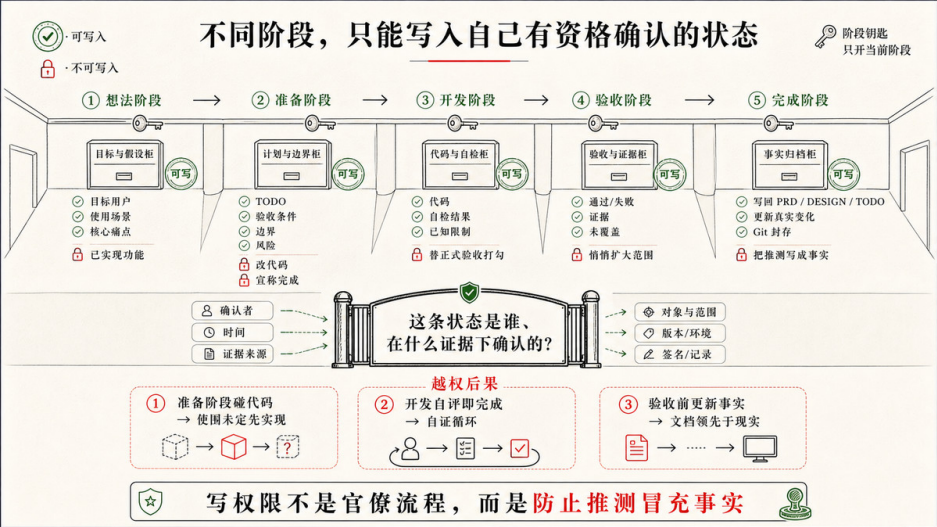
上一节把 CodeNow 看成一条状态交接的流水线。这一节退一步，讲这条流水线为什么要这样拆。

冷启动最难控制的不是代码质量，而是 Codex 对状态的写入。我在带新人、复盘项目时反复遇到同一类失控：不是某段代码写错了，而是几种性质完全不同的状态被混写在了一起——“可能要做”“应该做”“正在做”“已经做完”“已经验证”，这五种状态在 Codex 手里很容易压成一种：此刻写。后果是具体的：备选方向被写进模块文档，变成项目已经确认的方向；自检通过被当成正式验收，TODO 的勾就打上了；聊天记录里说过的计划，下一个会话进来当成已实现的项目状态继续往下推。

这是冷启动失控的根本机制。不是 Codex 能力不够，而是没有告诉它什么时候只能写什么。

我把 CodeNow 冷启动的设计压缩成一个判断：每个阶段只写自己有资格写的东西。

想法阶段写意图。
准备阶段写契约。
开发阶段写代码和自检结果。
验收阶段写证据和状态戳。
完成以后写事实和差距。



这五行是对 Codex 写入权限的分配。CodeNow 的四段冷启动流程，每一段只守住其中一行：

```
模糊想法
-> 01-mvp-prd-builder
-> prd.md
-> 02-project-prepare
-> TODO.md + prd/<module>/acceptance.md + design-system/DESIGN.md
-> 03-project-develop
-> 可运行首版 + Codex 自检 + 运行说明
-> update-prd
-> prd/<module>/README.md + technical.md + TODO 差距 / 归档
```

下面顺着这四段，看我当初面对什么问题、做了什么选择。

01：为什么只生成一个 prd.md

01 阶段最容易踩的陷阱，是看起来"更完整"。如果 01 一开始就把想法拆成模块文档，用户会马上看到一整套 PRD 目录：`prd/feature-a/README.md`、`prd/feature-b/acceptance.md`.....看起来进入了工程状态。

我在 CodeNow 里刻意没有这么做。理由只有一个：在想法还没有稳定下来之前，过早拆模块会把不确定性伪装成事实。一个模块文档一旦落地，下一轮 Codex 就会把它描述的方向当作项目已经确认的事实。文档会约束后续的判断，哪怕它描述的还只是某个下午尚未确认的想法。

对用户来说还有另一层。在没有看到项目跑起来之前，人其实也想不清楚自己到底要什么。如果一开始就摊开十几份还填不满的 README 和 technical，用户面对的是虚构的"事实"，既无从确认，也容易被绑定。一个根目录 `prd.md` 把决策压力收敛到"这个产品故事对不对"，用户此刻有能力回答的，恰好就是这一层。

六个维度，不是固定问卷

01 的核心工作是信息充分度评估，围绕六个维度进行：

工具形态 → 决定框架选择 (Next.js / Electron / 浏览器插件)

目标用户 → 决定功能优先级和界面密度

使用场景 → 决定主流程如何闭合

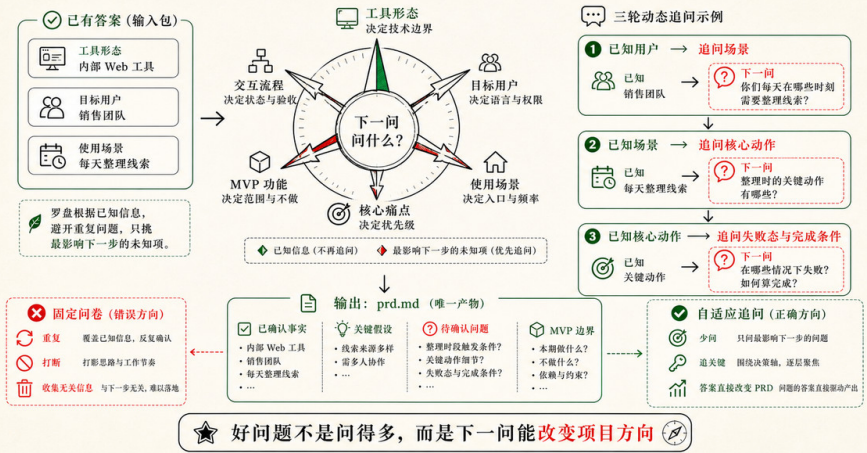
核心痛点 → 决定哪些功能进 MVP、哪些不进

MVP 功能 → 决定第一版的范围边界

交互流程 → 决定页面结构和数据流向

这六个维度不是可以独立填写的表格字段，它们之间相互影响。工具形态变了，交互流程就变；目标用户变了，MVP 功能的优先级就变。所以 01 不用固定问卷，而是先给六个维度各打一个状态（已充分、不够具体、缺失、冲突），挑出当前最影响 MVP 边界的那一个来问；用户每答一题，它就重新评估六个维度，再决定下一题。

冷启动追问不是固定问卷，而是一枚自适应罗盘



工具形态为什么排在第一个

六个维度里，工具形态的优先级最高，原因很具体：它不只是技术选型，它决定了产品能进入什么场景，能用什么平台能力，以及用户和产品之间是什么关系。桌面软件可以离线运行、直接读写本地文件系统；浏览器插件可以注入任意网页、捕获用户的浏览上下文；网站最容易分享和部署，但受限于浏览器沙箱。这三种形态对应的价值主张根本不同，一旦确认，后续的框架、打包方式、API 接入路径和验收场景都随之锁定。

这也是大多数现成 Vibe Coding 工具（Lovable、AI Studio、v0 等）的天花板所在：它们基本只能输出网站，用户的想法被工具边界提前收窄了。

CodeNow 明确支持三条路：网站走 Next.js，桌面软件走 Electron，浏览器插件走 vite-web-extension。所以当用户描述的想法更适合桌面端或插件形态时，01 不会把它往网页方向推，而是先问清楚，再让后续整条流水线对齐。

用户没选的选项也不会浪费。它们被当作弱信号降权——意味着后面的问题不再默认往那个方向展开。等六个维度都打到“已充分”，立刻停手，不为了流程完整继续追问。这样，用户回答的过程本身就是在收敛方向，而不是被动填表。

□0□ 和 □1□：作者与评审分开

信息足够后，01 会派出两个 SubAgent：`mvp-shaper` 和 `mvp-reviewer`。

`mvp-shaper` 是作者角色。它拿到用户材料和所有澄清答案，一次性产出：用户和场景的判断、MVP 范围和模块优先级、从首页开始的核心交互流程，以及一份 ASCII 流程图草案。这些全在同一个上下文里完成，是为了让成形稿内部自治——不要在同一个 MVP 里既说“个人工具”又说“团队协作”。

`mvp-reviewer` 是独立评审角色。它拿到同一份用户材料和 `mvp-shaper` 的成形稿，从外部视角检查三件事：一致性（场景、功能、流程有没有互相矛盾），范围克制（低优先级方向有没有偷渡进主线），ASCII 信息密度（流程图是否太空泛或太复杂）。

分开的意义在于：一个方案刚刚成形时最容易显得合理。独立评审在成形稿写进 `prd.md` 之前，先拦住范围膨胀和假设偷渡。作者擅长让方案自治，评审擅长发现作者的盲区。

最终生成的 `prd.md` 是叙事性总目标，不是模块规格，也不是验收合同。它只需要回答：这个产品是给谁的，在什么场景下解决什么问题，第一版围绕哪条路径闭合，哪些方向暂时不做。用户上传一张参考图时，01 还可以基于 `ui-ux-pro-max` 设计知识库生成视觉设计系统，写入 `design-system/DESIGN.md`；但这件事不是 01 的主线，也不影响 `prd.md` 的叙事职责。

02：为什么准备阶段不能碰代码

02 有一条硬约束：不写业务代码，不安装依赖，不初始化框架。

这条约束的设计理由是：开发前合同必须先于开发动作出现。如果准备阶段就开始实现，最容易把还没确认的任务顺序、设计边界和验收标准偷偷写进代码里。等用户想确认“这些验收场景对不对”的时候，代码已经按某个未经确认的理解走了一段，改回来的代价远高于没写。

五个专家 Worker，并行但只有一个合并者

02 把规划工作拆给五个 SubAgent，每个只懂一件事：

```
requirement-analyst → 把粗粒度 MVP 拆成子功能、主流程、状态（空/加载/成功/错误态）和数据读写
architecture-planner → 选主框架、定模块边界、划安全边界、处理平台能力
qa-acceptance-planner → 为每个功能写 GWT 验收契约（prd/<module>/acceptance.md）
ui-interaction-planner → 规划界面入口、组件、状态和演示内容（按需启用）
api-integration-planner → 规划 API 清单、鉴权方式、降级策略和 Key 边界（按需启用）
```

这五个 Worker 可以并行发出，但有一条写入规则：`TODO.md` 和最终开发规格只由主会话写。Worker 只输出自己的专业切片，主会话负责合并、消冲突，把五份输出变成一份前后一致的开发计划。并行提效，但合并权只有一个——防止多个会话同时改同一份共享文件导致的状态错乱。

验收契约为什么在写代码之前

`qa-acceptance-planner` 在规划阶段就写出 `prd/<module>/acceptance.md`，格式是 GWT：先写 SHALL 需求（这个功能必须满足什么），再写 GIVEN/WHEN/THEN 场景（在什么前提下，做什么操作，看到什么结果）。场景包括主流程、异常路径、空状态、加载态和未配置态。

这些契约在 03 开始之前就存在，不是为了形式完整，而是为了给开发设定门禁。03 在实现每个功能前，第一步就是读对应的 `acceptance.md`——契约不存在或太空泛，先补契约再写代码。这个顺序很重要：验收标准一旦在开发后才补，几乎不可避免地会按代码已有的行为来写，等于把测试改成描述，而不是检查。

`TODO.md` 和 `acceptance.md` 的分工同样刻意：`TODO.md` 只放任务和一行指向 `acceptance` 的指针，不复制验收标准。这样下一轮任务面对的是唯一事实源，不会出现"TODO 说一个标准，`acceptance` 说另一个标准"。

框架选择也在这一步锁定

02 会把 `prd.md` 里的工具形态翻译成唯一一个技术方向：网站走 Next.js (App Router + TypeScript)，桌面软件走 Electron (electron-vite)，浏览器插件走 vite-web-extension (Manifest V3 + React)。这个映射写进 `TODO.md`，03 直接照着做，不再临时决定。

`design-system/DESIGN.md` 在 02 这里也被当成一份工程合同。如果 01 阶段用户传过参考图、已经生成了视觉设计系统，02 只在上面对补工程章节：UI 技术栈、图标库 (React 栈默认 `lucide-react`)、占位图策略、数据与演示约定。如果文件不存在，02 从零构建一套完整的设计系统，把视觉 token 和关键界面状态一起补齐。目的是让 03 开始写代码时，有一份可以直接照着安装依赖、落地组件、对齐 token 的设计依据，不必一边实现一边临时发明视觉规则。

02 写完必须停下来，等用户确认 TODO、`acceptance` 和设计系统。03 开始之后，用户注意力会转向"项目能不能跑起来"；在此之前把方向、任务顺序和验收合同摆到台面上，是最后一次以低成本修改这些判断的机会。

03：为什么首版开发是一个独立类别

把首版开发和日常迭代分开，是因为这两种任务的结构完全不同。

首版要从零搭地基：脚手架、包管理器、框架配置、API Key 保存方式、示例数据或未配置态、全局样式、设计系统落地、启动命令和基础验证。日常迭代通常只是在这些地基上继续做一项 TODO、小改一个交互、修一个已定位

问题。如果每次小改都重新走首版流程，项目会被反复当成冷启动；如果首版开发直接用日常迭代方式，又容易漏掉运行入口、依赖安装和基础验证。

ATDD 门禁：写代码前先读验收契约

03 的第一个机制是在实现每个功能前，先过一道门禁：读

`prd/<module>/acceptance.md`，确认契约存在且包含具体可观察的 GWT 场景。契约缺失或太空泛，先 spawn `qa-acceptance-planner` 补，再写代码。不能在没有契约的情况下实现——因为实现完再补契约，结果几乎只是在描述代码行为。

这个门禁的设计目的是让验收先行而不是验收后补。Web UI 项目的 acceptance 场景需要稳定的 selector（优先语义选择器，必要时用 `data-testid`），这些要在实现时预埋，不能等 `project-verify` 来了再临时打补丁。

设计系统是代码的依据，不是参考

`design-system/DESIGN.md` 在 03 开始之前就应该存在。它有两条来源：01 阶段用户上传了参考图，01 会基于 `ui-ux-pro-max` 设计知识库生成视觉设计系统，直接写入 `design-system/DESIGN.md`，不放进 `prd.md`；如果 01 阶段没有参考图，02 会在准备阶段从零构建这份文件，把视觉 token、组件气质和关键界面状态一起补齐。无论哪条来源，`prd.md` 里都不包含设计规格——设计始终只在 `DESIGN.md` 里。

03 在开发 UI 时，颜色、字体、间距、圆角、图标库、占位图策略，全从 `DESIGN.md` 取。`prd.md` 是产品意图的叙事，描述的是“做什么”；`DESIGN.md` 是工程合同，描述的是“怎么做出来”。03 实现 UI 的依据只有后者。

可选的并行模块开发

默认情况下，03 是串行的——一个会话顺序做完 TODO。当多个 MVP 模块真正独立（文件所有权不重叠、无运行时数据依赖、每个模块有独立 acceptance）时，可以用 `module-developer` 这个 SubAgent 并行实现不同模块。每个 `module-developer` 只写自己的文件白名单，共享地基（路由、全局类型、主布局）在并行开始前由主会话串行锁定，最后统一接线和全局验证。这和 02 的多 Worker 模式结构相同：并行执行，但共享状态只有一个写入者。

03 的边界

03 只做脚手架、核心功能实现、依赖安装、Codex 自检和基础运行说明。正式验收、`update-prd`、README 完整重写、`prd.md` 归档，全部留给后续独立轮次。Codex 自检不等于正式验收：自检说明首版能运行（lint 通过、核心路径 smoke 可以走通）；acceptance 场景有没有全部通过、状态戳有没有写、证据有没有留，是 `project-verify` 的事。混同这两件事，会让 TODO checkbox 失去可信含义——你不知道勾了的任务是“真验收过了”还是“跑过一遍没报错”。

update-prd：为什么事实只能在开发后写

代码写完了，为什么还要回头整理 PRD？我的判断是：正因为代码写完了，PRD 才终于有资格记录事实。

三层事实的分离

这里的关键是三种文件代表三种不同性质的信息：

```
prd.md                = 产品意图 / 总目标 / 叙事主线    ( 01 生成, 开发前 )
prd/<module>/acceptance.md    = 开发前合同 / 做完标准 / GWT 场景 ( 02 生成, 开发前 )
prd/<module>/README.md + technical.md = 开发后事实 / 当前能力 / 实现上下文 ( update-prd, 开发后 )
```

`prd.md` 代表“我们想做什么”；`acceptance.md` 代表“什么算做完”；模块 README 和 `technical` 代表“代码现在真实实现了什么”。如果在开发前就把全部功能写成模块文档，每改一次代码都要回头改一摞文件，文档就开始追代码而不是记录代码。等 MVP 真正定稿再固化事实，文档才追得上。

模块 README 和 `technical` 的分工也是刻意的：README 写产品视角（用户现在能做什么、入口在哪里、哪些能力已经实现），`technical` 写代码视角（入口路径、数据结构、调用链、日志、测试和边界情况）。验收标准不复制进 README，README 里只有一行指向 `acceptance.md` 的指针——因为验收标准已经有了唯一事实源，不需要第二份。

□0□ 是真相，不是记忆

`update-prd` 不靠记忆决定要更新哪些文档。它从上次同步 PRD 的 commit 比到当前 HEAD，先看出这段时间代码改了哪些文件和范围，再据此增量更新对应模块，不是每次把整套文档重写一遍。未实现、部分实现、降级实现的内容，写回 `TODO.md`，不包装成已完成。

当一轮要更新三个以上模块的 as-built 时，`update-prd` 会 spawn `prd-module-writer` 并行处理各模块——每个 worker 只写自己模块的 README 和 technical，全局索引（`prd/README.md`）和 `TODO.md` 仍由主会话串行写。这和 02 的规划阶段、03 的开发阶段用的是同一套模式：并行叶子，单写者合并。

TODO 归档和 `prd.md` 归档

`update-prd` 还顺手解决了 TODO 越写越长的问题。某个功能块开发完、验收闭环，它把这一块从 `TODO.md` 归档进对应模块的 `prd/`，TODO 里只留一行索引。TODO 始终只承载“还要做什么”，已成为事实的部分搬去 `prd/`，两边各司其职。

在收尾轮次，`update-prd` 还会把根目录 `prd.md` 归档为 `prd.original.md`：原始总目标被保留下来供人回顾，后续 Codex 不再把它当开发依据。真正的当前项目状态，转移到 `prd/` 和 `TODO.md` 中。`update-prd` 不重建 acceptance 契约——那是 02 的产物，除非发现状态戳与代码不一致，否则它只补 as-built，不动做完标准。

设计的真正目的

CodeNow 的冷启动设计，目的不是让文档更齐，而是让每一份文档在正确的时间出现。

01 的 `prd.md` 不早拆模块，是因为模块文档在想法阶段没有资格出现；02 的 `acceptance.md` 不等代码写完才补，是因为做完标准必须先于实现；03 的自检不冒充正式验收，是因为证据只有经过完整验证才算数；`update-prd` 不把未来计划写成已实现事实，是因为事实只能描述当前代码里真实存在的东西。

这几条约束约束的不是用户，而是 Codex 的写入权限。AI 太容易把“可能要做”“应该做”“正在做”“已经做完”“已经验证”混在一起。脚手架的价值，就是把些状态分开放在不同位置，让 Codex 在每一轮开始时都能读到当前最可信

的事实，而不是在旧愿望、当前代码和聊天记录之间猜哪个才算数。

对没有工程基础的用户来说，这才是真正的减负：不是没有过程，而是不要他先懂所有过程——过程由脚手架接住，每个阶段只把他此刻有能力回答的判断交给他。

从这里往后，CodeNow 才能进入更细的一层：冷启动把约束写入了阶段边界，日常迭代同样需要约束。下一节会讲我如何把迭代、验收、调试和文档同步拆成四个职责清晰的节点，以及为什么每个节点的"不能做什么"和"能做什么"同样重要。

5.3 冷启动之后，日常迭代的四个节点是怎样分工的

上一节讲到，CodeNow 的冷启动设计是在约束 Codex 的写入权限：每个阶段只写自己有资格写的东西。这条逻辑不会在首版 MVP 跑起来之后就结束。真正的挑战在后面：项目有了代码、有了 TODO、有了验收契约，用户开始提持续的日常改动。这时候，如果没有对应的约束，Codex 在迭代中会系统地混淆五件事：

做了一点 / 做完了 / 自己检查通过了 / 正式验收通过了 / 文档已同步

每混淆一次，项目的可信状态就污染一处。TODO 的勾不再可信，acceptance 的状态戳不再可信，PRD 的描述不再可信。等你想知道"这个功能到底算不算完成"的时候，已经没有地方可以信任了。

CodeNow 的回答是把日常迭代拆成四个节点，每个节点有明确的能做什么，更重要的是有明确的不能做什么。

四个节点，四道隔离墙

project-iterate
 读取 TODO 与 acceptance
 实现功能
 补 selector / 日志 / fixture / 未配置态
 做 Codex 自检
 回写必要进度

project-verify
 按 acceptance 逐条执行
 采集截图、日志、命令或测试证据
 回写状态戳与 TODO checkbox

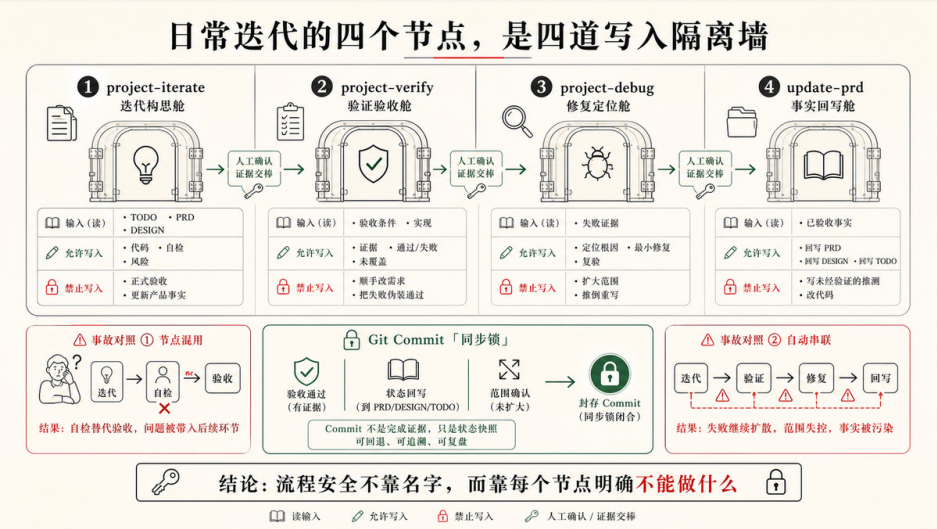
project-debug
 验收失败时复现、采证、最小修复

 修完再回 project-verify 复验

update-prd
 按 git diff 识别代码变化范围
 把已实现能力写入 prd/<模块>/README 和 technical
 把差距和未完成项写回 TODO

设计四个节点不是为了让流程更复杂，而是为了让每个节点都不越权：

节点	明确不做事
project-iterate	不给 acceptance 盖 □，不跑 update-prd
project-verify	不修代码，不做调试
project-debug	不打 §2 checkbox，不把"修了"当"通过"
update-prd	不把未实现的目标写成已实现能力



越权往往不是大动作，而是顺手发生的：iterate 顺手给 TODO 打勾，debug 顺手把修复标成通过，verify 顺手改几行代码，update-prd 顺手把意图写成事实。每一次"顺手"都在悄悄侵蚀可信度。侵蚀是隐蔽的——表面上进度在推进，实际上你已经不知道"完成"意味着什么了。

有顺序，但不自动链

四个节点之间没有自动调用——每次从一个节点切换到另一个，都需要用户或 Codex 明确发起。这同样是刻意设计的。如果 project-iterate 自检通过后自动触发 project-verify，"自检不能盖 □"这条规则就在工程上失效了——用户没有决策机会，就已经进入了正式验收。什么时候"算完成了可以正式验"，是人的判断，不是 Codex 自己的判断。

典型的日常迭代链路是这样走的：

用户需求

- project-iterate (实现 + 自检)
 - ↓ 用户看了觉得可以，说"验收一下"
- project-verify (证据化验收)
 - ↓ 某个场景失败
- project-debug (复现 + 最小修复)
 - ↓ 修完，回去复验
- project-verify (再次验收该失败场景)
 - ↓ 全通过，继续下一个 TODO
 - ↓ 若干轮之后，TODO 变长 / 里程碑到了
- update-prd (同步 as-built 文档)

整条链是"实现 → 人决策 → 验收 → 失败修复 → 复验 → 周期性归档"。

project-iterate：在实现时就定好"做完是什么意思"

用户不会在开始之前主动想"做完是什么意思"——没有理由，也没有时机。功能跑起来，他认为完了。等到验收才发现：按钮没有稳定的定位方式，失败了只显示"请求失败"，没有 Key 的环境根本无法触发未配置态。验收变成了补债。

我把解法拆成两步：在实现之前替用户锁定"做完的定义"，然后在实现过程中把验收需要的入口顺手埋进代码。

第一步是锁定定义。project-iterate 动手前，先确认对应 `prd/<模块>/acceptance.md` 存在且有具体可执行的 GWT 场景。契约缺失时，它会 spawn `qa-acceptance-planner`——这个 Subagent 只做一件事：把功能规格转成 GIVEN/WHEN/THEN 场景，所有场景初始状态是 □ 未实现，没有权限写 TODO 或改代码。契约就绪，iterate 再开始。

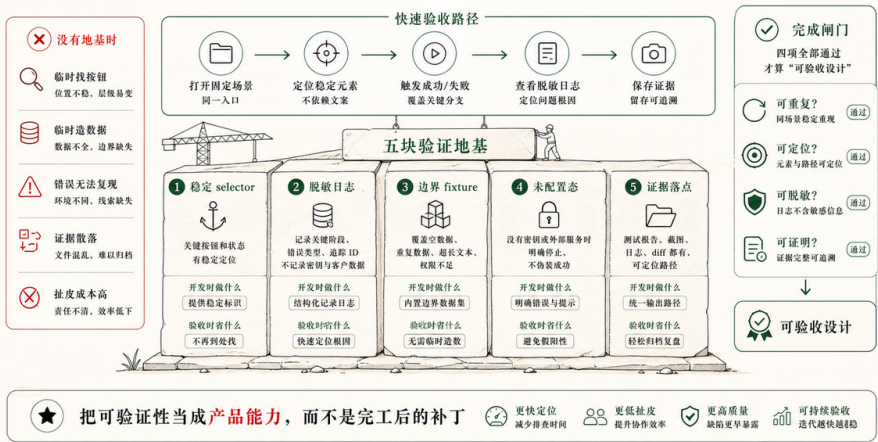
第二步是预埋。有了可执行的场景，实现阶段顺手把验收需要的入口一起埋进代码：

- 稳定 selector：acceptance 会触及的 UI 元素，优先语义选择器 (`role/label/accessible name`)，必要时加 `data-testid`。按钮文案一改测试就丢，同一页面两个"确认"自动化就懵——验收时才暴露就晚了。
- 脱敏日志：API、IPC、第三方请求、降级逻辑，记录入口、参数摘要、成功 / 失败结果，不记录 Key 和隐私原文。失败时只看到"请求失败"、分不清是

401、超时还是结构变了，调试路就堵死了。

- 可复现状态：空态、未配置态、错误态要能被主动触发。没有真实 Key，验收环境也能验证"未配置态行为正确"。
- fixture 和测试入口：边界条件和异常返回用集中 fixture 覆盖，不靠真实数据碰运气。

验收效率，来自开发时提前铺好的地基



实现时，iterate 按改动类型渐进式加载参考资料：改 UI 才读设计系统，改 API 才读日志规范，改平台边界才读框架参考。做完功能后只做 Codex 自检，不给 acceptance 场景盖 □，不回写 TODO §2 checkbox。自检是"本轮改动基本正确"，不是"场景逐条通过留下证据"——用户不需要理解这个分层，他只需要在觉得可以了的时候说"验收一下"，系统自然走到下一个节点。

project-verify：让"打勾"这个动作有可核查的含义

"我点了几下，好像没问题"——在用户体感上，这和通过验收是同一件事。没有证据，没有可追溯记录，TODO 的勾打上去之后没有人知道它代表什么。他也没有理由在意"能跑"和"按合同定义通过了"的区别——直到几轮之后出了问题，才发现根本不知道哪个状态是可信的。

所以我让"打勾"必须附带可追溯证据。`project-verify` 只在特定条件下触发——用户说"验收一下"、声明某个功能完成、准备发布，或触及高风险边界需要证据。触发后，它按 `acceptance` 场景逐条执行，把状态戳写回 `acceptance.md`，把 `TODO` checkbox 打勾。这是"打勾"的全部含义：不是"改了代码"，不是"smoke 没报错"，而是"按合同场景逐条通过，证据留在案"。

证据类型要标清楚：

<code>static</code>	lint/test 命令输出，可回归
<code>browser-smoke</code>	一次性浏览器走查，不可回归
<code>e2e-test</code>	Playwright 测试，可回归
<code>manual</code>	必须人工确认
<code>skipped</code>	条件不足（如 API Key 不可用）

"一次走查"和"可以纳入 CI 的测试"是性质完全不同的东西，混在一起叫"已验收"，可信度会随时间衰减。跨多个模块时，`verify` 可以 fan-out 给多个 `acceptance-verifier` worker 并行执行——每个 worker 只验一个模块，独占端口和证据目录，回写状态戳后返回摘要；主会话 `reduce`，统一更新 `TODO` checkbox。对用户来说，说"验收一下"，系统在背后协调，他只看到结果。

`project-debug`：先收束根因，再修复，而不是"改一下试试"

用户遇到问题，第一反应是"改一下试试"。改了一个地方，试一下；不行，再改另一个。改来改去，根因不明，最后不知道哪个改法有效，也不知道有没有顺手引入新问题。更麻烦的是：修完了，现象消失了，就把 `TODO` 打勾——但根因可能还在，只是暂时没触发。

我把 `debug` 的顺序强制反过来：先取证，再修复。`project-debug` 的流程是收束，不是发散：复现原失败路径 → 读 `console/network/server` 日志 → 定位最小可疑区域 → 最小修复 → 复验 → 回 `project-verify` 复验 `acceptance`。"改了"不等于"通过了"，`debug` 节点只负责找根因和修复，盖 □ 是回 `verify` 之后的事。

`debug` 按问题类型渐进式加载参考资料：描述模糊时先用三个问题对齐预期和实际；浏览器问题采集 `console/network` 证据；常见症状——页面空白、数据不变、登录失败——直接走对应 `playbook`；需要独立验证 API 或数据链路

，写最小 TypeScript 验证脚本。

临时探针有自己的生命周期：修复后，纯为定位而加的噪声日志必须删掉；只有确认属于根因链路、有长期理解价值的日志，才整理成正式脱敏日志保留。

`project-iterate` 在实现时预埋的 `selector`、日志和 `fixture`，在这里发挥第二次价值：`debug` 可以沿验收路径直接复现，从日志里看清失败发生在哪层，再做最小修复。有埋点和没埋点，`debug` 的效率差距可以是十倍——这也是为什么实现时顺手埋，不只是为了验收方便，也是为了让失败不可怕。

`update-prd`：让 Codex 下一轮进来时读到的是真实的项目状态

用户不维护文档——这是正常现象，不是他的问题。但文档不更新有一个他感知不到的后果：几轮迭代后，Codex 下一次进来读 PRD，读到的是三周前的状态。Codex 以为某个功能已经实现，其实那次迭代里被降级了；以为某个 API 字段还在，其实早改了。用户看不到这个“知识腐烂”在发生，但每一次 Codex 的判断都在基于越来越虚假的上下文，偏差越来越大。

我的做法是用 `git diff` 自动识别这段时间代码变了哪些范围，再增量更新对应模块的 `as-built` 描述。`update-prd` 从上次同步 `commit` 比到当前 `HEAD` (`git diff <lastPrdSyncCommit>...HEAD --stat`)，先知道哪些文件变了，再按范围更新，不增量重写。上次同步的 `commit SHA` 存在 `prd/update-prd-state.json` 里；不存在时改走全量扫描。需要同步三个以上模块时，`fan-out` 给多个 `prd-module-writer` worker 并行处理，主会话 `reduce` 后合并索引、把差距写回 `TODO`。

触发时机也是设计的一部分：不是每次 `iterate` 完就跑，而是在里程碑、首版收尾、大版本交付，或 `TODO` 过长需要归档时触一次。频率低，但每次同步的都是当前代码里真实存在的东西。三层文件的分工：

```
prd.md / acceptance.md      = 意图与做完定义（开发前合同，不改）
prd/<模块>/README.md + technical = 当前代码事实（as-built，update-prd 写这里）
TODO.md                     = 还需要做什么（差距写回这里）
```

未实现、部分实现、降级实现的内容写回 `TODO.md`，不包装成已完成。这和冷启动的逻辑一脉相承：文档必须等到有资格写的时候才能写。

git commit 是这套流水线的同步锁

四个节点最终需要一个工程层面的同步锁：每一段确认的进展，应该落成一个 commit。这不只是代码管理习惯，而是 `update-prd` 能正常工作的前提。

`update-prd` 的核心机制依赖 `lastPrdSyncCommit`：它从上次同步 PRD 时的 commit 比到当前 HEAD，才知道"这段时间代码改了些什么、要更新哪些模块文档"。如果长时间不 commit，这个 diff 窗口就会越来越大，`update-prd` 一次性要处理的变化范围会变得难以驾驭。如果 commit 粒度太混，一个 commit 里同时有代码改动、acceptance 状态戳和 PRD 更新，diff 就失去了语义，很难判断哪些变化是"功能实现"、哪些是"验证记录"、哪些是"文档同步"。

一个简单的 commit 分层习惯，可以让整条链路更清晰：

```
feat: 实现 AI 建议展开来源功能    ← project-iterate 完成、自检通过后
verify: AI 建议模块 §2.3 验收通过 ← project-verify 打勾、回写状态戳后
fix: 修复建议确认状态写入错误     ← project-debug 修复并复验后
docs: 同步 AI 建议模块 as-built    ← update-prd 完成文档同步后
```

这样的分层让 `git log` 本身就能看出这段时间经历了哪些开发节点，而不是全混在一条"update various things"里。`update-prd` 读 diff 时，也能更准确地判断哪些变化是代码功能改动（需要更新 `as-built` 文档），哪些是验证证据或文档同步（不需要重新触发模块 PRD 更新）。

commit 的节拍可以这样理解：project-iterate 的 Codex 自检通过后，提交代码；project-verify 打勾并回写状态戳后，提交验证记录；project-debug 修复并复验后，提交修复；update-prd 完成文档同步后，提交 PRD 更新。不需要每个小步骤都 commit，但每个节点完成的边界处，commit 一次，是让后续节点有可靠基线的最简单方式。

git 在这套系统里不是事后备份，而是流水线的时间轴。同步点越清晰，四个节点之间的交接越可信。

设计自己的系统时，先抓五个维度

如果你不准备完整复制 CodeNow，也可以先把这一节的判断拿走：不要等验收时才思考怎么验。任何日常迭代系统都需要在五个维度保持可验收性。

第一，UI 主流程有没有稳定 selector。不是所有元素都要加 testid，但 acceptance 会点击、填写、选择或断言的元素必须可定位。第二，外部服务、storage、IPC 和 API 调用有没有脱敏日志。日志不需要多，但关键入口、成功、失败和降级要能看见。第三，数据来源有没有说清楚。真实数据、fixture、空态、未配置态和示例模式不能互相冒充。第四，证据有没有落点。截图、console 摘要、测试结果和验证 summary 应该有固定目录，而不是散在聊天里。第五，日常迭代和正式验收有没有分开，PRD 有没有随代码同步。平时自检要轻，正式验收要有证据，失败修复要回到验收复验，文档要跟着代码事实走。

这五个维度不会替你保证产品正确，但会显著降低“正确无法证明”的概率。对 Agentic Coding 来说，这是很大的变化：Codex 不只是写出一个看起来能用的功能，还在写代码时顺手为下一轮验证、debug 和协作留下可接住的结构。冷启动的约束在日常迭代里以这四个节点的形式延续下去。

下一节会继续往前看一层：如果这些动作要稳定发生，只靠 Skill 还不够。你还需要一个薄的 AGENTS.md 做入口路由，也需要 rules 把长期边界放到该出现的位置。

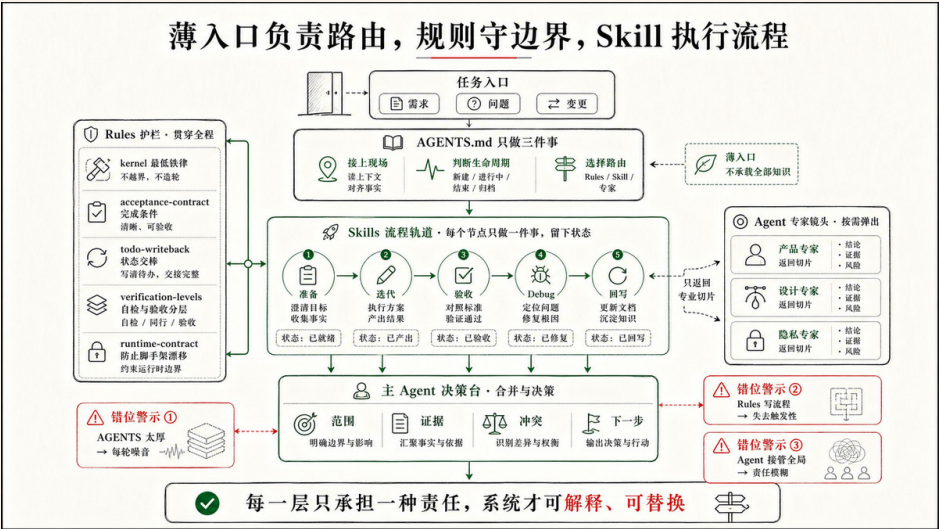
5.4 如何设计 AGENTS.md 和 rules

上一节讲完了日常迭代的四个节点，结尾提到一个悬而未决的问题：这些动作要稳定发生，只靠 Skill 还不够。project-iterate 必须先看验收契约、不能把自检当正式验收、要把进度写回 TODO——如果这些约束每次都靠用户提醒，CodeNow 并没有真正减轻他的负担，只是把写代码的压力换成了背流程的压力。

问题的根源在于：Skills 告诉 Codex 怎么执行一项任务，却不能告诉它现在该进入哪个 Skill，上一轮到哪里了，哪些约束每轮都必须保持。这两件事，

Skills 都没有能力负责，需要另外两层来分别承担：

- Skills → 怎么做一件事（执行方法）
- rules → 什么不能做、做到什么程度（持久边界）
- AGENTS.md → 现在在哪、该走哪条路（启动入口）



三层各司其职，才能让一个没有工程背景的用户，不靠记忆就能让系统维持健康状态。

在逐条展开之前，先分享一组设计每条 rule 时都会问的问题——这也是读这一节最有用的视角：

- 如果没有这条 rule，项目状态会怎样失控？
- 它拦住哪种具体失控、保护哪个事实源？
- 哪些任务不适用，避免把轻量任务变重？

带着这组问题看 CodeNow 的 AGENTS.md 和七条 rules，就不再是看一组文件清单，而是看我在反复被同类问题绕过之后，做出的每一次设计选择。

AGENTS.md：接上现场，走对路径

用户做完第一版 MVP，代码已经跑起来了，第二天他发了一条消息：“继续做下一项。”

Codex 看到这句话，可以有五种完全不同的理解：继续昨天被打断的那项 TODO？做新开的下一个功能？修一个刚暴露的 bug？整理 PRD 文档？还是说项目还有什么准备工作没做完？如果没有任何上下文，Codex 的选择只能靠猜，或者从上一轮聊天记录里猜——而聊天记录可能早已被截断。

这就是 **AGENTS.md** 要解决的第一层问题：让 Codex 知道这个项目现在在哪个位置，当前任务状态在哪里找。

CodeNow 的 **AGENTS.md** 开头写了几件看起来很朴素的事：项目初始化后以根目录 **TODO.md** 为准；开始工作前按需读取 **TODO.md**、模块 **acceptance.md** 和 **design-system/DESIGN.md**；当前仓库以 **AGENTS.md** 与 **.codex/** 为主事实源；旧兼容目录是历史路径，不要新增引用；代码默认生成在仓库根目录，不要自动创建 **web/**、**client/**、**frontend/** 子目录。

这些不是装饰性说明，每一条都在拦一个真实发生过的错误。用户说"继续做下一项"，Codex 如果不先读 **TODO.md**，就不知道上一轮哪些任务完成了、哪些被阻塞、下一项应该从哪里恢复。没有指定子目录，Codex 凭习惯创建一个 **web/**，后面的启动命令、路径引用和文档说明就会全都绕一层。仓库里残留历史兼容路径，Codex 如果不知道主事实源在哪里，就可能把旧路径当成当前规范继续扩写。

第二层动机，是降低用户的表达成本。只读评估、轻量维护、实现 TODO 第 x 项、正式验收/打勾、教学模式——这些省话口令不是为了看起来像命令系统，而是让用户不用背完整流程。一个不懂 ATDD、不懂状态戳、不懂 **project-verify** 是什么的用户，也可以用"正式验收"表达清楚：这次不是普通自检，要按验收契约留证据。口令的本质是把专业意图压缩成日常语言。

第三层动机，是让路线在行动之前就可见。**路由判断：... → 使用 ...** 这行声明的价值不在于格式，而在于把 Codex 的判断放到修改发生之前。用户在代码改动之前就能看到：你现在把这件事当成冷启动、日常迭代、正式验收、debug，还是轻量维护？如果路线判断错了，人可以立刻拦住，而不是等返工时才发现走偏。

三层加在一起，**AGENTS.md** 做的事是：识别当前阶段、声明主事实源、读取 TODO 作为任务控制面、降低口令成本、暴露路由判断。细节都交给 **rules** 和

Skills 处理；**AGENTS.md** 只负责让每轮任务从正确的位置开始。

lifecycle.md：同一句话在不同阶段是不同任务

有了 **AGENTS.md** 解决"接上现场"，还有一个问题没解决：各个 Skill 之间的边界由谁来说清楚？

如果每个 Skill 自己维护一份"我在什么时候用、什么时候改走别的路"，就会出现这种局面：**project-iterate** 说"根因不明的问题先去 debug"，**project-debug** 说"问题修完去 iterate"，**project-verify** 说"失败了去 debug 修"，**update-prd** 说"日常小改别来找我"。用户或 Codex 想确认"某件事该走哪条路"，需要把四个 Skill 的描述拼在一起才能读懂。某一天某个 Skill 的描述变了，其他 Skill 没有同步，路线图就漂移了。

lifecycle.md 作为唯一事实源解决这个问题。所有 Skill 涉及"与其他 Skill 的分工"时，引用这份规范而不是自己复述。路由表只有一份，每次调整只改一个地方。

这条规则同时解决另一个问题：同一句用户输入，在不同项目阶段对应的任务完全不同。

```
"继续做"——还没有 prd.md：先让想法成形
"继续做"——有 prd.md 没有 TODO：先做项目准备
"继续做"——首版还没跑起来：进入 03 首版开发
"继续做"——已有代码和 TODO：才是 project-iterate
"继续做"——行为异常根因不明：先 project-debug
```

对新手来说，这种区分尤其重要——他不一定知道"现在还不能写代码"，不一定知道报错应该先复现采证而不是直接重写。生命周期规则把这些判断从用户的记忆里拿出来，让 Codex 按当前项目状态自动路由。

lifecycle.md 还管另一类边界：subagent 的写入隔离。主会话是唯一 orchestrator，叶子 worker 只写各自的物理文件范围，共享文件由主会话 reduce 阶段单写。并行执行可以提效，但不能变成"多个执行者同时改 TODO、PRD、acceptance"——共享状态只能有一个最终写入者。

kernel.md：每轮都要进入上下文的最低铁律

有了路由，还有四件事比路由更基础——忘了不只是走错路，而是项目状态立刻变得不可靠。这四件事必须每轮都记得，但我又不想让 Codex 每轮读完所有 rules。

这就是 kernel.md 只保留四条常驻铁律的原因：

改了产品行为或关键代码，要回写 TODO。
写 §2 功能前，先确认 acceptance 存在且具体。
本轮没做 TODO 回写或没补契约，最终回复要说明原因。
开始修改、开发、验收或同步前，先声明路由。

四条分别对应四种常见失控。

"代码变了，项目记忆没变"：本轮改完，下一轮 Codex 只看到旧 TODO，重复做、漏做，或把未完成当已完成；回写 TODO 是让项目状态从聊天里落回仓库。

"先写代码，再补完成标准"：acceptance 后补几乎必然变成描述既有行为，不再是开发前合同。

"沉默掩盖了没做的事"：没有回写 TODO、没有补契约，如果最终回复不说明，用户无法区分"刻意跳过"和"Codex 忘了"。

"任务还没开始，路线已经错了"：路由声明让错误在行动之前暴露，不是等代码改完才发现走偏。

kernel.md 不追求完整，只保留每轮都值得进入上下文的最低边界。其余细节按需加载——如果把所有规则都放进常驻层，规则会把 Codex 的上下文占满，真正该做的事反而被挤掉。

acceptance-contract.md：把"做完"从感觉里拿出来

如果只能选一条最影响项目可信度的 rule，我会选 acceptance-contract.md。

新手项目里，“做完”最容易变成一种感觉。页面能打开，按钮能点，lint 也过了，于是很自然地说“完成了”。但一个实际的产品功能不是只看按钮能不能点。AI 建议有没有来源？沟通记录为空时会不会编造？没有 API Key 时会不会伪装成真实分析？销售确认前会不会进入已执行记录？这些才是功能真正不能被接受的标准。如果这些标准没有在开发前写清楚，“完成”就只是那天 Codex 自我评估的结果。

`acceptance-contract.md` 的动机，是让完成标准先于代码出现，并且只在一处定义。CodeNow 规定 `prd/<模块>/acceptance.md` 是唯一验收事实源，TODO 只放任务和一行指针，`TESTING_CHECKLIST.md` 和 `test-results/` 只记录证据，模块 README 和 `technical` 只记录 as-built 事实——不复制验收清单。

这条规则拦住三种失控：验收后补（代码写完再写 acceptance，最容易把现有行为包装成标准，约束能力消失）；标准多头（TODO 一版、README 一版、测试清单一版，产品方向一变，多处复制的标准就开始漂移）；自检盖章（`project-iterate` 跑完 Codex 自检，不能把场景标成 ☐；只有 `project-verify` 按场景执行并留下证据，才能写状态戳）。

这条 rule 同样写了什么时候不适用：只读评估、规则讨论、错别字、纯格式、小样式、不改变数据语义的内部重构，不应该被拉进 ATDD。好的规则不是把所有任务都变重，而是在真正会影响完成定义的地方变硬。

`todo-writeback.md`：让下一轮不用翻聊天记录

很多人第一次用 AI 做项目时，会低估“聊天记录不是项目状态”这件事。本轮 Codex 知道自己做了什么，用户也知道，因为刚看完回复。可是换一个会话、过一天、改另一个模块以后，这些状态就开始模糊：哪个功能做完了？哪个只是部分完成？哪里被 API Key 阻塞？哪个验收失败还没修？如果只留在聊天里，项目很快就会失去连续性。

`todo-writeback.md` 解决的就是这种跨会话失忆。CodeNow 要求：完成或部分完成 checkbox、修改产品行为、API、平台能力、主流程、验证失败或跳过、发现新任务或规格差距——这些都要回写 TODO。核心不是“勤写文档”，而是让下一轮能恢复现场。

但这条 rule 同样写了什么时候可以不回写：只改错别字、纯文案、纯格式，只回答问题，只读评估，或本轮修改不对应任何开发计划变更。这一半同样重要。如果每个小动作都往 TODO 里塞记录，TODO 会从任务控制面变成流水账，新手打开以后更不知道下一步做什么。

这条 rule 想同时拦住两个方向相反的问题：一个是 silent drift，代码和 TODO 悄悄不一致；另一个是文档噪声，TODO 被低价值记录撑爆。把"必须写"和"可以不写但要说明"分清楚，才能让 TODO 始终只承载"还要做什么"。

verification-levels.md：自检要轻，验收要硬

验证最容易出现两个极端。太松：跑过 lint，页面 smoke 一下，就说功能验收通过——TODO checkbox 很快失去可信度。太重：改一个文案也跑完整 build、全量测试、浏览器回归和正式验收——用户觉得脚手架笨重，最后什么都想绕过去。

verification-levels.md 的动机，是把验证从"跑得多不多"改成"语义清不清"。CodeNow 只暴露两类验证动作：

Codex 自检证明的是本轮改动在当前上下文里基本正确、未明显破坏相关入口。强度随风险伸缩：可以是 lint、类型检查、相关单测、局部 smoke 或动态走查，但默认不把 build、全量测试、浏览器回归和 project-verify 串成固定收尾。

正式验收证明的是某个功能按 prd/<模块>/acceptance.md 逐条通过，并留下可追溯证据。必须由 project-verify 执行，写状态戳、证据路径，并同步 TODO checkbox。

这条 rule 同时规定下界和上界。高风险主流程、API、安全边界不能只靠"我看起来可以"；但低风险规则维护、文案和小样式也不应该被过度验证拖住。这种区分同时守住两端：平时改动不会被重流程吓住，正式完成时有证据可以信任。

还有一个细节：project-verify 失败时不修 bug，只记录失败证据，交给 project-debug 复现和最小修复，修完再回验收复验。失败记录、修复动作、复

验证证据分开，“完成”才不会变成一句乐观判断。

runtime-contract.md：脚手架自己也会漂移

前面几条 rule 管的是项目功能状态，runtime-contract.md 管的是脚手架自身。

CodeNow 不是一套永远不变的文件。它会迁移路径，删除旧目录，调整 subagent 格式，决定哪些东西应该复制到新项目、哪些只留在脚手架开发仓库。如果没有一条规则管这些，脚手架自己也会慢慢变成一团历史痕迹。

这条 rule 守住几个具体的漂移点：当前主路径是 .codex/，旧兼容目录只是历史路径；AGENTS.md 是主协作入口，其他运行环境需要的兼容镜像不能成为新的事实源；.codex/agents 的真实文件扩展名是 .toml，文档中不能把 subagent 继续描述成 Markdown；当前没有内置 .codex/settings.json 与 hook，不能假装这些机制仍然存在。

这些约束看起来像维护细节，但它们直接影响读者能不能复制 CodeNow。一个新手 clone 仓库，如果文档里一会儿说 .codex/，一会儿说旧路径，一会儿说有脚本门禁，仓库里却根本没有，他会立刻失去信心。

我也在自己的仓库里看到过这种漂移：AGENTS.md 里残留了对一个已删除速览文档的引用。问题本身不大，但它说明了一件事：rules 会腐烂，入口也会腐烂。runtime-contract.md 要求修改 .codex/ 或 AGENTS.md 后，人工通读一遍，确认引用路径存在、规则和 Skill 不冲突、没有悬挂引用。这不是形式检查，而是维护脚手架自身的可信度。

这条 rule 还承认了一件事：当前没有机器门禁，一致性检查主要靠人工核对。不要把没有自动化的地方伪装成已经自动化；等复杂度上来，再引入脚本和 hook。

coach.md：新手路径与执行流程分开走

最后一条 rule 最短，但体现了一个重要取舍。

CodeNow 的目标用户很多没有工程背景，所以需要有一个教学模式。但教学和执行如果混在一起，每次开发、验收、PRD 同步都会变成冗长教程，真正要做的事被挤到后面。

`coach.md` 只有用户明确说"教学模式"、"一步一步讲"、"帮我理解为什么"，或自称零基础、看不懂代码，才按这条规则启用。启用后的原则是：先保护信心，术语先落地，只讲当前问题，给安全小实验。边界是：教学不改更多代码，解释和修改仍然遵守当前任务路由；用户只要结果时，先完成结果，再简短解释关键原因。

新手友好不是把所有流程都讲慢，而是在用户需要理解时有一条温和的入口；在用户只想完成任务时，不把解释变成新的负担。

rules 的设计方法：先写失败，再写边界

把这七条 rules 放在一起，设计方法是共同的：不要先问"我要写哪些规则"，先问"如果没有这条规则，项目状态会怎样失控"。

<code>lifecycle.md</code>	→ 拦住走错阶段
<code>kernel.md</code>	→ 留住最容易忘、忘了代价最大的动作
<code>acceptance-contract.md</code>	→ 拦住完成标准漂移
<code>todo-writeback.md</code>	→ 拦住跨会话失忆
<code>verification-levels.md</code>	→ 拦住自检冒充验收，以及过度验证
<code>runtime-contract.md</code>	→ 拦住脚手架自身漂移
<code>coach.md</code>	→ 拦住教学和执行互相污染

每条 rule 对应一种反复发生过的失控，边界都包含两面：触发后必须做什么，以及哪些任务不适用。只有当某个问题会反复出现、会影响状态可信度、会让新手更难继续，才值得沉淀成 rule。一次性偏好放在任务说明里，当前进度放在 TODO 里，产品范围放在 PRD 里，完成标准放在 acceptance 里，流程步骤放在 Skill 里。

`AGENTS.md` 和 rules 不是为了把提示词写得更完整，而是为了把最容易失控的边界放到项目系统里——让一个没有工程背景的用户，不靠记忆就能让系统维持健康状态；让 Codex 每轮开始时都能读到当前最可信的事实，而不是在旧愿望、当前代码和聊天记录之间猜哪个才算数。这套逻辑不只适用于 `AGENTS.md` 和 rules，也适用于整套脚手架的设计——下一节把它展开到

Skills、SubAgents 和其他部件上。

5.5 如果你要设计自己的 Agentic Coding 脚手架，应该考虑什么

写到这里，CodeNow 的几个关键部件已经摊开了。如果你要为自己的项目从 0 到 1 设计一套 Agentic Coding 脚手架，应该从哪里开始？

我的答案是：不要从目录开始，也不要从工具清单开始。先问你的项目最容易在哪里失去状态。

不要先设计目录，先找到失控点

很多人一听到脚手架，第一反应是建目录。

是不是要有 `AGENTS.md`？要不要放 `PRD.md`？要不要写一组 Skills？要不要接 MCP？要不要加 hooks？要不要安排几个 SubAgent？

这些问题都重要，但顺序反了。

一个 Agentic Coding 脚手架真正要解决的，不是“项目里有哪些文件看起来像高级工程”，而是“当 Codex 开始拥有一定自主性时，哪些事情会失控”。目录只是答案的一种形状，不是问题本身。

如果你做的是一个简单脚本，可能根本不需要完整的 PRD、DESIGN、acceptance 和多阶段 Skill。一个清楚的 `AGENTS.md`，一条运行命令，一组最小验证，就够了。

如果你做的是一个长期产品，需求会变，页面会改，外部 API 会失效，用户会反馈，验收标准会漂移，那就不能只靠聊天记录。你需要让 Codex 每一轮都能知道：现在做到哪里，哪些事实已经成立，哪些只是计划，哪些场景必须被验证，哪些风险不能越过。

如果你做的是面向新手的项目脚手架，问题又会更具体。新手不是懒得写 PRD，而是不知道什么叫可执行的 PRD；不是不愿意验收，而是不知道什么

才算验收；不是不想维护状态，而是不知道状态应该放在哪里。CodeNow 想解决的“无痛”，不是让 AI 神奇地一次生成一切，而是让这些原本需要工程经验才能完成的动作，被脚手架一步一步接住。

所以，设计脚手架的第一步不是建文件夹，而是列出你最害怕反复发生的失败。

例如：

- Codex 每次进入项目，都不知道上次做到哪里。
- 用户说了一个想法，Codex 直接开始写代码，没有先确认目标、非目标和验收标准。
- 页面看起来能用，但缺少稳定 selector，正式验收时只能肉眼判断。
- 外部 API 没有 Key，Codex 把错误状态当成业务失败处理。
- TODO、PRD、代码事实互相冲突，但没有规则说明哪个更可信。
- 开发自检通过以后，Codex 直接勾掉正式验收。
- 新手问“帮我继续”，Codex 不知道该进入冷启动、日常迭代、Debug 还是验收。
- 脚手架自己改了入口或 rules，却没有检查悬挂引用和过期路径。

这些失败比目录更重要。目录只是把失败分门别类地收住。

一条好的 rule，不是因为它看起来严谨，而是因为它能拦住一种真实发生过的漂移。一个好的 Skill，不是因为它写了很多步骤，而是因为它把某个阶段的输入、输出、边界和退出条件讲清楚。一个好的 SubAgent，也不是因为它让系统显得更“多智能体”，而是因为某类工作确实适合被隔离到独立上下文里完成。

从失控点出发，脚手架才会长成项目需要的样子。

先把任务入口、验证和边界做扎实

设计脚手架时，最容易被复杂架构吸引。看见 Skills，就想把每个动作都写成 Skill；看见 SubAgents，就想让多个 Agent 并行工作；看见 MCP 和插件，就想把所有外部系统都接进来。可真正决定 Agentic Engineering 是否可靠的，往往不是这些更高阶的部件，而是几件更朴素的事有没有做扎实。

第一，任务入口必须同时交代目标、上下文、限制和完成条件。只说“帮我做一个客户线索工具”，Codex 得到的是一个方向；补上“给销售新人用”“先做线索录入、AI 建议和来源追溯”“不能调用真实付费接口”“当这三个验收场景可运行并留下证据时才算完成”，Codex 才得到一个可以执行、可以收束的任务。入口写得越像愿望，执行越容易漂；入口写得越像合同，后面的实现、验证和汇报越容易对齐。

第二，项目入口要短而准。AGENTS.md 的价值不是把所有事实都塞进去，而是让 Codex 知道先读哪里、怎么判断当前阶段、哪些命令可以运行、哪些行为必须确认、完成以后要把状态写回哪里。在支持分层指令的工具里，越靠近当前目录的入口越会影响实际行为，这意味着入口文件本身就是工程配置。它应该像配置一样被维护：短、准、可验证，出错以后再补规则，而不是一开始写成一部百科。

第三，验证必须从“人来感觉”前移到工作流里。一个脚手架如果不能告诉 Codex 如何自己运行检查、保存截图、记录命令、复现失败和归档证据，那它最后还是会把压力还给人。人会被迫一遍遍问：“你怎么知道做完了？”所以，测试、构建、lint、截图、fixture、日志和人工确认点，都不只是交付前的附加动作，而是脚手架设计的一部分。它们决定了“完成”是不是可以被别人复查。

第四，复杂度要从风险里长出来。一个小任务不需要完整生命周期，一个临时脚本不需要多 Agent 协作，一个内部原型也许不需要复杂发布门禁。反过来，涉及真实数据、支付、隐私、权限、生产环境和长期维护的项目，就不能靠“自检通过”草草收场。Agentic Engineering 的成熟，不在于所有项目都用同一套重流程，而在于你能判断什么时候轻，什么时候重，什么时候必须停下来让人确认。

第五，权限边界不能只靠审批按钮。沙箱、审批和权限控制很重要，但它们只是防线的一部分。真正可靠的系统还需要项目内规则、外部 API 降级策略、日志脱敏、真实数据隔离、危险命令确认、脚本可审查和人的最终责任。否则，Codex 即使没有直接越权，也可能通过修改配置、脚本、测试数据或项目内文件，把风险带到你没预料的地方。

第六，把每个阶段的写入权限想清楚。AI 最容易把五种状态混成一种：可能要做、应该做、正在做、已经做完、已经验证——它们在 Codex 手里很容易压成“此刻写”。设计脚手架时，一个根本性的问题是：每个阶段有资格写什么？

想法阶段写意图
准备阶段写契约和计划
开发阶段写代码和自检结果
验收阶段写证据和状态戳
完成以后写当前事实和差距

这五行是对 Codex 写入权限的主动分配，不是文档纪律。如果准备阶段就把未确认的实现细节写进模块文档，下一轮 Codex 会把计划误读成事实继续推进；如果开发阶段自检通过就在 acceptance 打状态戳，“完成”就失去了可信含义；如果完成阶段把意图和代码事实写进同一份文件，下一个会话就得在“我们想做什么”和“代码真实实现了什么”之间猜哪个才算数。每个阶段的写入权限清楚，脚手架的约束力来自结构本身，而不只来自 rules 的数量。

把这些判断放回 CodeNow，就能解释我为什么没有一开始追求“全自动”。我更关心的是：当一个新手说“我想做一个项目”时，脚手架能不能把他带到一个足够具体的位置；当 Codex 说“我做完了”时，我们能不能知道它到底做了什么；当下一轮继续时，新的 Codex 能不能不依赖上一轮聊天记忆，也能接住当前项目状态。

迁移到你的项目之前，先问六个问题

如果你准备为自己的项目设计脚手架，我建议先回答六个问题。

第一个问题：你的项目最常见的入口是冷启动，还是迭代？

CodeNow 把冷启动做得很重，因为它面对的是“一个不会工程化表达需求的人，如何把想法变成可运行项目”。但如果你的团队已经有稳定 PRD、设计稿、issue 和 CI，你的脚手架入口就不必从 01-mvp-prd-builder 开始。你可能更需要的是“拿到一个 issue 后，如何读取相关代码、列出影响面、补测试、跑验证、更新文档”。

如果你的项目主要问题是冷启动，就要重视意图澄清、非目标、最小可行范围和验收场景。如果主要问题是迭代，就要重视 TODO、变更影响面、回归测试、文档同步和发布前检查。入口不同，脚手架重心也不同。

第二个问题：你的项目最容易丢失哪类状态？

可能是产品意图丢失。第一次说得很清楚，三轮以后 Codex 开始追逐局部实现，忘记这个功能为什么存在。

可能是进度状态丢失。上次完成了一半，但下一轮不知道哪些任务已完成、哪些只是计划、哪些失败后还没修。

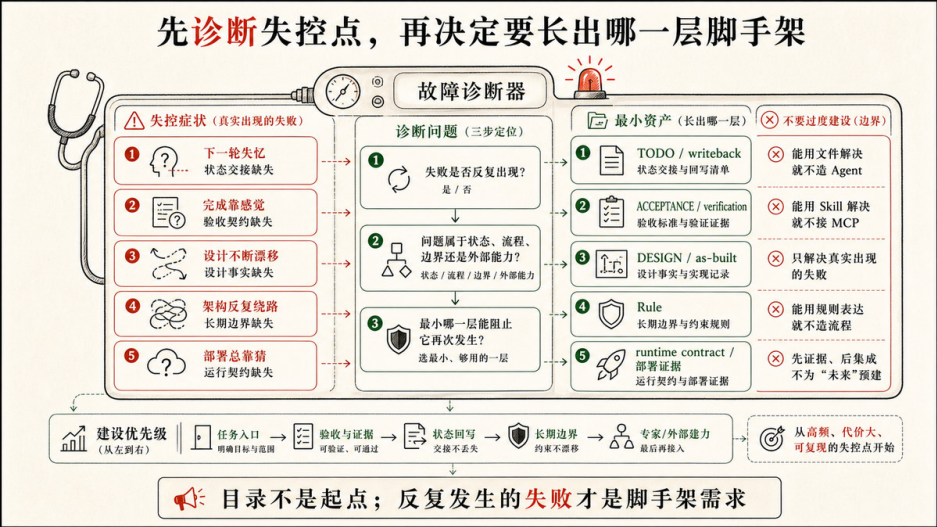
可能是证据状态丢失。测试跑过，但没有记录命令、结果、截图、日志和失败原因，过两天谁也不知道“通过”是什么意思。

可能是设计状态丢失。页面一轮一轮被改，视觉语言、组件语义和交互规则逐渐漂移。

可能是架构状态丢失。为了一小功能临时绕开模块边界，后面所有 Agent 都把这个绕法当成正常结构。

也可能是部署状态丢失。环境变量、外部权限、第三方服务和回滚步骤都在人的脑子里，Codex 只能猜。

不同状态要放在不同地方。产品意图适合放在 PRD 或模块说明里；当前执行适合放在 TODO 里；正式验收适合放在 acceptance 和证据目录里；设计约束适合放在 DESIGN 里；架构事实适合放在技术文档或 ADR 里；部署和密钥边界适合放在运维文档和规则里。把所有东西塞进一个 AGENTS.md，最终只会让入口变厚、事实变乱。



第三个问题：哪些事情必须在写代码前被确认？

不是所有任务都需要严格前置确认。改一个错别字、补一个按钮 hover 状态、调整一个字段名，直接做可能更好。但有些事情如果不先确认，后面改起来会很贵。

例如：目标用户是谁，功能边界在哪里，哪些数据不能上传，外部 API 没有 Key 时怎么表现，AI 生成内容是否需要来源提示，哪些页面必须适配移动端，哪些验收必须由人确认，哪些失败可以降级，哪些失败必须阻断。

这些问题不是“需求文档写得漂不漂亮”的问题，而是“Codex 在动手之前有没有足够的边界”。边界越模糊，Agent 的自主性越容易变成自作主张。

第四个问题：哪些场景可以 Codex 自检，哪些必须正式验收？

这也是前几节反复强调的分界。

Codex 自检适合确认本轮改动有没有明显错误：类型检查能不能过，页面能不能打开，关键路径有没有报错，单元测试有没有失败，日志有没有泄露敏感信息。

正式验收适合确认用户价值和合同场景：LeadFlow 的 AI 建议是否真的基于输入线索，来源追溯是否可见，预算不足时是否明确提示，外部模型未配置

时是否进入预期降级态，用户是否能按验收场景完成任务。

如果你把所有自检都写成正式验收，流程会重到没人愿意用。如果你把正式验收降级成“我自己看了一下”，脚手架就会失去可信完成的能力。真正的设计不是二选一，而是给不同风险配不同强度。

还有一个容易被忽视的设计点：验收效率不是在验收阶段才决定的，而是在开发阶段就被决定了一大半。如果实现时没有给 UI 主流程留稳定的 selector、没有给 API 调用和异常路径留脱敏日志、没有准备可以主动触发的边界 fixture，到了正式验收时，要么只能靠人眼点页面，要么发现哪里缺什么再临时补，验收反而变成了补债。好的脚手架会在 Skill 或 rule 里明确：实现阶段就应该为验收需要的入口顺手埋好位置——稳定 selector、关键路径日志和可复现的边界状态，在写功能时一起落地，不留到验收时才想起来。

第五个问题：UI、外部 API、隐私和合规有没有特别边界？

很多 Agentic Coding 的失败不在算法，而在这些“看似边缘”的地方。

UI 没有稳定 selector，自动化验收就很脆。没有设计系统，Codex 每轮都会发明一种新样式。没有截图证据，视觉问题就只能靠回忆。

外部 API 没有模拟数据和未配置态，Codex 就可能把认证失败、配额不足、网络错误、权限不足混成一种“功能坏了”。如果日志没有脱敏规则，调试越努力，泄露风险越高。

隐私和合规更不能只靠“请注意安全”。哪些数据不能写入日志，哪些文件不能上传，哪些操作必须用户授权，哪些测试只能用 fixture，哪些真实接口不能在默认流程里调用，都要变成可执行边界。sandbox、approval、permission gate 都很重要，但它们不是万能的。真正可靠的做法，是把权限防线、项目规则、工具边界和人工审查一起设计。

第六个问题：你的脚手架同时服务新手和专家吗？

如果只服务专家，脚手架可以更简洁。专家知道 PRD、TODO、acceptance、fixture、CI 和回归测试分别意味着什么，只需要你把入口和标准说清楚。

如果要服务新手，脚手架就要多做一层翻译。它不能把所有工程词汇一次性甩给用户，也不能假装这些东西不存在。更好的方式是：让新手用自然语言表达想法，由 Skill 把它逐步转成工程结构；让 Codex 在需要确认时问少量关键问题；让验收标准用用户能理解的场景表达；让证据路径保留给专家复查。

这就是 CodeNow 的双重目标：对新手来说，它降低表达门槛；对专家来说，它保留可追溯的工程证据。一个好的脚手架，最好也能同时照顾这两种人。

一条克制的建设顺序

如果让我给一个通用建设顺序，我会从轻到重这样做。

第一步，写一个薄的 `AGENTS.md`。

它只需要回答几个问题：项目如何启动，主要事实在哪里，任务开始前读什么，常用验证命令是什么，什么情况必须停下来确认，完成以后要回写哪里。它不是项目百科，也不是所有规则的仓库。它的职责是让 Codex 一进门不要走错。

一个实用的 `AGENTS.md` 可以包括：

- 项目结构的最短地图。
 - 当前任务状态的入口，例如 `TODO.md`。
 - 产品事实和已实现事实的入口，例如 `prd/`。
 - 验收标准和证据的入口，例如 `acceptance.md` 或 `test-results/`。
 - 常用命令：安装、启动、测试、构建、lint。
 - 明确禁止或需要确认的行为：真实付费 API、删除数据、改安全配置、提交或推送代码。
 - 完成汇报格式：改了什么，跑了什么验证，证据在哪里，剩余风险是什么。
- 写到这里就够了。不要一开始就把所有最佳实践都放进去。入口太厚，Codex 会读得慢，人也维护不动。

第二步，建立少数几条 rules。

rules 要从重复失败里长出来。比如：

- 生命周期路由：冷启动、日常迭代、Debug、正式验收、文档同步分别走不同入口。
- TODO 回写：代码行为、关键路径、验收状态变化后，必须更新当前执行状态。
- 验收契约：正式验收只能由指定流程写状态戳，自检不能冒充验收。
- 证据边界：截图、日志、命令输出、失败原因放在哪里，日志如何脱敏。
- 外部 API 边界：没有 Key、权限不足、网络失败时进入什么状态，默认流程能不能调用真实服务。
- 脚手架自检：改入口、rules、Skills 后，要检查悬挂引用、旧路径和相互冲突。

每条 rule 都要能回答三个问题：它什么时候触发，它拦住什么风险，它什么时候退出。如果回答不了，先不要写。

第三步，做三到五个高频 Skills。

Skill 不是长版提示词，而是可复用 workflow。它应该写清楚：输入是什么，先读哪些文件，产出哪些文件，哪些事不负责，什么时候要停下来问用户，什么时候可以继续。

对很多项目来说，最早值得做的 Skills 不是高级工具，而是最普通的几类：

- 冷启动：把模糊想法变成目标、非目标、约束和初始验收。
- 准备开发：把目标拆成 TODO、acceptance、设计约束和实现计划。
- 日常迭代：读取当前状态，实现小步变更，做自检，回写 TODO。
- Debug：复现失败，收集证据，做最小修复，不顺手重构。
- Verify：按验收场景执行，留下证据，写正式状态。

这些 Skill 不漂亮，但很有用。它们接住的是每天都会发生的工作。

第四步，给 UI 和体验建立设计事实源。

如果项目有界面，不要只让 Codex “做得好看一点”。这种指令太抽象，也太容易漂移。

你需要一个 DESIGN 或 design-system 文档，说明颜色、字体、间距、组件语义、按钮状态、空状态、错误态、加载态、移动端约束。也可以借用公开的 UI/UX Skill 或设计知识库，但要标清来源，并把它定位为设计辅助，而不是替代你自己的产品设计事实源。

UI 验收还要提前考虑 selector、截图、关键路径和视觉回归证据。视觉质量不是主观玄学，它至少应该能被场景、状态和截图部分地固定下来。

第五步，等流程稳定后再加 SubAgents、hooks、MCP 和插件。

SubAgent 适合读多、查多、互不干扰的任务：代码影响面扫描、验收场景规划、日志分析、安全检查、文档差异总结。它不适合一开始就被拿来并行改一堆共享文件。写入越多，协调成本越高，主会话越需要承担合并和裁决。

还有一种值得沉淀的设计模式，它不会增加写入协调成本：当某个任务需要产出一份重要规划——比如 MVP 方案、验收契约或架构决策——可以把作者和评审分成两个 SubAgent。作者 SubAgent 基于用户材料和澄清结果一次性产出自治的方案；评审 SubAgent 拿到同一份材料和方案，只读取、不改文件，从外部视角检查一致性、范围克制和关键假设。作者擅长让方案内部自治，评审擅长发现作者的盲区。一个方案刚刚成形时最容易显得合理，独立评审能在内容写进文件之前就拦住假设偷渡和范围膨胀——这种分离不是为了多派一个 Agent，而是在重要规划写定之前多一道可见的检查。

hooks 适合那些必须每次都执行的确定性动作，例如格式化、lint、规则检查、敏感信息扫描、路径漂移检查。它不适合替代人的判断，也不适合悄悄执行高风险操作。

MCP 和插件适合把外部上下文接进来，例如设计稿、文档库、浏览器、工单系统、监控、数据库或内部 API。但外部连接越多，权限、隐私、速率限制和误操作风险就越高。接入之前，先问清楚：Codex 需要读什么，能不能写，写之前需不需要确认，失败时如何降级，日志能不能记录。

第六步，定期给脚手架做体检。

脚手架也会腐烂：文件会改名，路径会失效，rules 会重复，Skill 会过期，TODO 会越积越长，acceptance 会跟代码事实脱节，SubAgent 的写入边界会慢慢变宽，原来只服务新手的 coach 模式可能混进正式执行流程。

所以，你需要周期性检查：

- AGENTS.md 里的路径是否仍然存在。
- rules 是否仍然对应真实风险。
- 是否有多处保存同一事实，导致冲突。
- Skills 是否写清“不负责什么”。
- TODO 是否还能作为当前状态，而不是历史垃圾桶。
- acceptance 是否仍然能被执行，而不是愿望清单。
- 证据目录是否能让新人理解某个功能为什么算完成。
- 脚手架是否仍有轻量路径，能处理小任务。
- 高风险任务是否仍然需要正式验收，而不是被“自检通过”吞掉。

这一步很容易被忽视。但脚手架越成功，越会被频繁使用；越被频繁使用，越需要维护。Agentic Engineering 不是一次搭好一个系统，然后永远享受自动化。它更像持续维护一种工作秩序。

每一层都要有边界，也要有删除标准

设计脚手架时，一个很实用的原则是：每加一层，就写清它不负责什么。

- AGENTS.md 不负责保存所有项目事实，它负责入口和路由。
- PRD.md 不负责记录每一次代码变化，它负责表达目标、非目标和用户价值。
- TODO.md 不负责解释产品为什么存在，它负责保存当前执行状态。
- DESIGN.md 不负责描述所有业务逻辑，它负责约束体验和界面语言。
- acceptance.md 不负责收集所有测试细节，它负责定义什么场景算完成。
- Skill 不负责所有临场判断，它负责一个可复用阶段。
- SubAgent 不负责最终决策，它负责在边界内完成局部调查或执行。

- hooks 不负责理解业务，它负责机械一致性。
- MCP 不负责证明外部事实永远正确，它负责把外部上下文接进来，并暴露必要约束。

如果一层没有边界，它就会开始吞掉别的层。AGENTS.md 会变成百科，Skill 会变成杂物间，TODO 会变成历史档案，SubAgent 会变成失控并行写手，hooks 会变成黑箱自动操作。

所以每一层也要有删除标准：

- 一条 rule 如果三个月没有触发过，也没有拦住任何真实风险，可以删掉或降级成参考。
- 一个 Skill 如果只有一句“请认真完成任务”，它不是 Skill，只是换了位置的提示词。
- 一个 SubAgent 如果每次都需要主会话重新做一遍判断，说明它没有提供足够清晰的局部价值。
- 一个 hook 如果经常误报，导致人习惯性忽略它，它就已经失效。
- 一个 MCP 连接如果读写边界说不清，宁可先不接。

轻量不是偷懒。轻量是为了让脚手架长期可用。复杂也不是错误。复杂必须来自真实风险，而不是来自“我听说 Agentic Coding 应该这么高级”。

脚手架要按真实风险逐层增加，也要能随时拆除

克制建设顺序

- 先入口与验证
 - 明确任务入口
 - 快速验证能否推进
- 再状态交接
 - 交接上下文与结果
 - 让下一步接得住
- 再长期边界
 - 设定行为与质量边界
 - 减少反复越界与失败
- 最后专家与外部能力
 - 专家视角与独立判断
 - 连接外部系统与权限

原则：能少一层就少一层
先内部可控，再向外扩展

有效/保留 观察/优化 过度/闲置

CodeNow 不是标准答案，只是一种把责任写清楚的方法。

6	MCP 外部系统与能力	加入条件 确有外部能力 与授权需要	拆除条件 连接闲置或 权限风险过高
5	Hooks 自动检查与触发	加入条件 关键检查常被 漏掉或晚发现	拆除条件 相关节点 不再存在
4	SubAgents 独立专业视角	加入条件 关键视角带来 明确收益	拆除条件 主 Agent + Skill 已足够
3	Skills 流程与步骤	加入条件 流程稳定重复 或易遗漏步骤	拆除条件 很少触发或 维护成本更高
2	Rules 规则与边界	加入条件 失败反复出现 或越界频发	拆除条件 风险消失或被 更好机制替代
1	薄入口 AGENTS.md	加入条件 任务需要入口与阶段定位	拆除条件 任务结束或不再需引导

三问维护闸门

- 1 仍解决真实问题？
如果移除会出现风险或失败？
YES NO
- 2 仍被触发？
近期是否被实际使用？
YES NO
- 3 收益大于成本？
价值 > 上下文/维护/
权限等综合成本？
YES NO

任一为 NO
→ 收窄、降级或删除

收窄范围 降级替代 拆除移除

写清意图，最小足够；保持轻盈，随时可撤。

最好的脚手架不是层数最多，而是每一层都有存在理由与退出条件

CodeNow 不是答案，而是一种写法

我写 CodeNow，不是为了给所有项目提供一个标准答案。CodeNow 的取舍有它自己的场景：它面对的是很多没有工程基础的人，他们有想法，也愿意动手，但不知道如何把想法变成可执行任务，不知道如何验收，不知道下一轮怎么继续。于是它把冷启动、准备、开发、迭代、Debug、验收和回写拆开，用 `AGENTS.md`、rules、Skills 和少量 SubAgents 把这些阶段串起来。

如果你的项目不同，你的脚手架也应该不同：

你可能不需要 `01-mvp-prd-builder`，因为你的团队已经有产品经理和 issue 模板；你可能不需要复杂的 UI 设计辅助，因为你的项目是后端库；你可能最需要的是安全规则、性能基准和发布门禁；你可能只需要一个很小的 `AGENTS.md`，加上一个 `debug` Skill 和一组 CI 命令；你也可能需要比 CodeNow 更严格的权限分层，因为你的项目涉及真实用户数据、支付、医疗、金融或公司内部系统。这些都正常。

真正可迁移的不是 CodeNow 的目录，而是它背后的几个判断：

- 让模糊想法先变成可执行意图，再变成代码。
- 让当前状态留在文件里，而不是留在聊天记忆里。
- 让自检和正式验收分开。
- 让失败进入 Debug 回路，而不是直接扩大修改范围。
- 让事实只放在最合适的位置。
- 让每条规则拦住真实风险。
- 让每个 Skill 有清楚的输入、输出和边界。
- 让证据成为“完成”的一部分。
- 让高风险任务保留人的确认权。
- 让脚手架自己也接受检查和维护。

这些判断组合起来，就是我理解的 Agentic Engineering——它不是一个比 Agentic Coding 更玄的词。它只是提醒我们：当 AI 不再只是补全几行代码，而是能读文件、改项目、跑命令、调用工具、写文档、修失败、回写状态时，我们就不能只设计一次对话。我们要设计一套工程系统，让它知道目标、知道边界、知道当前状态、知道如何证明、知道什么时候停下。

结尾：把能力变成可以托付的系统

Vibe Coding 很重要。它让想法迅速可见，让一个模糊念头变成可以点击、可以演示、可以继续讨论的东西。很多项目如果没有 Vibe Coding 的第一步，甚至不会开始。但一个项目不能永远停在“看起来有了”的阶段。

当你想继续改它、交给别人用、修复真实错误、接入真实 API、保护真实数据、面对真实用户、积累真实版本时，问题就变了。你需要的不只是生成能力，而是工程秩序。

这就是 Agentic Coding 的价值：人定义目标、边界、规则、权限和验收标准；Codex 在这些约束里读取上下文、选择路径、实现、验证、修复和汇报。它不是人退场，也不是 AI 失控。它是把人的判断外化成一个 AI 能读懂、能执行、能被约束的系统。

再往前一步，就是 Agentic Engineering。它关心的不只是这一轮代码能不能写出来，而是这一轮之后，项目有没有留下更好的状态。需求有没有更清楚，TODO 有没有更新，验收有没有证据，设计有没有收敛，规则有没有沉淀新的风险边界，下一轮 Codex 能不能接住今天的结果。

好的脚手架最后会变得有点朴素。它不会每次都炫耀自己用了多少工具，也不会让用户背一大堆术语。它只是让一个想法进来时不再那么飘，让一个任务开始时不再那么乱，让一个功能完成时不再只靠感觉，让一次失败出现时不再只能重来，让下一轮继续时不再从零开始。

对新手来说，这是一条更平缓的路。他不必先学会所有工程名词，才能做出自己的项目。但他也不会被“一句话生成产品”的幻觉带走，因为脚手架会把目标、边界、验收和证据一点点摆到他面前。

对专家来说，这是一条更可审查的路。他不需要手把手盯着 Codex 每一步，但可以看到 Codex 读了什么、改了什么、跑了什么、留下了什么证据、还有什么风险。

对项目来说，这是从一次性生成走向长期生长的路。每一轮之后，代码不只是多了一项功能，上下文更清晰了，验收更有凭据了，下一轮的起点更确定了。

你不需要一开始就搭出一个完整的 CodeNow。你可以从很小的地方开始：一个薄 `AGENTS.md`，一个当前 `TODO.md`，一个最重要的验收场景，一条从真实失败里长出来的 rule，一个能跑得起来的自检命令。

然后，在每一次失控之后问：这次问题应该留在聊天记忆里，还是应该沉淀到脚手架里？

- 如果只是偶然误会，解释清楚就够了。
- 如果它反复发生，就给它一个位置。
- 如果它影响完成标准，就写进 acceptance。
- 如果它影响当前进度，就写进 TODO。
- 如果它影响长期行为，就写进 rules。
- 如果它是一组稳定工作流，就写成 Skill。
- 如果它需要独立上下文，就交给 SubAgent。
- 如果它必须每次机械执行，就考虑 hook。
- 如果它依赖外部事实，就接入合适的工具或 MCP，并设计权限边界。

这就是 Agentic Engineering 的日常形态。不是一次宏大的架构设计，而是在一次又一次真实任务之后，把人的经验沉淀到系统里。

AI 编程真正成熟的标志，不是 AI 终于能替我们做所有事，而是我们终于学会怎样把自己在意的目标、边界和证据放进一个它能读懂、能在约束内工作的系统里。

当你做到这一点，Codex 的自主性才不再只是令人兴奋的能力。它会变成可以托付、可以限制、可以验证，也可以被下一轮工作继续接住的工程力量。

这本书写到这里，真正希望交给你的并不是一套目录模板，而是一种设计工作的方式：先看见想法，再安放事实；先定义边界，再释放自主；先留下证据，再相信完成；先承认人仍然负责，再让 AI 成为可靠的工程伙伴。

从这里开始，你就可以设计自己的 Agentic Coding 脚手架了。